

**Proceedings Scientific Training in Multi-Agent Systems Workshop
STMAS 2021**

Danny Weyns, Federico Quin, Omid Gheibi

Introduction

Scientific training is an essential part of academic education. The STMAS workshop provides a forum for academic students to present research and engineering contributions in the domain multi-agent systems. The event was organized by the Computer Science Department of the Katholieke Universiteit Leuven, Belgium.

The concrete goals of the workshop are:

- To present new research results and engineering efforts on multi-agent systems;
- To participate in the review of research papers submitted by other students;
- To present the research results via a short movie of 5 to 10 minutes (facultative).

In total STMAS accepted 16 papers. The best paper was awarded to: Dooms Lena, Ravoet Yannou, Tysebaert Stefaan, and Van Campenhout Simon for the paper: Communication-Based Interaction Protocols to Solve Packet-World Multi-Agent Environments. We hope you will enjoy reading the proceedings.

Leuven, 8 June 2021

Danny Weyns, Federico Quin, Omid Gheibi

Website event

<https://people.cs.kuleuven.be/danny.weyns/events/MASE/MASE-21.htm>

Table of Content

- *Wolfgang Mollmann, Wouter Jochems, Joao Cruz Ferreira Matos, The Packet-World: Developing Efficient and Robust Algorithms for Multi-Agent Systems*
- *Timen Celis, Thomas Decoster, Petra Fridrichova, Joshua Schroijsen, A* Based Approach to a Situated Multi-Agent System in the Packet-World*
- *Jan Wellemans, Maarten Van den hof, Arne Wouters, Michiel Pieters, Design of agent interaction protocols and patterns in a multi agent system*
- *Colpaert Bjarne De Heel Benson Echevarria Herrera Inaki, Direct and indirect communication combined with agent memory: our experience*
- *Hrvoje Abramovic, Joanna Kiesiak, Ward van Gerwen, Multi-Agent Systems: Enabling energy management and collaboration between agents in Packet-World*
- *Inias Peeters, Kristof Van Loock, Seppe Van Steenbergen, Development of a Multi-Gradient-Based Situated Multi-Agent System and Analysis of its Performance and Real-World Applicability*
- *Ioan-Alexandru But, Mateo Duren, Sebastian Michels A., Marion Waterkeyn, Multi-Agent A* based grid search implementation and evaluation in the context of package delivery*
- *Dooms Lena, Ravoet Yannou, Tysebaert Stefaan, Van Campenhout Simon, Communication-Based Interaction Protocols to Solve Packet-World Multi-Agent Environments*
- *De Smet Lennert, De Spiegeleer Pieter-Jan, Globevnik Jonas, Leenknecht Arno, Solving the abstract Packet World Multi-Agent System through Direct and Indirect Communication Patterns*
- *Jesse Geens, Joris Ignoul, Wouter Lenaerts, Emeric Goossens, Implementing Multi-Agent System behaviours for overcoming energy constraints and obstacles in The Packet-World*
- *Pieter Claeyss, Mathijs Hubrechtsen, Seppe Plomteux, Michiel Provoost, A comparison of environment-optimised algorithms for situated multi-agent systems in Packet World*
- *Gilles Meyhi, Miguel Russian Rojas, Rajat Sharma, Venkataraman Subramanian, Multi-Agent based Warehouse Management Automation*
- *Georgios Kouros, Gregoris Malekos, Bolat Tleubayev, Victor Van Wymeersch, Collaborative Situated Agents for Baseline Logistics Problems on the Packet-World Testbed*
- *Kristof De Troch, Robin Spiltijns, Emma van Doren, Maarten Wyns, Refining autonomous and collaborative behaviour in the Packet World*
- *Tobias Baert, Kobe Kwanten, Sofie Landuyt, Kobe Reygel, Introduction and evaluation of strategies and protocols for situated multi-agents with the use of the Packet-World test bed*
- *OmerYilmaz, Multi-Agent Systems - Basic Delivery and Energy Handling with Packet-World*
- *Aymeric Barbin, Mihaly Galambos, Exploring Multi-agent Systems solutions in the Packet-World paradigm*

The Packet-World: Developing Efficient and Robust Algorithms for Multi-Agent Systems

Wolfgang Mollmann, Wouter Jochems, João Cruz Ferreira Matos

Department of Computer Science, KU Leuven

Leuven, Belgium

wolfgang.mollmann@student.kuleuven.be, wouter.jochems@student.kuleuven.be,

joao.cruzferreiramatos@student.kuleuven.be

Abstract—The application of a decentralized multi-agent system (MAS) with situated agents can allow for much greater optimization and modularity than a centralized approach for certain applications. It is important that algorithms for MAS are efficient and robust in order to deal with all the uncertainties that can occur in real-life scenarios. This is still an active research topic and definitely not an easy task to solve. In this paper we will present and design a MAS from three angles with increasing complexity. The first stumbling block is that agents should be able to make autonomous decisions. Our solution to this problem is based on communication between agents, smart use of the agents memory and an efficient path-finding algorithm. The second problem lies in the fact that agents should be able to manoeuvre more difficult terrain and manage their own resources. We solved this issue by introducing a stochastic path finding algorithm and by extending our current communication algorithm. The third obstacle will be that the system should be able to handle restrictions on certain agents and allow for collaboration between agents to solve difficult tasks. Our solution extends the core algorithm from part one with these restrictions and introduces a help call in order to pass collaboration information to other agents. In conclusion, we believe that the solutions presented in this paper will be an inspiration for future applications of MAS in real-life scenarios.

Index Terms—Situated Multi-Agent System, Packet-World, Communication, Stochastic Path Finding, Memory, Collaboration

I. INTRODUCTION

A Multi-Agent System (MAS) can be defined as: "a loosely coupled network of problem-solving entities that work together to find answers to problems that are beyond the individual capabilities or knowledge of each entity" (Rocha et al., [1]). Over time, extensive literature on the study of MAS has been developed [2] [3] [4]. In this paper we will design a fully functional MAS and focus on three different angles. First, we introduce the notion of autonomous agent decisions. Secondly, the agents should be able to manage their own resources in particular energy consumption. Thirdly, agents should be able to collaborate for certain tasks. The implementation of our solution will be done in the Packet-World [5] which can be used as a test bed for investigating situated MAS. By using this approach we can conduct experiments that can quantify our found results. The remainder of this paper will be organized as follows. In Section II we introduce the Packet-world. Subsequently, we describe the problems in detail in Section III. In Section IV, we present our solution to these

problems. Afterwards, we discuss our results in Section V. In Section VI we discuss related work. Finally, we conclude our paper with the conclusions possibilities for future work in Section VII.

II. INTRODUCING THE PACKET-WORLD

The packet-world is a grid-based field where packets of different colors lay around. The goal of the agents is to pickup all these packets in the environment and to deliver them to their corresponding colored destination. An agent is allowed to manipulate the environment by placing down stigmergy in order to facilitate indirect communication. In the next subsection we introduce some further basic characteristics of the packet-world.

A. Main Concepts and Characteristics

- Each agent has a limited perception [6]:
The agents can only see the immediate environment around them. The view range determines how far away an agent can perceive the world.
- No global control:
There is no centralized global entity that decides the actions. Each agent decides their best course of action for themselves. But sometimes providing synchronization can be beneficial when multiple agents want to collaborate with each other. This happens locally between individual agents. However using this type of synchronization drastically can reduce the scalability of this world compared to a global clock
- Data communication:
Agents can communicate with each other directly in the packet-world. This allows them to pass information about packets and/or destinations to each other. This communication can also be used for more advanced types of collaboration like passing packages to each other. The packet-world also allows the use of indirect communication in the form of stigmergy. The paper discussed the use of different stigmergy: flags, pheromones, gradient fields, etc.

B. Real-World applications

The importance of situated multi-agent systems comes from the many real-world scenarios it can be applied to. In this

subsection two packet world applications to the real world will be presented. We refer to the paper "Situated agents can have goals" [7] for more background information on situated agents.

1) Self Driving Taxi-service:

One great example of a real-world application is a self-driving taxi-service. A series of recent studies has indicated that a MAS could indeed be a possible solution [8] [9] [10]. The taxis would be the agents, the clients would be the packets to be picked and the client's destination would map to the packet destination. The environment in which they perform are the public road rules, which strongly resemble the systematic rules in place for the grid in the packet-world. On top of this there are also the non-agent vehicles to look out for. Communication between the agents could warn for traffic jams, road blocks and new people to pick up.

2) Autonomous Transportation inside a Hospital:

Another example would be the autonomous transportation of patients and medicine in a hospital. A more comprehensive description can be found in the paper "A Multi Agent System for Hospital Organization" [11]. In this case there would exist delivery robots who would be the agents. The patients and medicine would be the packets to be picked and would be delivered to different rooms in the hospital, which would be the destinations. The environment would consist of several different floor plans with a limited number of elevators who also have limited capacity. Communication between the agents could be used to share the position of the elevators, as well as inform idle agents of jobs.

III. PROBLEMS DESCRIPTION

A. Autonomous Agent Behaviour

Each agent must be able to work autonomously. It will search the space for packets, pick up them up and then deliver them to a correct destination. An environment is considered solved when all packets have been delivered.

Agents have to deal with certain restrictions:

- Restricted vision: agents have a limited view of the space around them and will have to move around to get a more complete picture of their surroundings.
- Coloured packets: packets have a certain colour and can only be delivered at destinations of the same colour.

B. Agent Energy Management

The agents should be able to work with two new challenges introduced: Energy Management and Walls.

- Energy management is merged in the MAS as the key component able to increase the agent's efficiency, reliability and sustainability. The quantity of energy used it is dependent on each action taken, because different actions have different costs (Table I).

If an agent runs out of battery, this agent stops until the end of the simulation.

To charge their batteries, the agents must go to one of the energy stations and skip their turn. For each turn the battery will charge 100 until a maximum of 1000.

TABLE I
ACTIONS ENERGY COSTS.

Action	Cost
Step with Packet	20
Step without Packet	10
Skip	5
Other Actions (Pick Packet, Put Flag, ...)	5

To locate the stations agents are allowed to broadcast their location to every agent independent if they are within vision or not.

- The other challenge is in a specific environment ("energy-3", Figure 1). This environment contains walls, which are cells the agent cannot step into. The challenge of this environment it is due to the position of the energy stations. There is only one way to get to each charging station which is very specific and narrow as it is shown in Figure 1.

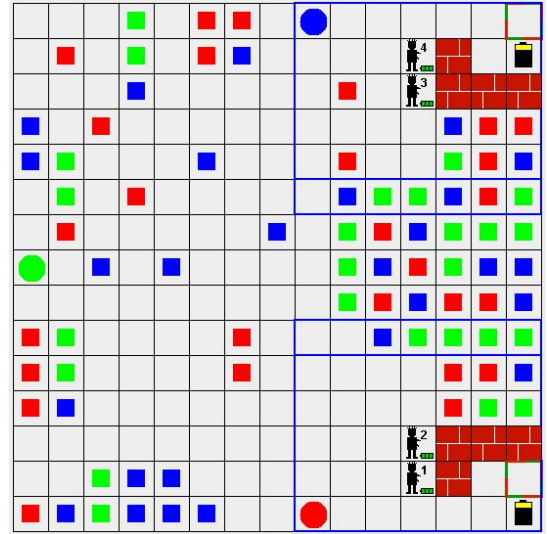


Fig. 1. "energy-3" environment

C. Agent Collaboration With Task Delegation

This part deals with colour restrictions and delegating tasks to other agents.

- Coloured agents: Agents can have a specific colour now which means they can only pick up and deliver packets of their own colour. They ignore packets of different colours even if no more packets of their own colour are present.
- Task Delegation: Agents might find their path blocked by packets that they are not allowed to pick up. In this case, another agent of the right colour needs to be made aware of the blocking packet and remove it.

IV. OUR PROPOSED SOLUTIONS

In this section we will explain and compare our proposed solutions to the problems of Section III.

A. Autonomous Agent Behaviour

1) Design:

This solution uses three main strategies: Efficient walk, memory and communication.

- Efficient Walk:

It is not always obvious how to implement a walk that is efficient enough to explore the whole map while not remembering anything but its last position. However there are still multiple ways to tackle this.

For this solution different walks were implemented and tested for different functions: wander (for when it is searching for the target) and walk_0/walk_1 (two different ways to walk to a known location).

- Wander:

In this walk the agent is searching either for a packet or for a destination. He moves randomly to one of the walkable neighboring cells. To explore the map the maximum possible, the agent does not walk back to its previous position unless it has no other option.

- Walk_0:

When the agent already knows where to go (target within vision or not) it does not need to aimlessly wander anymore.

Instead, it can just take the shortest path to it. In this algorithm the agent performs a greedy search. It checks all the possible moves, grades them with a distance metric and then chooses the best.

The distance metric used in this walk is the Manhattan distance from Equation 1 (inspired by Manhattan where it is needed to walk around the blocks).

$$distance = |x - x_f| + |y - y_f| \quad (1)$$

Using a greedy search gives a very fast agent who goes straight to the goal location. However, when there are obstacles in the way it can be challenging to not get stuck in a deadlock always following the best metric.

- Walk_1:

With the limitations of the previous walking algorithm in mind, some changes were made: to the distance metric and a stochastic approach. The Manhattan distance metric is optimized for 4 point connectivity moves (like Manhattan blocks) and not for 8 point connectivity moves like the Packet World. A better alternative it is to use distance metric of Equation 2. This distance metric not only gives the correct amount of moves needed to arrive to the goal as it does as well give more possibilities of paths to arrive to the goal in the smallest amount of moves.

$$distance = \max(|x - x_f|, |y - y_f|) \quad (2)$$

To deal with obstacles in the direct path, it was taken a stochastic approach. Where instead of taking the last found closest cell. It creates a list with all the possible steps which will take the smallest amount of moves (often there are more than one possibilities) and randomly selecting from it. This is able to prevent most of the loops seen in Walk_0. To improve further what is already a very efficient walk, the agents are not allowed to walk back to their last position unless no other option. Even with all these measures, the agent still managed to get stuck in a triangle loop in some very specific environments. To avoid that, it is added a stochastic component. Where in 1% of the moves, the agent goes back to its previous position (breaking the loop).

This elaborate walking method, produced very good results for all the default environments, it is able to explore well and avoids getting stuck.

- Memory:

To better improve the agents efficiency, the agents have a limited memory.

This memory can be used in numerous ways, in this solution the memory was used to store the locations of the destinations and of the last seen packet. This prevents the agent from searching for already found destinations or packets, and instead they can retrieve the locations from memory and head straight there.

- Destinations:

The agent stores its last seen destination of each color. However, if an agent is heading for its saved destination and on the way it sees a new destination which is closer, it will shift route and go for this closest one and save this new destination to memory.

- Packets:

When walking around in the map the agent always stores its last seen packet (only one packet in memory at any time), so that it can head there when it is looking for a packet. When a packet that was in memory is picked by the agent, this agent removes that packet location from its memory, and will now memorize any new packet that sees on its way to the destination. If in the meantime another agent B picks the packet which agent A remembered, agent A will still go towards that packet, and when it sees it is no longer there, it will delete it from its memory and wander until a new one is found.

- Communication:

"The results show that Cooperative MAS model is suitable for distributed problems which can be divided to sub-problems. Solving and dividing these sub-problems can be done cooperatively." [12] This encourages us to use the communication of agents to cooperate with one another. We also chose to use asynchronous collaboration between agents as this meant we did not have to implement the complex simultaneous actions [13]. Before the acting

phase (where the agents choose and perform an action), there is a communication phase. During this phase the agent is allowed to send messages to agents which are within its vision. This messages can contain small strings of information. The communication capability creates many possibilities of interaction and cooperation between the agents. In this solution the communication is used to share locations of destinations and packets. The receiving agents can store this informations to memory, this will prevent them from wandering randomly searching, and instead the agents will use the already know locations.

- Destinations:

The agents can share the locations and colors of the destinations they have in memory (up to one destination for each color). The receiving agents will only write those destinations to memory if they do not already have one destination of that specific color in memory.

- Packets:

Every time an agent sees another agent, it shares its last seen packet. The receiving agent however, will only save the packets location to its memory if it does not already have a last seen packet saved. This helps the agents who are wandering randomly to follow other agents who know where packets are, while preventing if possible all agents to go the same packets.

- Optional Design Elements:

- Flags:

In the packet-world the use of flags (Figure 2) has been discussed as a possible way to enable indirect communication between agents. Currently when agents have no information on where the packets are located, they will wander around aimlessly until they find one. Flags could be a potential solution to this problem and could further optimize our solution. They can be placed in areas where there are no more packets to find, so that the agents do not have to search these areas. There are also countless other functions these flags can fulfil, like for example using color flags to communicate the path until a destination [14]. Our current implementation places the flags in a pattern that keeps a distance of 2 between flags on the same axis. The agents, when looking for new packets, will follow a path that stays clear of destinations, the barrier of the world and the flags. This makes sure that the agents will only look for packages in the open spaces and also utilizing the fastest path to this location. In rare cases in our implementation the agents can get stuck in a loop between flags which results in significantly worse energy consumption and amount of turns. This is why we have decided to add this design element as an optional mode to our solution.

2) Implementation:

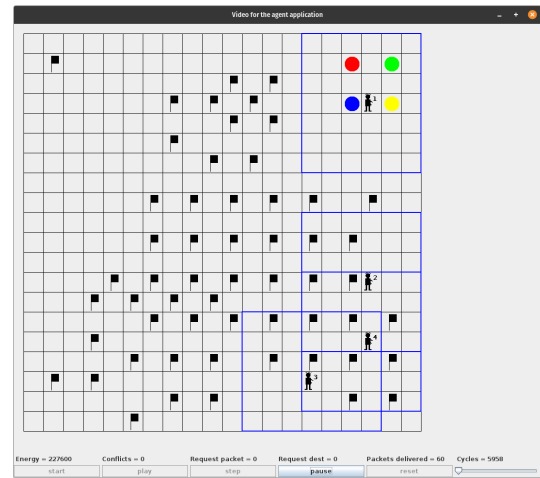


Fig. 2. "basic-3-color" with flags.

To implement this design solution, we used different behaviour classes:

- Search:

This behaviour class is used when the item that is searching (either it is carrying a packet and looking for a destination or it is not carrying a packet, and looking for a packet to pick) is not within vision.

It can be divided in two specific situations:

- The location of the wanted item is not known: There it uses the Wander mode, which walks randomly until the target item is observed.

- The location of the wanted item was retrieved from memory, but it is still not within vision: There it uses Walk_0 or Walk_1 to head straight to the correct location until the location or a similar target item is within vision. In both situations the use of the class finishes once the target item is within sight. If the target item was a packet then it goes to PickUp class, otherwise if the target item was a destination it goes to the Deliver class.

- PickUp:

This behaviour is for when the agent it not carrying a packet but it has at least one within vision.

In this class the goal is to walk to the packet and pick it up. Once that is done successfully, it passes to the delivery mode (if destination within vision) or to search mode (if destination not within vision).

- Deliver:

It is used when the agent is carrying a packet to deliver and has its destination already in sight. The goal is to walk in the smallest amount of moves to the destination and deliver the packet. Once that is accomplished, the agent goes back to looking for packets. If one is in sight it goes to the class PickUp, otherwise it goes to the class Search.

3) Test Results:

Our data resulted from averaging the outcomes of 500 runs.

- Walk_0 vs Walk_1:

From the results in Table II it can be seen that on average the two algorithms perform very similar. When comparing the worst possible results walk_1 performs much worse in amount of turns, because of the stochastic component of the algorithm. This is why we prefer to use the walk_0 algorithm for our part 1 program.

TABLE II
WALK_0 VS WALK_1

Walk	Map	Turns (Average)	Energy (Average)	Max Turns	Max Energy
walk_0	1-tutorial	165	2180	165	2180
walk_1	1-tutorial	168	2225	184	2530
walk_0	2-vision	392	9440	1167	25950
walk_1	2-vision	384	9437	1320	27680
walk_0	3-color	625	37433	1575	107600
walk_1	3-color	704	40470	2764	106860

- No Communication vs Communication: The results for this test are presented in Table III it can be concluded that on average not having communication performs slightly worse on the second map, and much worse on the third map. It was expected that the third map would perform considerably worse because of its bigger size where communication is the key for a good performance. For the second map not communicating does not change as much because the agents rely more on their own memory.

TABLE III
NO COMMUNICATION VS COMMUNICATION

Mode	Map	Average Turns	Average Energy	Max Turns	Max Energy
Comms	2-vision	392	9440	1167	25950
No Comms	2-vision	427	10386	1293	28520
Comms	3-color	625	37433	1575	107600
No Comms	3-color	1204	72170	4930	261520

- No Flags vs Flags:
The test of the flags is presented in Table IV. On average adding flags to our implementation only slightly improves the average amount of turns and energy. The main reason for these mediocre results is the fact that our agents can sometimes get stuck between flags. This can be seen in the worse case scenario results.

TABLE IV
NO FLAGS VS FLAGS

Mode	Map	Average Turns	Average Energy	Max Turns	Max Energy
No Flags	2-vision	392	9440	1167	25950
Flags	2-vision	388	9087	4821	95155
No Flags	3-color	625	37433	1575	107600
Flags	3-color	649	37469	4770	221470

Both our initial and structured implementation, as well as our optimised implementation eventually find and deliver all packets. From our test results it becomes clear that our optimisation outperforms the initial implementation, but still relies heavily on random wandering to find packets which impacts

overall efficiency. The basic-tutorial environment gives the exact same results for all runs and all behaviours because the random factor is not present.

B. Agent Energy Management

1) Design:

The design for this problem is separated in two distinct challenges: Energy Management and Walls Environment.

• Energy Management

This feature consists of giving the agents a battery level, which decreases depending on their actions (Table I). To charge the battery, agents must go to an empty energy station and stay there until the battery is full.

In this implementation the agents start looking for a energy station when they are low on battery (40% battery). This gives them time to go straight to the energy station. However if they still have to look for its position it is likely that they will run out of battery and be static for the rest of the simulation.

To make sure the agents get to the charging station before running out of battery, there are multiple solutions:

– Memory:

When a energy station is found, the agent can save the stations location to its memory, so it will know where to go when it needs to charge. However, this does not completely solve the problem which means additional solutions are needed.

– Communication:

To complement the memory solution, agents are allowed to broadcast the energy stations location to all other agents (within vision or not). With this additional solution the problem is solved and agents no longer run out of battery because of unknown charging locations.

A small solution which significantly increased the performance, was to have the agents drop their packets whenever they are low on battery. This way they save energy by walking without packets (Table I).

• Walls:

In this specific environment when the agents need to charge, they often struggle to find a path to the energy station, even with Walk_1 which already explores much more than the previous Walk_0. So the agents sometimes run out of battery before discovering how to get to the charger.

To solve the Walls environment (Figure 1) a better alternative to the current Walk_1 was developed:

– WallWalk:

The solution for this, was to create a more advanced walk, which is able to go around objects like in Figure 3. To implement this, it was created an optimized depth first search which tries all the possible routes to the charging station.

It searches neighbors that are walkable (no wall, no packet, ...), then for each of this neighbors it searches

solved very easily by the WallWalk, while Walk_1 fails to solve it.

TABLE VI
WALLWALK VS WALK_1

Mode	Map	Average Turns	Average Energy	Max Turns	Max Energy
WallWalk	1-energy	994	32347	1863	56920
Walk_1	1-energy	1077	35768	1663	54595
WallWalk	2-energy	751	53807	781	55120
Walk_1	2-energy	742	53661	866	60470
WallWalk	3-energy	622	28358	942	41050
Walk_1	3-energy	-	-	-	-

C. Agent Collaboration With Task Delegation

1) Design:

These colored agents are restricted with the packets they can pick and manipulate according to the color of agents. Whereas a regular black colored agent can pick up any color of packets, an agent with a specific color can only pick up packets with its color. Task delegation becomes a separate behaviour of the agents. This behaviour occurs when an agent finds a packet on its path to a destination. It will start sending messages towards agents in range. Agents receiving such a message will change their own behaviour to help the requesting agent, if they have the correct colour. If they receive a request they cannot help with, it is simply ignored. After completing the request they send an "ok" message to the requesting agent. After receiving an "ok" message, the requesting agent will revert back to search behaviour.

2) Implementation:

We adapted our original behaviours to take the colour restrictions into account and to check messages for help requests. We also added two new behaviours in which the agent either attempts to delegate a task or accepts and executes a task.

- Search:

The agent walks randomly in the environment to search for a packet or destination with the correct color by invoking the getColor() method for comparison.

The agent will also check received messages for help requests and will change to Help behaviour if it can complete the request.

- PickUp:

The agent will move towards and pick up a packet of a correct colour.

The agent will also check received messages for help requests and will change to Help behaviour if it can complete the request.

- Deliver:

While delivering the packet, the agent looks for the shortest path towards the destination. It then checks if the next step in this path contains a packet and if so, switches to TaskDelegation behaviour.

The agent will also check received messages for help requests and will change to Help behaviour if it can

complete the request. In that case it will also send a positive response towards the requesting agent, provided that it is still in range.

- TaskDelegation:

The agent will look for agents in range and sends a help request to any agent it finds. The agent also checks if it can pick up the blocking packet itself and if so, delegates the task to itself. The agent will move close to the center of the environment in hopes of finding other agents. If on its way there it encounters an agent which can complete its request, a "wait" message is received which means the encountered agent has accepted the task. This causes the requesting agent to wait for an "ok" message, signaling that the task is completed. This movement towards the center is similar to the "meeting" pattern described by Aridor et al. [15] but differs in the fact that agents might stop and wait before reaching the meeting point. The agent will also check received messages for a response to its help request and will abandon TaskDelegation behaviour if a response was received. If a help request from a different agent is received, it will change to Help behaviour if it can complete the request.

- Help:

The agent first drops its packet if it is currently holding one and if there are at least 2 free spaces neighbouring it. The two spaces are needed since otherwise the agent would block itself. If only one free space is available the agent will move to that space and attempt to put its packet down again.

When the agent is no longer carrying a packet it will move towards the coordinates it received in the help request. Once the coordinates are within its visual range it will check if there is still a packet in that spot and if so, the agent will move in and pick up the packet. After picking up the packet or when there isn't a packet in the spot anymore, the agent will update its task in order to move towards the center. When the agent who send the help request is in range a "ok" message is send. The helping agent will then remove the task from it's memory and skip a turn changing back to Search behaviour on its next turn.

3) Test Results:

The results for this experiment are presented in Table VII. The agents were able to easily solve the environment in relatively few moves.

TABLE VII
TASK DELEGATION

Map	Average Turns	Average Energy	Max Turns	Max Energy
task-delegation	303	11276	674	22120

The part of the task delegation to unblock all the passages is consistently completed in fewer than 70 turns. The rest of the turns are the agents delivering the colored packets. To optimise

this part agents remember one additional packet of each color and share these locations with other agents.

V. DISCUSSION OF THE RESULTS

Our solutions perform well in their respective environments. The added optimisations produce significant improvements in performance and prevent agents running out of battery in the charging environments. Committing locations of importance to memory yielded the greatest improvement overall, due to a lesser reliance on random wandering. The simple communication between agents reduced average turns and energy spent by up to 50%. This effect is more noticeable for agents with a smaller perception range in larger maps because under these conditions wandering agents will take a longer time to explore the environment. The use of flags can further enhance performance, but our implementation only slightly improved the results. This is likely due to agents getting stuck between the flags. While our specialised WallWalk function did not have a significant positive impact on performance, in some cases even proving less efficient than our original walk function, it was able to complete a more complex environment for which our regular walk could not produce a solution. Finally, our task delegation solution was also able to correctly solve its environment. We did not perform extensive experiments with this implementation, as most of its optimisations are variations of previous solutions. Other improvements resulted in the elimination of certain edge cases where agents would get stuck and failed to complete the environment.

VI. DISCUSSION OF RELATED WORK

Some of the most interesting questions in this context are the possible implementation of self-adaption in MAS [16] [17] [18]. Instead of configuring the agents with pre-defined parameters, we let self-adaptive agents tune these parameters continuously while collecting packages. This does not only ensure a higher degree of modularity, but also more robustness when dealing with uncertainties that were before unforeseen. In general there are still many tough challenges to solve in this domain in order to make self-adaptive systems viable in real-life scenarios. But we strongly believe as the field of self-adaptive systems does progress that the field of MAS will only benefit from this collaboration in design.

Weyns et al. [19] introduces a communication phase separate from the perception and action phases. Our solution also makes use of this but a critical difference is that our communication phase is asynchronous while their LT/D scheme uses synchronised communication consisting on multiple messages back-and-forth during a single phase. Their approach results in more reliable communication whereas with our solution multiple agents might receive and respond to the request for help which might cause them to interfere with one another. However, this more complex form of inter-agent-communication is needed in their scenarios where agents have to synchronise their actions to a specific time whereas in our scenario exact timing is not a concern.

In the paper "A Multi-Agent Framework for Packet Routing in Wireless Sensor Networks" [20] explains a framework that can assist existing communication in order to improve performance. This algorithm is based on building a model that is based on past routing in order to predict future message passing more efficiently. This could be a possible extension to our solution in order to further optimize energy consumption.

VII. CONCLUSIONS AND FUTURE WORK

The application of MAS allows for lot of flexibility when implementing solutions. We can clearly see that a decentralized approach can lead to great results when solving the packet-world. We have shown that implementing robust algorithms is a necessity for MAS in order to complete their task in a reasonable time. In designing our solution we have primarily focused on three important aspects that a MAS should be able to achieve. In order to achieve autonomous agent decisions our solution relies on communication, memory and an efficient path-finding algorithm. Here we can come to the conclusion that enabling these features drastically improves the performance of the agents and also brings great consistency into the results. The agents in our second part were able to manoeuvre around walks by introducing a stochastic walk algorithm that definitely improves our result in complex environments. Extending the communication algorithm also benefited the results well that allows the agents to make smarter decisions about their resource management. The agents in our third part were able to achieve collaboration by the means of collaboration. Our solution was constructed with modularity in mind so that further restrictions could be easily added.

In the bullet list we sum up future work in order to further optimize our presented solution.

- Improving Wander:

Currently the agents still wander too much around randomly. Here is definitely room for improvement in order to more efficiently explore the environment.

- Energy Management:

A possible improvement could be to communicate to which energy station agents are going, and naturally separate the agents from all going into a already very busy energy station. Another improvement could be to instead of purely managing their resources strictly by communication. The agents could be extended with indirect communication to further improve our results. One example is the use of a gradient field. The gradient values would indicate how far a certain position would be from the nearest energy station, informing agents of how much energy they would need to reach it. Agents could find their way to a charging station by following the direction of the decreasing gradients value.

- Flags:

Improving our current implementation in order to achieve much more consistent results without the agents getting stuck. Also an extension to this algorithm by using multiple colors in order to more efficiently guide the agents can be very beneficial.

Another important limitation that we have not taken into account in our solution is the uncertainty of the environment. For the application in real-life scenarios the agents should be able to handle unexpected events in a timely manner. This is a crucial step before MAS can actually be deployed in real-life scenarios. That is also why MAS to this day is still a very active research topic. In conclusion, we believe that the solutions presented in this paper will be an inspiration for future applications of MAS in real-life scenarios.

REFERENCES

- [1] J. Rocha, I. Boavida-Portugal, and E. Gomes, "Introductory chapter: Multi-agent systems," in *Multi-Agent Systems*. IntechOpen, 2017.
- [2] M. Wooldridge, *An introduction to multiagent systems*. John Wiley & sons, 2009.
- [3] J. Ferber and G. Weiss, *Multi-agent systems: an introduction to distributed artificial intelligence*. Addison-Wesley Reading, 1999, vol. 1.
- [4] D. Weyns, *Architecture-based design of multi-agent systems*. Springer Science & Business Media, 2010.
- [5] D. Weyns, A. Helleboogh, and T. Holvoet, "The packet-world: a test bed for investigating situated multi-agent systems," in *SOFTWARE AGENT-BASED APPLICATIONS, PLATFORMS AND DEVELOPMENT KITS, WHITESTEIN SERIES IN SOFTWARE AGENT TECHNOLOGY*, 2005.
- [6] D. WEYNS, E. STEEGMANS, and T. HOLVOET, "Towards active perception in situated multi-agent systems," *Applied Artificial Intelligence*, vol. 18, no. 9-10, pp. 867–883, 2004. [Online]. Available: <https://doi.org/10.1080/08839510490509063>
- [7] P. Maes, "Situated agents can have goals," *Robotics and autonomous systems*, vol. 6, no. 1-2, pp. 49–70, 1990.
- [8] M. Lu, M. Taiebat, M. Xu, and S.-C. Hsu, "Multiagent spatial simulation of autonomous taxis for urban commute: Travel economics and environmental impacts," *Journal of Urban Planning and Development*, vol. 144, no. 4, p. 04018033, 2018.
- [9] R. Vosoghi, J. Kamel, J. Puchinger, V. Leblond, and M. Jankovic, "Robo-taxi service fleet sizing: assessing the impact of user trust and willingness-to-use," *Transportation*, vol. 46, no. 6, pp. 1997–2015, 2019.
- [10] G. Inturri, M. Le Pira, N. Giuffrida, M. Ignaccolo, A. Pluchino, A. Rapisarda, and R. D'Angelo, "Multi-agent simulation for planning and designing new shared mobility services," *Research in Transportation Economics*, vol. 73, pp. 34–44, 2019.
- [11] H. Jemal, Z. Kechaou, M. B. Ayed, and A. M. Alimi, "A multi agent system for hospital organization," *International Journal of Machine Learning and Computing*, vol. 5, no. 1, p. 51, 2015.
- [12] A. A. Abusnaina and C. H. Yong, "Cooperative multi-agent system for solving packet world problem in grid," in *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*. IEEE, 2009, pp. 754–758.
- [13] D. Weyns and T. Holvoet, "Synchronous versus asynchronous collaboration in situated multi-agent systems," in *Proceedings of the second international joint conference on Autonomous agents and multiagent systems*, 2003, pp. 1156–1157.
- [14] P. E. Petrucci, "Dealing with norms within the packet world multi-agent simulator," Master's thesis, Universitat Politècnica de Catalunya, 2010.
- [15] Y. Aridor and D. B. Lange, "Agent design patterns: Elements of agent application design," in *Proceedings of the second international conference on Autonomous agents*, 1998, pp. 108–115.
- [16] M. Delabie, K. Kulak, and D. Weyns, "A modular approach for engineering situated multi-agent systems with self-adaptation."
- [17] D. Weyns, "A pattern language for multi-agent systems," in *2009 Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture*. IEEE, 2009, pp. 191–200.
- [18] D. Weyns and M. Georgeff, "Self-adaptation using multiagent systems," *IEEE software*, vol. 27, no. 1, pp. 86–91, 2009.
- [19] D. Weyns and T. Holvoet, "Model for simultaneous actions in situated multi-agent systems," in *German Conference on Multiagent System Technologies*. Springer, 2003, pp. 105–118.
- [20] D. Ye, M. Zhang, and Y. Yang, "A multi-agent framework for packet routing in wireless sensor networks." *Sensors (Basel)*, 2015.

A* Based Approach to a Situated Multi-Agent System in the Packet-World

Timen Celis, Ir. Thomas Decoster, Petra Fridrichová, Joshua Schroijsen
KU Leuven

Leuven, Belgium

timen.celis@student.kuleuven.be, thomas.decoster1@student.kuleuven.be,
petra.fridrichova@student.kuleuven.be, joshua.schroijsen@student.kuleuven.be

Abstract—Multi-Agent Systems (MASs) is a research field with a broad range of applications such as E-commerce, transportation system and air traffic control. In this paper, we demonstrate the development and evaluation of a Situated Multi-Agent System (SMAS) in the Packet-World model to which many real-world problems can be mapped. The Packet-World problem is a case study in the literature to examine the cooperative actions between agents. The main goal of the problem is to clean an environment from packets while minimizing the time, number of steps and energy usage. The autonomous behaviour of agents, coordination mechanisms and cooperation strategies are analyzed, discussed and supported with experimental results for different scenarios.

Index Terms—Multi-Agent Systems, Packet-World, Situated Multi-Agent Systems

I. INTRODUCTION

A large selection of problems can be regarded as a Multi-Agent System (MAS). In this situation, solvers called agents work together autonomously to solve a given problem that is beyond the capabilities of a single agent. Key characteristics of a MAS are:

- Limited agent's perception/knowledge,
- Lack of global control,
- Cooperative or competitive agent behaviour,
- Asynchronous agent action.

The concept of the MAS acts as a general design framework. It can be created with a wide variety of intra-agent or inter-agent structures. Agents can vary from simple reactive ones to complex reasoning ones. The population can be divided into selfish agents pursuing their own interests and into cooperative agents sharing a set of common goals that are realised by helping each other. [1] Although designing MASs is a complex problem, there are many design patterns in the literature that substantially ease this task. Furthermore, the benefits of MASs might easily overcome the design difficulties. Thus, many robust, open and flexible systems were invented using the MAS model that tackles intractable real-world problems, such as the autonomous guided vehicle systems in the logistics sector. In this paper, we will discuss and demonstrate the design, implementation and evaluation of a MAS within the Packet-World framework/model by Weyns, Helleboogh & Holvoet [2].

The (goal-oriented) autonomy of agents is a defining property of MASs. It is believed that such systems adapt their behavior better to changing circumstances. However, splitting up a

problem solving method into parts that can be executed by a set of agents is a challenge by itself, especially when the agents' communication options can vary. The traditional AI approach of symbolic reasoning based on a knowledge base suffers from issues such as brittleness, inflexibility and a lack of real-time reaction [3], [4]. As a result, in [5] Brooks adopts the radically new approach to defining autonomous behaviour called situatedness. Situatedness means that agents do not deal with abstract descriptions but are directly coupled with their dynamic environment and their perception of it (for which a models are provided by Weyns et al. [6] and Weyns, Steegmans and Holvoet [7] respectively). In other words, an agent's behaviour should be defined as a function that maps its immediate perception of the environment to an action. We will adopt this modern approach to define a simple but effective set of behaviours for our agents.

This paper is structured as follows: Section II briefly introduces situated MASs and how they can be reduced to the Packet-World. The different Packet-World problems that our approach has to overcome are covered in Section III. In Section IV, we elaborate on our solutions to the problems. Then the results of the implemented approaches are presented in Section V. This is followed by a discussion about the limitations and lessons learned in Section VI. Next, related works in the literature to this project are discussed in Section VII. Finally, a short conclusion is given in Section VIII.

II. THE PACKET-WORLD MODEL AS SITUATED MAS

MASs present a large class of distributed systems. Weyns, Helleboogh and Holvoet [2] present a model called the Packet-World. It provides an abstraction to a particular class of MASs referred to as Situated MASs (SMASs) [8]. The authors also provided a Packet-World-based test bed for tackling a number of open research questions related to SMASs and exploring SMASs development.

A. Key Concepts of SMASs

To be able to use the Packet-World-based test bed, we have to be able to convert a problem to the SMAS. Hence, we define the five key concepts of a SMAS.

1) *Situatedness*: The system's agents are situated in the present time, act within a local environment, encapsulate their

behavior and maintain their own state. A situated MAS is completely decentralized.

2) *Operational scarcity*: Agent's actions are limited by its consumption of an operationally finite resource, e.g. energy. Minimizing the consumption corresponds to increasing the system's efficiency.

3) *Stigmergy*: Stigmergy is indirect communication between agents, by placing markers such as flags or pheromones in the environment. Thus, the agents might influence each others behavior by the markers. This concept is borrowed from the study of social insect colonies [9].

4) *Protocol-based communication*: Besides stigmergy, agents communicate peer-to-peer by exchanging messages. To be able to read the messages, they have to follow a set of rules called the communication protocol. This concept was modelled and studied by Weyns, Steegmans and Holvoet in [10].

5) *Goal*: The purpose of agents is to execute a given task. Two real-world goals are presented in the following section.

B. Reducing SMASs to the Packet-World

The Packet-World consists of a two-dimensional rectangular grid containing coloured packets and destinations. These packets will be referred to as packages for the rest of this paper. The agents have to clear the grid by bringing the coloured packages to the destinations of the corresponding colour. The goal is to take advantage of the number of agents to do the task as fast and efficiently as possible.

The Packet-World test bed enables to easily evaluate new approaches. Therefore, it is advantageous to convert SMASs to the Packet-World. In this section, we present two real-world problems that conceptually map to the Packet-World test bed. On the one hand, we describe the food delivery problem in which packages have to be transported from a given place to a destination. On the other hand, there is a security guard problem in which the goal is to keep an event in order.

1) *Food Delivery*: In [2], an Automated Warehouse Transportation System has to deliver a load from a particular location to a given destination. In the Food Delivery problem, the same idea of package transportation is given [11]. Delivery persons owning a smartphone (agents) deliver food from a restaurant to customers. The agents cooperate to execute all orders in minimum time based on their position, capacity and battery level. Also, an agent can transport multiple orders at the same time if the customers are close to each other. When an agent takes an order, the other agents should not work on it anymore. Furthermore, if an agent runs out of battery, the other agents should be able to continue delivering all orders without that agent.

2) *Security Guards at an Event*: The second real-world problem is an arrangement of security guards at an event (e.g. a festival) [12]. The security guards, representing agents in the Packet-World, have to ensure safety of everyone at the event by intervening in situations where the safety cannot be guaranteed. The situated agents are scattered around the event area to have maximal coverage. When an unsafe situation is detected by one or multiple agents, communication might be

established to resolve the situation. There have to be enough agents to handle the situation safely while keeping plenty of agents on the lookout to identify possible other problems. Based on the observations, the MAS has to decide whether a backup agent is needed. Additionally, the intervening agents have to be chosen such that the problem gets resolved as fast and efficient as possible. It corresponds to the Packet-World problem where the agents have to collaborate to pick up and deliver packages to destinations efficiently to minimize time, steps and energy usage.

III. THE PROBLEM STATEMENTS IN THE PACKET-WORLD

For this project, different scenarios with increasing difficulty are considered. The scenarios are grouped by different problems that have to be overcome and they are split into 3 stages.

In the first stage, the MAS has to perform the basic task of delivering packages to the destinations. Next, rechargeable batteries for each agent and charging stations are introduced. Moreover, in this phase the agents might encounter some obstacles, such as walls. Finally, obstructions are introduced between the tasks of different agents, for whose resolution task delegation is needed. The different tasks are discussed separately in the following sections.

A. Autonomous Behaviour

In the first stage, a simple Packet-World test bed are considered. The agents move within a rectangular grid referred to as the environment. The environment contains only coloured packages and their corresponding targets. The goal is to provide a strategy that delivers all packages to their targets as fast as possible while minimizing energy consumption. The agents have to decide which action they are going to perform based on the area they can see in the environment called the perceptive field. Walking to the eight neighbouring cells or waiting are considered legitimate actions.

B. Energy Management

For the second task, an agent's energy and charging stations are introduced. In the MAS, energy management is crucial as the energy is required for an agent to perform its tasks. If an agent runs out of energy, it could stop functioning and possibly obstruct other agents or even crash and damage the environment, for example in the case of aerial agents [13]. Due to limited number of charging stations, agents have to take an energy management procedure into consideration. Furthermore, environmental obstacles such as walls are now introduced. In reality, agents are often confronted with such obstructions, such as racks in a warehouse.

C. Task Delegation

Up to this point, it is assumed that agents are able to pick up any coloured package and that all packages and destinations were reachable. However, the case in reality is not always that simple. Agents may be restricted in which tasks they can perform. For example, agents in warehouses have to transport

packages but they may not carry all types of packages. The robots can have a weight or size limitation. For some agents, the packages can be too heavy or too small to recognize. In this case, colour restriction is introduced. The agents may only manage packages of their own assigned colour. Moreover, the agents cannot always execute a certain task before a different one is solved. In the current stage, the path of an agent may be blocked such that they are unable to fulfill their task. This can be due to an obstruction on a route where packages of other colours have to be removed before a clear path appears between the agent and his target. Thus, task delegation is necessary to clear all obstacles such that the agents can fulfill all the tasks. For the sake of simplicity, the agent's batteries are not present in these scenarios.

IV. OUR SOLUTION OF THE PACKET-WORLD PROBLEMS

In this section, our solution on the Packet-World problem is proposed based on the three scenarios.

A. Autonomous Behaviour

Initially, an autonomous *Simple agent* behaviour set is proposed (having one behaviour class *Simple*). The agent randomly walks until it perceives an entity such as a package or a destination. If the agent does not carry a package, it picks up the first one it sees in his perception range. If it carries a package, it randomly wanders through the environment until it notices the right colored destination. When an agent moves towards a found package or destination, he moves straight towards it. However, when there is an obstacle in its way not allowing it to move straight towards the desired entity, the agent randomly chooses a walkable neighbouring position and then tries again to go straight towards its target. Next, the belief-desire-intention (BDI) pattern [14] is adopted as a framework for handling the agent's upcoming complexity. The single behaviour class from the *Simple agent* behaviour set is split into four behaviours:

- Look for package behaviour (class `LookForPackage`),
- Go pick up package behaviour (class `GoPickUpPackage`),
- Look for target behaviour (class `LookForTarget`),
- Bring package to target behaviour (class `BringPackageToTarget`).

This results in a new behaviour set, called *Structured Agent*, consisting of behaviour and behaviour changes. The behaviour changes are conditions that, when met, allow the transition from one behaviour to another behaviour.

The *Structured Agent* behaviour set is further enhanced with the addition of stochastic momentum, reducing the randomness of walking. Thereafter, the agent keeps moving in the same direction with a higher probability, which leads to rapid exploration of the environment. Moreover, the agent now uses a memory that stores locations of entities as they appear in the agent's perceptive field (class `Memory` containing several collections of the grid cells `CellPerception`) which leads to more deterministic agent's behavior. The agent does not waste time by random wandering and it goes directly to a

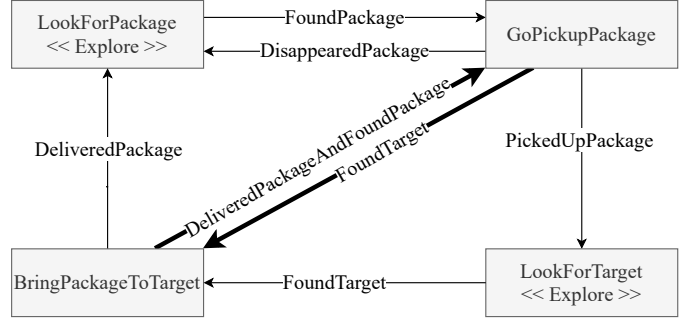


Fig. 1: State diagram representing an overview of the behaviours and the corresponding transition conditions for an agent in the *Optimized Agent* behaviour set. The thickness of the transition lines denotes their priority.

stored entity. When implementing optimized random wandering, an extra abstract behaviour (class `Explore`) is added, which handles the random exploration in `LookForPackage` and `LookForTarget` class. The result is a new behavior set, called *Optimized Agent* of which the behaviours and the conditions for a behavioural switch (represented via arrows connecting the behaviours) are depicted in Fig. 1. The conditions with higher priorities have thicker arrows.

In this phase, a problem was encountered. When an agent without memory saw an entity in the perceptive field, it sometimes moved away and lost the perception of the entity. However, the behaviour assumed that the entity was still within range and thus, the simple behaviour agent froze. Thus, the condition `DisappearedPackage` is added. The problem is also solved by the implementation of memory. Based on the testing, such issues have been detected and fixed.

B. Energy Management

In this stage of development, we continued on the previous implementation. Firstly, we realized that while the test bed already had an implementation of a memory as a string map, creating an abstraction layer for CRUD (create, read, update, and delete) operations on the agent's memory in the form of the `MemoryTool` class would be advantageous. The string map memory saves two categories: *destinations* and *packages*. The memory blobs are structured by special symbols. The main change introduced in this phase is battery limitation. Batteries are added to each agent and charging stations are added to the environment. Following the gradient fields pattern proposed by Steels [15], we added gradient fields around the charging stations which allows agents to identify their relative position to the station and the urgency with which they should go to one to recharge. The general idea is that an agent can travel in the decreasing gradient direction to find an energy station at the minimum. To deal with obstacles along the way, a simple bug algorithm was initially adopted. This bug algorithm would move in the maximal gradient decrease direction unless that direction was blocked. If it was it would rotate the desired direction counterclockwise until it found a direction it could

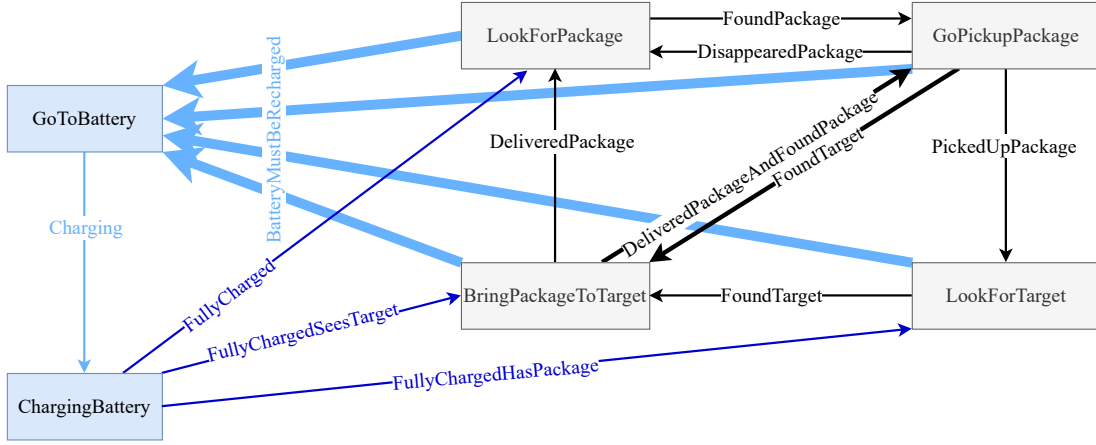


Fig. 2: Updated state diagram of the behaviours and the corresponding transition conditions for an agent with a battery limitation. The blue color represents transitions and behaviours related to the introduction of agent energy. The thicker line conditions have a higher priority.

move in. Then, it would proceed in that direction and repeat this procedure until it reaches the energy station. It turned out that this bug algorithm can have issues with non-convex obstacles. For that reason, we used a simple A* shortest path algorithm based AStar planner pattern. In this approach, the agent moves to the cell in its perception with the lowest gradient value. If the straight path to that cell is blocked, the agent determines an optimal route to that cell using the A* algorithm and follows it. This is again repeated until the energy station is reached. Every time an energy station is placed in the world, the gradient is updated. The gradient at a cell is defined as the minimum of the set of lengths of paths from every energy station to the cell using breadth-first search.

As agent energy is introduced, two additional behaviours are added:

- Searching for a charging station behaviour (class `GoToBattery`),
- Battery charging behaviour (class `ChargingBattery`).

In the Fig. 2, the current six behaviours are depicted. The blocks and arrows in blue represent the additions related to the agents energy. An agent switches from non-energy behaviours to the `GoToBattery` behaviour when its current energy level drops below a tunable multiple of the energy required to move the number of steps to the nearest charging station with no carry. An estimate of the number of steps required is the perceived gradient value by its construction. Notice the high priority of the conditions for switch to `GoToBattery` behavior because it is crucial to get agents to a charging station in time. Note also that any carried packet is dropped before the agent starts to look for an energy station so that reaching one does not consume unnecessary energy. When an agent goes towards a charging station and notices that it is already in use by another agent, it waits and stops moving until the station is freed and he then proceeds towards it. When an agent's battery is recharged, it goes back to its previous non-energy behaviour except the `GoPickupPackage` behaviour

because the agent might have walked a long distance to the charging station. Therefore, the agent may make a lot of effort to return back to its previous position. Thus, the agent switches to the `LookForPackage` behaviour allowing to find a closer package.

C. Task Delegation

The implementation of the task delegation consists of two phases. The first phase has involved no communication between agents. Following the situated commitment pattern, the basic behaviour set is extended with an additional loop to pick up package that block a path, called priority packages. When an agent wanders around and notices a destination, it checks the destination's accessibility using A* search algorithm [16], heuristically prioritizing paths with a package of the agent's color. If the destination is not accessible, the agent checks the first blocking package in the path. If the agent can pick up the package, it is added to the *packages-to-remove list* and the agent will change to the `GoPickupPriority` behaviour as a situational commitment. If not, the agent will continue wandering around. In the `GoPickupPriority` behaviour, the agent picks up the closest package from the *packages-to-remove list* and transitions to the `MovePackage` behaviour. Each agent may remember a best path to every destination separately, which is used to check if the agent is far enough from each path to drop the package. Additionally, it checks if there are enough free cells around it to prevent getting stuck. If both conditions are met, the agent drops the package in one of the free cells. If the agent no longer has packages in its *packages-to-remove list*, and it is no longer carrying a package, the agent transitions out of the `GoPickupPriority-MovePackage` loop. If the agent knows about a clear path towards its destination, it transitions to the `LookForPackage` behaviour, engaging its normal cleanup loop. If the agent's path towards its destination is blocked by a package it cannot clear, it transitions to the `Explore` behaviour, looking for blocked paths that it can clear.

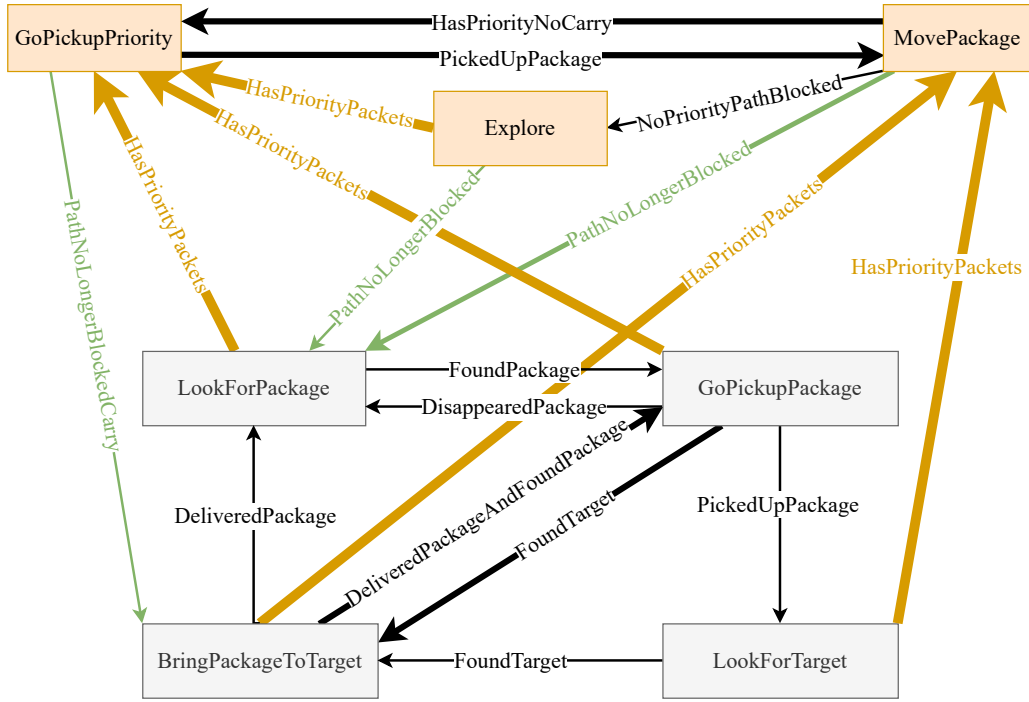


Fig. 3: Updated state diagram of the behaviours and the corresponding transition conditions for an agent with the implementation of task delegation to cope with task obstructions. The orange transitions indicate that agent needs to deal with a blocked path. The green transitions indicate that the blockage has been cleared and the agent can go back to its normal package cleaning routine.

In the second phase, the process is made more efficient in two manners. Firstly, agents can help each other significantly faster by enabling direct communication, following the protocol-based communication pattern. When an agent sees a path to a destination which cannot be cleared, it notifies another agent of the corresponding color that there is a blocking package. Subsequently, the receiving agent adds this package to its *packages-to-remove* list. Secondly, time and energy can be saved by moving the priority packages more intelligently. When an agent moves a blocking package, instead of wandering randomly until it is far enough from a path, it walks towards cells that are further away in a stochastic manner. It does so by determining which cells in its perception are far enough from the known paths, and selecting one at random. On one hand, the agent wastes less time by removing the random wandering. On the other hand, edge cases such as infinite loops can be avoided by adding a stochastic component.

V. RESULTS

For the testing of our approach, several scenarios were provided on which we did multiple experiments. For the first autonomous behaviour stage, three basic scenarios have been tested: Basic 1, Basic 2, and Basic 3, with increasing size and complexity. These scenarios only contained coloured packages and destinations and an optimized agent approach has been applied on them. For the energy management stage, agent's energy was introduced and two scenarios have been

tested: Energy 1 and Energy 2. The Energy 1 scenario just has the additional charging stations in the environment while the Energy 2 scenario also has some small walls inserted. For the last task delegation stage, 1 scenario is tested. Here, agent have to cooperate to cope with introduced package obstructions. However, in this scenario, agent's energy is not implemented anymore. All scenarios have been run 100 times. In all plots, the average values over the different runs are depicted.

A. Autonomous Behaviour

We will first measure the total cycles needed and energy consumed by the three different agent behaviour set implementations of Section IV-A for the three different scenarios provided by the Packet-World software. The plots in Fig. 4 lay out our results. As one can see, the total number of cycles and energy consumption increases with increasing size and complexity of the scenarios. Furthermore, as the environment becomes more complex and more agents are present, our implemented optimizations make an increasingly positive impact on the required time and energy to reach the goal. For the Basic 2 scenario, the optimizations allowed a reduction of more than 60% on the total number of cycles and energy consumption compared to the *Simple agent* behaviour set. For the Basic 3 scenario, the improvements were even more significant as the total number of cycles and energy consumption were reduced by 87% and 89%, respectively. One may notice that for the



Fig. 4: The total number of cycles and energy consumption needed to reach the goal, averaged out over multiple runs, for the three different autonomous behaviour scenarios with increasing complexity and size, shown on a logarithmic scale.

Basic 1 scenario, no real improvement can be seen for our implemented optimizations. This is because the vision of the agent in that scenario was the whole environment and thus the optimizations regarding the exploration while searching packets/destinations and memorizing them when found had no contribution.

B. Energy Management

Here, our experimental results for the two scenarios related to the energy management stage will be discussed. The plots in Fig. 5 lay out our results.

One can see that the more complex Energy 2 scenario required more cycles and energy consumption, as expected. However, the increase of both parameters is rather limited. Unfortunately, we noticed that the simulation of both scenarios related to the energy management were sometimes interrupted before all packages were delivered. This is because an agent sometimes ran out of energy when going towards a charging station and the fastest paths were obstructed. This could be avoided by letting the agents switch faster (with more remaining battery) to the `GoToBattery` behaviour. However, this comes at the cost of lower performance as the agents travel more often from/to a charging station. These interrupted scenario cases were discarded when constructing the plots of the experimental results.

C. Task Delegation

As for the task delegation stage only 1 scenario is considered, we will not discuss it in a separate plot. However, the results for three scenarios: one for each developed stage are shown together in Fig. 6. As one can see, The autonomous behaviour and task delegation scenarios were much faster solved than the energy management scenario. This is logical as the Energy scenario was the only scenario containing agent's energy and thus where the agents had to move around a lot to

recharge their batteries. Naturally, this comes at a performance cost.

VI. DISCUSSION

A. Limitations

The applied approaches for this project had their limitations/drawbacks. A first limitation occurring in all approaches is related to the searching for packets and destinations (when none are found in the agent memory). This is done by random wandering with stochastic momentum and is sub optimal. No indirect communication methods (such as stigmergy) have been applied to improve this. However, as the simulations progress and the agents move around, found packets and destinations are shared and stored in their memory and thus counters this limitation.

Another limitation of our approaches is the limited implementation of obstacle avoidance and pathfinding. For the first two stages (automated behaviour and energy management), when an agent encounters an obstacle in the direction it wants to move in, it moves along it until it reaches its end and can continue to its target. However, when more complex structures are present in the environment (such as concave packet structures, blockages that do not lead to destinations or mazes), our implementations is prone to failure with agents getting stuck in certain areas or running out of energy. Examples of this are the limited success and failure faced in the energy-1 and energy-3 environments respectively. For the last stage of our approach (task delegation), agents avoid walls more intelligently through the AStar planner pattern for paths within their perception. But this remains a brittle solution in light of continued failure in the energy-3 and maze environments. Unfortunately, there does not exist a limited knowledge pathfinding algorithm yet that works perfectly without making assumptions about the kind of environment an agent is navigating through. A higher degree of knowledge sharing through, for example, stigmergy

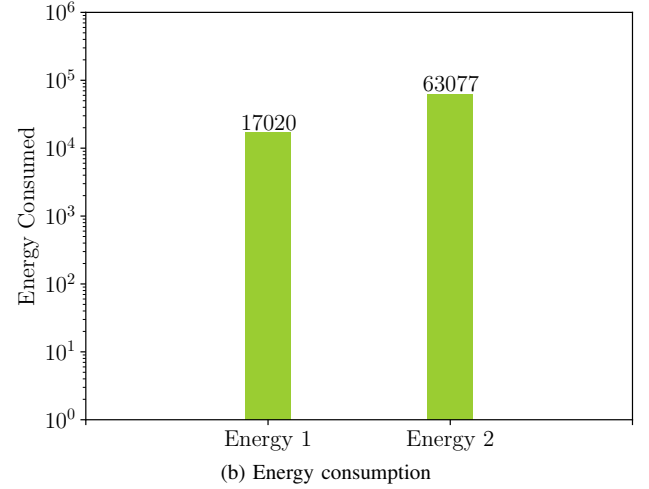
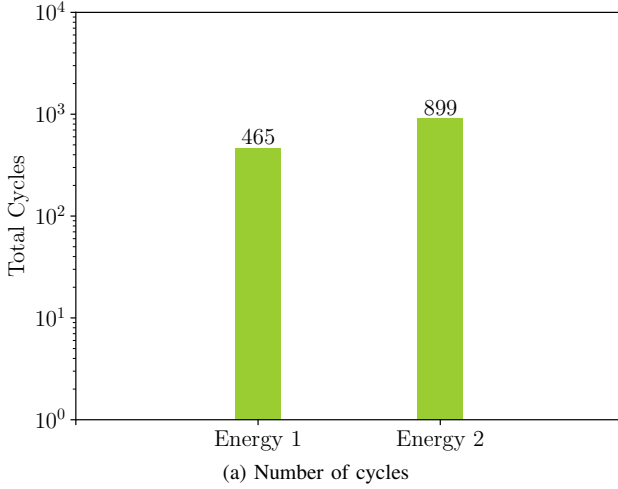


Fig. 5: The total number of cycles and energy consumption needed to reach the goal, averaged out over multiple runs, for the two different energy management scenarios with increasing complexity and size, shown on a logarithmic scale.

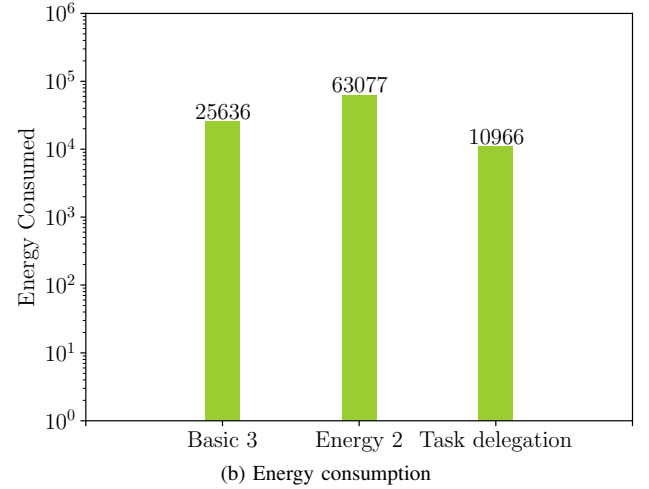
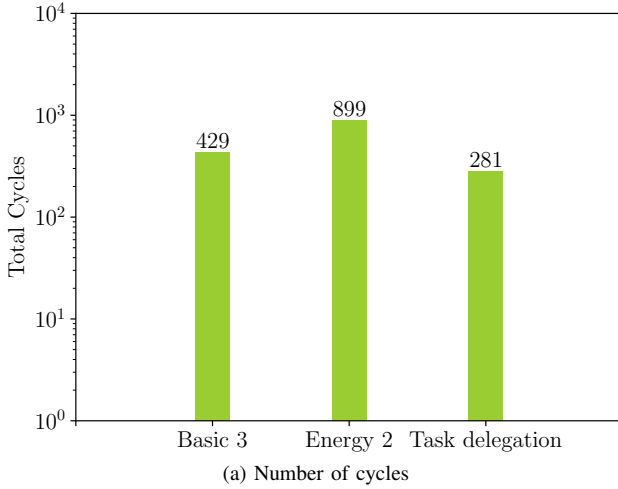


Fig. 6: Comparison of the performance of our approaches, averaged out over multiple runs, for scenarios of different stages, shown on logarithmic scale.

or even the middleware seems unavoidable for better system operation for a majority of the time.

Finally, the lack of strong and adaptive coordination schemes for scenarios such as battery charging proves very detrimental in certain environments. For example, in the energy-3 environment agents that needed to recharge would block agents that just did from leaving the energy station they required. This problem was so severe that our system continuously failed to succeed in this environment.

B. Lessons learned

A lot of things have been learned from this project. First, good knowledge has been obtained about MASs. Secondly, we have learned that exhaustive scenario testing is needed to ensure the correct working of our code implementation. It happened that the code worked perfectly for most cases but

then failed for very specific cases. Thus, good debugging was needed to correct the code. Related to this, we learned that a good structure in the code is essential. The recommended division of the agent behaviour into multiple behaviours helped significantly to easily add implementations to the code. Finally, it became apparent that it is challenging to develop a working system based on deterministic behaviour when multiple independent agents are combined and knowledge and perception are limited. Adding stochastic components avoids a lot of these pitfalls.

VII. RELATED WORK

A. Existing Approaches

The authors of [17] have proposed cooperative and non-cooperative MAS (C-MAS and NC-MAS, respectively) algorithms to tackle the Packet World problem. Both algorithms

consist of 2 stages. In a discovering stage, the agents wander around to search for all packages until all the environmental positions have been visited. In the second cleaning stage, the agents pick up all found packages and deliver them to their targets. The difference between the C-MAS and NC-MAS algorithm lies in the the presence or absence of cooperation during the different stages. The C-MAS algorithm turned out to be the best in terms of speed, energy consumption and total steps taken.

Another approach has been presented in [18] and it is called Self-Adaptation (SA). The SA approach consists of behaviour sets and a metabehaviour. It uses a MAPE (Monitoring, Analysis, Planning and Execution) loop to determine when the metabehaviour changes the used behaviour set. Each behaviour set consists of multiple behaviours and a set of transitions between the behaviours. A transition is triggered when the environment changes and the precondition of the next behaviour is satisfied. In the paper, the approach was applied to the Packet World problem with examples of two behaviour sets: a basic behaviour set and a flag behaviour set. This flag behaviour set is an extension of the basic behaviour set where the agent adds flags in the environment in locations where there are no packages. The external MAPE loop is then used to let the metabehaviour change from one behaviour set to another. This approach is modular, it allows to easily add or remove behaviour sets without making significant changes. The authors of [19] have extended time management support for MASs. Time management is crucial as it allows consistent and repeatable simulation runs. Semantic duration models have been introduced in this article to capture the timing requirements of the semantics of MAS activities. These models were then used as basis for a time management infrastructure to transparently integrate all time management functionality in a MAS. Their contributions have been successfully demonstrated by application on the Packet-World test bed.

B. Comparison of our methodology to existing approaches

One can also compare our used methodologies with the existing approaches mentioned above. Our approach in Section IV-A is similar to the approach from [17] but differs by not including a 'discovering stage'. The agents immediately bring packets to a destination when they find one. However, when other packets are seen while an agent is busy, their location is remembered and communicated to other agents. Thus, our approach seems to combine the 2 stages from [17] into 1 stage. Unfortunately, no details were found on the scenarios used for their performance assessment and thus we are not able to compare their performance with ours.

The SA approach from [18] is more sophisticated than the approach that we implemented for this project. As mentioned before, our implemented behaviour set from Section IV-A is very similar to the basic behaviour set from this paper. However, we did not implement flags to identify where there are no packets, like in the flag behaviour set, and we don't have an external loop for transition between behaviour sets.

Hereby, the performance of our solution will probably be worse than that of [18].

VIII. CONCLUSION

In this paper, we have introduced our approach on the SMAS problem in the Packet-World test bed. The approach is created in three stages, each one adding new concepts to the Packet-World. In the first stage, a straightforward solution is proposed to clear a classical Packet-World test bed from all packets. The agents randomly search for packets and for a destination afterwards. Based on testing, we improved the approach by adding a momentum to the agents movement when searching, which forces the agents to not change its direction extensively and allows better environment exploration. Moreover, agent memory is added to remember seen packages and destinations. These improvements allowed a significant reduction of the total number of cycles and energy consumption needed to reach the goal. For the Basic 3 scenario, these reductions were 87% and 89%, respectively, which is a very large improvement. In the second stage, agent energy and charging stations were introduced. It leads to the implementation of energy management where agents go to a charging station based on their actual battery level and their distance to the station (which is implied by gradient level). A simple bug algorithm was initially introduced to help agents move towards the gradient's source while avoiding obstacles. But this algorithm proved insufficient because it can only route around convex obstacles. An alternative was then adopted that uses the A* shortest path algorithm to guide an agent to the cell in its perception that is closest to its current goal if something blocks the direct path to that cell. This worked better, but also not perfectly. In the third stage, package obstructions were introduced. We let agents detect these obstructions and let them help clear them through the concept of priority packets: packets they need to move away from an obstruction but not to a destination. This approach is successful in solving the task-delegation environment efficiently. In future work, we can focus on optimizing memory usage. Each agent may remember only its coloured packets and destinations which might speed up the read operation. The agent sends a message to different coloured agents if it encounters an entity with a different colour. Additionally, agent coordination schemes for situations such as using energy stations deserve careful development and consideration. Furthermore, the exploration behaviour and path finding ability might be improved by introducing more complex algorithms using concepts like indirect flags communication (stigmergy), as done in [18]. Finally, peer-to-peer protocols involving agents and other active items could be developed to enable continuous knowledge sharing. When using algorithms that rely on memory in agents situated in a dynamic environment, avoiding stale knowledge in this way could boost system efficiency, autonomy and continuity.

REFERENCES

- [1] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.
- [2] D. Weyns, A. Helleboogh, and T. Holvoet, “The packet-world: A test bed for investigating situated multi-agent systems,” in *Software Agent-Based Applications, Platforms and Development Kits*, pp. 383–408, Springer, 2005.
- [3] P. Maes, “Situated agents can have goals,” *Robotics and Autonomous Systems Journal.*, vol. 6, no. 1, pp. 991–997, 1989.
- [4] K. M. Ford and Z. W. Pylyshyn, *The robot’s dilemma revisited: The frame problem in artificial intelligence*. Greenwood Publishing Group, 1996.
- [5] R. A. Brooks *et al.*, “Intelligence without reason,” *Artificial intelligence: critical concepts*, vol. 3, pp. 107–63, 1991.
- [6] e. a. Weyns, Danny, “Environments for situated multi-agent systems: Beyond infrastructure.,” *Environments for Multi-Agent Systems II*, vol. 3830, pp. 1–17, 2006.
- [7] D. Weyns, E. Steegmans, and T. Holvoet, “Towards active perception in situated multi-agent systems,” *Applied Artificial Intelligence*, vol. 18, no. 9-10, pp. 867–883, 2004.
- [8] D. Weyns and T. Holvoet, “A formal model for situated multi-agent systems,” *Fundamenta informaticae*, vol. 63, pp. 125–158, Dec 2004.
- [9] P.-P. Grassé, “La theorie de la stigmergie: essai d’interpretation du comportement des termites constructeurs,” *Insectes Sociaux*, vol. 6, pp. 41–81, 1959.
- [10] D. Weyns, E. Steegmans, and T. Holvoet, “Protocol-based communication for situated multi-agent systems,” in *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems, 2004. AAMAS 2004.*, pp. 118–125, 2004.
- [11] D. Reyes, A. Erera, M. Savelsbergh, S. Sahasrabudhe, and R. O’Neil, “The meal delivery routing problem,” *Transportation Science*, vol. 3415 LNAI, pp. 1–70, 2018.
- [12] T. Taylor and K. Toohey, *Routledge Handbook of Sports Event Management*. Routledge, 2015.
- [13] I. A. Mason, V. Nigam, C. Talcott, and A. Brito, “A framework for analyzing adaptive autonomous aerial vehicles,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 10729 LNCS, pp. 406–422, 2018.
- [14] M. Georgeff and A. Rao, “Modeling rational agents within a bdi-architecture,” in *Proc. 2nd Int. Conf. on Knowledge Representation and Reasoning (KR’91)*. Morgan Kaufmann, pp. 473–484, of, 1991.
- [15] L. Steels, “Cooperation between distributed agents through self-organisation,” in *EEE International Workshop on Intelligent Robots and Systems, Towards a New Frontier of Applications*, pp. 8–14, 1990.
- [16] W. Zeng and R. Church, “Finding shortest paths on real road networks: The case for a,” *International Journal of Geographical Information Science*, vol. 23, pp. 531–543, 04 2009.
- [17] A. A. Abusnaina and C. H. Yong, “Cooperative multi-agent system for solving packet world problem in grid,” in *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pp. 754–758, IEEE, 2009.
- [18] M. Delabie and D. Weyns, “A modular approach for engineering situated multi-agent systems with self-adaptation,”
- [19] A. Helleboogh, T. Holvoet, D. Weyns, and Y. Berbers, “Extending time management support for multi-agent systems,” *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 3415 LNAI, pp. 37–48, 2005.

Design of agent interaction protocols and patterns in a multi agent system

Jan Wellemans, Maarten Van den hof, Arne Wouters, Michiel Pieters

KU Leuven

Leuven, Belgium

jan.wellemans@student.kuleuven.be, maarten.vandenhof@student.kuleuven.be,

arne.wouters1@student.kuleuven.be, michiel.pieters@kuleuven.be

Abstract—The paper presents an elaboration on the Packet-World multi agent system problem. Agents in this test bed are limited due to their perception, energy level and environmental restrictions without access to a centralized system. Agents and their behaviours are locally designed and analyzed on a broad view, where a simulation shows the indirect effects of the different used architectures. The agents are first build, with an A* based path planning to manoeuvre in the world, instead of wandering. The agents are introduced to a depleting energy level and to make use of the charging stations. Actions are made more energy efficient and collaboration is introduced to avoid the depletion of agents. Further collaboration is elaborated on using Field-based Task Assignment (FiTA) to make agents collaborate to solve complex environments.

I. INTRODUCTION

A multi-agent system (MAS) [1] is a decentralized system composed of multiple agents with no controlling entity [2]. The main goal is to interconnect agents to make the system of agents more efficient at completing tasks than the agents alone [3]. Agents are autonomous entity, able to cooperate with other agents to accomplish its objectives. They can be physical robots or digital systems. These agents are capable of adjusting to changes in their environment, but each agent has only a limited view of the environment. Three core concepts of multi-agent systems: autonomous behaviour, coordination [4] and collaboration are looked at. Autonomous behaviour is about how the agent moves and reacts inside the environment using its own perception. The designed solutions are evaluated and assessed using the Packet World [5] test-bed. In this paper, three main problems were tackled, namely the agent's behavior to exploit its environment, coordination for energy management and delegation between the agents. First, an introduction of the Packet World [5] (section II) will be given, followed by a more in-depth problem description (section III) and our proposed solution of these problems with a MAS architecture based design [6] (section IV). The following contributions are implemented and discussed as solutions in the paper.

- A* path planning and autonomous behaviour
- Charging station gradient fields
- Indirect agent communication with flags
- Virtual hull environment to avoid collisions
- Agent communication

After discussing the results (section V), future and related work (section VII and VI) will be discussed, ending with the final conclusion (section VII).

II. THE PACKET WORLD

The packet world is a test-bed exercise to asses and evaluate MAS techniques. The basic setup of the world consists of agents designated to carry (coloured) boxes from one location to a drop-off point. The agents have limited action capabilities given their energy capacity. Additionally implemented the agents might, when low on energy, need to recharge at a battery station, indicated by a perceived gradient field. The environment is made flexible and modular, allowing the addition of other segments given their constraints with the current system. The test bed is designed in layers where constraints are defined within and between layers. As examples the constraints between the agent layer and the drop off layer (e.g. agents may drop but not collide) or possible constraints inside the agent layer (e.g. agents may not collide). This design pattern gives the implementation its modular characteristic, enabling quick and easy insertions of additional entities (e.g. battery boxes).

A. Agent characteristics in a MAS

1) *Perception*: The agent is aware of certain aspects of its surroundings. Mostly limited perception is integrated in the agent which allows it to react properly on given environment states or state changes [7]. The perception is modular implemented in the architecture, allowing it to be simplified and ordered in different perceptive layers. An example would be the perception of different boxes of the Packet-World context. The perception characteristic is used in the path planning, collaboration and the search for charging stations in the environment.

2) *Actions*: A variety of actions that the agent could perform are possible, given certain perceptive criteria. There are multiple ways to orchestrate the 'best' action, e.g. a decision tree or free-flow diagram. The actions on its own will change the environment in trying to achieve the agent's objectives. The actions in the environment are based on a chronological basis, where certain triggers initiate an action state change.

3) *Collaboration*: The world is often times more complex and the capabilities of agents often limited. Therefore a decentralized system needs cooperation to efficiently reach the

agent's objectives. This collaboration allows agents to solve complex environments where each agent their strengths are used to balance other agent flaws.

4) *Timing and execution control*: The timing an agent gets in a MAS world to perform a task is arbitrary because of the complexity of the world and the interactive collaboration it needs to do with the other agents in this environment. In the Packet-World three approaches are supported for controlling the execution time. Different methods are synchronization methodologies are possible. Global synchronization, Regional synchronization (They work together in clusters) and Logical Time where each agent gets a customized timing for each action.

B. Application examples

1) *Autonomous truck vehicles*: A self-driving truck is an agent with the goal to pickup goods and deliver them to a specific destination without the use of a human driver [8]. Driving from one point to another is not that simple because the environment is constantly changing [9]. Roads are blocked because of roadworks, accidents, etc. The agents can cooperate by notifying other agents (trucks from e.g. other companies) of these obstacles so that they can look for a better route to shorten the delivery time.

2) *Search and rescue automated vehicles*: Search and rescue operations tend to be in highly complex and dangerous situations. These operations have a high need to replace human interference with automated robots [10]. An example case is a search and rescue operation in an underground cave network. These under laying tunnels are often very complex, partially filled with water and have no communication with the surface. This lack of direct communication gives rise to the need of a decentralized automation system. Automation of this scene requires the collaboration of several specialized vehicles. For instance, the cooperation of a submerging underwater vehicle and a surfacing drone. The two agents have a mutual objective, although different methods of achieving it, to find missing persons. They could either work individually and find defined interesting flags or cooperate when the submarine can not communicate with the surface. A cooperation should be possible where the drone could fly up and send the signal of the submarine to the surface.

III. PROBLEM DESCRIPTION

The agents have as designated main objective to bring all the packages to a corresponding drop off points. The method to reach the goal is not directly stated in the script, making this a declarative task.

A. Autonomous behaviour

The agents operate in a multi agent system. This states that they are not connected to a centralized system, making them dependant on their perceptions and actions of the world. This has many implications. The distances to targets are hard to estimate as they have only partial information about their environment. They can go in directions that lead to useless

trajectories. They also have the potential to block other agents, reducing the system efficiency. Whereas the agents initially only wander around. They move in the world with an equal chance to move in each direction on the next turn. A quick look shows that this movement is very inefficient and makes the agent only walk around locally. Additionally, the world is filled with non walkable structures such as walls, packages and other entities which need to be evaded efficiently.

B. Energy dependence

The agents are introduced to a energy level. This limits their possible actions and makes it necessary to choose energy efficient actions. the agents now have the additional constraint to not deplete. The energy levels are rechargeable at charging stations but they need to be localized by the agents. Agents making mistakes or making things harder for other agents now have a higher influence as they might deplete the agent, rendering it useless for the rest of the iterations. Depleted agents can also have the possibility to block further paths, making this more inefficient.

C. Collaboration

Agents are limited in their actions. The color of the packages in further environments are agent dependant. This makes it impossible for agents to move other colored packages if they are blocking their way. Collaboration needs to be introduced to tackle the main objective, trying to solve these problems in a more efficient way using multiple agents that can interact with one another. This gives rise to a need of efficient action selection and agent collaboration to help each other for optimally completing their goals with limited resources.

D. Addendum

An other issue occurs when the problem is taken from its course context and placed in the real world. Robots or Automated Guided Vehicles (AGV) in the real world applications might suffer from memory dependence and have a high need for fast computing software to make decisions and do perception. Lag is an important problem and might freeze the agent when it is too large, rendering it useless. Long tunnels are a case specific problem that should be considered. The maze environment tests the robustness of the agent system on this issue. When agents make actions based on a probability to move a certain direction, it is hard to traverse such mazes or long and thin structures.

IV. SOLUTION

A. Autonomous behaviour

The agents are set to behave individually, responsible for their own actions and interacting with other agents their actions and the world. The starting point of the code are agents that wander around randomly. Agents pick up packets when they stand next to it. When they happen to be standing next to the correct destination while holding a packet, they deliver it. The ability of agents to work in the environment and indirectly

collaboration with other agents is heavily influenced by this non optimized agent movement.

1) *Design:* The first optimization is when a random move is performed, there is a 85% chance that the previous random move is repeated (i.e. when the agent goes left, the chance of going left again is higher if going left is a valid move). Next an 'optimal' path finding algorithm is implemented. The autonomous behaviour implementation starts by adding an 'optimal' path generation when new information about a object of interest location is presented.

There are 2 major issues that need to be handled

- An optimal path needs to be found capable of evading obstacles and guiding the agent's movements towards it's (sub-)goal.
- The path needs to be time-efficiently calculated to minimize the processing cost for the agents. (Important for further real-life cycle time)

A* [11] is proven to be a fast and efficient guided path searching algorithm. It uses a few basic principles to establish the optimal path. The implementation of the search is done using the Manhattan distance as the distance measure, given the tiled world. Two optimizations for this algorithm are engineered. The first optimization is following the shortest path towards a packet or desired destination when they are available in the perception of the agent. The second optimization caches 'interesting' locations (destinations and packets) that have been found while moving in the Packet World. This results in a faster delivery of the packet once it is picked up and reduces the time the agent needs to find a new package. Another issue for the agents is collision. The agents are made to plan their movements using hulls to prevent any collision in the Packet-World. This resulted in more energy consumption in the basic agent implementation, but the extra energy is not that significant anymore after optimizations. The hulls are an example of coordination through a virtual environment [12]. The hulls are generated using the A* path planning. Agents can check the hull virtual environment (world) to see whether their current paths would collide with other planned agent paths.

2) *Implementation:* The A* function needs two variables. The first variable is the heuristic, indicating how close it is, using the Manhattan distance from the target location (corresponds to assumption that this is always an underestimate). The second variable is the accumulated path Cost, which adds 1 at every move, in this case.

$$f = C_{acc} + heuristic \quad (1)$$

An example of the A* path finding is given in the path buffer brief example above. The example queue procedure uses figure 1 as model. The accumulated cost and heuristic values are given in the brackets (*c, h*), where the sum gives the function value.

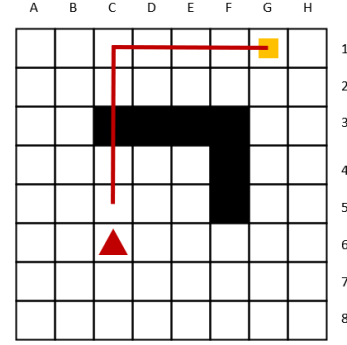


Fig. 1. Tile world representation, Manhattan distance in Red

Queue:

C6

C6,C5 | C6,D6 | C6,C7 | C6,B6
(1,8) (1,8) (1,10) (1,10)

C6,C5,C4 | C6,C5,D5 | ~~E6,C5,C6~~ | C6,C5,B5 | C6,D6 | C6,C7 | C6,B6
(2,7) (2,8) loop (1,10) (1,8) (1,10) (1,10)

~~E6,C5,C4,C3~~ | **C6,C5,C4,D4** | ... | C6,D6 | ...
Wall (3,6) (1,8)

~~E6,C5,C4,D4,D3~~ | **C6,C5,C4,D4,E4** | ... | C6,D6 | ...
Wall (4,5) (1,8)

~~E6,C5,D4,E4,E3~~ | ~~E6,C5,D4,E4,F4~~ | C6,C5,D4,E4,E5 | ... | **C6,D6** | ...
Wall Wall (5,6) (1,8)

Fig. 2. Queue path finding procedure A*

Paths containing loops are removed from the node buffer. It keeps searching the tree until the goal node with the shortest current path is found. When the algorithm cannot find a goal node, the agent wanders to a neighboring tile. In the second optimization the shortest path available is taken to a tile that is closer towards the destination in memory. It is possible for the agent to get stuck behind walls so a timeout mechanism is added. If the agent could not reach the destination in memory after a certain amount of moves, the agent will start wandering until it sees the destination in its perception. For the destinations that are cached in memory, also a special type of memory is available that can be limited in size. When this is full, it acts like an LRU cache and replaces a destination of the colour that is least recently used. Agents that are in range of each other will communicate their destinations in memory, this leads to a quick way of spreading information to optimise the behaviour of agents.

3) *Results:* A further look is taken at a few statistics for one of the standard environments (*basic-3-color*) and a more advanced environment. The average amount of cycles, average amount of energy consumed, average time to deliver a packet and the average time to pickup a packet for every implemen-

tation is measured. The average time to deliver a packet is the time it takes to transport the packet from its pickup point to the corresponding destination measured in cycles. The time to pickup a packet is the time it takes the agent to find a new packet after it delivered one. Four optimizations were implemented where every optimization also includes all the previous optimizations: *opt1* (this is the basic setup where moves are repeated with a certain percentage), *opt2* (here path finding is used in the perception to packets and destinations), *opt3* (the agent caches encountered destinations in memory) and *opt4* (the agent caches encountered packet in memory). First, a look is taken at the results for one of the standard environments (*basic-3-color*). In table I you can clearly see the impact that the first simple optimization has. The average amount of cycles is reduced to roughly one third. The average energy consumed, time to deliver and the time to pickup also follow this trend. This makes sense as that the agents now no longer wander around, which lost a lot of time and efficiency. The second optimization shows a drastic decrease in average cycles, energy consumed and time to deliver. The time to pickup does not follow the same trend. This could be explained by the more efficient path finding. The agent is now capable of deliberately following the correct trajectory without being dependant on a random wander. Time to pickup stays the same. This is because the time to find the package is not related to the path finding implementation. The agents still wander around, searching for a package. After that they have a known goal to find a path to. The third optimization impacts the time to deliver and not the time to pickup, this is expected since the path finding towards destinations is improved. The fourth optimization lowers the time to pickup, because the logic to find packages is improved. Figure 3 contains the more advanced setup which is used for evaluating the optimizations. This environment contains walls and 112 packets with all possible colors. In table II almost the same observations as can be made in the standard environment. The energy results are visualised in Figure 4. One thing that is observed is that the impact of the fourth optimizations is not that noticeable as in the standard environment. In both cases a decrease of around 97% is reached in the energy that is required to clean the environment of all the packets. It is known that there is a strong linear correlation between the amount of cycles and the amount of energy consumed. The agents are now capable of finding an efficient and valid path, given that walkability of the perceived tiles are incorporated in the A* design.

TABLE I
RESULTS FOR THE BASIC-3-COLOR ENVIRONMENT

Implementation	Avg. Cycles	Avg. Energy Consumed	Avg. Time to Deliver	Avg. Time to Pickup
basic	29029	1014393	1450	287
opt1	10145	366017	546	64
opt2	4094	142270	200	49
opt3	1250	35238	24	49
delegation	750	26036	22	24

TABLE II
RESULTS FOR THE ADVANCED ENVIRONMENT

Implementation	Avg. Cycles	Avg. Energy Consumed	Avg. Time to Deliver	Avg. Time to Pickup
basic	65655	3154338	2120	87
opt1	18641	704150	594	36
opt2	10916	416694	353	15
opt3	2936	109807	80	19
delegation	2315	90989	62	17

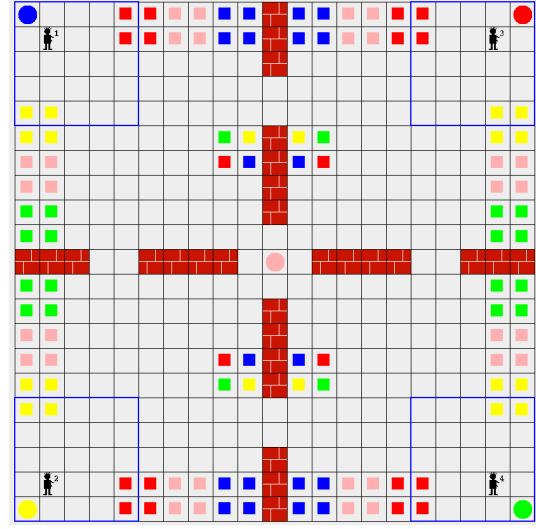


Fig. 3. Environment setup for the evaluation of the optimizations

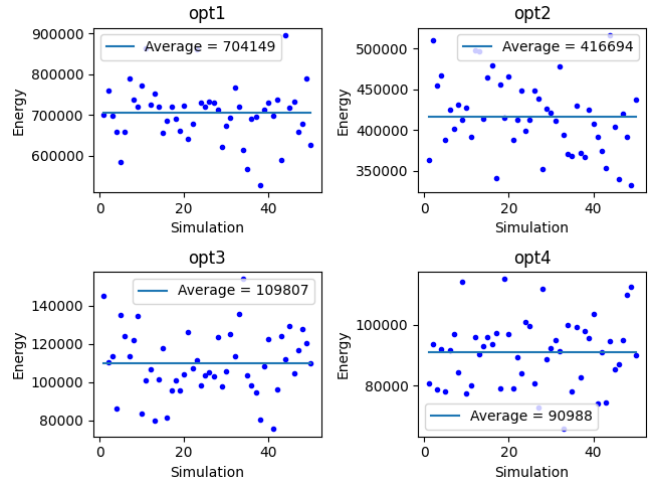


Fig. 4. Simulation results of the optimizations

B. Energy efficiency

The second stage of the project introduced a battery and energy value in the robot as a limiting factor. An important notion in figure 5 is the cost for skipping an action. This evades the problem of agents that keep standing still, as it is without the cost the generally most energy efficient action.

This gave a need for energy efficient behaviour as that agents can no longer carelessly perform actions. The energy

Step	10
Step with carried packet	20
Other action (pick packet, put flag, skip, ...)	5

Fig. 5. Energy limitation factors [12]

levels can be replenished by standing beside a charging station in the charging zone.

1) *Design*: Agents are now in function of the nearest battery locations. These batteries are perceived through their gradient field. The field values are not calculated using their radius from the battery but are generated to fill walkable roads. (see [4] FiTA: Dealing with local minima) The agent can perceive, if present, the field strength value. An example of a charging station setup with their gradient field is given below.



Fig. 6. Example of a charging station gradient field

The filling road gradients solve the problem where agents are misled by the gradient given in bird's eye view. The gradient value does not represent the current distance to the charging station. This might have led to agents depleting themselves because they were not aware of the current longer distances ahead, and worked with a underestimated value. The depleting energy makes agents dependent on batteries, making that their working area is influenced by the distance to batteries and their energy level. One important assumption is that the gradient fields do not always cover the whole world. It is possible that there exist areas where no gradients are available. This is done in the assumption that agents should not always know where charging stations are and that they need to actively find and collaborate in finding these points of interest.

2) *Implementation*: A safe working area is introduced, robots could keep moving away from a battery, 'hoping' to perceive a new one on its path. This is not a safe approach. It would make agents randomly walk away to their current targets until they end up depleted, unable to make further

moves. The safe working area is introduced, it is the area where a robot can move, a safe distance away from batteries (away from the gradient field). Such that they are still able to go back. This safe distance is directly definable. The perception field of a robot is limited and the distance to the battery is often larger. This makes the distance back having an uncertain part where objects like walls can alter the actual distance value. The distance is currently calculated to be the A* path distance in the perception field together with the hamming distance to the closest battery with an additional arbitrary safety value. The distance to a battery is not an ideal measure. There are many ways that the path can be blocked from this and storing the battery's location makes it no easier for the agent. Therefore the location of the battery is changed by the location of the last seen gradient. This ensures that the agent is more often capable to predict the correct distance to a battery. By adding the calculated distance with the distance given by the gradient. The agents are now able to cover a safe operating area while still being able to go back to the charging stations with or without delays on the road. Agents that are charging move away when they see agents approaching with very low energy, this prevents that agents deplete their whole battery.

3) *Results*: Tracking the average time to deliver and pickup is no longer possible since agents drop their packets when they go to the charging station. You can see the average amount of cycles, average amount of energy consumed and the average time between deliveries (in cycles) in table III.

The agents now cluster around their found charging station. A behaviour change is initiated when a low energy state is achieved. The agents will then stop their actions and head back to the closest charging station. As the amount of agents increase, more problematic situations occur where agents run out of energy. A small note, in environment 2 there is no implementation of the prioritization of low energy agents when checking the ideal next agent to charge at a specific station.

The occurrences where the system fails are when agents get stuck or when agents are depleted near the charging station, making other agents misinterpret the state for a needed prioritization. This can be optimized in future works.

TABLE III
RESULTS FOR THE ENERGY ENVIRONMENTS

Environment	Avg. Cycles	Avg. Energy Consumed	Avg. Time between Deliveries
energy-1	1465	29057	85
energy-2	2810	111047	107

C. Collaboration

The report follows the following coordination structure.

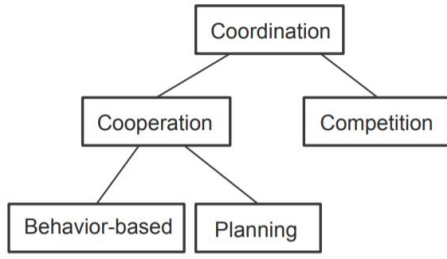


Fig. 7. Coordination differentiation tree [4]

Further improvements at this stage could be made on the agent awareness, of interesting targets in the world, and agents not taking their influence on other agents into account. These two problems could be solved with a cooperation method. Allowing agents to take others into account and giving other knowledge about their perceived environment.

1) *Design*: The multi agent system becomes more flexible and efficient if cooperation methods are introduced. This can be an unary operation where agents have an implementation only being influenced by themselves. Binary operation introduce communication between two agents trying to share information with each other, either direct or indirect. N-ary operation share information with a group of agents, for example a pheromone which affects all agents sensing it.

There are three major issues that can be solved using a cooperation operation. The cooperation solutions have two general methods to be implemented. Using a direct or indirect communication channel.

- Agents need to find a drop-off point of their color.
- Agents blocking passages
- Targets that are blocked from agents because of packages with other colors

2) *Implementation*: Three main cooperative schemes are introduced.

The first implementation is a direct communication operation. When the agents are initialized they have to search for their target drop-off points to be able to start their search. If it is spotted it is stored in the memory. But if drop-off points of other color are spotted, they are useless for the current agent. But they are stored and directly communicated to all the other visible agents.

Secondly, an unary operation is designed to let agents keep in mind to not stand in the potential way of other agents. This occurs when agent paths are blocked by a dynamical entity. For instance in this case this would be an other agent or other path hull. When this happens agents have to wait to be able to go further, until the blockage is removed, agents have passed. The position where the agent waits has an effect on other agents. It might block other agents by standing in their way. This can be evaded with an agent detecting if it blocks possible passages. One way to implement this is in a direct way by calculating if all possible walkable tiles around

the agent are reachable when it stands in the situation. But this is a computational heavy algorithm that can be replaced by a more generalizing idea. The possible blockage situations can be reduced to a 3x3 tile area where only 14 configurations are blocking the passage in one way or another. Checking this configurations is a quick process with no apparent delay for the processing time.

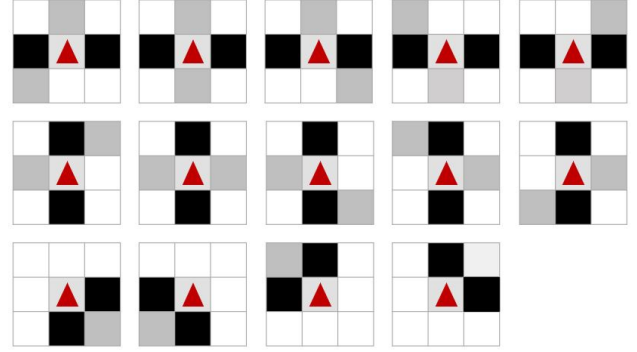


Fig. 8. The 14 configurations that need to be checked. Agent in red, passages in gray, non walkable in black and non-important in white.

The third implementation is the use of flags. When a potential target package is perceived by an agent but it is blocked by a package of different color it puts a flag beside it. The flag can be detected by other agents and preferably by the agent of the blocking passage. If there is a package in the direct vicinity of the flag, it will take this package and place it in a nonblocking location (see previous implementation). If the robot did not hold a package yet, it can take the newly detected package and bring it to its target location. When the agent detects a flag it checks if there are any packages in its vicinity (1 tile neighbors), if not it carries on.

3) *Results*: The system is now able to solve more complex problems. The agents are able to deliver their packages in an efficient way while prioritizing the other agents operation. If agents are blocked by other colored packages, they can communicate with other agents using flags. Enabling the agents to not lose further time and still solve the current blockage.

For the *task-delegation* environment, an average energy consumption of 31577 is achieved. There are a few outliers as seen in Figure 9. The outlier case is that the agent could in a given situation continuously walk from a package to a flag and back until the package was removed. Still most of the time the system can deliver all packets in a short amount of time.

V. DISCUSSION OF RESULTS

The agents are placed in a world where they are limited in perception and awareness. Therefore a thorough path generation is needed and management of these paths and destinations are in order. A* proofed to be a useful algorithm

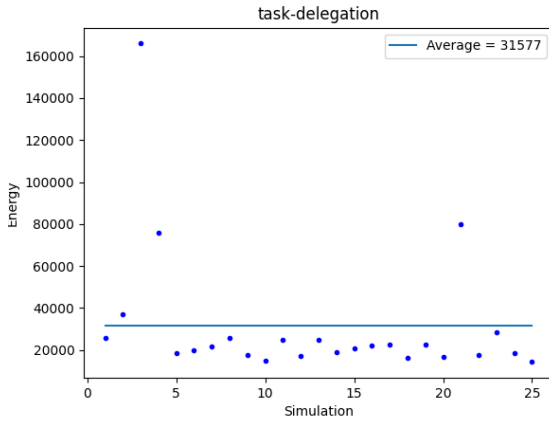


Fig. 9. Simulation results of the optimizations

in determining the shortest path. Agents are able to efficiently move towards destinations and can more accurately estimate distances. This increases the efficiency dramatically by replacing random movement with guided traversal. The agents were made aware of their energy values. This limited the agents and made them aware of their finite actions, making the step for more strategic thinking necessary. The final steps were to improve the system and make it more applicable on certain scenarios. Agents are now able to perform in more less ideal situations and generate thoughtful and collaborative actions.

In general is concluded that the direct communication is useful for local tasks with low amounts of information. It is a fast and robust method. But direct communication proved to be complex and needed logical reasoning in order to function efficiently. An example of an error occurs when an agent wants to have an interaction with an other agent but the agent does not perceive the message. This lets the message sender get stuck, not able to interpret the not conceived message. An example situation is given in figure 10. The agent is depleted just before being able to charge. This leads to an unanswered message which could freeze the agent.

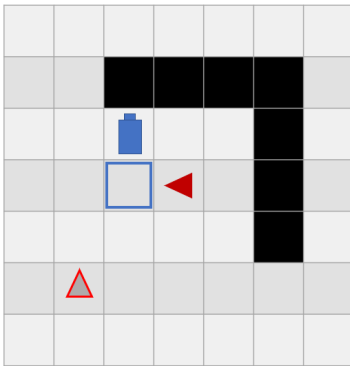


Fig. 10. Illustration of an agent communication deadlock

Indirect communication works well in the multi agent system. Agents are able to perceive information without the need for perceiving each other. This allows a much wider range of information flow.

VI. DISCUSSION OF RELATED WORK

Cooperative vs non-cooperative is discussed in [13]. It concluded that cooperation leads to a significant reduction in execution time. They used a blackboard algorithm that consists of two phases. In the first phase they have a discovery agent, looking for packets and storing their position. When the discovering is finished, the agent returns to a blackboard and stores all the information inside. Now the second phase begins where all the worker agents extract packet information of the blackboard and use this information to locate the packet. Once they reach the packet and deliver it at the designated destination, they return to the blackboard to extract information from another packet. It would be interesting to compare this with these implementations and compare the results.

Flags are used to attract agents but they can also be used to repulse agents. In [14] flags mark areas where there are no packets left and thus make agents search for packets in the direction where there are no flags. This could be a good addition to the current implementations.

Hulls were first introduced in [15], they are a great example of indirect communication. Path marking with direct communication would significantly increase the complexity because of all the extra required communication protocols.

Gradients can also be used for delivery and pickup locations [16]. This simplifies the path seeking as agents just have to follow gradients now as this is also the most efficient route. One thing extra that has to be taken into consideration is congestion control.

The paper written by Juziuk et al. [17] gives us a map with an overview of all the design patterns and their categories for a multi agent system. The implementation of communication used in this paper can be situated in the Focus dimension on this map, living as a subcategory of the behavioural category. This is also the case for other implementations used in this paper like the gradient fields (nature-inspired, Inspiration), requirements analysis (conceptual, Abstraction) and collaboration (nature-inspired, Inspiration).

VII. CONCLUSIONS AND FUTURE WORK

A decentralized system works for the given model. The methods applied for path planning (A*), energy level maintenance collaboration optimization showed immediate success. The result were promising and keeping collaboration in mind during the agent's local system design is an interesting exercise. The multi agent system proved to be a dynamic and moldable system given only a few criteria, making agents capable of executing strategic actions.

There were three implementations that were considered but time came in short.

- Agents with low battery should have a communication protocol, that enables an interaction to determine the agent with the lowest value. This agent will receive priority to charge firstly.
- A cluster of packets is interesting for all agents. This 'Honeypot' should be communicated with as much agents as possible. Therefore an indirect pheromone communication could be useful [18].
- Large world with agents having a relative small perception could benefit from having a wider communication range for sharing of drop-off point locations. A pheromone method can cover a wide area and reach a lot of agents that benefit from this information.

A further implementation on the maze problem could be included in future work. The flags used in the collaboration phase remain in the Packet World, which leads to unwanted additional checks after the obstacle problem has already been resolved. Future works should consider how to deal with these flags, whether they have a limited time in the world, agents removing this flags or other possible methods. The problem resulted in a problem that could occur, given by the outliers in figure 9.

REFERENCES

- [1] Michael Wooldridge. *An introduction to multiagent systems*. Wiley, Chichester, 2nd edition edition, 2009.
- [2] Jacques Ferber. *Multi-agent systems: an introduction to distributed artificial intelligence*. Addison-Wesley, Harlow, 1998.
- [3] Z Ren and Chimay J Anumba. Multi-agent systems in construction—state of the art and prospects. *Automation in Construction*, 13(3):421–434, 2004.
- [4] Wei Ren, Randal W Beard, and Ella M Atkins. A survey of consensus problems in multi-agent coordination. In *Proceedings of the 2005, American Control Conference, 2005.*, pages 1859–1864. IEEE, 2005.
- [5] Danny Weyns, Alexander Helleboogh, and Tom Holvoet. The packet-world: A testbed for investigating situated multiagent systems. In *Software Agent-Based Applications, Platforms, and Development Kits*, pages 383–408. Birkhauser Verlag, Basel - Boston - Berlin, 2005.
- [6] Danny Weyns. *Architecture-Based Design of Multi-Agent Systems*. Springer Berlin Heidelberg : Imprint: Springer, Berlin, Heidelberg, 1st ed. 2010. edition, 2010.
- [7] Danny Weyns, Elke Steegmans, and Tom Holvoet. Towards active perception in situated multi-agent systems. *Applied Artificial Intelligence*, 18(9-10):867–883, 2004.
- [8] Evelyne Tina Kassai, Muhammad Azmat, and Sebastian Kummer. Scope of using autonomous trucks and lorries for parcel deliveries in urban settings. *Logistics*, 4(3):17, 2020.
- [9] Bo Chen, Harry H Cheng, and Joe Palen. Integrating mobile agent technology with multi-agent systems for distributed traffic detection and management systems. *Transportation research. Part C, Emerging technologies*, 17(1):1–10, 2009.
- [10] A.J Murphy, M.J Landamore, and R.W Birmingham. The role of autonomous underwater vehicles for marine search and rescue operations. *Underwater Technology: The International Journal of the Society for Underwater*, 27(4):195–205, 2008.
- [11] Danny Deschreye. Course material fundamentals of artificial intelligence, 2021.
- [12] F. Quin D. Weyns and O. Gheibi. Multi agent systems - course slides, 2021.
- [13] A.A.A Abusnaina and Chan Huah Yong. Cooperative multi-agent system for solving packet world problem in grid. In *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pages 754–758. IEEE, 2009.
- [14] Patricio Emanuel Petrucci. Dealing with norms within the packet world multi-agent simulator. Master's thesis, Universitat Politècnica de Catalunya, 2010.
- [15] Danny Weyns, Tom Holvoet, Kurt Schelfhout, and Jan Wielemans. Decentralized control of automatic guided vehicles: applying multi-agent systems in practice. In *Companion to the 23rd ACM SIGPLAN conference on Object-oriented programming systems languages and applications*, pages 663–674, 2008.
- [16] Tom De Wolf and Tom Holvoet. Design patterns for decentralised coordination in self-organising emergent systems. In *International Workshop on Engineering Self-Organising Applications*, pages 28–49. Springer, 2006.
- [17] Joanna Juziuk, Danny Weyns, and Tom Holvoet. Design patterns for multi-agent systems: A systematic literature review. In *Agent-Oriented Software Engineering*, volume 9783642544323, pages 79–99. Springer Berlin Heidelberg, Berlin, Heidelberg, 2014.
- [18] John Sauter, Robert Matthews, H Van Dyke Parunak, and Sven Brueckner. Evolving adaptive pheromone path planning mechanisms. In *Proceedings of the first international joint conference on autonomous agents and multiagent systems*, AAMAS '02, pages 434–440. ACM, 2002.

Direct and indirect communication combined with agent memory: our experience

Colpaert Bjarne De Heel Benson Echevarría Herrera Iñaki
MAS Group 4

Abstract—This work designs, implements, and tests a Multi-Agent System (MAS) for a Packet-World environment. The goal of the MAS is to find and deliver packets as efficiently as possible. A Packet-World test bed application is used to test the proposed MAS design and implementation. The MAS solution consists of three steps: realizing (i) autonomous behaviour, (ii) coordination between agents for the purpose of charging, (iii) collaboration between agents. The proposed design focuses on the usage of agent memory combined with direct message based communication and indirect gradient based communication. Specific contributions are the usage of A* pathfinding, and the implementation of dynamically changing environmental gradients.

Index Terms—Situational Multi-Agent Systems, Autonomous Agents, Behavior-Based Systems, A* pathfinding, Dynamic Environmental Gradients

I. INTRODUCTION

Increasingly more jobs are being replaced by machines. The kind of jobs being automated can range from repeated tasks to responding in a dynamical environment. In the latter, the machines (agents) require a certain capacity to reason and respond to changes in task and environment.

When multiple agents are performing tasks towards a common goal, the system is called a Multi-Agent System (MAS). An example of a MAS is the task of finding and transporting packets in a warehouse. In such a MAS, both the requested packets and environment can change dynamically. The agents within this system have to be able to adapt to these changes. The Packet-World test bed is an application which simulates such a warehouse environment and allows agents to be placed inside of it.

The Packet-World test bed application [16] is used in this paper to design, implement, and test a packet-world MAS. In this system, agents have to find packets and bring these to the right destination point. This MAS is developed in three steps. First, autonomous behaviour for the agents is implemented. This gives an agent the capacity to perform the tasks on its own. In the second step, coordination between agents is implemented. This allows the agents to communicate with each other to keep their batteries charged. In the third and last step, collaboration is implemented. This step allows the agents to work together to solve a certain task, such as for example clearing paths to destinations.

Multi-agent systems are not limited to physical ‘robots’ driving around in an open environment, they are also used in other (virtual) applications such as cloud computing [2], heating management in buildings [13], or distributed energy storage in smart grids [9].

This paper is organized as follows: Section II elaborates the Packet-World test bed that is used to design and evaluate the MAS. Section III describes the problem that the MAS has to solve and also gives the different steps involved in the solution. Section IV explains the design and implementation of each step in the proposed MAS. Section V discusses the test results and limitations of each step in the solution. Then, Section VI discusses other related work and compares the MAS of this paper with other existing solutions. The paper finishes with a conclusion and some ideas for future work, Section VII.

II. THE PACKET-WORLD TEST BED

A. Concepts and characteristics

The Packet-World test bed is an application which is used to study and evaluate fundamental concepts of situational multi-agent systems. The test bed allows for conceptual exploration of MASs. The outcomes of the test bed can be used to infer insight in real-world situational MAS applications.

A MAS is essentially a system consisting of autonomous agents, which are able to flexibly adapt their behavior to changing operating conditions.

In a situational MAS, the agents are explicitly placed in an environment. They have a local view on the surrounding environment and can only communicate with other agents that are nearby [17].

There is no central entity which drives the agents. Instead, a decentralized approach is taken in which the agents act in ‘the present’, based on their position, perceived environment, and internal state. The intelligence of situational MASs does not arise from the capability of one agent. Instead, it derives from an efficient cooperation of all agents in the system. Benefits of this distributed approach include more efficiency, robustness, and flexibility as compared to a centrally-driven system.

In the Packet-World test bed, agents can perform a limited number of actions. All actions have a certain energy cost. Similar to most real-world applications, the agents have a battery which has to be recharged before running out. The packet-world contains charging stations which allow an agent to do so.

The environment has a grid-like structure which is modelled as a collection of grid layers. Each layer hosts a different kind of item such as agents, packets, and pheromones. Relations can be defined between layers, for example an agent in the agent layer can hold a packet in the packet layer.

B. Real-World Examples Similar to the Packet World

A first example which is similar to the packet-world problem is the coordination of straddle carriers in a port [5]. In such an environment, the agents are straddle carriers which are either autonomous, or driven by an operator. The packets are containers which can vary in weight and size. These containers are stacked in a storage area of the port and have to be transported to a container terminal, in order to be loaded onto a ship. An important difference with the packet-world application is that the containers in the port is a 3D-environment, while the packet-world test bed is a 2D environment. Not every agent can pick up an arbitrary container. Sometimes a container can be too heavy, or stacked too high for a certain straddle carrier. Effective communication and coordination between agents is crucial in order to obtain an efficient MAS. Currently, such a MAS is likely to be coordinated from a central dispatch. In the future, such a system might perhaps evolve into a situated MAS in which the agents communicate with each other to coordinate their actions.

A second example that conceptually maps to the packet-world problem is a warehouse system, such as the one developed by SSI Schaefer [6]. Such warehouse consists of a 3-dimensional environment in which packets are stacked in racks. In this particular example, there are two kinds of agents: driving agents which transport packets, and crane agents which fetch and bring packets from and to the shelves. The crane agents are locked in between shelves, so they cannot move around in the warehouse. To retrieve a certain packet from a shelf and bring it to a destination, both the driving and crane agents have to work together, similar to the packet-world test bed. The video shows that the driving agents have to charge, which is also a feature in the packet-world test bed. The driving agents are also communicating where they go, such that they don't collide. The agents have to plan ahead and find a path, which they will then have to share with other agents [7].

III. DESCRIPTION OF THE PROBLEM

The Packet-World environment contains coloured packets and coloured destinations. The agents are tasked to find and deliver packets to a destination with corresponding color. The agents in the system are situated agents which only have a limited perception of the environment. The perception of an agent is a module which the agent can use during execution. Every event in the environment that reaches the agent goes through this module, which is similar to the problem described in Kuiper and Wenkster [8]. They can perform a limited amount of actions: picking up a packet, dropping a packet, and moving one square in a certain direction.

In this work, no packets are added during runtime. The agents have to deliver all packets in the environment to the corresponding destinations while minimizing the elapsed time and energy consumption.

The packet-world problem is divided in three sub-problems. The most fundamental problem is realizing autonomous behaviour of the agents. The second sub-problem involves coordination between agents in order to deal with battery constraints. The third sub-problem concerns the collaboration to overcome the limitations of certain agents.

A. Realizing autonomous behaviour of agent

In the Packet-World environment, there is no central entity which guides the agents to do certain actions. Therefore, agents have to be able to reason and perform tasks on their own. The autonomous behaviour of agents is a fundamental piece in the MAS solution.

The agents have to explore the environment for packets. When they find a packet, they have to deliver it to the destination with the same color as the packet. If the agents cannot see the right destination, they have to explore the environment or ask other agents for seen destinations.

B. Realizing coordination to handle energy

The second sub-problem involves coordination to deal with a limiting battery. All actions that the agents perform cost a certain amount of energy: 10 energy for moving without packet, 20 energy for moving with packet, 5 energy for all other actions. The battery capacity of an agent is 1 000 units.

Charging stations are added to the environment. If the battery of an agent is low, it should go to a charging station to recharge. An agent charges at a rate of 100 energy units per time step.

This sub-problem requires the agents to be able to (autonomously) find a charging station in the environment before their battery runs out. There are less charging stations in the environment than agents. This means each agent should be able to coordinate in a queuing system for the charger.

C. Realizing agent collaboration

The last sub-problem requires the agents to collaborate in order to be able to solve the packet-world task. For this problem, packets are blocking the way to certain destinations. Agents are no longer able to pick up all packets. Every agent has been assigned a color and can only pick up packets of that specific color. Therefore, the agents will need to work together to overcome their individual limitations and unblock the paths to the destinations.

IV. THE PROPOSED PACKET-WORLD MAS

This section explains the proposed MAS for each sub-problem in the previous section. The first solution realizes fundamental autonomous behaviour of agents for the packet-world problem. The sub-sections that follow build upon this fundamental behaviour and increase the overall efficiency and capabilities of the MAS.

A. Realizing autonomous behaviour of agents

This step in the solution provides the most basic form of situational MAS one could implement. The agents are able to perform the Packet-World tasks without any interference of other agents or humans.

¹<https://youtu.be/S8zDRu72HD0>

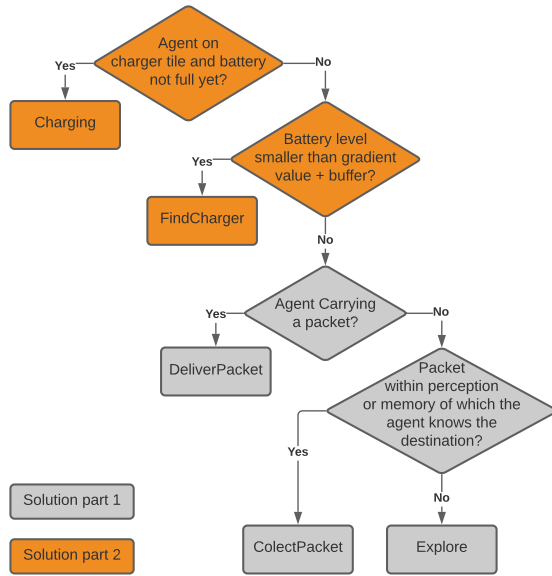


Fig. 1. Decision tree describing an agent's behaviour based on its state. The grey and orange part of the tree is implemented in part 1 respectively 2 of the solution.

1) *Design and implementation:* Agents are tasked to find packets and bring them to corresponding destinations. If the agent does not see any packet or destination, it has to explore the environment until it finds one. Based on the state of the agent, the task that it performs is either: exploring, collecting packets, or delivering packets.

In the solution these tasks are implemented as separate modules (behaviours). When an agent changes to another task, it changes its behaviour. This approach keeps the implementation code structured and relatively simple, as opposed to just one module which deals with all tasks at once.

The grey part in Figure 1 shows the decision tree that determines the behaviour of an agent. The subdivision of behaviours is inspired by the structure given in Chapter 2.3.1 of the course text [15].

There are two versions implemented for autonomous behaviour of agents: *simple implementation* and *optimized implementation*. Both implementations follow the grey part of the decision tree in Figure 1. However, the simple implementation does not utilize memory. It is therefore less complex than the optimized implementation, but also less efficient.

The memory of an agent is implemented as a 2D-grid which reflects the environment that it perceived. This 2D-grid grows as the agent explores new regions of the packet-world. Each iteration, this virtual environment is updated with new information from the agent's perception. This virtual environment keeps track of all representations in the packet-world such as packets, destinations, and walls. It also keeps track of which cells (locations) that the agent has seen. Packets that have been picked up at locations within the agent's perception are also removed from the agents memory.

Another difference between the simple and optimized behaviour is the usage of pathfinding. In the optimized behaviour, agents use the A* search algorithm to efficiently calculate an optimal path to a certain target location. The search algorithm uses the virtual environment in the agent's memory to incorporate obstacles (walls, packets, destinations) in the packet-world. Unexplored areas are assumed to not contain any obstacles. If such unexplored area turns out to contain blocking obstacles, the path is recomputed. The A* search algorithm is an efficient pathfinding algorithm. However a possible drawback is its memory footprint. A* stores all visited nodes in memory. For very large environments this can possibly lead to efficiency issues if the agents are hardware-bound (this is not the case in the packet-world test bed). Alternative pathfinding algorithms with a better memory footprint are iterative deepening A* for example. Regular A* was chosen in this implementation, mainly due to its simplicity.

Exploration behaviour: In the simple implementation, the exploration behaviour consists of moving to a random tile every iteration. Once the agent sees a packet or destination in its perception, it will move towards it and pick up or drop the packet.

Choosing a random tile to step to each iteration leads to a sporadic trail in which the agent often steps back to areas where it just explored. The optimized behaviour improves efficiency by making use of the agent's memory (and pathfinding). In the optimized behaviour, an agent chooses a random unexplored target location from its memory and moves towards this location. Each iteration, the memory is updated and all newly explored tiles within the agent's perception are marked as seen. If there are no more unexplored tiles left, the agent chooses a random location in the packet-world as a target and moves towards it.

Collecting behaviour: In this behaviour, the agent sets a packet location as target and walks towards it. When it is right next to the target packet, the agent picks it up and switches behaviour again.

In the the simple implementation, an agent switches to collecting behaviour if it sees a packet within perception. The optimized implementation on the other hand only switches to this behaviour when it sees (or knows) a packet of which it also knows the corresponding destination location (from memory or perception). This way the agent saves energy by not having to explore for the corresponding destination while carrying a packet.

In both implementations the agent goes towards the closest packet. The difference between both implementations is the distance metric being used. The simple implementation uses the bird's-eye distance to find the closest packet. The optimized implementation utilizes pathfinding, and selects the packet location with shortest path. In an environment without obstacles, both distances are the same. In a packet-world with obstacles such as walls and other packets, the latter distance is superior for the overall efficiency of the MAS.

Delivery behaviour: The delivery behaviour is very similar to the collecting behaviour. In this behaviour, the agent

TABLE I
SUMMARY OF THE PERFORMANCE OF THE SIMPLE AND OPTIMIZED
AUTONOMOUS AGENT IMPLEMENTATION.

Environment	Implementation	Energy	Cycles	First delivery
basic-1-tutorial	simple	2180	165	4
basic-1-tutorial	optimized	2175	165	6
basic-2-vision	simple	23778	864	44
basic-2-vision	optimized	6172	244	17
basic-3-color	simple	512596	7603	54
basic-3-color	optimized	20401	386	27

is carrying a packet and moves to the closest corresponding destination. When it is right next the destination, it drops the packet.

In the simple implementation, the agent switches to delivery behaviour if it is carrying a packet and the corresponding destination is in its perception. The optimized implementation always switches from collecting to delivery behaviour, because this implementation ensures that an agent only picks up a packet of which it knows the destination.

Similar to the collecting behaviour, the main difference between the simple and optimized implementation is the usage of memory and pathfinding. The simple behaviour will select the closest destination based on bird's-eye distance. The optimized behaviour uses the length of the path to the destination as selection criteria.

Finally, another addition in the delivery behaviour of the optimized implementation is communication between agents. When one agent found a destination, it will communicate the coordinates and color of this destination to all agents within perception. This way other agents are able to learn destinations without having to explore for them.

The communication is rather an addition on top of the autonomous behaviour of the agents. Without the communication, the agents are able to solve the Packet-World tasks autonomously. It just makes the task solving more efficient.

Only destinations are communicated as these don't change during execution. This makes them ideal to communicate to other agents. It is also possible to communicate packet locations in a similar way. However, packets are rather volatile to be explicitly communicated with messages. Therefore in the proposed implementation the agent don't share their known packet locations explicitly through message based communication.

2) *Test results:* The implemented MAS is evaluated in three different environments. The efficiency of the MAS implementations is expressed by the amount of cycles and energy it takes to find and deliver all packets. In this and the following experiments, all reported values are averaged over 20 executions. Table I summarizes the performance of both implementations for the different environments.

The first environment has only one agent, one color of packets and one destination. Furthermore, the perception of the agent covers the entire environment. This environment is named *basic-1-tutorial*. The simple and optimized behaviours

give almost the same results due to the fact that the agent has total information of the environment.

The second environment is called *basic-2-vision*. In this setup there are two agents, 37 packets of only one color and 2 destinations. The most important change with respect to the previous environment is that the agents can only see 3 tiles in all directions. This makes it harder for the agents to act in an optimal way.

The third and last environment goes by the name *basic-3-color* and introduces multiple colors to the environment. There are four colors in total. Every color has one destination and there are 60 packets in total. The perception range remains the same as in the second scenario at 3 tiles in every direction. The optimized behaviour is able to deliver all packets 25 times faster than the simple behaviour. Packet delivery evolution for this environment can be seen in Figure 4.

B. Realizing coordination to handle energy

This step in the solution introduces battery handling for agents. The MAS solves the same tasks as before, the only difference is that they now have a limited battery which has to be charged on time. If an agent runs out of battery, it is stuck for the rest of the simulation.

1) *Design and implementation:* The design in the previous subsection is augmented with routines for energy handling. Charging stations are added to the packet-world. These stations emit a gradient, which the agents can use to find the charger. The gradient value on a walkable tile corresponds to the amount of energy that an empty-handed agent would require to move from the tile to the nearest charging station. Non-walkable tiles such as walls, packets, or destinations have no gradient. When a packet is picked up, the tile becomes walkable and a gradient is added to this tile. The whole gradient grid is updated to incorporate the new walkable tile. Similarly when a packet is dropped, the gradient on that tile is removed and the whole gradient grid is updated again. A tile with an agent is considered walkable, as the agent is most likely moving. Tiles with agents still have a gradient.

The design of these gradients have two main advantages. A first advantage is due to the fact that the gradient value in a tile represents the amount of energy required to move to the closest (energy-wise) charging station. This allows the agent to easily derive whether it needs to go charge or not.

The second advantage is that the gradients implicitly emit the optimal path to the charger. Non-walkable tiles are incorporated in the gradient values. If an agent follows the gradients along the direction of steepest descent, it will automatically avoid static non-walkable tiles.

Two new behaviours are introduced: *FindCharger* and *Charging* behaviour. The orange part in Figure 5 shows the decision tree for the added behaviour modules.

FindCharger behaviour: When an agent's battery level is lower than the gradient at its current location plus a certain buffer (250 units in the implementation), it will switch to *FindCharger* behaviour. In this behaviour, the agent simply

follows the direction of steepest descent in gradient value towards the charging station.

Charger behaviour: When an agent is standing on a charger tile and its battery is not full yet, it switches to Charging behaviour. In this behaviour, the agent reserves the charging station for a certain amount of steps, *nbSteps*. Gradients emitted by the occupied charger within a radius of *nbSteps* are adjusted to account for waiting until the charger is free (5 energy units per step). If there are multiple charging stations in the environment, this mechanism implicitly promotes empty charging stations over occupied ones.

Using indirect communication in the form of gradients provides a strong baseline. Better results can be achieved by combining the gradients with direct communication. In particular, message based communication is implemented to construct a charging queue. This queue is useful, since it allows the charging agent to get a better estimate of the number of turns the charging station should be reserved for. For example, when a charging agent needs to charge for 5 more turns and two agents in the queue both want to charge for 10 turns, the charging agent will inform the charging station that it should be reserved for $5 + 2 * 10 = 25$ turns. This estimate is then used to adjust the gradients.

As a consequence, the gradients emitted by this charger will be quite high. The gradient of a nearby charger might be smaller and new agents will move towards the other charger, instead of simply going to the nearest charger.

A diagram of the interaction protocol that is used to construct the charging queue can be found in Figure 2. The implemented protocols are a simplification of the direct communication model given by [14]. The protocol starts when an agent approaches the charger and another agent is already present at the charger. In this diagram, the approaching agent is considered the participant.

The participant sends a message to the charging agent, asking to join the charging queue, see (1) in Figure 2. The charging agent then checks whether the participant is standing on a tile with a sufficiently large gradient, before sending back the number assigned to the participant in the queue, see (2) in Figure 2. If the value of the gradient is too small, the charging agent does not send a queue number to the participant, as this low gradient indicates that another charger would be a better choice for this agent. Since the participant did not receive a queue number, it will not wait and just keeps following the gradients to another charger.

If this initial interaction happened successfully, both agents wait until the charging agent is done charging. At that moment, the charging agent informs the participant with queue number 1 that it is next in line. The leaving agent also communicates the order of the other participants in the queue, see (3) in Figure 2. From that moment on, the agent at queue number 1 becomes the charging agent. Any new participants will communicate with this agent to obtain a queue number.

2) **Test results:** Similar to the experiment in sub-problem one, the energy behavior implementation is executed 20 times for each environment. The average amount of cycles and

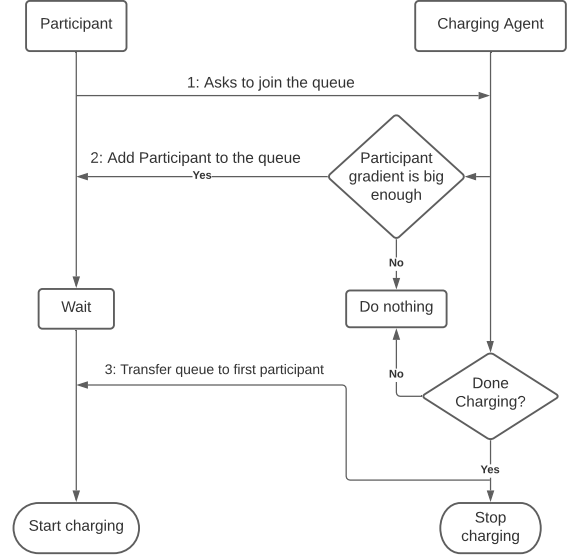


Fig. 2. Diagram of the interaction protocol used to construct the charging queue.

TABLE II
SUMMARY OF THE PERFORMANCE OF ENERGY HANDLING IMPLEMENTATION.

Environment	Implementation	Energy	Cycles	First delivery
energy-1	optimizedEnergy	13774	417	10
energy-2	optimizedEnergy	51938	807	10
energy-3	optimizedEnergy	19161	495	5

energy it takes to find and deliver all packets is again assessed. The results can be found in Table III.

The implementation for the *energy-1* and *energy-2* environments work as expected, but *energy-3* environment was sometimes left unsolved. This problem happens when more than two agents attempt to use the same charging station. The agents get trapped into a deadlock situation due to a bottleneck tile caused by the wall. When this happens, it is usually the first time agents search for a charger. Out of the 20 runs performed approximately 28,6 percent fail in the *energy-3* environment. An average of the successful runs in the *energy-3* environment can be found in Figure 4.

C. Realizing agent collaboration

At this point, the complexity of the problem increases by limiting the capabilities of individual agents. Every agent is assigned a color and can only pick up packets of this specific color. Now, the destinations in the environment are blocked by packets of multiple different colors. The agents need to work together to move the blocking packets in order to be able to reach their destinations and fulfill their tasks.

1) **Design and implementation:** The challenges introduced in this sub-problem mainly affect the behaviour of the agents in the beginning of the simulation. Three new behaviours are added here: *Clear path to destination*, *Pickup packet to be*

moved and *Drop packet that needed to be moved*. Agents will only use these new behaviours as long as their destination is inaccessible.

The interaction between these behaviours is shown in Figure 3. The agents start in the *Explore* behaviour, which is explained in the first sub-problem. Once an agent sees a destination of a corresponding color, it starts the *Clear path* behaviour.

Clear path to destination: This behaviour uses the A* path finding to construct a path towards the destination, where tiles which contain packets are given a higher weight than empty tiles. The agent then starts to move along the found path. If the destination can be reached, the agent starts collecting its packages (using the implementation from the previous parts).

If the agent is not able to reach the destination due to blocking packets, there are two options. When the blocking packet is of the same color as the agent, it switches to *Pickup packet to be moved* behaviour. In case the color does not match, the task of moving the packet is allocated to another agent.

Pickup packet to be moved: When an agent encounters a blocking packet of the same color, the packet is added to the *ToBeMoved* queue. Whenever this queue is non-empty, the agent switches to *Pickup packet* behaviour.

In this behaviour, the agent moves towards the packet using the A* path finding, picks up the packet and switches to *Drop packet that needed to be moved* behaviour.

The design allows other agents to also add packets to the *ToBeMoved* queue. This is implemented by using a *Task Communication* protocol. As such, task allocation to another agent simply consists of communicating the coordinates of the packet to be moved. Once the receiving agent parses this message and adds the packet to its *ToBeMoved* queue, the agent switches to *Pickup packet* behaviour to move the packet.

Drop packet that needed to be moved: This behaviour is the second part of the packet removal protocol. Once the agent has picked up a packet, it looks for an empty region within its perception to safely drop the packet. The agent drops the packet on a tile with empty tiles around it. This makes it highly likely that the dropped packet will not block other important paths.

Once the packet is dropped, the agent switches back to *Clear path* behaviour and check whether there are other known packets blocking a path. If no such packets are known, the agent starts delivering packets to the corresponding destination.

Task Communication: This is not a separate behaviour module, but an abstraction used in Figure 3. During task communication, the agent switches to *Explore* behaviour, in order to search for other agents. Once another agent of the required color is found, a single message is send to this agent. This message contains the coordinates of the packet to be moved. After sending the message, the agent waits in place for 10 time steps. This allows the other agent to move the

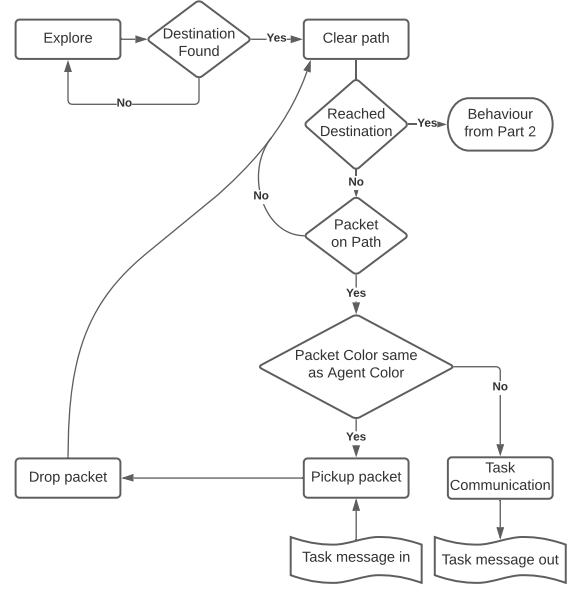


Fig. 3. Diagram of which indicates the interactions between the new behaviours added in the task allocation sub-problem.

TABLE III
SUMMARY OF THE PERFORMANCE OF THE TASK DELEGATION IMPLEMENTATION.

Environment	Implementation	Energy	Cycles	First delivery
task-delegation	taskdelegation	9446	264	68

packet. Once the 10 time steps have passed, the agent goes back to the *Clear path* behaviour.

2) *Test results*: The task delegation behavior is tested 20 times in the *task-delegation* environment, similar to the experiments in previous sub-problems. The total number of cycles and energy required to find and deliver all packets is again assessed. The results can be found in Table III.

Similar to the *energy-3* environment, there is also a situation in the *task-delegation* environment that the current implementation cannot solve. The problem occurs when an agent requests the support of another agent to remove a package from his path, but the asking agent blocks the path of the helper. As a result, the overall success rate in this environment is 66.1%. The packet delivery evolution for this implementation can also be found in Figure 4.

V. DISCUSSION

A. Test results

Based on the basic environments, there is a clear difference in performance when the viewing distance becomes limited. The optimized implementation is able to find the packets faster. The main improvement comes from the more efficient exploration behaviour. Furthermore, the optimized behaviour only requires the agents to find the destination once, after

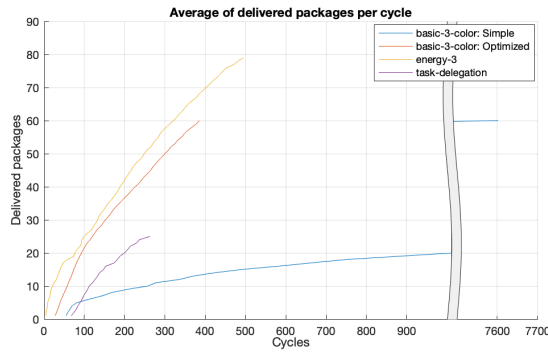


Fig. 4. Deliveries per cycle in three more challenging environments

which it is stored in memory. In contrast, the simple implementation makes the agents re-explore the destination every time, which causes them to waste a lot of time and energy. The performance gap between both implementations gets even bigger when the environment becomes larger and additional colors and packets are added.

The current implementation sometimes struggles in the *energy-3* and *task-delegation* environment. In these environments, two or more agents occupy a tile to which the other agent wants to walk to. The agents then wait until the other one moves, but no one moves. It is clear that a protocol to overcome these deadlocks is necessary in future work.

Figure 4 shows that the graphs of the optimized behaviour, the optimized energy behaviour and the task delegation behaviour all have the same slopes. This indicates that the constraints imposed by the battery and blocked paths do not significantly worsen the throughput of packets. It is however clear that the task delegation graph starts to increase at a later time. This is due to the paths that have to be cleared first.

B. Limitations

A first limitation arises in the exploration behaviour of the optimized implementation. Agents currently choose a random unseen coordinate from their memory to walk to, without keeping in mind the amount of surrounding tiles that have been explored. A better exploration implementation might look for unexplored areas instead of tiles. Another possibility would be to get rid of the randomness altogether and start exploring the environment in a consistent and deterministic way.

A second limitation is the usage of memory for packet locations. Packet locations are known to be volatile information, because packets can be picked up. The memory is only updated with information of the agent's perception. If another agent picks up a packet outside of an agent's perception, its memory becomes outdated. This can lead to an agent walking to an already-taken packet. However, the experiments showed that the effects of outdated packet memory are rather limited. Usually there are other packets around the removed packet. The agent changes its target packet once it notices that the current target packet is gone.

Another limitation manifests in the energy management and task allocation assignments. In the current implementation, 28% of the runs fail in the *energy-3* environment, due to the fact that three or more agents decide to charge at the same station. This causes a deadlock situation, because there is a bottleneck tile between the walls and the boundary of the environment. The *task-delegation* environment presents a similar challenge. When an agent requests the help of another agent, the helping agent sometimes is not able to reach the packet due to another agent standing in the way. Both agents get stuck and the environment can no longer be solved. This happened in 33.9% of the *task-delegation* runs.

C. Lessons Learned

The most straightforward way of implementing a MAS is to write the agent's handling in one method. This project demonstrated that this approach quickly becomes very convoluted and unclear. By structuring the agent's behaviour into different states, the whole implementation gets a clearer shape.

It was clear that utilizing the agent's memory, by storing locations of destinations and packets, helped a lot. It causes the agents to be less in the exploration state. The simple behaviour on the other hand constantly falls back to the exploration state, if the packet or destination is not within sight, which wastes a lot of time.

The combination of direct and indirect (gradient-based) communication worked really well to distribute the agents among the charging stations. This behaviour is most pronounced in the *energy-2* environment, where one or two agents move to the charger further away during the first charging round. This is preferable over having to provide a larger energy buffer and going charging earlier. Without both the direct and indirect communication, this behaviour would be a lot harder to implement and the agents would just clump up around the closest charger.

In the task allocation assignment, a very simple direct communication protocol was used. The implementation shows that simple communication can be combined with a smart subdivision in behaviour classes, to achieve a working solution without relying on complex communication protocols.

VI. DISCUSSION OF RELATED WORK

In order to meet its design objectives, multi agents systems should have three characteristics: pro-activeness (goal-directed behavior), reactivity (response to changes in environment), and social ability (interaction with other agents) [12]. Related projects have demonstrated that one of the most challenging aspects of dealing with autonomous behaviours is to build a system that achieves an effective balance between the goal-directed and reactive behavior, which strongly depends on the characteristics of the environment [12].

The first part of the project consists of implementing autonomous behavior for the agents. The optimized implementation works under the goal-directed behaviour and support changes in the environment (packages already delivered), but in a non-optimal way. An agent could approach a location

with the intention of picking up a packet, but another agent may already have picked it up before. This leads to the agent wasting energy and time. In the proposed implementation, there is no explicit communication of performed tasks, so keeping the position of packets in memory is an inherently volatile source of information.

In the exploration behaviour of the proposed implementation, all agents independently explore the environment. An alternative approach would be to temporarily convert the whole MAS into a mapping/exploration MAS, such as the one proposed by Scott and Yu [11]. In such exploration MAS, agents efficiently explore an area while communicating with each other. The exploration phase in autonomous behaviour has an important effect on overall efficiency.

The project started with wandering exploration behaviour, which resulted in sporadic movements that consumed a lot of unnecessary energy. Making the exploration behaviour more efficient was the first goal. This optimization resulted in at least 5 times less energy consumption compared to the simple behaviour. However, it is not fully optimized yet. Other related work, uses stigmergy to keep track of the regions which the agent has already explored, i.e. placing flags to indicate that a region is empty and should no longer be explored [16].

For the second task, agents have to coordinate in order to handle energy. The most common coordination would be communication between agents. In its simplest form, they share information about the environment and each agent manages this information on its own.

Direct communication is often more efficient, but requires advanced logic to process messages and additional support in case of message loss. On the other hand, indirect communication is easiest to follow and message loss has a minor impact on overall performance, but may be sub-optimal.

Other projects in this field use more complex types of cooperation like those based in bio-inspired algorithms (ants, bee swarms, wolves...) [3], the formation of chains to deliver packets more efficiently or indirect communication between agents.

The concept of stigmergy is inspired from social insect colonies. It is an example of indirect communication. In MAS it is applied as markers in the environment. These markers can for example be introduced as flags, gradient fields, or synthetic. In this way markers could be static or dynamic (disappear over the time) [16]. The proposed implementation in this work uses gradient fields to search for an energy source.

Our implementation takes a first step towards collaboration between agents and implements a simple communication protocol. In comparison to other related work, this version does not share packet locations. Instead, the agents have to find packet locations themselves, which might lead to a waste of energy. Not having this type of communication causes agents to spend a lot of time when there are only a couple of undiscovered packets left. But it also reduces the time spend on moving towards phantom packets, i.e. packets which were already moved by another agent.

In the energy management implementation, agents have to coordinate in order to handle energy. Indirect communication based on gradient fields in the environment is used to find charging stations. The emitted gradient field is dynamic and changes as the charger is being used. The gradients indicate the amount of energy needed to arrive to the charger.

On top of that, this work used cooperative collaboration in a peer-to-peer way. The agents communicate between each other to deal with a charging queue and decide who is next. We did not encounter related work that took a similar approach as we did. As we allow all agents to run low on battery at the same time. Hence, creating a rush to the chargers. A more advanced communication system that makes the agents go to the chargers at separate times, even if that means that the battery is not almost empty yet, might be an interesting concept to look at.

A classification of design patterns in multi agent systems has been done by Juziuk et al. [7]. Based on this classification, the proposed queue protocol can be seen as a human-inspired pattern. The proposed implementation used it for negotiating for a charging position. In more advanced scenarios, this pattern could also be used to construct a queue at the destination. For example, if the agent needs to wait a bit at the destination until a human picks up the packet. The focus of the proposed MAS lies mainly at interactions between agents. The proposed MAS implementation uses patterns for coordination, communication, negotiation (for a charger), and collaboration. The abstraction pattern that this MAS uses is a concrete one, because it was implemented in the test bed. Finally, the granularity of the MAS is mainly agent-level. Also some subsystem level patterns were used, such as a virtual environment in the memory of the agents.

For the task delegation part, the agents use direct communication. Indirect communication as described in [15] and [10] was also considered. These related works do indicate that indirect communication often leads to better solutions than direct communication. However, the indirect communication (using multiple gradients at the same time) was not directly supported by the Packet-World test bed. Therefore, some efficiency was sacrificed in favor for a reduction of complexity.

The task selection in the proposed MAS happens in a decision tree manner. Based on the state of the agent, it selects a suitable action to perform. This is still do-able for the packet-world problem of this paper. However, if additional (more complex) tasks would be added, a decision-tree type of approach becomes less feasible. In such case, it might be beneficial to consider other action selection methods such as fuzzy neural networks [6], or multi-agent reinforcement learning (MARL) [4].

VII. PRESENT CONCLUSIONS AND FUTURE WORK

A. Present conclusions

Based on the experiments held within the packet world test bed, it is clear that the optimized behaviour provides a solid basis which can be extended to deal with energy and task allocation concerns. The combination of direct and indirect

communication can result in a good strategy for dividing the agents among available chargers. Finally, this works uses a very simple communication protocol for task allocation. This is possible, since the underlying behaviour classes were designed with collaboration in mind.

B. Outline possibilities for future work

The main focus of future work should be on a mechanism to avoid or resolve deadlocks. These deadlocks pose the main problem in both the *energy-3* and *task-delegation* environment. When a deadlock does not occur there, the results seem satisfactory. On top of that, if this deadlock problem would be resolved, the proposed implementation would also be able to solve the *maze* environment, since the A* algorithm already provides very good path finding capabilities.

REFERENCES

- [1] Ahmed A. A. Abusnaina and Chan Huah Yong. Cooperative multi-agent system for solving packet world problem in grid. In *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pages 754–758, 2009.
- [2] Javier Bajo, Fernando De la Prieta, Juan M. Corchado, and Sara Rodríguez. A low-level resource allocation in an agent-based cloud computing platform. *Applied Soft Computing*, 48:716–728, 2016.
- [3] Omid Bozorg-Haddad. *Advanced optimization by nature-inspired algorithms*. Springer, 2018.
- [4] Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [5] Binghuang Cai, Shoudong Huang, Dikai Liu, Shuai Yuan, Gamini Disanayake, Haye Lau, and Daniel Pagac. Multiobjective optimization for autonomous straddle carrier scheduling at automated container terminals. *IEEE Transactions on Automation Science and Engineering*, 10(3):711–725, 2013.
- [6] K.G. Jolly, R. Sreerama Kumar, and R. Vijayakumar. Intelligent task planning and action selection of a mobile robot in a multi-agent system through a fuzzy neural network approach. *Engineering Applications of Artificial Intelligence*, 23(6):923–933, 2010.
- [7] Joanna Juziuk, Danny Weyns, and Tom Holvoet. Design patterns for multi-agent systems: A systematic literature review. In *Agent-Oriented Software Engineering*, pages 79–99. Springer, 2014.
- [8] Dane M Kuiper and Rym Z Wenkster. Virtual agent perception combination in multi agent based systems. *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AA-MAS)*, 12:611–618, 6 2013.
- [9] Cuong P. Nguyen and Alexander J. Flueck. Agent based restoration with distributed energy storage support in smart grids. *IEEE Transactions on Smart Grid*, 3(2):1029–1038, 2012.
- [10] Wannes Schols, Nelis Boucké, Danny Weyns, and Tom Holvoet. Gradient field based order assignment in agv systems. *Master's thesis, Katholieke Universiteit Leuven, Belgium*, 2005.
- [11] Adele F. Scott and Changbin Yu. Cooperative multi-agent mapping and exploration in webots®. In *2009 4th International Conference on Autonomous Robots and Agents*, pages 56–61, 2009.
- [12] Roman Vaculin and Roman Neruda. *Autonomous Behavior of Computational Agents*, pages 514–517. 01 2005.
- [13] Olaf van Pruissen, Armin van der Togt, and Ewoud Werkman. Energy efficiency comparison of a centralized and a multi-agent market based heating system in a field test. *Energy Procedia*, 62:170–179, 2014. 6th International Conference on Sustainability in Energy and Buildings, SEB-14.
- [14] D. Weyns, E. Steegmans, and T. Holvoet. Protocol-based communication for situated multi-agent systems. In *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems, 2004. AAMAS 2004.*, pages 118–125, 2004.
- [15] Danny Weyns. *Architecture-Based Design of Multi-Agent Systems*. Springer Berlin Heidelberg : Imprint: Springer, Berlin, Heidelberg, 1st ed. 2010. edition, 2010.
- [16] Danny Weyns, Alexander Helleboogh, and Tom Holvoet. The packet-world: A test bed for investigating situated multi-agent systems. In Rainer Unland, Monique Calisti, and Matthias Klusch, editors, *Software Agent-Based Applications, Platforms and Development Kits*, pages 383–408, Basel, 2005. Birkhäuser Basel.
- [17] Danny Weyns and Tom Holvoet. A formal model for situated multi-agent systems. *Fundamenta Informaticae*, 63:125–158, 1 2004.

Multi-Agent Systems: Enabling energy management and collaboration between agents in Packet-World

1st Hrvoje Abramović
Erasmus exchange student

KU Leuven
Leuven, Belgium

hrvoje.abramovic@student.kuleuven.be

2nd Joanna Kiesiak
Erasmus exchange student

KU Leuven
Leuven, Belgium

joannaetulalia.kiesiak@student.kuleuven.be

3rd Ward van Gerwen
Dept. of Computer Science

Master of Artificial Intelligence
KU Leuven

Leuven, Belgium
ward.vangerwen@student.kuleuven.be

Abstract—The paper focuses on researching Multi-agent system problems and presenting proposed solutions in the software Packet-World. Packet-World [1] is a software packet to simulate Multi-Agent systems situations in various environments with independent agents. The ultimate goal for agents is to clear all the packet in the environment by delivering them at their destination. We focus on three main problems: autonomous behaviour, coordination and collaboration. Autonomous behaviour of agents is necessary to let agents pick-up and deliver packets in an autonomous way, independent from other agents their behaviour. In coordination, the agents need to manage their energy level, coordinate their movements and have solidarity with other agents when recharging. In collaboration, the agents get a designated color and need to collaborate to deliver all the packets to their designated coloured destinations. We propose a solutions for agents to autonomously move around based on gradients and Dijkstra's algorithm. We propose a solution to energy management were agents keep track of their energy level and distance to charging station. At charging stations agent have solidarity with other agents. We propose a solution to gradients and direct messaging for agents to collaborate. Agents showed significant gains in efficiency after the proposed solutions. We conclude that it is necessary to manage the agent's behaviour with a structured and planned strategy, otherwise, the amount of cycles and used energy increases rapidly.

Index Terms—Multi-agent systems, Packet-World, Autonomous behaviour, Energy management, Collaboration, Path-finding, Gradient, Direct messaging.

I. GENERAL INTRODUCTION

The paper first provides an introduction to Multi-agent systems and Packet-World. In section IV we discuss the given problem statements related to autonomous behaviour, coordination in energy management and collaboration. We link this with the given applications and environmental situations in Packet-World [1]. Afterwards, we discuss the design our solutions these problem statements and their implementation. We present the results on all three implementations in section VI. The sections has one dedicated subsection for each problem statement. We finalize this paper with a conclusion, mention the limitations of the paper and propose subjects for future research.

II. INTRODUCTION TO MULTI-AGENT SYSTEMS

Multi-Agent System is a system composed of agents who interact among themselves and cooperate to solve complex

problems in the software system. Multi-agents systems are known to be suited for the development of decentralized software systems are flexible with dynamic environments and conditions and not dependent on overall known factors (eg components/information can come and go). In these systems many (intelligent) agents interact with each other to obtain their goal. Each entity is considered the operate autonomously, bases on its obtained knowledge at that point in time. To obtain their goal agents can work in collaboration, but also act in their own (selfish) interest. In a sense multi-agent systems "mirror" the global human population. Humans work towards their own goals, but interact with each other to obtain knowledge and work in collaboration if needed. A difference is that these system actors/agents can not intentionally act in bad faith and humans can [4] [7].

A. Software architecture for Multi-Agent systems

In the Packet World the architecture is based as three layered system: agents, ongoing activity and environment. Agents are contractors for the tasks and contain information of a current environment according to their perception. Possible activities are selected and decided by agents free-flow. The communication holds on incoming messages and sends messages as a protocol. The systems presents all ongoing activity as a concept of handling. It concerns operating tasks for instance: moving objects, evaporating pheromones etc. The software calculates reactions agents can do, based on internal rules and environmental input. [1]

B. Middleware for Multi-Agent systems

Multi-Agent systems were created as a result of the development of distributed systems with additional computational problem-solving skills. Implementation is based on a grid structure that creates layers for agents, ongoing activity etc. and defined relations between these objects. This structure has advantages, such as simplification of agent's perception either the possibility to extend the structure and re-usability. Communication takes mainly through protocol-based communication. It is important that tasks are under control and executed in a dedicated time. Multi-Agent systems have global synchronisation, regional synchronisation and synchronisation based on logical time.

C. Real world Multi-Agent system applications

We defined two real world examples that have similarities with the fundamentals of Packet-World, but do not match every aspect of Packet-World. This show how Multi-Agent systems are applied in various aspects of reality and not only in an Amazon warehouse.

1) *The Bitcoin network*: The peer-to-peer Bitcoin network is an example of a multi-agent system that has fundamental similarities with the Packet-world. We evaluate basic setup and objectives of agents and the architecture. [2]

Basic setup and objectives: The Bitcoin network defines two types of agents: miner nodes and network nodes. The miner nodes search for new blocks to add transaction and confirm added blocks by other miner while the network nodes only confirm the new blocks and included transactions. Miner nodes work in competition to find a new block as fast as possible, but together to confirm the blocks and the transactions on the network. Network nodes work together to confirm the transactions on the network. Transactions of Bitcoins can be seen as packets on the network, the agents are responsible for creating the blocks to move and confirming the move. [2]

Architecture: The architecture of the Bitcoin network has many similarities with the Packet-world. In the underlying paragraph we will highlight the communication protocol and synchronisation. The entire network works on the same communication protocol: Proof of Work. The moment a new block is created the miner broadcast this to the entire network. However, when 2 block are created at the same time, the network will convert into 2 regional synchronized chains (localised view), but with the next blocks the network will convert back to a global synchronised chain. [10]

2) *Market making agents (high frequency trading companies)*: In this example, agents are offices and trading entities of the same high-frequency trading company tied to different stock exchanges (for example: London, Amsterdam, Frankfurt). They trade on their own on each of the exchanges, making autonomous decisions to make profit. However they also communicate among each other in order to spot cross-exchange arbitrages (possibility of making risk-less profit due to different prices of the same security), and then cooperate to capitalize on those opportunities. Market making can effectively be viewed as delivering packets from origin to destination: the underlying assets (shares, bonds,...) are being moved from its original owner to a new one. This sets the value/price to a market price. The amount of assets moving enable the price to move from a high bid-ask spread to a small bid-ask spread. Additionally, agents are situated (placed really close to the stock exchanges), and act in present - in very short time frames. [3]

III. INTRODUCTION TO PACKET-WORLD

Packet-World [1] is a, flexible, software environment where Multi-agent systems can be tested in various (self) developed environments with different types of applications and different types of communication. Packet-World contains agents with a localized perception in a globalized environment, meaning bigger than their perception. The perception is visualised with a blue radius around the agent. As you can see in figure 1. Agent can memorize aspects in the environment. Apart from agents the environment contains packets and destinations. The packets need to be delivered, by the agents, at destinations that are spread within the environment. Agents can have a specific colour, introducing the difficulty that they can only pick-up and deliver packets of the same colour. Packets and destinations can have a specific colour, introducing the difficulty that they can only deliver packets at destinations with the same colour. As it can be seen in figure 2

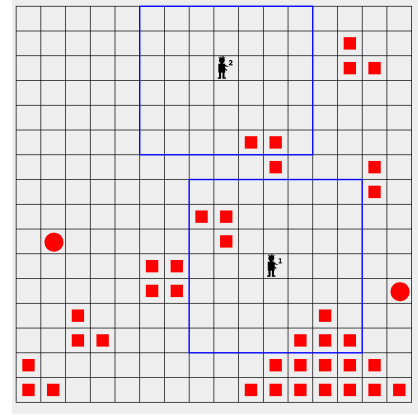


Fig. 1. Basic agents with perception and one color packets and destinations

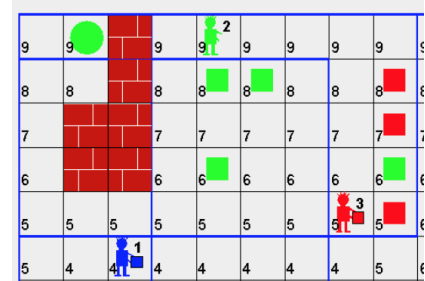


Fig. 2. Coloured agents with coloured packets and destinations

Additionally, charging station can be added to the environment. When these are added the agents in that environment need to manage their energy level so that they do not run out of energy. As you can see in figure 3. Agents can communicate with each other by using a broad spectrum of possibilities. For example by: direct-messaging, meaning when they are in each others perceptions, through broadcasting (not recommended for every situation) and

through pheromones, gradient fields and flags.

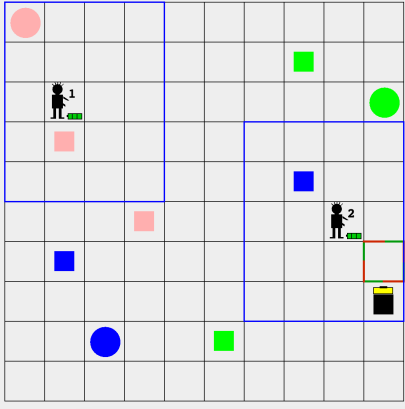


Fig. 3. Environment with uncoloured agents, local perception, coloured packets, destinations and charging station.

Finally, the overall goal of Packet-World is for agents need to find their way towards the packets and deliver them to their destinations. In this paper, we will introduce various environments with different levels of difficulty.

IV. ESSENTIAL ASPECTS TO IMPLEMENT EFFICIENT BEHAVIOUR IN MULTI-AGENT SYSTEMS

The paper divides the problem, in terms of the agents actions, into three steps : autonomous behaviour of the agents to solve tasks (A), coordination to handle efficiency in terms of energy management (B) and collaboration to operate efficiency in terms of teamwork (C). These steps are implemented in a cumulative manner.

A. Autonomous behaviour of agents

Agents need to behave in an autonomous behaviour way in multi-agent systems. Each individual agents needs to be able to carry out all of his tasks without collaboration with other agents. The agents need to pick-up and deliver packages in an efficient and autonomous manner. To increase the efficiency it is allowed for the agent to communicate through gradients and/or direct messaging, but in essence even if they did not communicate the agents should be able to finalize the task at end. We use basic items already provided in Packet-World and build more efficient solutions around this. Packet-World's current Java project only includes a random wandering action. Agents need simple behaviour actions to perform tasks such as picking up Packets, delivering Packets, dropping Packets. To obtain Packets agents wander around in the environment and use there perception to perform actions. This introduces an additional problem: the behaviour of the agent depends on it current action and current status. Agents are not allowed to perform multiple actions at the same time and can only wander one tile per instance. The action of the agent depends on its current status. For example: an agent that is wandering in the environment without a Packet can move towards a

Packet, when in its perception, pick up the Packet and switch to a wandering to deliver state. The agents keeps wandering with the Packet until the delivery tile comes into perception and drops the Packet at delivery. Afterwards the agent will wander again without a Packet, looking for the next one. The subsection of "Solution in Autonomous behaviour" proposes a solution by using specific structured behaviour classes. The agents wanders around in an environment with coloured packets and their destinations. For a visualisation of such environment we refer to figure 4

Finally, the random wandering results in an inefficient solution for task-oriented behaviour. Coordination, which is needed to perform tasks in an efficient way, will be discussed next.

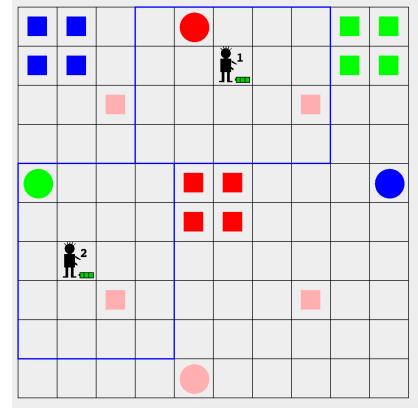


Fig. 4. Environment with uncoloured agents, local perception, coloured packets and destinations.

B. Coordination to enable energy management

Previously agents wandered randomly in the environment and consume a lot of energy with inefficient movements. Introducing energy constraints into the problem would result in agent losing all their power before the packets are delivered at their destination. Agents that memorize the destinations gain a significant increase in efficiency. To keep track of their lifespan agent have to engage in energy management which is a central subject of the second part in our paper. Agents get an initial energy bar of 1.000 on which it can move around given the energy costs listed below. Agents that are randomly wandering the environment, without memorizing the charging station and destination could lose all their energy without delivering all the packets. Therefore an agent must coordinate his moves in the environment and calculate distances to maintain a proper energy level. Charging stations can be sent as a message to the entire network, once a first agent encounters a charging station or while using gradient fields. For a visualisation of such environment we refer to figure 3. The energy costs are determined as show in table I.

Finally, it is more efficient if the coordinated agents work in collaboration with each other. The second introduced problem has additional agents in a larger space. When all of them

TABLE I
ENERGY COST TABLE

Actions	Energy cost
Step without carried packet	10
Step with carried packet	20
Other actions	5

want to charge at the same moment a bottle neck occurs and agent run out of energy. A first introduction of collaborative behaviour is necessary to solve this problem. A final form collaborative behaviour, around colored agents, is the main subject of the next subsection.

C. Collaboration between coloured agents

The first two problems focus on efficient wandering and energy wandering with a little bit of broadcasting collaboration. The final part of this paper focuses on the introduction of coloured agents that wandering around and can only delivering colour-dedicated packets to coloured destinations. The coloured agents introduce a need for delegations of tasks and a more advanced type of coordination and communication between agents. However, broadcasting is not used as a form of communication in this part. Agents before delivering the packet to the destination, have to operate obstacles such as walls and path with non-yours coloured packets. Directly, reaching coloured destinations is not possible in this environment. The task focus on picking up and deliver packets must be delegated to the right coloured agent to clear the path towards the coloured destinations. Agents have to be able to deal with this colour restrictions and collaboration exists as direct messaging.

The coloured agent settings take abstraction of the energy management problem and have, consequently, no charging stations nor energy. For a visualisation of such environment we refer to figure 2

V. DESIGN AND IMPLEMENTATION OF PROPOSED SOLUTIONS

A. Solution to Autonomous behaviour of agents in Packet-World

1) *Design*: The first and simplest approach is the following. We propose a structured behaviour design by using two states. In AgentWithoutPacket state an agent does not carry any packets. In this state, he has in his perception packet or he thus wanders around in the environment until a packet comes into the agent's perception. AgentDropsPacket state defines agents who carry a packet, in this state agent randomly wanders in the environment to find the destination, and moves towards destination in his perception to deliver the packet at the target. After the packet is dropped at the destination the agent enters the first defined state. The switches between states are based on conditional whether the agent is carrying a packet or not.

However, this approach is not efficient due to the random wandering. The optimization of agents behaviour can be designed in various manners. Firstly, we designed a solution where agents remember the destination to drop packets. The second designed optimization solution takes into account direct messaging between agents, to share knowledge about seen destinations. This optimization will discussed in subsection C "Solution to Collaboration". However, this design did not solve the random wandering. Our most efficient solution is designed and discussed in part two. The solution relies heavily on gradients, with both packets and destinations being labeled, with different gradient "layers". In this solution, agents don't communicate between each other, all the knowledge about the world is contained in gradients. In essence an agents needs to pick-up packets, mark packets that are not picked-up with gradients and deliver the packets at destination. Agent also marks unmarked destinations with a gradient. For a visualisation we refer to figure 5.

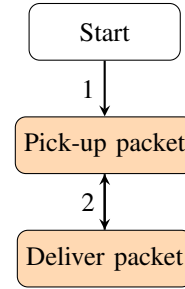


Fig. 5. Flowchart of Autonomous behaviour.

Figure 5 legend:

- (1) Look for packet and destinations. If encounter a destination memorize the location. Pick-up the first packet within perception. ;
 - (2) When destination is know, move towards destination and deliver the packet. When destination is unknown search for the destination. In "OptimizedGradient", Mark encountered packets and destinations with gradients.;
- In "Optimized2", when encountering another agent, share the know destinations through direct messaging.

2) *Implementation*: Firstly, we implemented agent behaviour named "Simple". The simple mode provides the following features: moves an agent is allowed to make and a check for viable moves, agent behaviour when it is not carrying a packet but has one in its perception, behaviour when an agent carries a packet and sees a destination and random wandering. An agent is allowed to move one tile at the time and is only able to make a move when there is no obstacle on the tile and it stays in the environment. When agents see a packet or destination, depending on its state, it takes the shortest path, by sorting possible moves with respect to their distance towards the destination or packet. This approach to path-finding works in the environments

without walls and structures that require a more complex approach to path-finding. If no destination or packet is in sight the agent will wander randomly in the environment. All the features are structured in one class.

Secondly, we implemented agent behaviour named "Structured". The structured implementation has two classes "AgentDropsPacket" and "AgentWithoutPacket" and two behavioural changes: PacketCarried and PacketNotCarried. The "AgentWithoutPacket" class implements the same allowed movements as in our "simple" implementation, but focuses on agent behaviour when it wanders around without a packets and notices a packet in its perception. the "AgentWithPacket" defines the same, but when an agent is carrying a packet. The change classes define the behavioural changes of agents when they pick up (PacketCarried) or drop (PacketNotCarried) a packet.

Afterwards, we implemented an optimization of agent behaviour named "Optimized1". This implementation its set-up is equal to the structured implementation, but optimised the agent its behaviour by memorizing a packet destination location the moment it enters an agents perception. Destinations are, as string representation of a list of destinations, memorized in the given memory map. Parsing methods are implemented in AgentImpl class. Memorizing this destinations increased the speed of our solution significantly. However, this was improved through coordination, namely messaging and gradients.

B. Solution of coordination to enable energy management

1) *Design:* Firstly, we introduced coordination for the "basic" environments (first part of the project). The first approach to coordination that we take is messaging. Agents communicate among themselves to exchange information about destination locations, consequently they do not need to discover everything on their own anymore. Agent are allowed to broadcast the location of charge stations. Nevertheless, the most efficient solution that we designed relies heavily on gradients. Whenever agent sees a packet or destination that is not previously tagged, he marks it adequate gradient center. In that case, all agents can sense those gradients and act accordingly in a given environment.

To take care of the **energy management** and charging in the second part of the project, we designed the following behaviour: Agents first search for a charge station and memorized it. When an agent finds a previously unknown energy station he memorizes the location and broadcasts the coordinates to other agents. To differentiate energy station location message from destination location message, a message coding standard is introduced. Destinations are still memorized along the way and exchanged with other agents through direct messages. Direct messaging is only possible when agents are in each others perception. While agents are

looking for packets to pick up or deliver they monitor their energy levels and keep track of their distance towards the closest energy station. Every step, the agent calculates if the energy level drops below the energy needed to move towards the energy station plus a certain static threshold value. The threshold value takes into account waiting for other agents to charge and certain possible delays along the way (obstructed path). In case an agent needs to recharge he drops the packet he is carrying, if applicable, and moves towards the charging station. However, due to the large number of agents that need to recharge at the same time, only threshold is not sufficient. Therefore, we introduce a **solidarity mechanism**. If an agent is charging he keeps track of the number of agents waiting to recharge and adapts his charging time accordingly. He does not recharge for 100% if there are more agents waiting, this prevents agents from running out of energy. Additionally, we designed and implemented an **emergency recharge message system** in case an agent is on the brink of running out of energy. The charging agent can step away from the charge station and other waiting agents would restrain themselves from charging next term. However, this system did not prove useful and was therefore deprecated. The solution for energy management is visualised in figure 6

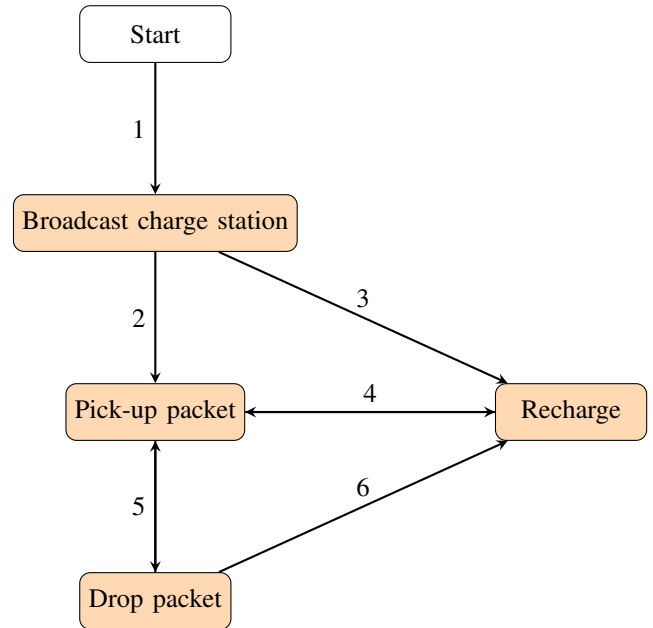


Fig. 6. Flowchart of the energy management coordination.

Figure 6 legend:

- (1) Look for charge station;
- (2) Look for packets;
- (3) Move to recharge, if no packet is found but recharge is needed;
- (4) Move to recharge, if carrying a packet drop it and move to charge station. Once recharged move to pick up a packet, following the fitted movements;
- (5) Move towards the closest viable destination and drop the

packet. Afterwards, start looking for a packet again and pick it up;

(6) Move to recharge, if no new packet is found but recharge is needed.

2) *Implementation:* Firstly, to introduce coordination in the basic environments, we implemented the optimized behaviour **optimized2**. Optimized2 uses memory to remember destination and direct messaging that allows agents to share destinations. To prevent unnecessary messages, agents keep track of destinations that are already sent to other agents. Agents only send a message if there is new information for that particular other agent. In other words, agents are not allowed to send out multiple messages with the same information to one specific agent, but is allowed to send the same message to multiple agents. Methods that achieve message parsing and coding are implemented in AgentImpl.

Secondly, we implemented the optimized behaviour **optimizedGradient** which introduces gradients, which is the fastest implementation for the basic environments. The gradient implementation allows for multiple values to be stored, this enables us to save gradient values for different packet colors and destination colors. Therefore, the visual representation of gradients (values presented on map) does not show the real gradient values, but agents can sense them. All methods were added to the Gradient World class to allow gradient propagation throughout the environment and to remove gradient centres in a way that re-evaluates other gradients. In OptimizedGradient agents add gradient center whenever they see an untagged packet or destination. Agents follow the gradients if they need to pick up a packet or deliver it to their destinations. This implementation is the most efficient for Part 1 of the assignment (basic-2 and basic-3 maps). The results of optimizedGradient are shown in the table III.

For the environments equipped with energy stations, we implemented **battery1** behaviour. This behaviour has four states: AgentLookForBattery, AgentWithoutPacket, AgentDropsPacket, and AgentRecharge. Subsequently, it has four classes that handle the behavioural change. Agents share energy station location by broadcasting the coordinates. To differentiate destination and battery messages, we enabled messages to start with a prefix "BATTERY—" for battery location or "DEST—" for destinations. The threshold value is static at 250, and set so all agents survive in the energy-2 environment. The solidarity mechanism subtracts the remaining energy level to fully charge with the number of waiting agents and multiplies this with a solidarityAmount parameter. Agent recharges until he gets to the following value: $[fullBatteryLevel - amountOfWaitingAgent * solidarityAmountParameter]$. *solidarityAmount* parameter is a static parameter that is tuned so that all agents in energy-2 survive.

C. Solution to enable agents to collaborate

1) *Design:* To let agents collaborate in order to complete their task we designed the following solution. First, the agent looks for the destination of its colour. Using a path-finding algorithm, the agent evaluates if the destination is blocked. Afterwards the agent checks if the first packet along the way is of his own or other agents colour. If the packet is his colour, he moves the packet out of the way and repeats the path-finding process. This enables the agent to free the path as far as possible towards his own destination. If the packet is of different colour, he tags it with the gradient of the other agent colours. In this design, gradients are an indicator of help. After tagging, the agent waits or helps other agents in need (if he senses a gradient of his own colour). In case agent's destination is blocked, but someone needs help, the agent will move its packet to the closest free cell. To make the packet finding more efficient after his destination is clear, the agent also tags his own packets, as a way to remember the locations. That enables an agent to efficiently deliver the remaining packets. When the path towards a destination is cleared, the agent tags it with an adequate destination gradient, this allows for easier path-finding. For a visualisation we refer to figure 7

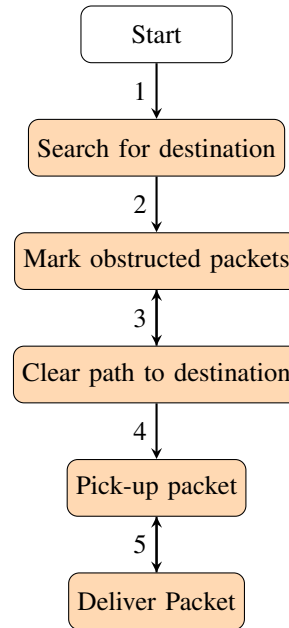


Fig. 7. Flowchart of coloured agents collaborations.

Figure 7 legend:

- (1) Look for destination, mark them with specific coloured gradients;
- (2) Look for packets that obstruct the way towards the destinations and mark them with a gradient;
- (3) Pick-up the packets that obstruct the path and drop them were they do not obstruct a path;
- (4) When the path towards the destination is cleared, pick-up

the packets;

(5) Deliver the packets at their destination and afterwards move to undelivered gradient marked packets. If no gradient marked packets, look for unmarked packets.

2) *Implementation:* This behaviour is implemented as **task-delegation**. The behaviour has five states: AgentLookForDestination, AgentHelps, AgentPicksPacket, AgentDropsPacket, and AgentResolveBlocked.

AgentLookForDestination is the first state that an agent enters, and is active until the agent finds a destination of his own colour. Agent enters AgentResolveBlocked if the destination of his color is blocked. In that state, agent checks what color is the first packet that blocks the path and then either moves the packet of his own color, or tags the packet for another agent (so the other agent can help him). If the agent is waiting for help, he checks the gradient field, and if he senses gradient for his color he switches to AgentHelps state. In that state agent follows gradient to get to his packet, and then either moves it out of the way or delivers it to his destination (if any destination is free). Once the agent can't sense any gradients (gradient center is removed when the packet is picked up) he either switches to AgentResolveBlocked state (if there is a blocked destination) or AgentPicksPacket (where he searches for packet). The state AgentDropsPacket is activated whenever agent holds a packet.

The path-finding for an obstructed path is based on breadth-first search. To find the best possible path Dijkstra's algorithm is used: if certain coordinates are updated they are pushed into breath-first search queue. The path-finding algorithm returns the shortest free (non-obstructed) path. When there is no free path available, the algorithm returns the path with the least number of obstructions (packets) and also a list of the packets on this path. This enables the agent to act accordingly. To encapsulate the result of the path-finding, a new class "PathfindingOutput" is introduced. This class provides path length, packets along the shortest path and the possible next moves.

Also, the gradients are spread in a breadth-first search manner to increase the efficiency of the proposed solution. The gradients are used as an additional path-finding tool, they avoid walls and show the number of free steps an agent needs to take to reach its goal. For a visualisation we refer to figure 2, even though there are various layers of gradients (for every packet and destination color).

, When the agent needs to move some packet of his own colour out of the way (for himself, or to help another agent), he will leave the packet at the closest free cell. Preferably all all the neighbouring cells are free, when this is not possible the agent drops the packet in a cell with only one neighbour. This method assures that dropped packets not block any path. To differentiate between blocked destinations and free destinations (for easier implementation), agent has two different memory locations. They are, however, not implemented as string representations (as before), but

as lists of coordinates (simple memory objects).

VI. RESULTS

A. Results - Autonomous Behaviour of agents

We analysed the results of our structured, optimised and optimised gradient implementation by doing five iterations on the basic-3-colour map. We analyse the results on basis of cycles performed and energy consumption. We noted that cycles are decreasing from thousands to hundreds, the same conclusion holds for energy. With this implementation, we achieved a significant improvement in both cycles and energy consumption. Moreover, we noted that whenever one agent memorizes a destination first it will gain a substantial advantage with regards to the other wandering agent(s). Consequently, allowing agents to pass along coordinates of destinations would increase the efficiency even further. At this stage, the results are without the implementation of coordination and collaboration solutions. We refer to table II for detailed results of the five iterations.

TABLE II
AUTONOMOUS BEHAVIOUR
STRUCTURED (S), OPTIMISED (O) RESULTS

Value	Cycles(S)	Energy(S)	Cycles(O)	Energy(O)
Average	8 095	540 024	2 300	108 830
Decrease	-	-	71.6%	79.9%

B. Results - coordination to enable Energy management

The results of optimizedGradient after five iterations is show in table III. OptimizedGradient enables agents to move around in the environment in an efficient manner. While optimized2 usually finishes in between 1800 and 2500 cycles, optimizedGradients usually finishes in between 300 and 450 cycles. We use this optimizedGradient behaviour is our battery1 implementation. The battery1 implementation, together the solidarity mechanism and static energy threshold, prevents all agents from running out of energy. This enables the agents to complete their tasks in both, energy-1 and energy-2, environments. After ten iterations agents deliver all the packets in on average 1095 cycles and 72 409 total energy usages. We can not show these results for individual agents and we can not show the results of energy lost while waiting to charge. The results for behaviour battery1 in the energy-2 environment is shown in table IV.

TABLE III
AUTONOMOUS BEHAVIOUR
OPTIMISED GRADIENT (OG) RESULTS

Value	Cycles(OG)	Energy(OG)
Average	363	20 204
Decrease	95.5%	96.3%

TABLE IV
ENERGY MANAGEMENT
BATTERY-1 BEHAVIOUR RESULTS

Value	Cycles - (B)	Energy - (B)
Average	1 095	72 409
Median	1 104	72 920

C. Results - collaboration between coloured agents

Agents collaborate to finalize the global and local task of delivering all the packets. We can not show the results of these local tasks and therefore present them on the global scale (clearing the total environment). We noted that the blue agent will always deliver all his packets first, before assisting the green and red agent. Agents clear the environment after on average 416 cycles and an average total energy usage of 13 960. We can not show results of the amount of placed gradients, level of collaboration etc. Agent always manages to complete all of his tasks. The exact amount of cycles and energy spent is shown in the table 4. For a visualisation of this environment we refer to III.

TABLE V
SOLUTION TO COLLABORATION
TASK DELEGATION(D) RESULTS

Value	Cycles - (D)	Energy - (D)
Average	416	13 960
Median	381	13 363

VII. RELATED WORK

Firstly, A. Baumann, B. Fabian, and M. Lischke show an approach with a deterministic solution for pathfinding. The solution are the deterministic algorithms PHA* and MAPHA* with the use of pheromones [10]. Each agent has a memory of their own graph with visited vertexes and edges. The large pheromones are a trail put on the ground, where agents communicate by writing and reading data at the nodes of the graph that states their environment. This solution is well applicable in dynamic environment and big networks.

Secondly, a developing feature is the application of reinforcement learning in cooperative multi-agent systems [11] [12]. In this case, the behaviour of agents would be to to maximize their result in each step. Before each step, an independent agent will calculate all possible actions. The agent will pick an action, which brings the maximum calculated reward after taken action. After taken this action the agent will be rewarded (good moves) or punished (bad moves). This reward/punishment scheme initiates a learning curve for the agents to find optimal movements.

Thirdly, The implementation proposed in the article of Claus, Caroline Boutilier, Craig [13] is based on a group-consensus problem with hybrid protocol. When the agents react asymptotically in a undirected network. The researchers

propose a novel group consensus hybrid protocol to solve the couple-group average-consensus problem.

Furthermore, researchers from Massachusetts described an optimization in knowledge exchange in cooperative multi-agent systems [14]. Hong-xiang Hu, Li Yu, Wen-An Zhang and Haiyu Song determined that decentralized control in communication between agents is a decision-making problem because sharing knowledge between agents is not feasible. In this paper, they explain the trade-off between deriving action policies and communication policies to obtain a maximum of the objective function.

Finally, We use a real-world example in which a multi-agent system is used throughout their distribution process. The distribution centres of Amazon are an example where multi-agent systems perform various tasks from packing all the different products off an order to allocating the packet towards the correct zip-code division [8]. In some locations the robots work together with humans to perform tasks, but have an entire environment solely for their agent interactions and movements.

VIII. CONCLUSION, LIMITATIONS AND FUTURE WORK

Firstly, we conclude that to obtain efficient autonomous behaviour of agents it is key to implement a good path-finding algorithm and introduce memory to remember specific locations and/or gradients to mark specific locations. Besides that we conclude that sharing knowledge amongst agents increases the overall efficiency of agents. When sharing destinations through direct-messaging agents became more efficient and this contributed to better overall result to deliver all the packets.

Secondly, we conclude that, in addition to the conclusion reached in part 1, a good knowledge of the environment is key when managing energy levels. Agents need memorize the location of charge stations in order to manage their energy level. Increasing the knowledge about locations of charging stations will help agent to manage their energy levels in a more efficient manner. This increased knowledge will give the agents a more distributed behaviour among charging stations and decrease the queue at certain stations. We note that agent need to have a certain degree of solidarity when charging to avoid that waiting agents run out of energy. The implementation of this solidarity contributes to a lower efficiency of the agents that need to stop recharging earlier but increase the overall efficiency.

Finally, we conclude that when agents need to collaborate to reach their individual goals a decent messaging solution is necessary. We used the gradients to mark certain packets and free the path towards the coloured destinations. Without this agent would not obtain the necessary knowledge to collaborate by moving the marked packets. Additionally, the marked packets increase the efficiency of agents because they obtain knowledge over the locations of their packets. We note the

selfishness of agents in our solutions as well, were the first agent to have a clear path (blue one in our case) delivers all its packets before assisting other agents. This selfish behaviour reconciles with the stated aspects of multi-agent systems in our introduction. The collaboration needed to free up paths will enforce that all agents work indirectly together on the same global goal of delivering all the packets while maintaining there selfish behaviour of delivering their own packets.

A. Limitations

Firstly, the solutions are based upon theoretical examples. This ensures that we encounter problems that in practice rarely occur. One limitations here is the queue of agents that is created when they need to recharge, especially with the first recharge. All agents are initiated at the same time, with the same energy level, this makes sure that, especially the first time, all the agents need to recharge at approximately the same moment. This results in a higher energy threshold which makes the solution less efficient. We created an, unsuccessful, emergency scream method to try and tackle a problem that in practice could easily be solved by releasing the agents separately. Packet-World could tackle this by releasing the agents in a sequence after the initiation or by giving agents different levels of energy with the initiation. Additionally, we did not place gradients on the charge stations. Placing gradients on these stations could enable agents to find the charging stations in a more efficient manner before broadcasting them to the network.

Secondly, the use of static thresholds and parameters in energy and solidarity, potentially, have significant impact on the efficiency and/or practical use of our solutions. Developing functions to dynamically determine the thresholds and parameters increases the robustness of solutions and decreases the need to tune these variables. These robust functions could be implemented in other situations and environments, without the need to tune the values.

Thirdly, in the collaboration solution, when the agent is clearing the path towards a destination, the agents can only place a packet when maximum one neighbour packet is around. In the current environment this was a sufficient solution, however when you move towards a maze or an environment with narrow paths this solution will not be sufficient. To make the implementation useful in practice this issue should be resolved.

Finally, the limited amount of study points (4) introduce a time constraint, which decreases the option to research on other advanced solutions. We encountered a decrease in team members which, in combination with time constraints, has a significant limitation to explore optional assignments and more advanced solutions. this is an course/team related limitation.

B. Future Work

Future work and research around multi-agent systems can be widespread. To stay in the concept of Packet-World and possible improvements on our solutions we propose the following suggestions. We propose to research the use of dynamic thresholds and parameters when working with multi-agent systems. Dynamic variables could increase the efficiency and will increase the robustness of the presented solutions.

Secondly, we propose to expand our work to test the robustness of the current solutions. In practice this means running the proposed solutions on different environments. We propose to run the current solutions in big environments with limited charge stations, because we did not place gradients on the stations agents might run out of energy before encountering one. However the implemented path-finding algorithm should be efficient enough to cover this. We propose to run the collaboration solution in a maze like environment, with small paths. It could be that the agents are not able to clear the environment in an efficient manner and this gives potential for future work.

Finally, The use of pheromones is not used in our solution and could be a useful expansion. When agents use pheromones they mark the most chosen paths to deliver or search for packets. It would be useful the assess whether this solution is more efficient than the implemented path-finding algorithm based on Dijkstra [9].

REFERENCES

- [1] Weyns D., Helleboogh A., Holvoet T. (2005) The Packet-World: A Test Bed for Investigating Situated Multi-Agent Systems. In: Unland R., Calisti M., Klusch M. (eds) Software Agent-Based Applications, Platforms and Development Kits. Whitestein Series in Software Agent Technologies. Birkhäuser Basel. pp 383-408
- [2] Nakamoto, S., 2008. Bitcoin: A Peer-To-Peer Electronic Cash System. I've added b12 for switching it, maybe
- [3] McGroarty, F., Booth, A., Gerding, E. et al. High frequency trading strategies, market fragility and price spikes: an agent based model perspective. *Ann Oper Res* 282, 217–244 (2019).
- [4] An Introduction to MultiAgent Systems 2nd edition by Wooldridge, Michael (2009) Paperback (2nd ed.). (2021). Wiley.
- [5] Felner, A., Shoshani, Y., Altshuler, Y. et al. Multi-agent Physical A/* with Large Pheromones. *Auton Agent Multi-Agent Syst* 12, 3–34 (2006). <https://doi.org/10.1007/s10458-005-3943-y>
- [6] Chen et al., 1999 Q. Chen, P. Chundi, U. Dayal, M. Hsu. Dynamic agents International Journal of Cooperative Information Systems, 8 (1999), pp. 195-223
- [7] Weyns, D. (2010). Architecture-Based Design of Multi-Agent Systems (2010th ed.). Springer.
- [8] Simon, M. (2019, June 5). Inside the Amazon Warehouse Where Humans and Machines Become One. *Wired*. <https://www.wired.com/story/amazon-warehouse-robots/>. maybe better this one? D Jain, Y Sharma. Adoption of next generation robotics: A case study on Amazon, *Case Research Journal*, Posted: 2017
- [9] Cormen, T. H. (2009). Introduction to Algorithms, 3rd Edition (The MIT Press) (3rd ed.). MIT Press.
- [10] A. Baumann, B. Fabian, and M. Lischke, "Exploring the bitcoin network," in *Proc. 10th Int. Conf. Web Inf. Syst. Technol.*, vol. 1. Barcelona, Spain, 2014, pp. 369–374.

- [11] Felner, Ariel Shoshani, Yaron Altshuler, Yaniv Bruckstein, Alfred. (2006). Multi-agent Physical A* with Large Pheromones. *Autonomous Agents and Multi-Agent Systems*. 12. 3-34. 10.1007/s10458-005-3943-y.
- [12] Lauer, Martin Riedmiller, Martin. (2000). An Algorithm for Distributed Reinforcement Learning in Cooperative Multi-Agent Systems.
- [13] Claus, Caroline Boutilier, Craig. (1998). The Dynamics of Reinforcement Learning in Cooperative Multiagent Systems. *The National Conference on Artificial Intelligence (AAAI 1998)*.
- [14] Hong-xiang Hu, Li Yu, Wen-An Zhang, Haiyu Song, Group consensus in multi-agent systems with hybrid protocol, *Journal of the Franklin Institute*, Volume 350, Issue 3, 2013
- [15] Claudia V. Goldman and Shlomo Zilberstein. 2003. Optimizing information exchange in cooperative multi-agent systems. In *Proceedings of the second international joint conference on Autonomous agents and multiagent systems (AAMAS '03)*. Association for Computing Machinery, New York, NY, USA, 137–144.

Development of a Multi-Gradient-Based Situated Multi-Agent System and Analysis of its Performance and Real-World Applicability

Inias Peeters, Kristof Van Loock, Seppe Van Steenberghe

Department of Computer Science

K.U.Leuven, B-3001 Leuven, Belgium

{inias.peeters,kristof.vanloock,seppe.vansteenbergen}@student.kuleuven.be

Abstract—Developing general, real-world applicable, situated agents for multi-agent systems is not a straightforward task. Many factors need to be taken into account in order to solve the different challenges agents could face. In this paper, an agent is developed over three iterations, each solving a sub-problem. Their performance is evaluated using the Packet-World.

Each iteration aims to solve a specific problem. The first serves as the foundation of the subsequent iterations. Its goal is to implement reasonable autonomous behaviour. The second iteration incorporates battery awareness and charging, with each agent responsible for maintaining its battery level. The final iteration integrates cooperation among agents. An agent is able to ask others for help if it is unable to proceed with its task.

The first iteration uses semi-deterministic wandering to find items in the environment and moves towards them using an A^* -based algorithm. Chargers emit a gradient field the agents use to move towards them. Charger contention is avoided using a naive policy that does not extrapolate well to other environments. Using repulsive gradient fields could be more elegant. The final iteration exploits environment-mediated communication through a multi-gradient field to address coordination. An agent can request help, tagging packets to make them emit a coloured gradient field. Corresponding agents react to this field by moving said packets.

The approach developed throughout this paper efficiently solves some proposed worlds, dealing with the difficulties they pose. However, the use of A^* search is problematic in environments with limited agent perception and containing many obstructions.

Index Terms—situated multi-agent system, packet-world, action selection, task assignment, memory fragments, protocol-based communication, multi-gradient fields

I. INTRODUCTION

A multi-agent system (MAS) is a system in which several components (agents) work together to solve a complex problem. This paper will look more specifically at the Packet-World [1] as a test bed for situated MASs. An important aspect of situated agents is that they do not create an internal model of their observed environment, but act directly on perceptions from their surroundings [2]. This prevents the agents from making long-term decisions, but the goal is that the emergent behaviour from the cooperation between them results in an intelligent system [3].

Developing an agent behaviour for solving the Packet-World can be accomplished by dividing the problem into three steps. The first step consists of realising autonomous behaviour of

agents to solve the task. The second step introduces coordination to deal with the limited amount of energy an agent can store. The third and final step introduces collaboration between agents to help each other. Each of these steps increases the real-world applicability of the solution over the previous ones. These steps are laid out in Section IV. Additionally, a section is devoted to the livelock problem.

II. THE PACKET-WORLD

Section II-A summarises the aspects of the Packet-World and Section II-B lays out two real-world problems related to it.

A. Summary of the Packet-World

The Packet-World is an extensible virtual environment designed to evaluate and experiment with multi-agent systems. In the Packet-World, battery-powered agents are tasked with moving packets scattered throughout the environment to their designated destination points. The limited amount of energy contained in the battery forces the agent to regularly visit one of the charging points in the environment. This constraint increases the applicability of the Packet-World to real-world situations. The Packet-World models the environment as a first-class entity with multiple layers that each contain a single type of items and can be used to implement environment-mediated communication approaches [4]. These items include agents, destination, packets, gradient fields, etc.

A variety of real-world problems can be interpreted as a variant or extension to the Packet-World. Two examples are detailed in Section II-B. The Packet-World consists of several layers, each containing one specific type of item. Examples of these types are agents, packets, destinations, etc. This makes the implementation easily extensible as the addition of new layers is straightforward. A possible extension could be a pheromone system. Another advantage of the layered system is the simplification of the agent's perception and the effects of actions that are taking place. Agents perceive only parts of the environment, hence only the layers necessary for this perception need to be taken into account. An agent's actions are limited by the currently active constraints. Constraints are either specified on a specific layer or on the interaction between different layers.

The original Packet-World paper [1] already provides a possible solution. In this solution agents dynamically assign themselves a role. These can be individual roles they fulfill on their own, or social roles where they work together with other agents as social entities. When the agents are acting as social entities, they use protocol-based communication [5] to establish a situated commitment to each other. As the environment may change dynamically while the agents are operating, a flexible mechanism is used by the agents to decide their next action. To this end, the Packet-World uses free-flow trees [6]. The flow of activity in the tree determines the agents' next action [7]. When an agent is committed, actions that fulfill its role in the commitment receive an additional weight in the tree. In a situated commitment, as opposed to a traditional commitment, the agents decide individually when to end the commitment. This approach increases robustness and enforces the situated nature of these agents.

B. Real-world problems mapped to Packet-World

Real-world problems come with some additional challenges. In Packet-World, the amount of tasks is finite but in the real-world, there is often a continuous stream of tasks that will be available.

1) *Self-driving taxi service*: An example of a (futuristic) application that can be approximated with Packet-World is a self-driving taxi service. This can be modelled by considering the autonomous vehicles as the agents and customers as packets. A minor difference is that each agent will be able to carry multiple people/packets. The need for energy, whether electric or fossil fuel, is analogous to the Packet-World. The limited view of agents also applies. It would be wasteful to let each taxi consider all customers in the world. They should only consider customers within a certain range. Cooperation between several agents could occur in the same way as with heavy packets in the Packet-World. A large group of people could be transported to the same place by cooperating vehicles driving together. The impact of such a system is discussed in [8]. A similar idea is worked out in [9]. They propose a MAS for taxi scheduling only, where the taxis themselves are still driven by humans. This is a less ambitious but currently more viable scenario.

2) *Automated shopping cart*: Another example is a system where automated guided vehicles drive around in a store or warehouse to pick products that have been ordered by customers. These are mapped to the Packet-World's agents and drive around in a virtual environment. A central system receives all orders and controls this environment. Each order can be mapped to a Packet-World colour. When a new order comes in, the central system allocates a destination box where its products should go and adds it into the virtual environment perceived by the agents. For each product in the customer's order, a virtual packet of the same colour is spawned at the location of that product. This leads to a scalable solution for automatic order picking. In earlier work, [10] explores the trade-offs between the use of conventional conveyor belts and AGV approaches in warehouses. Furthermore, [11] develops a

MAS which automates product selection for grocery shopping based on user preferences. This could be combined into a fully automated grocery experience with a MAS to pick the actual products from a warehouse.

III. PROBLEM DESCRIPTION

The problem to be solved in the Packet-World is transporting coloured packets from their original location to their corresponding destination. The following sections describe the three constituents of the problem: realising autonomous behaviour, coordination to handle energy and collaboration between agents.

A. Autonomous behaviour

The first problem tackled is for each agent to be able to work on the Packet-World problem autonomously. It should do so without relying on external control, information it cannot perceive or communication with other agents. Without autonomy, agents need central control to be able to solve the problem. This would go against the definition of a situated multi-agent system. Being able to function without relying on communication increases the robustness of the solution. Even when communication results in a more efficient solution, communication can fail in a real-world scenario. The agents should therefore at least be able to fall back on reasonable individual behaviour.

B. Handling energy

The limited battery capacity of the agents has a significant impact on the behaviour of the agents. Not only do they have to charge when their battery gets too low, but they also have to find and navigate to a charging station. The burden of having to charge is not to be underestimated. When an agent fails to charge in time, it runs out of energy and cannot move anymore. However, it does not disappear from the Packet-World. An agent that has failed to charge in time becomes an obstruction to the other agents by occupying space in the Packet-World. In a real-world scenario, the *dead* agent would likely require manual intervention.

C. Collaboration

An important and sometimes necessary aspect of an agent's behaviour is the ability to cooperate with other agents in its environment. An example of collaboration between agents that is necessary to solve the Packet-World is when agent can only pick up packets of certain colours. In these types of environments, an agent can become blocked or have its path towards a target position obstructed by packets it is unable to move out of the way. In these situations, the agent in question will have to rely on the appropriately colored agent to help them out. This will either require agents to be able to figure out when another agent is stuck or to be able to communicate its situation to other agents. The former approach is hindered by the limited perception of the agents. Communication can happen either directly or through the environment, by for example influencing gradient fields or other environmental features in the world. Even when not strictly necessary to prevent

these situations, communication between agents has potential to greatly improve the efficiency of the overall solution. Agents can, for example, share some of their knowledge such that new information is distributed quickly. For these reasons, proper agent collaboration is an important aspect of overcoming the agents' otherwise inherently selfish tendencies, improving efficiency and enlarging the set of solvable environments.

D. Livelocks

One persistent problem that can still easily show up after the fulfillment of the solution steps outlined above is the occurrence of livelocks. A solution can be very efficient in most of its runs but get stuck indefinitely in a slightly altered world or after an unlucky series of random moves. Most of these livelock situations are very hard to predict beforehand and tedious to design around. It therefore seems to be beneficial to insert slightly random behavior, even when there is an obvious course of action. Take, for example, any situation in which an agent does not see a superior action to perform. Performing one of a few candidate actions in a deterministic way will obviously lead to a higher chance of entering into a livelock than choosing one randomly. Similarly, skipping a turn and waiting for other agents to make a beneficial change in the environment may seem alluring in this situation, as it costs less energy than moving. However, this behaviour is again dangerous from a livelock perspective. Without additional measures, it is easily possible for all agents to end up skipping and waiting forever when doing this. Randomness is a key aspect in robust emergent behaviour. Section IV-D discusses how this applies to our implementation of the three steps previously discussed.

An alternative approach to handling livelocks is through communication between the locked agents. This, however, is often more complex and still prone to some livelocks, because it relies partially on the designer being able to foresee the different ways in which a livelock can occur.

IV. SOLUTION

This section describes the respective solutions to the three-fold problem described in Section III.

A. Autonomous behaviour

The problem of writing efficient autonomous situated agents can be solved in an iterative manner. The Packet-World includes an agent that merely wanders around without performing any actions. To get a first working implementation that can serve as a baseline, this agent is extended such that it can solve the Packet-World. The new agent still wanders around, but picks up packets if it happens to end up directly next to one. An analogous strategy is used to deliver packets to their destination. The results of this short-sighted agent are discussed in Section V, together with the more optimal agent described next. Key to solving the Packet-World efficiently is letting agents fully use their perception, as then they can spot items of interest sooner and navigate directly towards them. Considering all perceivable tiles in

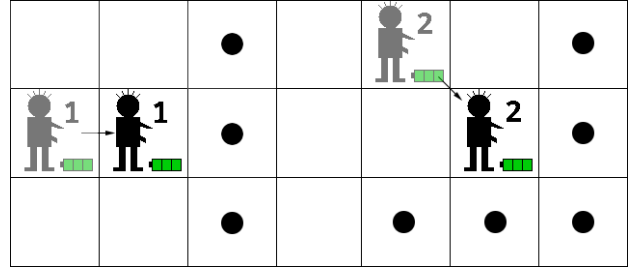


Fig. 1: Updated wandering behavior for the energy agent. The dots represent the options for the wandering agents' next moves.

this baseline agent can be made efficient by using different behavioural states. Each agent individually determines which behavioural state to follow depending on its current perception. This autonomous behaviour consists of three states: *FindingPacket*, *FindingDestination* and *Wander*. The first of these states is chosen as the initial state. However, the initial state is not of great importance, because the agents change states dynamically. The most basic of the three possible states is the *Wander* state. An agent enters this state when it is searching for an item that it can currently not perceive. When in this state, an agent performs random moves in order to explore the environment. There are two concrete ways an agent can enter the *Wander* state. A first condition for this transition to occur, is when the agent is currently not holding a packet but looking for one. The other way an agent can enter the *Wander* state is when it is currently holding a packet, but it cannot see a correspondingly coloured destination. The agent switches state to *FindingPacket* or *FindingDestination* when the item it is currently looking for appears in its sight. When this happens, the agent will take the most direct route to this item. If there are objects blocking its path, it will try to perform different moves to advance further in the direction of said item. When this is not possible, the agent will take a random step in order to avoid the obstruction.

For the energy agent, the *Wander* state has been adapted to allow the agents to discover their surroundings more actively. Rather than making completely random moves, the agents now always move away from their previous position when possible by randomly choosing one of the squares furthest away from this previous position. When this is not possible, e.g. when an agent hits a wall, a random move is made and the process repeats. This way the agent covers an increased area within the same time span. Whereas random wandering is neutral on average, this approach makes agents actively move away from where they previously were. Fig. 1 illustrates this behaviour.

Another improvement incorporated in the energy agent is the use of memory. Agents can store up to ten memory fragments, allowing them to *remember* previously observed items. The energy agent makes use of this capability by storing up to four observed destinations. Besides remembering destinations, the energy agent also only picks up packets for which it knows the location of a respective destination. Moving

with a packet costs twice the energy of moving without a packet. The goal is for the agents to continue exploring without a packet rather than picking up a packet for which they do not know the destination, previously leading to expensive wandering.

The agents have very limited memory. Each destination takes up a memory slot and the amount of memory slots to be used for destination is a parameter. The memory policy is to prefer to keep as many different coloured destinations in memory as possible. A colour that is not in memory will always replace a colour with multiple occurrences. More generally, upon encountering a destination, an entry of the colour that occurs most often in memory is replaced. Upon a tie, they are all considered. The specific entry among those that is replaced is the one furthest away from the current position of the agent.

The memory management strategy outlined above can still result in a state where the world is unsolvable. The situation that can arise is that an agent is not remembering the appropriate destination when there are only packets of one colour left and there are more destinations than an agent can remember. It can only occur in a world where an agent traveling along any path from the correct destination to the remaining packets always comes across enough destinations of other colours to clear out the correct one. This results in agents never picking up the packets. Once they find the destination, they will forget about it by the time they reach the remaining packets. This is illustrated by the example in Fig. 2. A straightforward but naive solution is to set the amount of destinations to remember higher than the amount of distinct colours expected to be seen in the world. A more realistic solution in specific environments where either not all colours are known beforehand or the amount of colours is high, is that agents also remember the colour of the last observed packet that they are not carrying. This extra information is then taken into account by the memory replacement policy. A destination with only one occurrence is never removed from memory if it matches that colour.

Another aspect improved in the energy agent is its pathfinding capabilities. The previously described agents (the short-sighted and full perception agents) used a basic algorithm which led them to their destination in an optimal way, assuming there would be no obstructions (e.g. other packets, destinations, etc.). The energy agent improved on this by using A^* as pathfinding algorithm (implementation based on [12]). However, to successfully implement this algorithm, an important assumption is made: all positions outside an agent's current perception are free. This assumption serves as a heuristic to be able to handle computing the path from the current position of an agent holding a packet to a remembered destination outside of the agent's current perception. As a consequence, the computed path towards the destination is only an approximation, but it dynamically updates as the agent moves closer to the destination. The same algorithm is used for navigating towards packets, but the approximation has less of an impact in this case because an agent only goes to packets

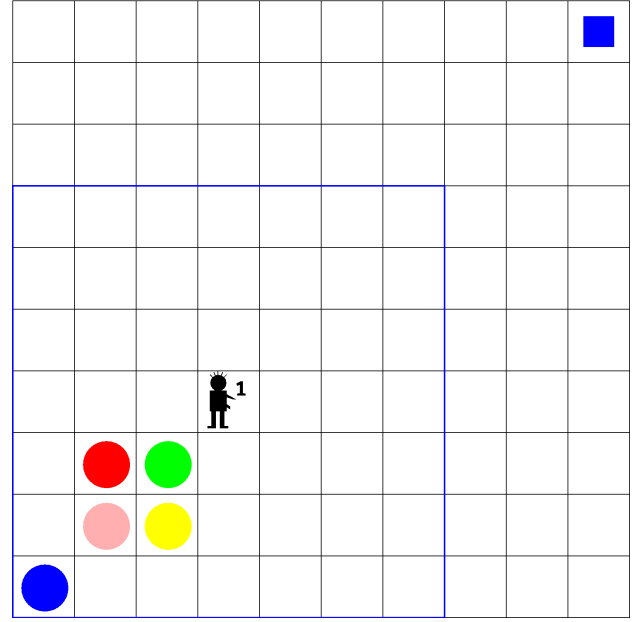


Fig. 2: This figure illustrates the memory problem. The agent in this figure has its memory configured to retain four destinations. Once the agent moves towards the blue packet, it will see only the four other destinations and remove the blue one from its memory. The blue packet is then never picked up.

in its perception. The algorithm is also used to find chargers in combination with a gradient field, which is explained in the next section. It is clear that the utility of this approach increases with the size of the agents' perception. It is also at risk of getting stuck, especially in highly obstructed worlds (e.g. lots of walls) with a small perception.

With the task delegation agent, the need arose to have a more flexible A^* implementation. Agents need to be able to calculate paths through packets to be able to determine which packets need to be removed first. Therefore, the A^* implementation was extended such that it can take custom cost functions. The energy agent still uses the algorithm in the same way, except for that it explicitly uses a distance-based cost function. To compute which packets need to be removed, the routing algorithm for the task delegation agent allows paths through packets. A custom cost function that incorporates colour as well as distance is used to penalise packets on the computed path. The penalty of a packet is smaller for packets of an agent's own colour than for differently coloured packets. This way, the path to be cleared is the most efficient one that requires the least help from other agents.

B. Handling energy

A crucial aspect of handling energy is locating the chargers in the environment. One can consider two possible approaches to find the chargers: either letting an agent broadcast the position of the charger to all other agents or using gradient fields. Here, the latter approach is used such that agents are not dependent on other agents to find the charger. The

gradient field is represented by a layer in the Packet-World. It holds a certain value for each position in the Packet-World, corresponding to the distance to the closest charger from that position. When the environment is created, the gradient field is computed using a flood-fill algorithm.

Before going towards the charge point, agents drop their packet if they are currently holding one to save energy. The agents then proceed to find the charger by walking down the gradient field. They do this by computing a route to the position with the lowest gradient value in their perception using the earlier described A^* pathfinding algorithm. When the charging station is in the agent's perception, the agent only continues to walk towards it if the station is free and *skips* if it is currently not free. This has two significant advantages: the agents use minimal energy when they *skip* and it avoids the cluttering of agents around the charge station.

C. Collaboration

Currently, the agents can solve the world without communicating. However, making use of communication can drastically improve the quality of the solution. The energy agent is the first agent to incorporate basic communication. If an agent has destinations stored in memory, it sends this information to the other agents in its perception. Therefore all agents know the destinations as soon as possible, even without having seen them, and can start moving packets towards them.

In the *task-delegation* world, more elaborate coordination between agents is a necessity. In this world, only coloured agents are present and their correspondingly coloured destinations are blocked by walls and packets. To solve this new kind of problem, the task delegation agent is introduced. This agent is built on top of the previously discussed agents. The main additions to this agent are the two new behaviours *ClearingPath* and *HelpingOtherAgent*. To accommodate these new behaviours, adaptations had to be made to the source code of the Packet-World. The two main additions to the code are a new layer that holds a *MultiGradient* field and a new action *tagPacket*. The new *MultiGradient* field layer is based on the already present *Gradient* field layer in the source code and on previous work by Weyns et al. [13]. The new layer mainly works in the same way, except for that it can hold multiple fields at once. The goal of this new layer is to be able to have gradient fields for packets. Therefore, the *MultiGradient* field is implemented to separate the different fields present based on colour. The way this is represented in the GUI is shown in Fig. 3. In order to update the *MultiGradient* field, agents can use the aforementioned *tagPacket* action. When an agent picks up a tagged packet, the field emitted by that packet is removed automatically.

The *ClearingPath* behaviour is responsible for clearing a path towards an agent's own destination. When the outermost blocking packet is of the same colour as the agent, it removes this packet to unblock the path. When the outermost packet cannot be moved by the agent, it uses the *tagPacket* action in order to let the other agents know that they should come

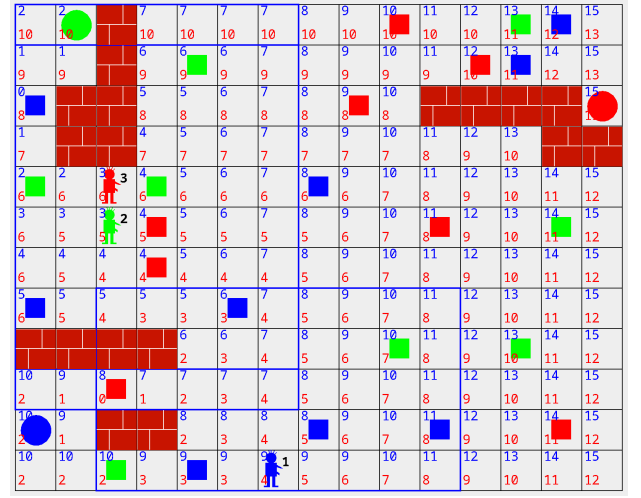


Fig. 3: This figure illustrates the way the *MultiGradient* field is visualised in the GUI. It is analogous to the original *Gradient* field but with the value of the individual fields written in their corresponding colour.

and remove this packet if they can. By tagging the packet, a gradient field is created in the correct colour and the agents of that colour will transition to the *HelpingOtherAgent* behaviour. This agent will go to said packet by traversing the *MultiGradient* field based on its colour the same way the energy agent finds its charger. Once arrived, the agent picks up the packet and will try to deliver the packet to its destination.

D. Livelocks

One of the main pitfalls to work around is the possibility of livelocks. Below are some ways in which the occurrence of livelocks influenced the solution proposed in this paper.

- Randomness is introduced in many situations where there is no obvious move. Most behaviour rules do not exhaustively select an action in every possible situation and then simply end with *if no action has been selected yet, take a random valid step*.
- The situation in Fig. 4 initially led to a livelock. This was fixed by allowing an agent who is helping another agent to deal with obstructions in its path as well. It can either tag obstructing packets of another colour or deal with obstructing packets of its own color by itself.
- Another livelock was caused by the determinism of the *HelpingOtherAgent* behaviour. Figure 5 shows the situation. This livelock was once again solved by introducing randomness. When an agent in the *HelpingOtherAgent* behaviour decides to move to the position it just came from, it will instead move in a random direction instead.
- The memory problem from Section IV-A and Fig. 2 leads to a livelock when unsolved.

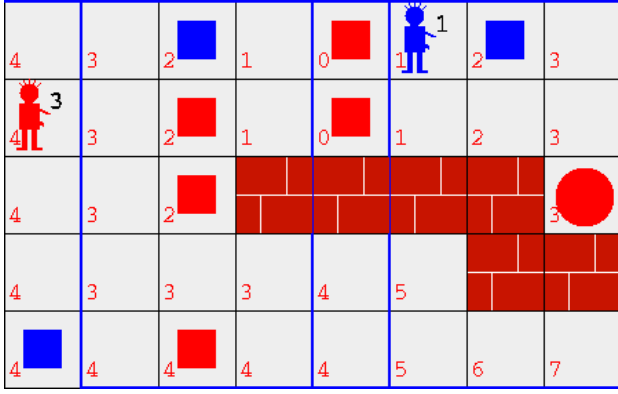


Fig. 4: The red agent has dropped what it was doing to come and help the blue agent who is stuck and has tagged two red packets that are in the way. However, the red agent is stuck as well.

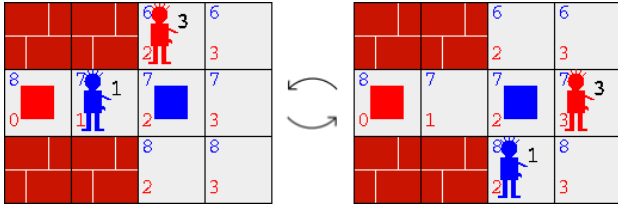


Fig. 5: In this livelock, two agents are in each other's way and continuously side-stepping each other.

V. DISCUSSION OF THE RESULTS

This section discusses the results obtained using the solutions described in Section IV. The metrics given in Tables I to VI are those provided by the Packet-World application. The “Total cycles” and “Energy consumed” are the number of time cycles and the total amount of energy consumed by all agents after delivering all packets. For both metrics, a minimal and maximal bound is shown. To obtain these estimates, the simulation was run for one hundred consecutive runs. All environments are depicted and described in Appendix A.

A. Basic environments

Tables I and II contain results obtained using respectively the short-sighted agent and the agent using its full perception described in Section IV-A on the provided basic environments. The former serves mostly as a baseline to situate the performance of further implementations. Tables III and IV show the results obtained by the energy and task delegation agents on these basic worlds.

TABLE I: Data collected from the short-sighted agent solving the different basic environments (Figs. 8 to 10).

Metric	<i>basic-1-tutorial</i>	<i>basic-2-vision</i>	<i>basic-3-color</i>
Total cycles	1499 - 4876	1609 - 4896	11378 - 20987
Energy consumed	23860 - 81460	50230 - 120790	773820 - 1371130

TABLE II: Data collected from the full perception agent solving the different basic environments (Figs. 8 to 10).

Metric	<i>basic-1-tutorial</i>	<i>basic-2-vision</i>	<i>basic-3-color</i>
Total cycles	165	570 - 1559	5971 - 10621
Energy consumed	2180	15095 - 38195	406015 - 723990

TABLE III: Data collected from the energy agent solving the different basic environments (Figs. 8 to 10).

Metric	<i>basic-1-tutorial</i>	<i>basic-2-vision</i>	<i>basic-3-color</i>
Total cycles	160 - 169	198 - 634	675 - 1013
Energy consumed	2110 - 2220	4730 - 14360	31780 - 45360

TABLE IV: Data collected from the task delegation agent solving the different basic environments (Figs. 8 to 10).

Metric	<i>basic-1-tutorial</i>	<i>basic-2-vision</i>	<i>basic-3-color</i>
Total cycles	161 - 168	192 - 647	583 - 1031
Energy consumed	2110 - 2220	4590 - 14580	28230 - 46130

The full perception agent is fully deterministic in an environment where it can see everything. This can be seen for *basic-1-tutorial* in Table II. Interestingly, the more advanced agents that use A^* pathfinding do show some spread on this world. The reason for this is that the algorithm considers the neighbours of the current position in random order. This means it does not always select the same shortest path. Because it cannot look that far ahead, one of the paths could end up next to the packet in a way that positions the agent further away from its next target.

Increasing the size of the world increases the amount of wandering the agents will need to perform in order to discover all of the packets. This means the amount of randomness and with it also the spread of the performance metrics across different runs increases.

As one would assume, the agent that can use its entire perception has a significant advantage over the short-sighted agent. When comparing both agents in the basic environments, it is clear that the full perception agent uses significantly less cycles and energy overall. The main reason behind this is that the full perception agent moves with a purpose and decreases the amount of wandering necessary by using its full perception.

When comparing the task delegation agent to the energy agent in the three basic environments, the results are almost identical. This can be explained by the fact that their behaviour is identical in these environments. If the charging aspect of the energy agent is never activated and paths are never blocked, they are essentially the same agent.

B. Energy environments

Table V illustrates the results obtained by agents finding their paths using the A^* algorithm and managing their own energy by utilising the chargers' gradient fields introduced in Sections IV-A and IV-B. The total amount of cycles and energy consumed mainly depends on the number of chargers available and their respective position but also the perception of the agents.

TABLE V: Data collected from the energy agent solving the different energy environments (Figs. 11 to 13).

Metric	<i>energy-1</i>	<i>energy-2</i>	<i>energy-3</i>
Total cycles	596 - 1321	939 - 1145	462 - 707
Energy consumed	19440 - 40120	60350 - 71460	21995 - 29540

In *energy-1*, the spread observed is mainly related to the fact that only one charger is available and the perception of the agent is limited. It induces that the number of cycles can significantly increase depending on charger contention.

The impact of the perception of agents is clearly visible in *energy-2* even with a larger world. In addition to that, the position of one charger close to the middle of the world helps with cleaning the world efficiently. Given the agents' wide perception, the charging station is visible from nearly every position in the world.

Most of the time, the *energy-3* world seems to be solved quite efficiently by the energy agent. However, rare livelocks still occur with the energy agent.

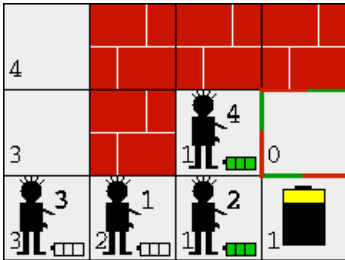


Fig. 6: A livelock occurs in the *energy-3* world. Agent 1 has died in a particularly unlucky position and is blocking two other agents from moving back out into the world. The two remaining agents stay alive forever, but they will not be able to clean up any more packets.

This happens most commonly when all four agents decide to go for the same charger. The excessive contention for the charger will lead to agents *dying*. If an agent happens to *die* right in the opening to the charging area, the agents inside will get stuck and the charger will be unreachable from the outside as well. Figure 6 illustrates an example. A solution to this problem would have to deal with charger contention effectively. One approach could be that agents, while going for a charger, emit a repulsive field on the same layer as the chargers. This would cause the other agents to automatically avoid crowded chargers when looking for one.

C. Task delegation environments

The results for the agent incorporating task delegation in the *task-delegation* environment are presented in Table VI.

TABLE VI: Data collected from the task delegation agent solving the task delegation environment (Fig. 14).

Metric	<i>task-delegation</i>
Total cycles	210 - 402
Energy consumed	7980 - 13985

Because the task delegation agent is the only one that can solve the *task-delegation* environment and there is only one such environment, it is difficult to assess the efficiency of the agent. The observed spread in cycles can be accounted for by the randomness in the agents' behaviour to avoid livelocks.

In terms of livelocks, the *task-delegation* environment can be considered solved by the proposed task delegation agents. Over several thousand runs, no livelocks are encountered. Nevertheless, it is not possible to completely exclude the occurrence of extremely rare livelocks without a formal proof of correctness. However, one can state with 99% confidence that the probability of a livelock occurring in the current solution is less than 0.02%. This result is obtained by performing a binomial hypothesis test over 25000 runs where none of them was observed to get stuck.

In the *maze* environment, the reliance on the A^* algorithm shows its downsides. The fact that the agents assume any coordinate beyond their perception does not contain anything, prohibits them from navigating an environment with limited perception and many obstructions. Livelock quickly occurs as agents walk back and forth, changing their mind on what the shortest route is. This could be solved by extending the *MultiGradient* field to incorporate destinations, as described in Section VII-B. Because the full perception agent does not use A^* , it is able to solve the *maze* environment. However, it does so very inefficiently.

VI. RELATED WORK

In this section, other works related to the problem posed in this paper are discussed. These range from directly handling the *Packet-World*, to solving a related multi-agent problem with methods that could be adapted for use in the *Packet-World*.

Abusnaina and Yong [14] propose a solution for the *Packet-World*, focusing on the communication between agents with a predefined hierarchy within the system. One of the agents, the discovering agent, is in charge of travelling across the entire world and reporting the numbers and locations of the discovered packets to a communication platform, called *Black Board*. This platform is shared with the other agents, who take on the role of working agents. Once the entire grid has been covered by the discovering agent, the working agents retrieve the data stored in *Black Board* in order to pick up the packets and deliver them. This mechanism is illustrated in Fig. 7. The dedicated role of a discovering agent improves energy consumption, even for large worlds. Both forms of communication play a significant role in reducing the number of steps an agent takes. On the intercommunication layer, all the packets' coordinates and their numbers are stored in the *Black Board*. The intracommunication layer allows the working agents to exchange information to e.g. prevent them from simultaneously going for the same packet. One aspect this paper ignores, is the perception of the discovering agents. This has a significant impact on the performance of the proposed system. The time required by the discovering agent to go through the entire environment and report relevant information

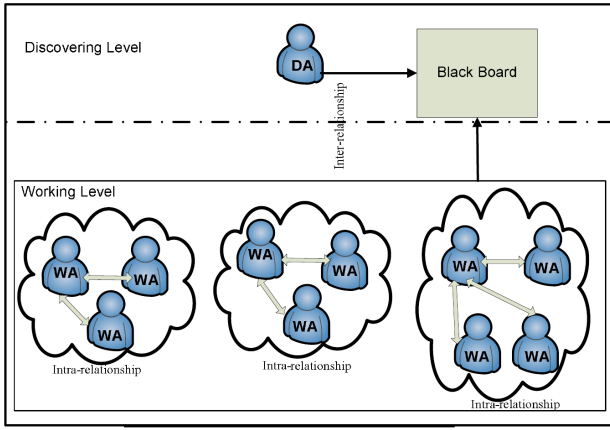


Fig. 7: Cooperative MAS Model [14]

to the *Black Board* strongly depends on its perception. Another restriction in this paper is that there is only a single destination where all the packets have to be delivered. The efficiency of the model they developed is also dependent on the assumption that the agents can remain stationary without consuming energy. This allows the working agents to remain in place during the discovery phase, fully conserving their energy. In an application where waiting and doing nothing consumes energy, this could cause problems. Another problematic situation is when a part of the environment is completely blocked off by packets (as in *task-delegation*, Fig. [14]). The enclosed part of the environment could remain uncharted during the discovery phase.

Moon et al. [15] propose a method for path planning, collision avoidance and task assignment in unmanned aerial vehicles (UAVs). While the concrete application is not directly related to agents in the Packet-World, they employ similar ideas to those proposed here. Their UAV system uses the A^* algorithm for navigating the static parts of the environment, but combines it with repulsive fields emitted by the UAVs to incorporate dynamic avoidance of other UAVs. This idea can be applied in the Packet-World, and is further explored in Section VII-B. It provides a solid approach to independent pathfinding. The Packet-World is a discrete environment, rather than a continuous one, which simplifies many aspects. However, it has its own unique challenges. The UAVs' environment has local tasks in fixed environments. They do not transport packets, but simply go to a *destination* and perform some task there. They are also designed for open spaces where it is possible to leave ample margin between themselves and obstructions in the environment. A three-dimensional variant on the narrow passages in *energy-3*, *task-assignment* and *maze* (Figs. [13] to [15]) would be difficult to handle. Limited perception is also not considered, while it does have an impact on the applicability of the A^* algorithm (as argued in Section IV-C). Finally, there is no need for collaboration in their application.

An alternative approach to task assignment that is also applicable in the Packet-World is proposed by Sariel and

Balch [16]. In auction-based task assignment, an auctioneering agent assigns tasks to all other agents. When an auctioneer proposes a task, interested agents respond with their estimate for what their cost will be to execute it. The auctioneer then selects the one with the minimal cost to carry out the proposed task. In the Packet-World, the environment itself, being modelled as a first-class entity, could take on the role of the auctioneer. In the Packet-World, the remaining challenge would then be for the agents to accurately estimate how much energy it will take to deliver a certain packet. Additionally, the proposed model requires agents that are on their way to execute a task to regularly broadcast their estimate of the remaining cost. If another agent receives this broadcast and determines that it can do the same task with a smaller remaining cost, it will send a *task exchange* message, causing the other agent to cancel its pursuit of that task. The agent that sent the *task exchange* message is then expected to fulfill the cancelled task. This allows the system to deal with more dynamic conditions. In the Packet-World, it would mean, for example, that an agent does not have to be able to produce an accurate estimate right away. After some cost-impacting event, e.g. the discovery of a new destination, an agent can adapt its cost estimates and potentially take over another agent's task or have its task taken over by another agent through the task exchange system.

Juziuk et al. [17] present an overview of design patterns currently in use for multi-agent system design. They consider four dimensions of classification: inspiration, abstraction, focus and granularity. The multi-gradient fields proposed here are a stigmergy pattern similar to normal gradient fields and can be classified as nature-inspired (inspiration), at the level of the environment subsystem (granularity), structural (focus) and conceptual (abstraction). Their overview of patterns includes some that apply to the work in this paper and to the Packet-World's design in general. The multi-gradient fields are a specialization of the *stigmergy* pattern. The Packet-World as a whole can be seen as a *virtual environment* system that is designed around the *multi-agent layer pattern*. It provides a separate virtual layer for each type of object in the world. The multi-gradient fields were introduced as an additional layer in the Packet-World's structure. To navigate towards their destination, agents make use of the A^* *planner* pattern.

VII. CONCLUSIONS AND FUTURE WORK

A. Conclusion

In this report, different implementations of situated MASs were investigated and evaluated. The three implementations of agents are the full perception, energy and task delegation agent. Even though each of these agents is engineered for a different kind of environment, the optimisations of the previous agent are propagated to the next one. One of the most impactful optimisations is the use of the A^* algorithm in order to let agents efficiently traverse their environment. This, in combination with remembering and sharing knowledge about destinations, boosted the performance of the solution drastically, although it is not applicable in all environments. Another

improvement that increased the efficiency of the agents greatly was the less random wandering behaviour, especially in larger worlds with limited perception range.

The different environments used to evaluate the agent each represent a different kind of real-world scenario. How the energy world provides a connection to real-world applications is natural and briefly described in Section III-B. The task delegation environment represents real-world applications where agents have to work together to reach their goal. An example of such a real-world application is order-picking where there are two types of agents, each capable of carrying a different type of container. The energy agent can handle the proposed energy environments, but has trouble solving charger contention in general. The current energy agent is unlikely to perform as well in more challenging environments.

The current code, however, cannot be used directly in the real world. It relies too much on random movement to avoid livelocks. Random movement of agents may seem odd to those who are not familiar with the agents. For instance, in Section II-B, a self-driving taxi service is suggested as an application of MASs. If the vehicles would rely too much on random movement, clients using the vehicles would not be at ease when taking such a taxi. A more efficient and less alarming way to behave would be to more extensively make use of the communication capabilities of the agents.

B. Future work

The proposed method to avoid charger contention is overfitted on the test environments used in this paper. It relies heavily on the size of agents' perception, as they stop moving once they observe the charger is occupied. In environments where agents have a rather limited perception size, this could be problematic as they will walk relatively close to the charger before realizing it is occupied. A better, more stigmergic solution could be to alter the gradient-field emitted by the charger, such that it emits a repulsive field when occupied. This way, agents are more encouraged to look for other chargers when another is occupied. The magnitude of the repulsive field could be based on the time left to charge of the currently charging agent such that other agents do not unnecessarily go looking for other chargers. An additional problem caused by charger contention is that an agent should leave the charger when another agent approaches that needs to be charged more urgently and might run out of energy before it gets the chance to do so. This could be resolved by using a communication based policy in which agents that need urgent charger access communicate their estimated remaining time to live to the agent currently using the charger.

The MultiGradient fields were introduced in a later stage. However, they have the potential to completely replace the A^* algorithm and cover some of its weaknesses. They could be extended to allow destinations to emit separate fields as well. These packet and destination fields, like the fields emitted by the charging stations in the proposed solution, would be present from the beginning of the run. Then, there would need to be a mechanism for the agents to still meaning-

fully tag a packet that's in their way by, for example, adding an additional priority MultiGradient layer that only contains gradients for tagged packets. In this way, agents could consider that layer before looking at the general field layer. This could lead to a simpler implementation in general. The randomness approach to avoid livelocks could be generalized as well. Any agent behaviour could perhaps always be expected to determine a list of possible actions in order of preference, with the last action in the list always being "move randomly". Then, an exponential ranking scheme or similar could be implemented to determine the chance for each of these actions to be selected. This way, the chance of selecting a very inefficient action is limited, while livelocks are eventually escaped through random movement.

Although these proposals seem promising, they are nothing but ideas and their actual performance should be properly evaluated in further research.

REFERENCES

- [1] D. Weyns, A. Helleboogh, and T. Holvoet, "The packet-world: A test bed for investigating situated multi-agent systems," in *Software Agent-Based Applications, Platforms and Development Kits* (R. Unland, M. Calisti, and M. Klusch, eds.), (Basel), pp. 383–408, Birkhäuser Basel, 2005.
- [2] D. Weyns, E. Steegmans, and T. Holvoet, "Towards active perception in situated multi-agent systems," *Applied Artificial Intelligence*, vol. 18, no. 9–10, pp. 867–883, 2004.
- [3] D. Weyns and T. Holvoet, "A formal model for situated multi-agent systems," *Fundam. Inform.*, vol. 63, pp. 125–158, 12 2004.
- [4] D. Weyns, A. Omicini, and J. Odell, "Environment as a first class abstraction in multiagent systems," *Autonomous Agents and Multi-Agent Systems*, vol. 14, pp. 5–30, Feb 2007.
- [5] D. Weyns, E. Steegmans, and T. Holvoet, "Protocol-based communication for situated multi-agent systems," in *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems, 2004. AAMAS 2004.*, pp. 118–125, July 2004.
- [6] E. Steegmans, D. Weyns, T. Holvoet, and Y. Berbers, "A design process for adaptive behavior of situated agents," vol. 3382, pp. 109–125, 07 2004.
- [7] D. Weyns, E. Steegmans, and T. Holvoet, "Towards commitments for situated agents," vol. 6, pp. 5479–5485, 01 2004.
- [8] G. S. Bauer, J. B. Greenblatt, and B. F. Gerke, "Cost, energy, and environmental impact of automated electric taxi fleets in manhattan," *Environmental Science & Technology*, vol. 52, no. 8, pp. 4920–4928, 2018. PMID: 29589439.
- [9] A. Glaschenko, A. Ivaschenko, G. Rzevski, and P. Skobelev, "Multi-agent real time scheduling system for taxi companies," pp. 29–36, 01 2009.
- [10] L. Ribas-Xirgo, J. M. Moreno-Villafranca, and I. F. Chaile, "On using automated guided vehicles instead of conveyors," in *2013 IEEE 18th Conference on Emerging Technologies Factory Automation (ETFA)*, pp. 1–4, Sep. 2013.
- [11] K. Joo, T. Kinoshita, and N. Shiratori, "Agent-based grocery shopping system based on user's preference," pp. 499 – 505, 11 2000.
- [12] Wikipedia contributors, "A* search algorithm — Wikipedia, the free encyclopedia." https://en.wikipedia.org/w/index.php?title=A*_search_algorithm&oldid=1019089087, 2021. Online; accessed 21-April-2021.
- [13] D. Weyns, N. Boucké, and T. Holvoet, "Gradient field-based task assignment in an agv transportation system," in *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS '06*, (New York, NY, USA), p. 842–849, Association for Computing Machinery, 2006.
- [14] A. A. A. Abusnaina and C. H. Yong, "Cooperative multi-agent system for solving packet world problem in grid," in *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pp. 754–758, 2009.

- [15] S. Moon, E. Oh, and D. H. Shim, “An integral framework of task assignment and path planning for multiple unmanned aerial vehicles in dynamic environments,” *Journal of Intelligent & Robotic Systems*, vol. 70, pp. 303–313, Apr 2013.
- [16] S. Sarel and T. Balch, “Real time auction based allocation of tasks for multi-robot exploration problem in dynamic environments,” in *Integrating Planning into Scheduling: Papers from the 2005 AAAI Workshop*, ed. Mark Boddy, Amedeo Cesta, and Stephen F. Smith, American Association for Artificial Intelligence (AAAI), 2005.
- [17] J. Juziuk, D. Weyns, and T. Holvoet, *Design Patterns for Multi-agent Systems: A Systematic Literature Review*, pp. 79–99. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014.

APPENDIX A ENVIRONMENTS

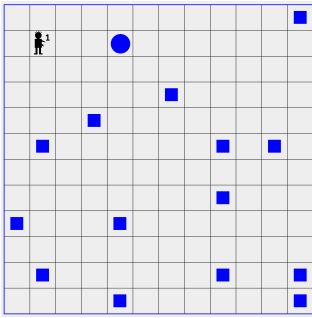


Fig. 8: Representation of the *basic-1-tutorial* environment. The environment contains 14 blue packets, one blue destination and a single agent. The agent’s perception covers the entire wold.

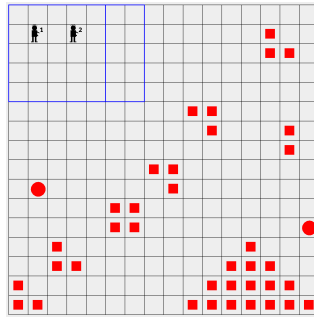


Fig. 9: Representation of the *basic-2-vision* environment. The environment contains 37 red packets, two red destinations and two agents. The agent’s perception covers a 7x7 area.

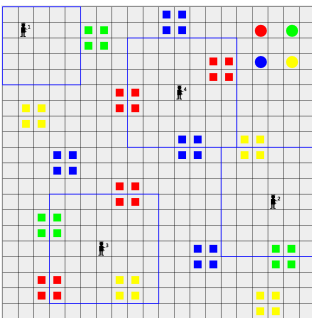


Fig. 10: Representation of the *basic-3-color* environment. The environment contains 60 packets of four different colours, four corresponding destinations and four agents. The agent’s perception covers a 7x7 area.

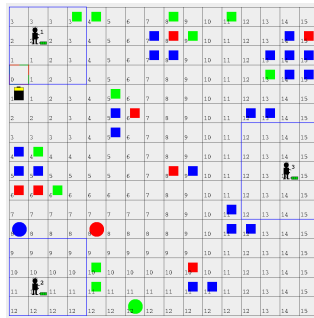


Fig. 11: Representation of the *energy-1* environment. The environment contains 40 packets of three different colours, three corresponding destinations, one charger and three agents. The agent’s perception covers a 5x5 area.

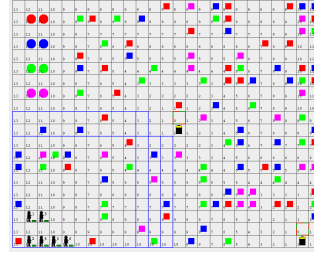


Fig. 12: Representation of the *energy-2* environment. The environment contains 110 packets of four different colours, eight corresponding destinations, two chargers and six agents. The agent’s perception covers a 17x17 area.

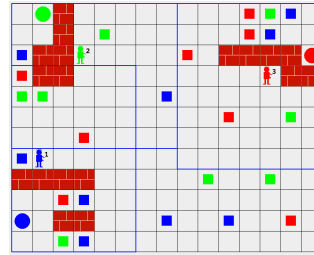


Fig. 14: Representation of the *task-delegation* environment. The environment contains 25 packets of three different colours, three corresponding destinations and three coloured agents. The agent’s perception covers a 9x9 area.

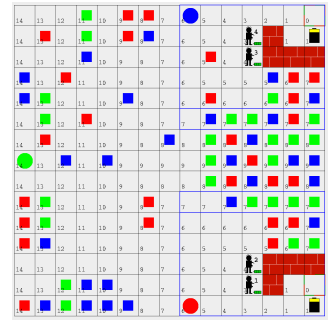


Fig. 13: Representation of the *energy-3* environment. The environment contains 79 packets of three different colours, three corresponding destinations, two chargers and four agents. The agent’s perception covers a 7x7 area.

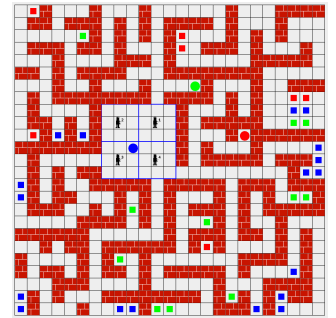


Fig. 15: Representation of the *maze* environment. The environment contains 36 packets of three different colours, three corresponding destinations and four agents. The agent’s perception covers a 3x3 area.

Multi-Agent A* based grid search implementation and evaluation in the context of package delivery

Ioan-Alexandru But, Mateo Duren, Sebastian Michels A., Marion Waterkeyn

Master of Artificial Intelligence

Katholieke Universiteit Leuven

Leuven, Belgium

Abstract—In this paper a multi-agent system approach is selected to solve a package delivery abstraction called The Packet-World. The implementation can handle environment-specific needs such as walls, energy management, blocked paths and task delegation. These constraints are handled by agent autonomous behaviour, communication, memory, and coordination via gradient fields. Performance of the solution is evaluated against a baseline behaviour based on random motion. A key contribution of this paper is the implementation of A* within agent navigation for optimal path search.

Index Terms—Multi-Agent Systems, Autonomous behaviour, Packet-World, A*

I. INTRODUCTION

This paper describes the design, implementation and testing of agent behaviours within the Packet-World environment. The Packet-World in [1] is a software test bed for investigating situated Multi-Agent-System frameworks (hereinafter referred to as MAS). A MAS is a decentralised approach to problem solving, where independent agents work towards achieving a common goal by interacting with each other and their environment. Problem resolution within a MAS comes from 3 main categories of behaviour: autonomous behaviour, coordination, and collaboration. In the Packet-World, these behaviours are oriented to the common goal of delivering all packages in the environment to their respective destinations. This must be done in an efficient way while handling agent constraints (e.g. battery) and environment constraints (e.g. expected time for task completion, walls and other obstacles).

This paper is organised as follows: Section II introduces the Packet-World environment. Section III introduces two real-world examples of a MAS. Section IV provides an overview on the functional requirements for efficient goal completion (e.g. delivering all packages) and the quality constraint that need to be handled (qualitative requirements). Section V describes the design approach selected (e.g. specific routines, patterns) to fulfil the requirements, as well as their implementation in the Java Packet-World application. In Section VI an analysis of the quantitative results is given. Some important limitations of the proposed implementations are outlined. Section VII provides a brief literature review on alternative solutions to the proposed implementation and discusses future work. Finally, sections VIII summarizes the work done and draws a conclusion on the results.

II. THE PACKET-WORLD

The Packet-World application allows to investigate different architectures, behaviours, interactions with the environment and communications schemes. The Packet-World is composed by several working elements as the environment, the agent implementation, the ability to perceive, act and communicate and finally, a set of simulations tools to configure the environment to the specific research task [1]. The overall goal is to deliver all the packages in the environment at the right destinations by using a minimum amount of energy. In a situated-MAS this goal is reached through the cooperation of the agents. The characteristics of the Packet-World are described below.

A. Environment Representation

Components are implemented in a layered architecture where each layer is devoted to one element type from the environment. The base layer is a two-dimensional grid that contains the geometry of the environment. All the other elements are related to this grid which makes them *situated* in the environment. One of these layers consists of the labelled packages, while another one consists of the specific destinations. Other layers contain elements such as: obstacles, power sources, agents etc. Elements in one layer can interact within the similar layer as well as with elements of a different layer. In addition to interaction, there are also layer and inter-layer constraints defined. Examples of constraints are: "Two packages cannot be situated on the same location" and "An agent's destination cannot be the same position as the location of another element". The environment handles the influences from agents and ongoing activities so that a new state can be provided (dynamic evolution). The environment serves as middleware for the direct and indirect interaction and advanced collaboration such as flags, gradients and pheromones.

B. Agent Abstraction

An agent is deployed in the environment as an autonomous entity which has the capability of interacting with the environment by using its perception and actions. The agent architecture contains different sub-modules that handle perception, knowledge, decision making, communication and execution of tasks. For example, an agent is able to perceive a flag, recharge itself, leave pheromones trails, pick, carry and drop a package, etc. These actions change the state of elements

towards the goal. The agents are also capable of holding information (memory) and communicating.

C. Architectural Characteristics

The Packet-World environment is *customizable* and allows the researcher to define types of interactions, size, steps, synchronisation and communication boundaries. The architecture in [1] includes several ways of interaction between the environment and the agents. The sections IV and V provide a description of the problem and how the agent implementation of the Packet-World has been adapted to fulfil the functional and qualitative requirements.

III. REAL-WORLD EXAMPLES

In this section, two real-world examples of MAS that map to the Packet-World environment are presented. The first real-world application concerns smart manufacturing, while the second one tackles the supermarket shelf refilling application.

A. MAS for Smart Manufacturing

Smart manufacturing reflects to the application of interconnected and collaborative machines in manufacturing processes. There is an increasing interest in applying multi-agent intelligent design in the so called "Industry 4.0" [2]. The aim is to adapt the manufacturing process in real time to changing demand conditions. This can be achieved through the implementation of big data analysis, artificial intelligence and advanced robotics technology. This section draws a parallel between the smart manufacturing application and Packet-World.

In our proposed smart manufacturing system, automated guided vehicle (AGV) agents are employed for the logistics and handling steps between different 3D printers, processing and assembly stations. Agents automatically run between these 3D printers and the assembly points to form a flexible assembly chain.

The main similarities of this smart factory and the Packet-World abstraction are:

- In both, AGVs are required to perceive and make rational decisions in order to achieve a goal (e.g. bring raw material 'X' to machine 'Y').
- The goal of an agent is to deliver as many packets as possible (e.g. raw material, finished parts) to their required destination within a minimum number of steps, packet manipulations and message exchanges. The productivity of the factory shall be maximized.
- Agents need to manage energy consumption.
- Agents face obstacle avoidance problems.
- An agent can perform individual tasks as well as cooperative tasks (e.g. inform that machine 'X' has piece 'Y' ready to be unloaded).
- The environment evolves dynamically and can be represented as a multi-layered 2D grid.

However, limitations of the Packet-World representation to this real-world example are:

- The amount of spawn locations for packages is fixed and does not change.
- Packets must be removed from the spawn location (e.g. 3D printer) in a certain time.
- Two sequential packets from a certain printer may have different destinations (e.g. two distinct assembly points).

B. Supermarket Shelf Refilling

Supermarkets pose several highly dynamic logistic and supply chain problems. One of those is shelf refilling [3]. It is very important to periodically refill the stock in order to avoid sale losses. However, shelf stock is stochastic and dynamic over time according to consumer demand. Furthermore, in a common supermarket, thousands of products are handled, with different "shelf life" values and even product care needs (e.g. refrigeration). Nowadays, this problem is tackled by humans and is therefore very labour intensive. Opportunities for automation arise as this problem is suited for a MAS approach and could be abstracted into the Packet-World. The analogy with the Packet-World is made below:

- Agents could perform the shelf stock monitoring (perception).
- Agents are requested to handle refilling of shelves in an efficient way (package delivery task).
- Agents need to manage uncertainty and incompleteness of information, making it difficult to plan long-term actions.
- Social interaction is desired and agents could learn from each other as which shelf requires refilling.
- AGVs need to manage all the challenges regarding physical interaction with an environment (energy conservation, moving and dealing with obstacles).
- The Packet-World architecture could allow packets with specific constraints (e.g. maximum delivery time for refrigerated articles).

IV. DEFINITION OF REQUIREMENTS

The main goal of the Multi-Agent System is the efficient delivery of packets. It can be broken down in functional and qualitative requirements, which define the design space for the agent behaviour.

Functional requirements are the specific tasks performed by the agents in order to complete the goal through collaboration. Also, these functional requirements are subdivisible in simple goals. Examples of sub-goals are: "determine which is the closest package, choose the most efficient direction to a target, walk a predefined path, pick the package or simply explore." In other words, the agent should:

- Autonomously determine a set of actions to perform.
- Perceive and interact with agents, targets and other objects.
- Move, explore and execute actions to influence the environment.
- Modify its behaviour according to the present system state, history and conditions. Use perception, knowledge (memory) and communication to influence the decision flow.

The key attributes that agents need to have are autonomy, interactivity and robustness. However, according to [4] mobility and reactivity are also paramount for this problem. Given a defined set of environments, different solutions are possible (with varying complexity). Solutions based on random movement are very energy consuming and may not be robust, more complex behaviour patterns (e.g. A* route calculation, memory) are aimed to increase system efficiency. Some environments can't be solved with simple behaviours and advanced coordination and cooperation is mandatory (e.g. task delegation). Qualitative requirements are mainly the handling of agent energy and packet delivery performance indicators (i.e. total cycles to complete).

All requirements need to be satisfied in the predefined set of environments. Some environments can be very complex and have specific characteristics such as labelled agents or blocked destinations. The implementation should be robust enough to succeed without modification to the source code (even though in real life, agents are normally customised to work in a specific environment).

A. Autonomous behaviour of agents

The agents should not rely on an external intervention (besides the information they can exchange) in order to decide their actions. More specifically, the agent implementation shall:

- Find a sub-goal and allocate priorities among set of possible actions given the current state (e.g. pick up a package, charge the battery).
- Optimise the set of actions to reach the current sub-goal (e.g. shortest path to destination).
- Discriminate validity of an action (e.g. when moving in a direction, the cell must be walkable).
- Perceive targets (e.g. destination, packages, energy stations) in its proximity and interact with them if required.
- Manage battery State-of-Charge to ensure continuous availability and efficient use of the charging stations.
- Be capable of advanced behaviour such as memorising location of stations or delegating a task to another agent.

The implementations of these functionalities are covered in detail in section V.

B. Coordination between agents

The coordination between agents ensures that the agents are well organised so that they harmoniously work together towards effective results. Coordination is achieved by combining communication between agents (coordination using direct message exchange) and a gradient field (coordination through a virtual environment). The coordinating agents will:

- Communicate by exchanging direct messages about the location of packages, destination, and charging stations.
- Understand and interact with the priority order of the messages received.
- Use the implementation of the gradient field, a virtual layer in which gradients indicate the path towards the charging stations.

C. Collaborative behaviours

Collaboration between agents is necessary to achieve certain tasks such as clearing a path to a temporarily blocked destination. By working together, the agents will succeed in completing tasks that cannot be completed by an individual agent [5]. The collaborating agents will:

- Collaborate by detecting an agent-specific task and assigning the task to the right agent through communication.
- Accept the received tasks and solve them accordingly.

V. DESIGN AND IMPLEMENTATION

A. Autonomous behaviour

The autonomous behaviour of the agent managed with different processes: 'Goal allocation', 'Navigation', 'Interactions' and 'Energy management'. The goal allocation processes decide what the agent should do, given a certain status. In Java, this is managed through behaviour changes. Processes related to 'Navigation' aim to find a suitable path and move the agent through the environment. 'Interactions' explicitly influence the environment, objects and/or agents in it, while 'Energy management' manages the on-time recharging of agents.

The agent is mostly reactive to the environment and has only short-term planning capacity (used for optimal path routing and coordination). Actions are determined for each cycle as the systems evolves. The agent can use its memory to perform short-term planning of actions like following a path determined by A* or waiting for another agent to clear the path. Transitions between behaviour types (e.g. between movement and interaction) are carried out by verifying specific satisfiability criteria (e.g. Is there a package neighbouring the agent?). The final agent implementation can autonomously navigate according to the immediate goal priorities, avoid all kinds of obstacles, pick-up, drop and deliver packages, manage the battery charge and share information with other agents. The four kinds of processes are explained below.

The implementation allows for a smooth transition from navigation to interaction and vice versa. However, energy management routines are prioritized and other interactions (unless they are required to charge the battery) are not possible until the agent has been recharged. High-level diagrams of the different states and transitions of our implementations are presented in figure 1 and 2.

The goal allocation revises the current status of the agent and selects the sub-goal according to predefined conditions (e.g. if the agent is carrying a blue package and a blue destination is neighbouring, deliver this package). Navigation routines have three high-level functions: 1) determination of the target path, 2) verification of path and detection of 'stuck' states, and 3) movement execution. The first part calculates a direction to follow given a certain rational (e.g. random direction behaviour being subordinate to the more complex A* based shortest-path behaviour). The second part verifies if the immediate action is feasible (e.g. is the cell of the next step walkable). If the direction is not walkable, this function

will override the first direction and propose a new feasible one. Lastly, the action is executed and the agent is moved.

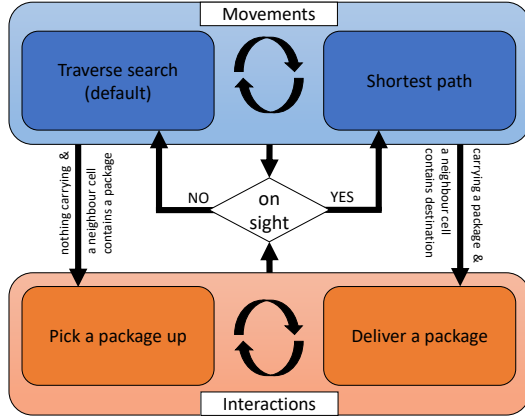


Fig. 1. Representation of the relationships between the different behaviours.

The implementation of the navigation routines has been approached in an incremental way, starting from the simplest kind of motion, i.e. randomly wandering. Three distinct incremental improvements are implemented: 1) Traverse search and 2) Shortest (straight) path determination and 3) Shortest (A* based) path determination. In traverse search behaviour, the agent searches the environment in straight line motions instead of random motions. In case no progress can be made as the direction is blocked, the agent will bounce randomly. This kind of random behaviour is important to avoid deadlocks. In the second implementation, the agent makes use of its perception capabilities to determine if there is a packet or destination within sight (depending on whether a agent is carrying a packet). If a target is detected, the agent will calculate the vector coordinate that is closest to the straight-line direction of the target and walk towards it. Random behaviour takes place in case straight line direction is invalid. When a package is delivered, an additional check is carried out to ensure that the destination and the package have the same label. The final and most complex movement behaviour is optimal path search with A* similar to reference [6]. The A* search algorithm will perform a heuristic search within the agent's perception to determine: 1) if there exists a walkable path and 2) the optimal path itself. Our A* method receives a target coordinate and returns an array representing the path to the target. In contrast to the algorithm of [6], the heuristic function, $h(i)$, is defined as the Chebyshev distance between cell i and target t : $h(i) = \max(|x_i - x_t|, |y_i - y_t|)$. This approach has the main advantage that it is able to route the agent through obstacles. Beside the built-in loop detection, path deletion and branch-and-bound principle of the A* algorithm, the implementation is made particularly robust by: 1) updating the previous computed path if it is obstructed and 2) directing the agent to a target out of sight by applying the same algorithm for the 'nearest' cell that is in sight.

In a combined case, the agent performs traverse search as long as it does not have targets within perception or its

memory. As soon as the object enters the perception, a straight or A* path towards the object is taken. One of the key advantages of this framework is that it is extendable to other navigation behaviours and its target independent.

The interaction routines allow the agent to pick-up, drop and deliver packages. No other interactions were implemented (dropping flags, pheromones) as the coordination is reached via gradient fields and communication.

The energy management routine verifies the current State-of-Charge (SoC), gradient field and, if possible and within sight, the availability of a energy station. When agents perform actions, they consume energy from their battery. This energy consumption depends on the performed action and is defined by table I. In all circumstances, the implementation should prevent agents from running out of battery.

TABLE I
ENERGY CONSUMPTION PER ACTION EXPRESSED IN ENERGY UNITS

Action	Energy consumption
Step with carried packet	20
Step without carried packet	10
Other action	5

The 'energy management' category contains three behaviours: 1) when to start charging and whether the current task should be interrupted, 2) navigation to the station and charging queue, and 3) charging the agent. A threshold strategy, based on [7], is implemented to decide the charging of the agent. Four State-of-Charge thresholds determine when the agent enters and leaves the 'energy management' mode: SoC_{LOW1} , SoC_{LOW2} , SoC_{LOW3} and SoC_{HIGH} . The relation between the four thresholds and the different modes are illustrated in figure 2.

SoC_{LOW1} threshold indicates 'opportunistic charging'. The agent will go to charge only if a station is on sight and no other task is assigned. The SoC_{LOW2} threshold indicates 'low battery'. The agent will go to charge if it is not already in its way to a destination. The SoC_{LOW3} threshold indicates 'very low battery' and its able to interrupt tasks immediately and proceed to the nearest station. Once in 'energy management' mode, the agent will navigate to a station using the gradient

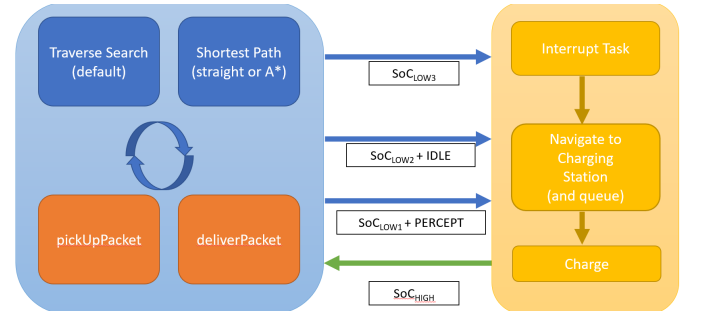


Fig. 2. Representation of the relationships between the different behaviours with 'energy management'.

field. [8] suggests two options for selection of the charging point: nearest station and lowest waiting time (queue). In our implementation, agents will navigate to the nearest station as performance gains are limited for such reduced environments and the implementation is simpler. If a package blocks the path defined by the gradient field (walls are taken into account by the gradient itself), the agent is capable of autonomously clearing the path. When the station enters the perception, A* path calculation is triggered. When approaching the station, the agent will check if the charging point is occupied. If yes, the agent will queue in few cells away not to block the entry point to the station. The waiting cell is selected through a simple algorithm that takes into account gradient and perception. The agents will charge based on the system turns - no other rules to handle priority were added. This avoids agents blocking the flow from and to the station. As the intelligence of the queuing algorithm is very limited, some environment configurations may result in agents to run out of battery due to long queues. Once the agent is in charging mode, it will remain until the battery level is higher than SoC_{HIGH} . The study in [8] suggests that a maximum State-of-Charge between 80% to 90% increases agent availability.

Our solution opted for conservative and robust values for the SoC thresholds, in order to avoid agent unavailability due to discharging: $SoC_{LOW1} = 60\%$, $SoC_{LOW2} = 50\%$, $SoC_{LOW3} = 40\%$ and $SoC_{HIGH} = 90\%$. These values have the disadvantage of reducing the number of actions between charges. However, further tuning can be done but is environment specific.

The concrete implementation into Java has been done ex-

tending the provided classes, following a structured approach that allows easy customisation and extensions of the existing routines. An overview of the implementation in Java is presented in figure 3 through an UML diagram. The UML diagram describes the relationships between the new and existing classes and their methods, that compose the agent's behaviour. Discussion of implementation results will be presented in the next section.

B. Memory, communication and coordination using gradients

Inspired by the LIDAR-based mapping of real AGVs [9], it is interesting for each agent to generate an own map about the environment using their perception, memory and communication. This map can be used to guide the agent towards targets outside its perception in a much more efficient way. We have implemented an environment map based in a JSON-object grid. The JSON-object is stored in one of the agent's memory cells. All agents start with an empty map of the environment. Each cycle, the agent's map is updated with the objects in its perception. In addition, agents communicate to other agents when objects are found and packages are picked up, dropped or delivered. Communication both accelerates the map generation, continuous update and reliability.

The communication between agents is based on the Agent Communication Language (ACL) proposed by FIPA [10]. The structure and subject of each possible message is defined using enums, in order to guarantee correct reading by the receiving agent. The communication occurs in each cycle just before the agent performs his action. The agents have to understand each others messages, so encoding and decoding steps are

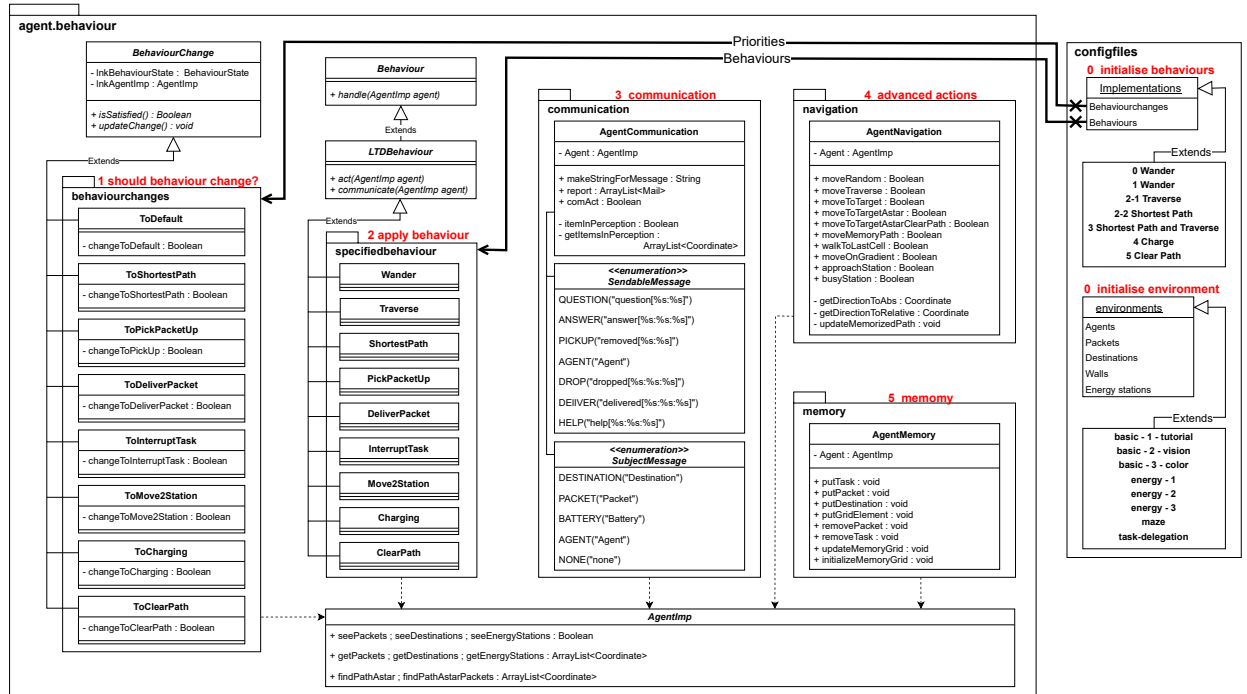


Fig. 3. UML representation of the relationships between the different classes.

introduced into the software architecture, following the design presented in chapter "Protocol-Based Communication" from [11] Ch. 3.9]. During the communication phase, the following actions are performed: update of the memory grid according to perception and send a new message (1) about the action they are going to perform or (2) to request all agents about specific information from their perception. Then, the agent will retrieve all messages directed to it and finally, the agent will act on these messages. This action can include: "replying on an information request, updating its memory or reacting on a collaboration request."

The coordination between the agents is performed in a decentralised manner, regardless of the interaction type between agents or between agents and the environment. [12] describes the use of gradients as a decentralised coordination design pattern. In our work the concept is used to facilitate agents in being aware of the energy source at all times. In order to coordinate the charging of agents, a gradient field is set in place at the initialisation of the environment. Gradients are marked in each cell representing the Chebyshev distance to the charging station. However, they consider walls to facilitate agent navigation. At any point the agent requires a recharge it is enough to descent on the gradient field in order to arrive at a charging position.

C. Collaboration through task assignment

A collaborative architecture of agents is necessary for each environment where task-specific agents must work together to achieve a common goal. (i.e. in the task delegation environment agents can only pick up packages that match their label.) In this case, a task schedule will be necessary as one agent will only be able to perform its task if the previous task was completed by another agent, and so on. Solving this issue required a mechanism of task delegation between agents. The solution of "Smart Message" proposed in [13] was adapted to our case. The agent will request help to the other agents when required to clear a path, and the agents accepting the tasks will change the behaviour to clear the path. We can say that a "migrating behaviour" mechanism is created. In other words, the receiving agent will assume the behaviour from the sender, while the sender will take on a different task, thus changing its behaviour.

The 'task delegation action' category contains three behaviours: 1) Whether to start clearing a path, 2) clearing the path, and 3) finding an agent to which the task can be delegated. A diagram of the different states and transitions of the task delegation implementation is presented in figure 4. The delegation of tasks is implemented by means of communication, see figure 5. First, the agent is in its normal mode (trying to pick up and deliver packages). Once it notices that the path towards a destination is blocked by packages it adds the task "clear package" with corresponding label in its memory. Once a task is in the agent's memory, the agent shifts to the clear path behaviour. If the agent memory contains a "clear package" task with the agent's label, the agent starts cleaning the path (picks packages up and drops them in a cell

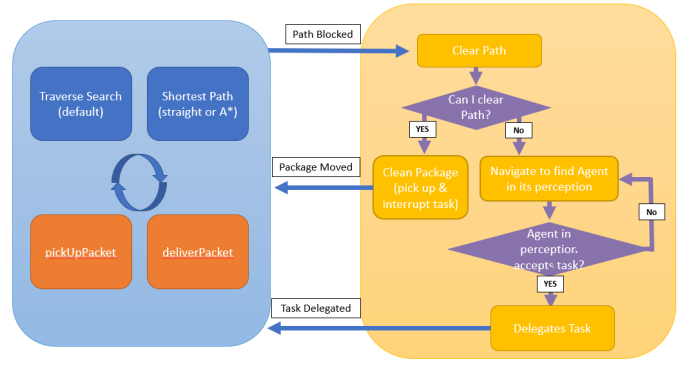


Fig. 4. Representation of the relationships between the different behaviours with 'task delegation'.

that does not obstruct the path). However, if the agent memory contains a 'clear package' task where the label of the packet is not the agent's label, the agent will traverse or wander until another agent enters its perception. Once another agent is in its perception it sends out a message asking for help. If the other agent accepts the help request (package has the label of the helping agent), the task is removed from the memory and the agent can carry on in normal mode. On the other hand, if the label of the request is not the same as that of the agent, the help request will be rejected.

VI. ANALYSIS OF RESULTS

This section discusses the performance of the solution in the environments defined in the problem requirements. Parameters such as package delivery, energy management and coordination are evaluated in this section.

A. Package Delivery

The different improvements were benchmarked against the wandering behaviour to evaluate their performance ('base-line'). The benchmark was performed in the three default

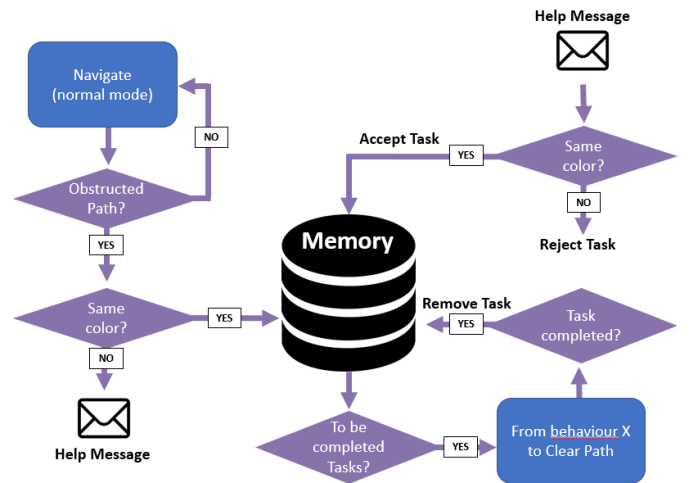


Fig. 5. Representation of the memory usage in 'task delegation'.

environments (B-1, B-2 and B-3). Each setting was executed fifteen times as the simulation results have some degree of stochasticity. As performance indicators, the average energy consumption is computed and normalised according to the baseline results. As expected, the environment size and features, the perception and the agent behaviour strongly influence the performance of the implementation.

B. Baseline

The baseline is used to measure the performance improvement of each implementation. As in the future there may be trade-offs in energy consumption, it's important to define a basic level of performance. Figure 6 illustrates the amount of energy consumed by the 'wander' behaviour to deliver all packages. In B-1, the smallest environment, the mean energy consumption of the wander behaviour is approximately 50 kilo-units. For B-2, the mean energy consumption is 84 kilo-units and for B-3, the mean energy consumption is 1048 kilo-units. These averages show only part of the picture, as the results of the 15 runs showed a significant variation. It is observed that the amount of energy consumed increases exponentially with the environment complexity. The environment complexity is mainly defined by the size, the number of packets and labels. The number of destinations plays a positive role in performance as well as the extent of the agent's perception.

In terms of the environment's size, the energy consumption will be higher in a large environment as the agent has to wander around for a longer time before it reaches an object. Despite the additional destination having a positive effect, the 68 percent increase in energy consumption that is seen for environment B-2 with respect to B-1 is not surprising, as the size of the environment and the number of packets increases. Surprisingly, environment B-3 shows an increase of the energy consumption of 21-fold with respect to B-1. It is suspected that besides the increase in size and packages to carry (60 in B-3 vs 14 in B-1), the added complexity of labels plays a key role in the overall energy consumption.

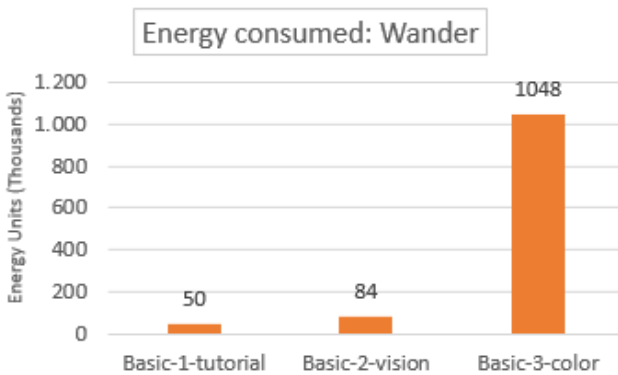


Fig. 6. Average energy consumed when running the Wander Package implementation in the default environments.

As such, the configuration of the environments is very different, and therefore it is uncertain what the influences are of individual factors. However, these quantitative comparisons are useful to understand the system dynamics of the Packet-World.

C. Improved Navigation

The energy consumption performance of the traverse search, shortest straight path, and shortest A* path with map memory behaviours are analysed in this section. When applying the 'Traverse' implementation in the B-1 environment the energy consumption decreases to 33 percent as the search becomes more efficient and the agent covers more area with less steps. Yet, while the 'Shortest Straight Path' shows an almost perfect performance the 'Shortest A* Path' actually shows a perfect performance in B-1. In both cases, the agent profits from its large perception (as large as the environment's size). This allows the agent to take the most direct and therefore, least energy consuming path for each package delivery. The small energy consumption difference occurs due to the lack of validation whether the route towards a target is blocked. More precisely, with this implementation, the energy consumption in B-1 is reduced to only 4.6 percent (Shortest Straight Path) and 4.3 percent (Shortest A* Path) of the baseline.

In environment B-2, a similar trend is observed. However, the reduced energy consumption gains not as high as in B-1, as the environment is more complex and agents have a limited perception. In B-2, both 'Traverse' and 'Shortest Straight Path' achieve similar performance (40 percent and 30 percent respectively). On the other hand, the 'Shortest A* Path' implementation again achieves an immense increase in performance as the energy consumption is reduced to 9.0 percent.

In B-3, the pattern is partly inverted. 'Traverse' performs better than 'Shortest Straight Path'. It is believed that the large size of the B-3 environment and reduced agent perception favours blind but efficient search. The mean energy consumption for 'Traverse' is reduced to 23 percent of the baseline, whereas 'Shortest Straight Path' only achieves 48 percent. As there is probably a trade-off between perception and cost for implementation (in real-life AGVs), it can be concluded that if the 'cost' of having a high perception is high, then it is better to have a more efficient exploration, whereas if having larger perceptions is cheap, then the agents could afford less sophisticated exploration methods. However, as can be observed in figure 7 the 'Shortest A* Path' still outperforms the two other search techniques. The energy consumption compared to the baseline is reduced to 2.0 percent. The use of efficient memory is the reason for this exceptional performance. Whenever an agent carries a package and has the location of the destination in memory, it can execute A* towards a destination that is not in its perspective. In this case, A* will ensure that the agent finds a path towards the cell that is closest to the destination, this action will be repeated until the destination is finally in sight.

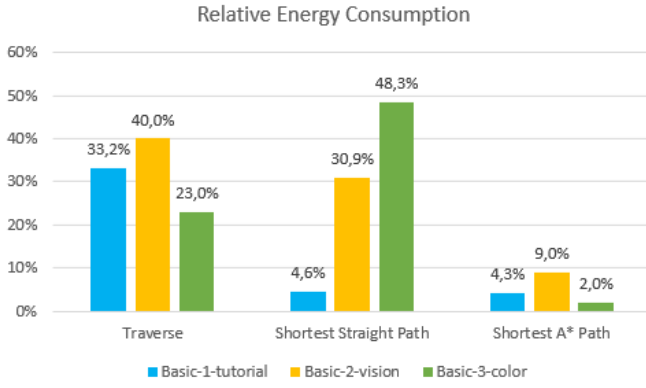


Fig. 7. Mean energy consumed in 'Traverse', 'Shortest Path' and 'Shortest A* Path' - relative to the baseline.

Finally, the combined behaviour performance is analysed and results are shown in figure 8. When combining 'Traverse' and 'Shortest Straight Path' we clearly see an energy consumption reduction as compared to both behaviours separately. However, this combination still underperforms compared to the 'Shortest A* Path' behaviour. Yet, the 'Shortest A* Path' can still slightly be improved when combined with 'Traverse'. As illustrated, the combination of the 'Traverse and the 'Shortest A* Path' achieves the highest reduction of energy consumption in all the three environments, the energy consumption is reduced to 4.3, 7.9, and 1.8 percent, respectively. The increased exploration capabilities of 'Traverse' have no additional value in B-1. This is in line with our previous conclusion regarding the trade-off between perception and movement complexity.

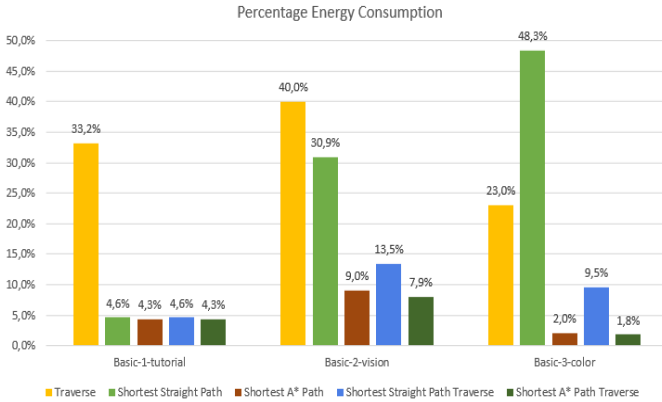


Fig. 8. Performance of all 3 behaviours for each environment - relative to the baseline.

D. Energy Management

Our implementation of energy management was able to successfully resolve all the three environments that required energy management. Several challenges were present to have a coordinated use of the stations. For example, agents need to be able to clear a packet obstructing the gradient to the station. Agents also need to wait for their turn without blocking the access to the charging point. Finally, walls must be sorted out.

These final challenges were tackled via a specific gradient that takes walls into account. In figure 9, the energy use of the implementation is solved for the three energy environments. One of the difficulties of Energy 2 is the extension of the environment: if the SoC parameters are not correctly tuned, the agents risk to run out of battery, before they can reach the station. In the Energy 3 environment, the walls and the fact that agents can block the access to the station are the main challenge.

E. Task delegation

The task delegation environment has been resolved using the implementation explained in section IV. One of the main challenges when unblocking a path through task delegation is to decide where to leave the package to avoid that the cleared path does not get blocked again. The assignment of task priority was also a challenge, but a simple approach was taken, i.e. the most important task is fixed to help another agent to clear a path. The average energy consumption in this environment is 12,997 energy-units.

F. The maze

The maze environment was also resolved by applying the simple random motion. Given the low energy performance of this approach, an improvement is implemented. The agent prioritizes any other cell that is walkable, before returning to its previous cell. This very simple improvement resulted in an approximately 3-fold performance increase (from 3,536,995 to 1,436,870). Another way to reduce the energy consumption would be to only allow for horizontal and vertical random movement. The implementation of this improvement would decrease the energy consumption to approximately 627,429 energy units.

G. Limitations of the implementations

Even though the behaviour implementations succeeded in the various environments, some limitations have been evidenced.

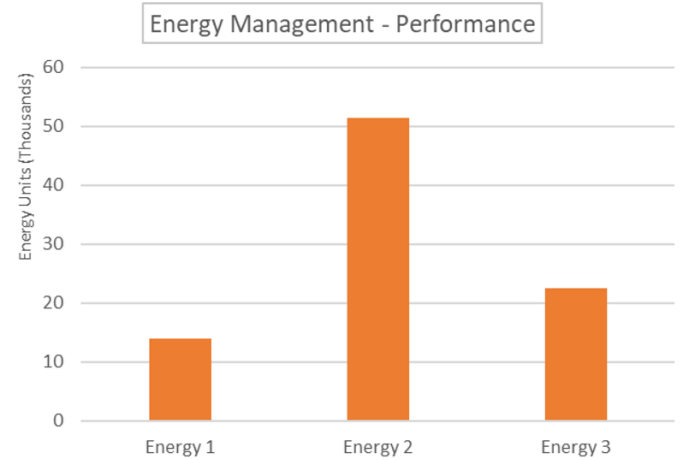


Fig. 9. Performance of energy management behaviour for the three energy environments.

The implementations were tested in custom environments that include obstacles. In some situations, the implementations lead to a 'stuck' loop, especially the 'Shortest Path' behaviour in a custom, so-called 'impossible world'. As illustrated in figure 10, the perception of the agent in this environment is as large as the size of the environment. Given this situation, the agent finds the package by using the shortest path, thereafter it immediately detects and takes the shortest path to the destination. However, the walls block this shortest path causing the agent to be stuck in an infinite loop (see fig 10 right). In other words, the agent gets stuck in an unfeasible situation and does not find another path that leads to the point of interest (POI). With this implementation, the agent will never reach a feasible route to deliver the package.

This limitation was the motivation for implementing the 'Shortest A* Path' behaviour, which successfully and efficiently solves this previously 'impossible' environment. Nonetheless, the A* implementation has two main disadvantages: 1) large computation time (exponential to perception size) and 2) falling into local optima when the target is not within the agent's perception.

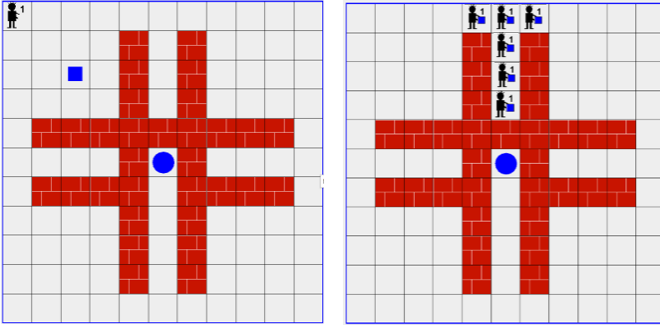


Fig. 10. Representation of the impossible world, an environment where the agent does not reach its goal.

Figure 11 illustrates an additional limitation where two agents get stuck due to the 'Shortest A* Path' behaviour. This situation occurs in the left right corner of the task delegation environment, where two agents are surrounded by walls and two paths (when cleared) to their POI exists. In this case, a loop occurs when one agent operates in 'Shortest A* Path' to pick up a packet at the right side, while the other agent tries to reach the destination at the left side using the same behaviour.

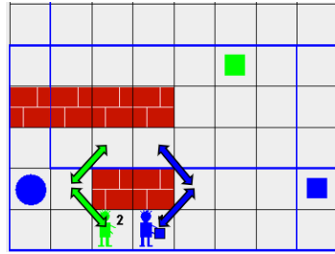


Fig. 11. Illustration of the stuck agents.

H. Key aspects determining performance

From the result discussion, it is clear that the Packet-World environment can quickly become complex and that

this complexity increases as more features are added to it. Further, the analysis shows that the energy consumption can be drastically improved by implementing simple behaviours such as 'Traverse', 'Shortest Straight Path', and 'Shortest A* Path', or even the combination of two. However, the lesson learned is that these improvements come with drawbacks in certain environments such as the impossible world. Thus, in certain environments careful analysis will be necessary when implementing specific implementations. Another solution would be to further improve our current implementations to surpass these limitations.

VII. DISCUSSION OF RELATED WORK AND APPLICATIONS

Our approach relies on communication, mapping and intelligence. The agents are exchanging information about the environment (one to many) or containing task delegation (one to one). The mapping mechanism is implemented by either explicitly memorising observed points of interests or implicitly by laid-out gradients. The intelligence is represented by the path finding algorithm embedded in the behaviour. The used architecture is fully decentralised and homogeneous: each agent follows the same behaviour pattern without creating any delusional hierarchies.

Based on the agent typology spectrum presented in [14], the proposed approach can be positioned in the multi-agent pattern map as follows:

- By **Focus**: the design mainly focuses on the behavioural aspects and the interactions between agents, which are done through communication and have a collaborative nature. The use of processing power (communication, path finding) is essential to our approach, but it is important to mention that alternatives to heavy communication exist in the literature. In [15] the authors propose a virtual environment with fields for coordination instead of messaging between agents. This has the advantage of relieving the agents from processing the communication tasks.
- By **Inspiration**: the model fits together with the nature inspired patterns where individuals delegate tasks and use gradients to guide themselves. An alternative of nature-inspired pattern could be the work presented in [16] where the authors describe how pheromone is used to coordinate the multi-agent system to scan the entire environment for packages. Considering that the pheromone prohibits the agents from entering a visited cell the agents will continue forward and only turn once no pheromone is detected, ensuring that a constant spiral of searching agents is formed around the starting point.
- By **Abstraction**: The current paper merely tries to evaluate the potential of using different behavioural concepts in solving the Packet-World problem. The behaviours are assessed in isolation with a single behavioural pattern for all agents, keeping the assessment at conceptual abstraction level. If we are to put this work in a methodology as the one described in the Tropos project [17], the progressive addition of conceptual behaviours for performance

comparison in an simplified simulated environment is characteristic to an early stage, conceptual design stage.

- By **Granularity**: The scope of the pattern deployed is kept at agent level. Although we use communication, there is no coordination that involves parts of the agent group, such that the implementations are treated at a system or subsystem level. Treating the implementations at agent level does not limit the overall system performance to be less complex. The authors of [18] studied the package-exchange robot-routing problem. In this case agents can exchange packets with each other. This application would be extremely convenient for solving the maze environment, as agents cannot jump over each other.

VIII. CONCLUSIONS

Several challenges within the package delivery context were analysed. A simple random motion behaviour was used as benchmark for more advanced behaviours (search, A*, messaging, memory). Some extra challenges, requiring energy management, coordination and/or task delegation, were tackled. It was observed that simple random motion can deal with fairly complex environments (as the maze) but more advanced behaviours are required to reach acceptable system performance. The environment complexity plays a key role in energy consumption to complete the goal. It is observed that energy consumption can grow exponentially in complex environments, particularly for very simple agent behaviours.

Using the agent's perception for navigation purposes is very efficient when the perception is large. However, when the perception is limited, smarter exploration results in higher performance gains. In particular, the A* algorithm provided a large performance increase to overcome path obstructions. Although, the biggest performance gain is achieved when agents can form a map and exchange information with each other. In real-world use cases, mapping of the environment can be a key element for AGV navigation.

Agent coordination, when charging, proved to be challenging, as both deciding when to charge the battery and the queuing of the agents needed to be solved in a correct way. First implementations had the caveat of blocking the access to the charging point. As future work, message-based charging coordination could increase system performance. In this case agents would not only decide based on the nearest station, but would take also the station's waiting time into account.

The main challenge for the task delegation was the restriction that no broadcasting was allowed, which forced the agent to search for other agents to send out the help messages to clear the path. In addition, packages blocking the path had to be moved to a cell where no other path was potentially blocked. A future possibility would be to use Field-Based Task Assignment, where tasks would emit a gradient field in a virtual environment. This emission would allow agents to be guided towards a task that needs to be executed.

REFERENCES

- [1] D. Weyns, A. Helleboogh, and T. Holvoet, *The Packet-World: A Test Bed for Investigating Situated Multi-Agent Systems*, pp. 383–408. Birkhäuser Basel, 01 1970.
- [2] M. Dornhöfer, S. Sack, J. Zenkert, and M. Fathi, “Simulation of smart factory processes applying multi-agent-systems—a knowledge management perspective,” *Journal of Manufacturing and Materials Processing*, vol. 4, no. 3, 2020.
- [3] “Robotics Enabling Fully-Integrated Logistics Lines for Supermarkets.” URL: <http://www.refills-project.eu/> (accessed Feb. 28,2021), 2018.
- [4] J. Odell, “Agent technology: What is it and why do we care?,” pp. 1–25, 2007.
- [5] D. Weyns and T. Holvoet, “Synchronous versus asynchronous collaboration in situated multi-agent systems,” 05 2003.
- [6] W. Zeng and R. L. Church, “Finding shortest paths on real road networks: the case for a*,” *International Journal of Geographical Information Science*, vol. 23, no. 4, pp. 531–543, 2009.
- [7] D. Colling, J. Oehler, and K. Furmans, “Battery charging strategies for agv systems,” *Logistics Journal : Proceedings*, 2019.
- [8] X. Zhan, L. Xu, J. Zhang, and A. Li, “Study on agvs battery charging strategy for improving utilization,” *Procedia CIRP*, vol. 81, pp. 558–563, 01 2019.
- [9] Z. Rozsa and T. Sziranyi, “Obstacle prediction for automated guided vehicles based on point clouds measured by a tilted lidar sensor,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 8, pp. 2708–2720, 2018.
- [10] S. Poslad, “Specifying protocols for multi-agent systems interaction,” *TAAAS*, vol. 2, 11 2007.
- [11] D. Weyns, *Architecture-Based Design of Multi-Agent Systems*. Springer, 2010.
- [12] T. Wolf and T. Holvoet, “Design patterns for decentralised coordination in self-organising emergent systems,” pp. 28–49, 05 2006.
- [13] T. Holvoet, D. Weyns, and P. Valckenaers, “Delegate mas patterns for large-scale distributed coordination and control applications,” *EuroPLoP*, vol. 2010, 07 2010.
- [14] J. Juziuk, D. Weyns, and T. Holvoet, “Design patterns for multi-agent systems: A systematic literature review,” in *Agent-Oriented Software Engineering*, vol. 9783642544323, pp. 77–97. Springer, 2014.
- [15] D. Weyns, T. Holvoet, K. Schelfhout, and J. Wielemans, “Decentralized control of automatic guided vehicles - applying multi-agent systems in practice,” *Object-Oriented Programming, Systems, Languages & Applications*, 2008.
- [16] O. Zedadra, H. Seridi, N. Jouandeau, and G. Fortino, “S-masa: A stigmergy based algorithm for multi-target search,” *2014 Federated Conference on Computer Science and Information Systems, FedCSIS 2014*, pp. 1477–1485, 10 2014.
- [17] H. Mouratidis and M. Weiss, “Patterns for modelling agent systems with tropes,” vol. 3914, pp. 207–223, 05 2005.
- [18] H. Ma, C. Tovey, G. Sharon, T. Kumar, and S. Koenig, “Multi-agent path finding with payload transfers and the package-exchange robot-routing problem,” 02 2016.

Communication-Based Interaction Protocols to Solve Packet-World Multi-Agent Environments

Dooms Lena*, Ravoet Yannou[†], Tysebaert Stefaan[‡] and Van Campenhout Simon[§]

Department of Computer Science, KU Leuven. Leuven, Belgium.

Email: *lena.dooms@student.kuleuven.be, [†]yannou.ravoet@student.kuleuven.be,

[‡]stefaan.tysebaert@student.kuleuven.be, [§]simon.vancampenhout@student.kuleuven.be

Abstract—Multi-Agent Systems(MAS) is a challenging field in computer science with many unrevealed fundamental issues. To explore these issues further, virtual test beds for Multilayered Multi-Agent Situated Systems like Packet-World are available. This paper aims to solve three sets of environments in Packet-World with increasing complexity. In the first set of basic environments, autonomous agents equipped with pathfinding and scouting algorithms and a limited memory succeed in efficiently delivering all packets to the corresponding destinations without inter-agent coordination or collaboration. Next, the agents are extended with communication protocols to participate in interaction sessions and broadcast information. These extensions were necessary to solve the second set of environments where agents have to maintain their battery-levels and navigate around walls. Finally, the third set of challenging environments, including a maze and color specific agents, is solved by delegating tasks between agents and employing influences (Crumbs and Flags) on the environment. In this paper we have shown that the system dynamics are influenced by the behavioral models of the individual agents and that the overall performance of the MAS considerably increases due to the interaction of the agents in interaction sessions, broadcasting and task delegation.

Index Terms—multilayered multi-agent situated system, test bed, protocol-based communication, synchronization, simultaneous actions, stigmergy, task management, interaction sessions

I. INTRODUCTION

This paper discusses the usage of the Packet-World Test Bed to solve three sets of environments with multi-agent systems. The first environments require agents to deliver packets to a destination of the corresponding color. The second set adds the need for agent to maintain their battery levels. The final set consists of a level that requires agents to collaborate to clear a path to the destination and a maze where agents with very limited vision have to deliver packets throughout the maze. Throughout the environments we focus on the direct communication between the agents rather than utilizing a predetermined gradient. Our motivation for this is that we felt direct communication is available more often in more multi-agent problems than a predetermined gradient field is.

Section II provides an introduction to the Packet-World Test Bed. Section III briefly states the challenge of each set of environments. In section IV, we outline the design, implementation and results of our solution for each set of environments. Section V discusses the systems that were used to solve the environments whilst section VI situates those systems in the related literature. Finally, section VII concludes the paper and section VIII outlines ideas for future work.

II. PACKET-WORLD

Packet-World is a virtual test bed for Multilayered Multi-Agent Situated Systems (MMASS) [1] that can be used to model and study the efficiency of cooperation between agents [2]. Situated agents are directly placed in the dynamic environment they operate in. They are autonomous agents with a specific behaviour pattern according to their own individual goals. They act in the present and only have access to local information of the environment. Interactions between agents are key in situated MAS as it determines the intelligence of the system more than the individual capabilities of the agents.

A. Characteristics

The Packet-World is a 2D-grid of packages and destinations. Agents need to move packages to their correct destination taking advantage of information provided by their surroundings and by other agents. Packet-World implements a model of action selection based on Ferber's theory of influences and reactions, where the state of the world is altered as a result of multiple influences. This altered state changes an agent's perception of the world, which will influence their future actions. The interplay of influences and environmental changes allows multiple activities at the same time by different agents.

The Multilayered representation of the environment allows agents to use Selective Perception, sensing the environment at a specific layer relevant to their task. Each layer contains a specific type of object with relations to objects of both its own and other layers.

Two forms of inter-agent interaction are supported in Packet-World: direct Protocol-Based Communication for knowledge sharing and setting up collaboration as well as indirect Stigmergic coordination.

The layered architecture and the separation of concerns in the design patterns make the system easily *modifiable* and *extendable* [3].

B. Real-world Applications

A real-world example of a MAS that relates to the Packet World is found within the biological world, where biological cells can be modelled as a MAS [4]. Although the complexity of the cellular dynamics exceeds the Package World test bed, the main principles are very similar. Like agents within the Packet-World, a cell is constructed as a component aware solely of its local environment. An interconnected

structure of cells representing tissue acts as an environmental context. Furthermore, cells are modelled with a specific perception system, a decision making system, a protocol-based communication system and advanced collaboration systems with multiple individual and group goals changing over time. The proposed MAS is designed with multiple modules: a physical model, behavioural model and interaction model are defined resembling the layered architecture design of the Packet-World.

Another problem that can be mapped to the Packed world is a ground search [5]. The area is divided in grids and the search party starts from assigned areas without full knowledge of the domain. Just like agents in the Packet-World can be assigned different roles, volunteers and police are different roles in ground search. Similar to the pheromones in the Packet-World, volunteers can mark the ground with tape when they notice something suspicious, motivating police to visit the area. If a team of the search party has covered one of the grids, they can notify the other teams of this through their communication channel (e.g. walkie-talkies). In both the Packet-World and a ground search, the shared goal is accomplished by the interaction of entities working towards their own goal, i.e. clearing a sector.

III. PROBLEM STATEMENTS

We separate the task of creating a multi-agent system into three subtasks. Firstly, we create autonomous agents that solve a task without coordination or collaboration in the simplest set of environments. Secondly, we increase the complexity of the environment by adding a need for the agents to maintain their battery-levels and navigate around walls. Finally, we further increase the complexity by limiting agents to Packets of a specific color and navigating through more complicated environments.

IV. SOLUTIONS

A. Autonomous Agents

1) *Design:* We create autonomous agents by iteratively improving upon simpler agent versions. The first version has a single behaviour that handles all possible situations. In the second version, we divide the behaviour of the agent into smaller Behaviours between which we switch using BehaviourChanges. BehaviourChanges model the conditions where an agent is allowed to switch from a source to a target Behaviour. This method of combining small behaviours allows for easy modifications in specific parts of the agent making it easier to optimize the agent. In the third version, the agent is improved by optimizing the *pathfinding* used to navigate around obstacles as well the *scouting* algorithm used to find new Packets or Destinations. The optimized agent is also able to *memorize seen packets and destinations*.

2) Implementation:

a) *Version 1:* “Fig. 1” shows the decision logic of the first version of the agent. The target variable is a set of coordinates saved in memory. It contains the coordinates of the packet or destination itself, as well as the coordinates of an open cell next to it. In the diagram, yes/no-questions are modelled as diamond shapes. Actions that result in the ending of a cycle for that agent are marked as grey dashed rectangles. The ‘target still valid?’ question is answered yes iff the cell at the target coordinate contains a packet or destination. When we save a target, we pick one random packet or destination out of the set of all valid packets or destinations. Clearly, there is a lot of room for improvement, but even with a simple agent we quickly lose sight of edge cases. Thus, the second version divides the decision tree into multiple smaller Behaviours linked by BehaviourChanges.

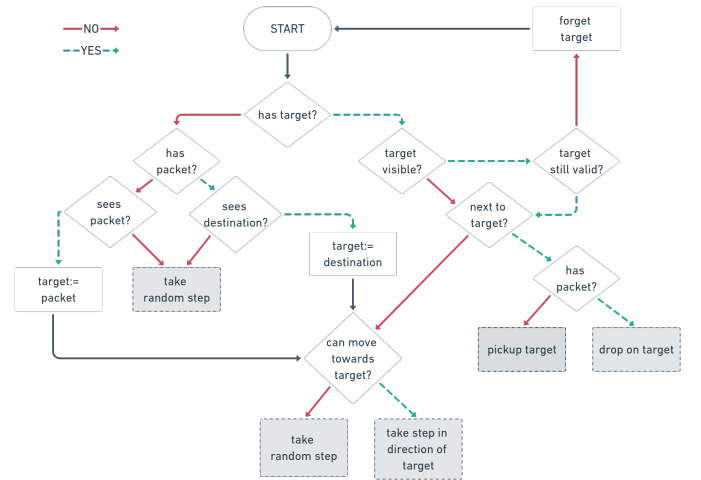


Fig. 1. Decision Tree of the autonomous agent Version 1

b) *Version 2:* “Fig. 2” shows the changes of Behaviour according to BehaviourChanges. Version 2 of the autonomous agents has 5 behaviours:

- **Wander:** The agent wanders around randomly looking for a packet or a destination with the same color as the packet it is carrying;
- **MoveToPacket/MoveToDestination:** The agent moves towards a packet or destination it has seen or remembers;
- **PickupPacket/DropPacket:** The agent is on a neighbouring tile of a packet/destination and picks up/drops the corresponding packet.

By default, the agent starts with the Wander behaviour. BehaviourChanges define how to switch between the Behaviours:

- **NeedsAndNotSeesPacket/NeedsAndNotSeesDestination:** The agent forgets or wanders out of sight of the packet/destination and switches back to looking for a packet/destination location;
- **NeedsAndSeesPacket/NeedsAndSeesDestination:** The agent sees a packet/destination and switches from wandering by going into a specific direction;

- **NeedsAndNextToPacket/NeedsAndNextToDestination:**
The agent is situated on a neighbouring tile of a packet/destination and switches to picking up/dropping the packet.

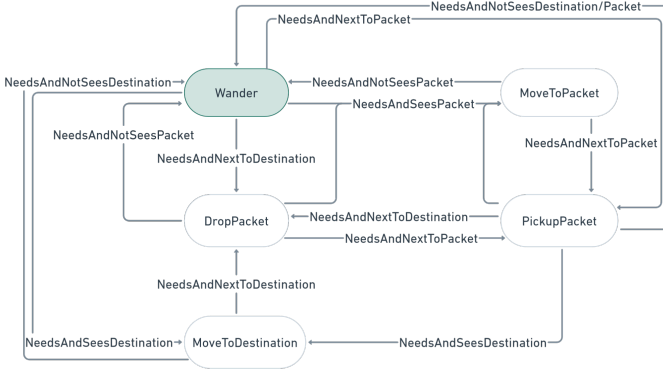


Fig. 2. Flowchart of the behaviours of the autonomous agent Version 2

c) *Version 3*: The third version enhances the agents with optimization algorithms to increase the individual performance. The following optimizations are implemented:

- **Scouting:** Instead of wandering with random steps, the agent scans the Packet-World systematically. This helps speed up the rate at which new packets and destinations are found and is used in the *version 3* scouting behaviour. To realize this behaviour, the agent is extended with a horizontal and a vertical direction and the ability to switch between these.
- **Pathfinding:** to optimize the route of the agent towards a target coordinate, the optimal route is calculated with the A*-algorithm using the Chebyshev distance as a heuristic.
- **Memory Management:** Since the memory of an agent is limited to 10 slots, we define a value function that values *ItemRepresentations*. In version 3, packets that are far away from the agent have a higher value. Destinations have the maximum value since these are most important to remember.

Although the individual agents can be optimized further, it is of importance to stay true to the concept of commitment: the balance between reactivity and goal directness will define the stability of the agent behaviour [6]. Furthermore, adding complex planning behaviour that takes long sequences of actions into account are not always valuable, as the system dynamics change rapidly - making the long action sequences unstable.

3) *Results*: “Tab. I” shows basic measurements of the different agent versions on the “Basic3” environment averaged over 100 playthroughs as well as the effects of enabling each optimization of the third agent version separately. An increase in cycles in the second version compared to the first version agents is due to the fact that, unlike the first version, the second version forgets the current target when it loses vision of it. The *pathfinding* optimization on its own provides little

benefit to the agent since there is a high probability that the agent passes another packet on its way to the target. Thus, the agent usually stops at the closest packet, in which case the a*-pathfinding algorithm can be reduced to simply taking a step in the direction of the target. However, pathfinding is very helpful in situations where the agent has to navigate around a destination to reach a packet. In such a case, we improve from randomly taking a step until having passed the destination to calculating a path around the destination and following that path. *Scouting* provides a big benefit to the agent as it ensures that the last remaining packets are found within a reasonable number of cycles. Adding *memory* to the agent has a similar effect, since it is highly likely that by the end of the level the agent will have passed every packet at least once. This makes it likely that the agent knows the location of the last packets.

“Fig. 3” shows more advanced insights where we see that memorization of packets and optimized scouting behaviour ensure that packets and destinations are found quickly regardless of their density in the environment. Whilst the more random oriented approach to finding packets and destinations from version 1 and 2 suffices to solve the environment, it can lead to a high variance in the number of cycles needed to deliver a packet or reach a destination as can be seen in the bottom graphs. The bottom plots are smoothed out to improve readability meaning negative values can occur due to large differences between values near each other.

TABLE I
BASIC MEASUREMENTS ON THE “BASIC3” ENVIRONMENT

Agent	Energy Consumed	Cycles
Version1	525.610 (100%)	7619 (100%)
Version2	580.796 (110%)	8732 (115%)
Version3 (OnlyScouting)	84.606 (16.1%)	1278 (16.8%)
Version3 (OnlyPathfinding)	512.437 (97.5%)	7670 (101%)
Version3 (OnlyMemory)	83.076 (15.8%)	1367 (17.9%)
Version3	22.300 (4.24%)	396 (5.20%)

B. Handling Energy

1) *Design*: To implement energy handling in our autonomous agents we add a set of new Behaviours that get called whenever the agent is running low on energy. These consist of finding an EnergyStation, moving towards it and staying there until they are sufficiently charged. Agents are extended to have basic communication, where they can broadcast the location of EnergyStations and share locations of other items with agents within their perception. The third environment provided to test our energy handling agents increased the complexity by making the EnergyStations only accessible by a narrow path. To prevent agents from blocking each other, we added *InteractionSessions* where multiple agents discuss through a communication protocol who gets priority to charge their battery, and who waits in line at a distance from the EnergyStation.

2) *Implementation*: To manage energy maintenance the agent should be able to switch back-and-forth between

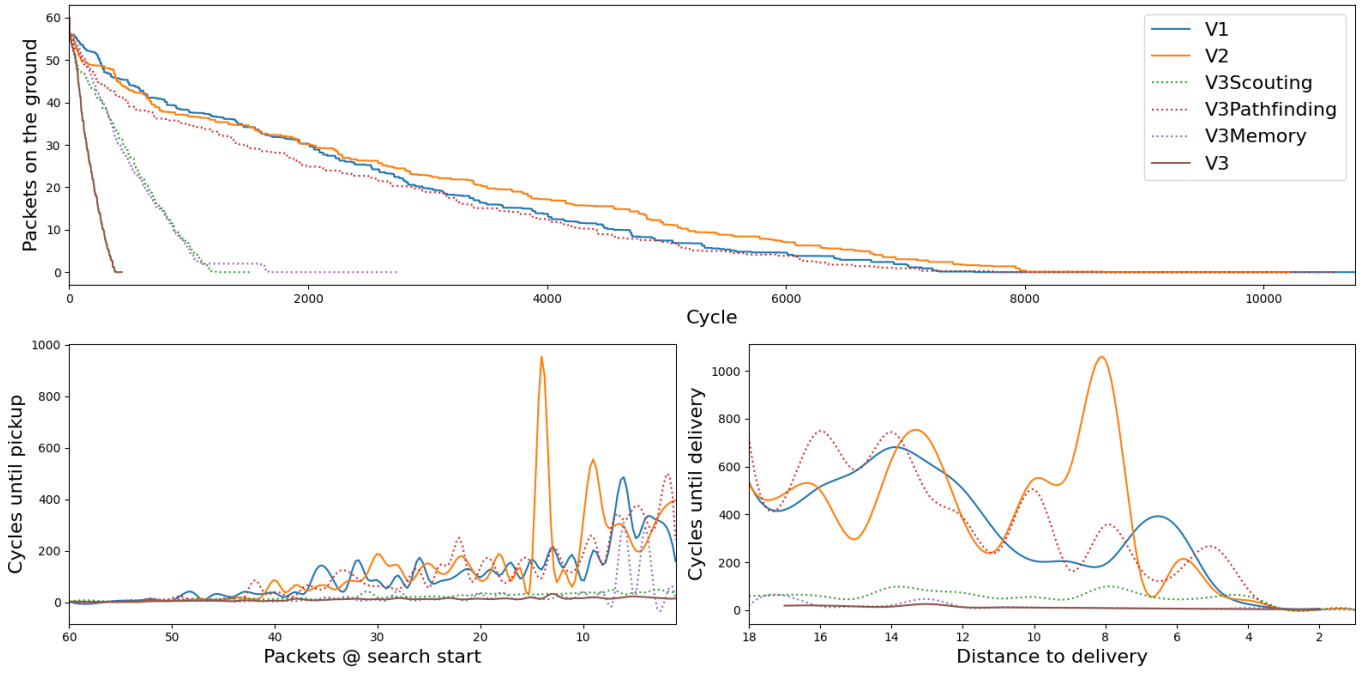


Fig. 3. Advanced Insights on autonomous agents on the basic-3-color environment. The results are averaged over 10 playthroughs

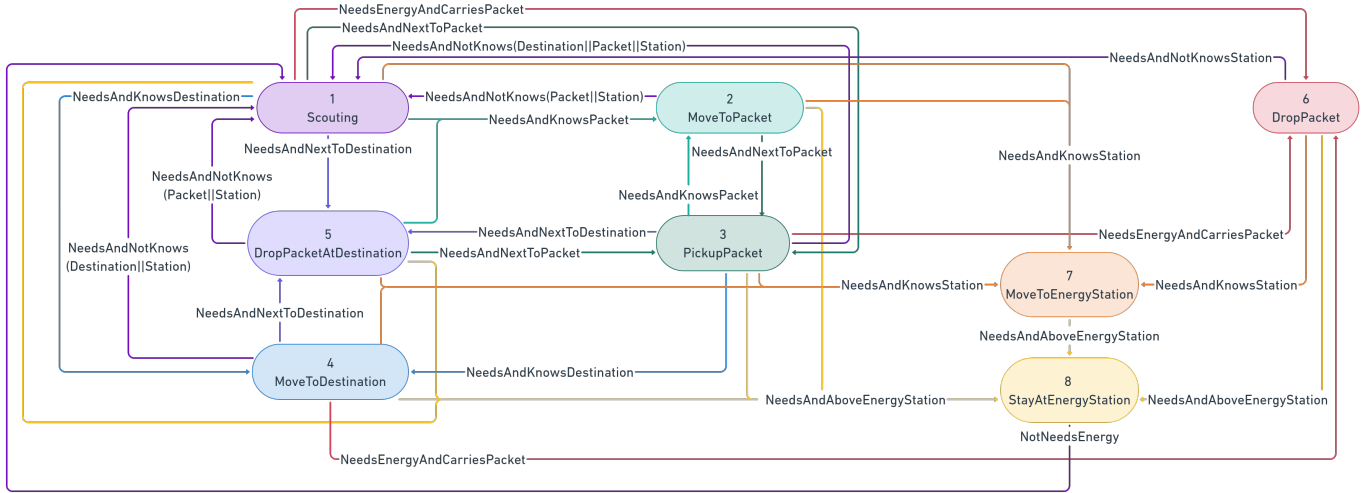


Fig. 4. Energized behaviours en behaviour changes

work-related and energy-maintenance-related behaviours. The implemented Behaviours and BehaviourChanges between them are shown in “Fig. 4”.

Specifically the behaviours are extended with three new behaviours that ensure agents do not run out of energy:

- DropPacket: The agent drops any packet he is currently holding;
- MoveToEnergyStation: The agent moves to a known energy station;
- StayAtEnergyStation: The agent waits at the energy station until fully charged.

Five new behaviour changes are introduced to switch between working and maintenance behaviours:

- NotNeedsEnergy: The agent is fully charged and can go back to work related behaviours;
- NeedsEnergyAndCarriesPacket: The agent needs to charge and is carrying a packet;
- NeedsAndNotKnowsStation: The agent needs energy but doesn’t know an energy station;
- NeedsAndKnowsStation: The agent needs energy and knows the location of at least one energy station;
- NeedsAndAboveEnergyStation: The agent needs energy and is on the correct location to charge.

The implementation of direct communication between agents is facilitated through *InteractionSessions*. A session, constructed during the communication phase, consists of an initiator and a group of participants. An optional resource subject can be added if the discussion requires this. It has a corresponding view - a radius to limit the participant group in function of the initiator or the targeted resource. During the communication phase, the active agent checks if sessions need to be initiated in function of certain boundary conditions - in this example when multiple agents are travelling to the same *EnergyStation*. Multiple types of sessions are possible, such as a Q&A Session, Negotiation Sessions - similar as seen in KQML performative categories [7]. In the case of the *EnergyStation* availability, a Q&A session is initiated. The initiator starts by querying all participants for their battery level and current behaviour. The participants reply to the outstanding *QueryMails* of the particular session. The replies are then processed by the initiator who decides which agent receives priority to move based on the battery level and behaviour of each of the participants. The other agents will have to wait. The initiator sends an *InfoMail* to update all agents on the decision, and the session is terminated. The information of this decision is then propagated to the act phase. The switch to the act phase is only allowed after all sessions have been terminated. Each cycle, sessions are re-iterated if required.

3) *Results*: The new energy maintenance behaviours ensure the agents are going into maintenance mode regularly and in time. The *AvoidCollisionSessions* lead to a setting where the *EnergyStation* prioritization is updated constantly, resulting in waiting lines at frequently visited stations. Especially in the "Energy-3" level, the lines are necessary to avoid agents blocking each other inside the walled *Energy Stations*. Averaged over 100 playthroughs, we solve the "Energy-3" level in 1144 cycles, consuming 24.319 energy. Because agents are modelled to always go to the nearest *EnergyStation*, this can cause long waiting queues at one *EnergyStation* while another one sits idle. We see this in the "Energy-2" level, where there is an *EnergyStation* located in the center and a second one in the bottom right corner. In most cases, the centered one will be the closest *EnergyStation* for an agent. An agent could, when arriving at an *EnergyStation*, decide whether it would consume less energy to wait at this station or move to another one. However, since the agent doesn't have global knowledge, it would be complex to make an accurate estimate about the energy needed to move to another station and start charging.

C. Collaboration

To test collaboration between multiple agents, we solve two new environments. The first environment consists of agents that can only carry packages of a specific color. However, destinations are blocked by packets of other colors.

The second environment consists of a maze with a few open space and multiple narrow corridors. Theoretically, the maze is solvable without any form of collaboration between the agents.

However, since agents cannot pass each other on the same tile, the agents would be blocking each other more often than not, making the use of multiple agents counterproductive.

1) *Design*: To solve the first environment, we created a system that allows agents to recognize a destination that is blocked by packets and create a Task related to unblocking that destination. If the task can be solved by the agent, it will perform the task itself. If not, the agent will have to look for another agent and propose this task to the other agent.

To solve the second environment, we implemented two separate systems. A first system uses *Flags* to mark maze sections that lead to a dead end. This ensures that agents do not enter sections of the maze that have been scouted and are known to not contain any more packets or destinations. The second system uses *Crumbs* to mark when an agent enters a section of the maze. This lets the other agents know that if they enter that section, they are likely to block another agent and should thus avoid the section if possible.

2) *Implementation*: To delegate task between the agents, we gave each agent a *TaskInterface* component. This component keeps track of the tasks the agent has to complete himself as well as tasks the agent has to pass to other agents. Whenever an agent has a task that needs to be completed or passed to another agent, the task gets priority. In the "Task-delegation" environment, the task consists of unblocking the path towards a destination. The system we implemented consists of recognizing the color of the first packet that blocks a destination, finding an agent of the appropriate color and having that agent move the packet to a designated area in the middle of the environment. Having a designated area to collect blocking packets also has the benefit that agents regularly see each other and tasks are quickly passed to the correct agent. To pass a task, we created a simplified version of the *DynCNET* discussed in [8] using the *SessionInterface* discussed in the previous section. The initiator is an agent that sees potential participants and has a task that needs to be passed. It passes a message to the participants asking them if they are able to solve the task. The participants respond whether or not they are able to solve the task and the initiator then picks one of the positive answers as the agent responsible for the task. All participants are then informed whether they are responsible for the task and the session is closed. The flowchart for the task-delegation agent is visible in "Fig. 5".

The "Maze" environment requires the agent to navigate a maze with a very limited range of vision. To limit the search space we created a system to mark dead-ends. When an agent arrives in a dead-end, it starts to retreat along the path until it reaches a split. It then marks its previous cell with a flag, informing all other agents who reach this split that they should not bother following the flagged path. This reduces the search space enough to solve the maze. To prevent agents from bumping into each other too often, we added a system to mark how many agents are currently in a section of the maze.

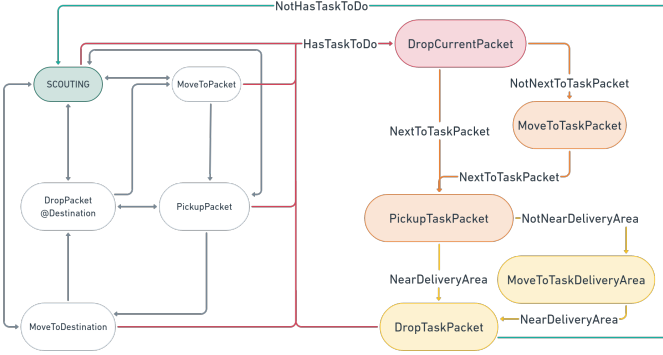


Fig. 5. Flowchart of agents that give priority to tasks

Every time an agent leaves an open space to enter a corridor, it places a crumb on the entrance of that corridor. Whenever an agent leaves a corridor to enter an open space, it picks up the crumb. If an agent does not carry a packet, it is not allowed to enter maze sections that are marked by crumbs. If it does carry a packet, it might have to reach a destination inside that maze section and is thus allowed to enter the section. Finally, to explore open spaces more efficiently, we adapted the scouting function to step in a straight line until a wall is hit. The agents then follow the wall to the nearest open corridor entrance.

3) *Results*: “Fig. 6” shows the number of packets on the ground at each cycle averaged over 100 playthroughs in the “Task-delegation” level. We see that in the initial cycles, the agents are focused on bringing the packets blocking destination to a central area of the environment. Once the destinations are cleared, the packets are steadily delivered to the appropriate destination. On average, we solve the “Task-delegation” level in 206 cycles consuming 8088 energy. The maze is solved in 3010 cycles when only using Flags, and in 2805 cycles when also using Crumbs (6.81% improvement).

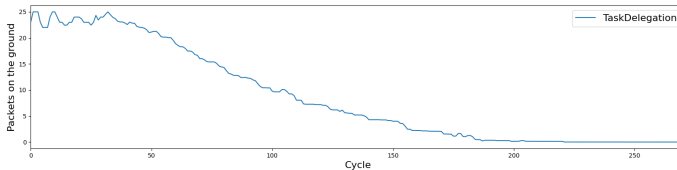


Fig. 6. Packets on the ground at each cycle

V. DISCUSSION

A. Agent Behaviours and System Dynamics

The implementation of the behavioural model for agents in the Packet-World displays some interesting insights. Simple behavioral rules can create complex system dynamics, resulting in emergent behaviour such as waiting lines and work synchronisation. It turns out emergence is a surprising phenomenon in complex adaptive systems and a key element in understanding self-organisation in MAS [9]. In the

following sections, we discuss some dynamical features within the Packet-World MAS.

1) *Memory Management*: The Memory Management functionality in agents is based on a value function approach. In each cycle, an agent will update his memory and retain only the 10 most valuable items. The value function can be tuned to match the environmental challenges. For example, in large levels with packets located at the border, it is more important to remember far away packets than packets close to destinations. This will prevent new and lengthy scouting sessions, when the central packets have been cleared. In levels with scarce EnergyStations and Destinations, the value function should favour these representations. The choice of the value function will determine the agents behaviour and system dynamics: by tuning the value function to memorize all packets and destinations, the agents will behave exploitative with maximal knowledge. By tuning the value function towards maintenance related representations, the agents will lean more to exploratory behaviour with more scouting. This is a trade-off that needs to be made in function of the available memory and EnergyStations, level size and amount of Packets and Destinations.

2) *Energy Management*: The Energy Management functionality in agents is based on the knowledge of EnergyStation locations, as well as a battery threshold, triggering recharging behaviour. Again, the choice of the threshold function will influence the agent behaviour and overall system dynamics. This can be seen in level 2 of the Energy Levels: a correct choice of battery charging threshold will optimally synchronize the agents to perform work and recharge. A too high threshold leads to long waiting lines and inefficient level clearance, while a too low threshold leads to agents running out of energy and failing clearance of the level. In the optimal situation, the waiting lines are just long enough to ensure the minimal amount of cycles and energy to clear a level. In fact, we can see that there are actually too many agents in combination with the centrally located EnergyStation: there will be scenario's where there is constantly minimally 1 agent waiting. Removing 1 agent or letting 1 agent go flat does not influence the overall performance in such cases. When tuning correctly, emergence of waiting lines and work synchronisation is visible. Another aspect in “Energy-2” is the high chance of an agent going out of battery due to the level size and scarce Energy Stations. However, due to the communication protocol, it is guaranteed that the agent with the lowest battery level can charge, minimizing the chance of going flat. In fact, in our solution this does not happen often - and is completely resolved when a battery safety factor of 6 is chosen. Of course, this influences the performance, so this is a trade-off to be evaluated.

3) *Communication Management*: The Communication Management functionality in agents is based on interactive sessions. An agent can initiate a session in a particular situation, where other agents within the view radius of the session are invited to participate. In the "Energy-3" level, an AvoidCollisionSession is introduced for EnergyStations, to ensure no blockage is created at the walled Energy Stations. The session follows a particular protocol to process all required information. From that information, the initiating agent processes a decision based on a rule set. The rule set decides which agent can move along, and whether an agent has to wait. Within the rule set there is room for optimization, as the minimal waiting distance can be chosen close or far away from the Energy Station. An optimal situation is found when the waiting distance is chosen as 4 - in combination with a sufficient time to let the charging agent walk away from the walled environment. This results not only in a "gentleman-style" waiting line, but also synchronises the work rhythm.

4) *Task Management*: The Task Management functionality uses the QA protocol to delegate tasks to other agents. In the case of the "Task-Delegation" level, this is used to tell agents to clear the path to certain destinations. By moving a Packet to the center of the level, we can observe that the task delegation goes very effectively - by coming together in the center of the level not only Packets are removed from the Destination blocking, but agents get in range of each other to communicate regularly.

5) *Signaling Management*: A special challenge is found in the "Maze" level, where the perception of agents is severely limited, and the environment is constructed as a maze. To resolve this level, agents have to rely on signaling techniques besides direct communication, memory and task management - as these techniques add little value in clearing the level. The choice of signaling techniques influences the agent behaviour. By using flags, the search space of the maze is reduced by marking dead-ends. This reduction leads to a better search for packets and destinations, but also leads to potential collisions and deadlocks when 2 agents meet in the same pathway. Crumbs are therefore used to mark the current active pathway of an agent - leaving a crumb at the entrance of each maze section an agent chooses. This signal warns other agents to leave this path for the time being - as it is already being explored by another agent. Lastly, when a successful path is discovered, pheromones can be used to signal packets - destination paths. This leads to structured behaviour, where agents can clear rooms of packets in an optimal way. However, due to time constraints, the pheromones were not implemented in our solution.

B. Performance Assessment

We have shown that the system dynamics are influenced by the behavioral models of the individual agents. As the models consist of value functions, heuristics and other parametric

logic, we can evaluate the influence of these parameters on the overall level performance. As a key performance indicator, we define the Performance Index (PI) as the number of cycles multiplied by the energy consumed. This function takes into account the consumed energy, as well as the time aspect defined by the cycles. For readability, the PI is scaled per level with a fixed constant value per level.

In the customized "MemoryEvaluation" level shown in "Fig. 7", we evaluate the value function from the memory management related to the PI. As tuneable parameter, the cost for remembering a packet is varied between equal cost and cost in function of the distance to the agent. The results of running 10 playthroughs for each scenario is shown in "Tab. II".

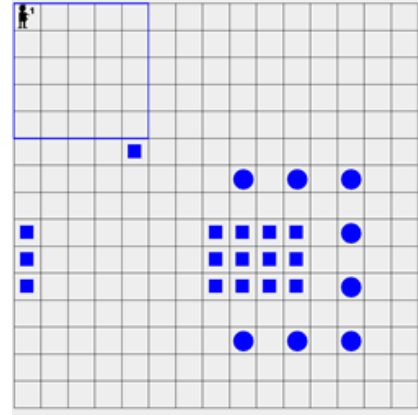


Fig. 7. Memory Evaluation Level

TABLE II
PERFORMANCE EVALUATION - MEMORY VALUE FUNCTION TUNING

Value Function	Packets Delivered	PI
Constant Packet Cost	100%	1.00
Variable Packet Cost	100%	0.78

We see that the choice of value function influences the performance: an optimal scenario is found for a value function favouring far away packets for levels with packet groups at the borders of the level.

In the "Energy-3" level, we evaluate the battery threshold from the EnergyHandler related to the PI. The threshold is used in a heuristic function, returning the maximum value of either a constant (when no EnergyStation is known), or a value calculated from the Chebyshev distance to the closest station multiplied with a safety factor. This safety factor can be tuned for performance evaluation, constructed from the result from 5 batches per safety factor. An optimum is found at a safety factor of 3,7. Any number lower will worsen the performance due to agents failing to charge their battery in time. Any higher factor will create long waiting periods resulting in inactivity. "Fig. 8" illustrates the tuned optimum.

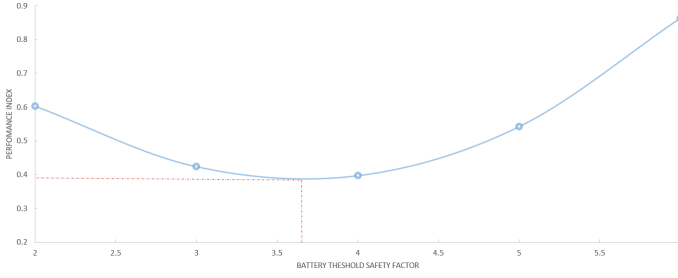


Fig. 8. Performance evaluation - Energy Safety Factor

Finally, since the project consisted of more environments than the ones discussed in the previous sections, “Tab. III” shows the results of the best agent versions on each of their corresponding environments.

TABLE III
RESULTS FOR ALL ENVIRONMENTS IN THE PROJECT

Environment	Agent Version	Cycles	Energy
Basic1	Version3	164	2.163
Basic2	Version3	239	6.306
Basic3	Version3	396	22.300
Energy1	EnergyHandling	582	15.630
Energy2	EnergyHandling	805	57.205
Energy3	EnergyHandling	1144	24.319
TaskDelegation	TaskDelegation	206	8.088
Maze	FlagsAndCrumbs	2805	161.408

C. MAS Architecture

When designing a MAS, a well thought architecture is of the essence to ensure an interpretative environment. Not only should a layered approach be favoured in terms of perception, influences and behaviours, but a robust middleware layer should be designed to serve the agent-agent and agent-environment interactions optimally. To achieve this goal, a middleware layer setup was designed providing the necessary interfaces for achieving Memory Management, Energy Management, Communication Management and Task Management. Each required functionality has its own dedicated interface, connecting Middleware objects such as InteractionSessions or Tasks with the Agent Perception, MailBuffer and Memory. The Middleware Interfaces are also synchronized in some cases, to ensure a proper time sequence is respected - as is required in certain communication protocols. A visual representation of the Middleware architecture is found in “Fig. 9”.

Lastly, the behavioral architecture is designed according to two different methods: for the Basic levels, a decision tree pattern where each cycle runs from the root node down the tree was used to find the action needed in that state. In the other levels, a finite state machine (FSM) approach is used where, starting from a start state, the state of the agent, modelled as a Behaviour, is maintained until a transition-condition, modelled by a BehaviourChange, is true. This

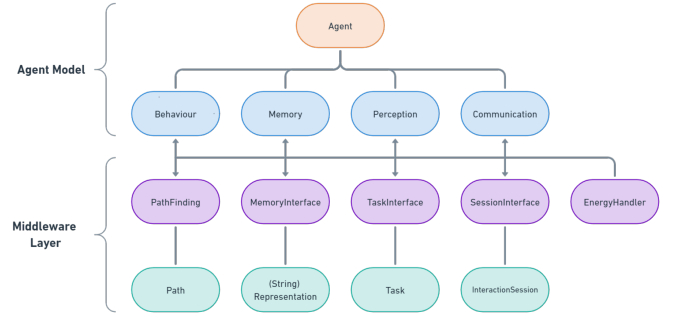


Fig. 9. Middleware Architecture

is advantageous as certain behaviour combinations can be expressed with less nodes. However, the FSM approach leads to harder to identify edge cases that can cause behaviour loops. These edge case often require additional attention during the design phase.

VI. RELATED WORK

Multiple design patterns for multiple agent systems have been proposed [10] in different projects, where several important clusters have been identified. Our approach can be mapped onto the Communication/Sessions/State cluster. An important insight in our work is indeed that a MMAS is driven by local information flows between agents and the environment, followed by learning or processing of this information for future decision making and behaviour choices. In this section we focus on how information flow techniques are designed by others.

A. Signaling Techniques

Multiple signaling techniques are available in the Packet-World test bed. Besides flags, crumbs, pheromones and gradient methods, other techniques are existing to pass on information through the environment to an agent. One particular example is in the field of infectious disease modeling, where MAS are used to simulate vector born disease transmission [11]. Here, the transfer of parasitic and viral objects are used as environmental mediated information flows.

B. Communication Protocols

For agent communication, we rely on interactive sessions to ensure proper information exchange. We have established a specific QA protocol, providing a framework for several potential sessions - such as the AvoidCollisionSession and TaskSession. Other communication protocols have been designed to provide frameworks for more complex social behaviour. The DynCNET protocol [8] - based on the contract net protocol - provides a framework for negotiating communication context. The framework gives room for task assignment based on negotiation and voting. It is dynamic, meaning task assignments are checked each cycle to ensure the best agent-task match is selected. Another interesting framework is found in competitive interactions, where game theory can be used to

achieve negotiation protocols within a MAS [12]. By using a two-stage negotiation strategy based on the Stackelberg equilibrium - where a leader moves first and knows the other agents observe his actions - win-win situations between agents are established. Similar to the protocol-based communication discussed in [13] the information in the messages sent between agents is encoded before being communicated to the receiving agent who then decodes it.

C. Processing and Learning

The information that is exchanged due to either environment mediated techniques or dynamic communication, is of value for an agent. An agent has several options of what to do with the information: it can process the data for direct decision making, or use the data in a broader learning context. The process and learning capabilities can be supported through Markov Decision Processes, which can further support Reinforcement Learning (RL) procedures [14]. In fact, much work has been done to integrate RL into MAS. It has been shown that RL can be used in combination with experience/information sharing, teaching and imitation in order to speed up learning convergence [15]. Both cooperative and coordinated goals can be used as objectives with this method.

VII. CONCLUSIONS

MAS is a broad and challenging field in Computer Science, where aspects of Artificial Intelligence, Software Architecture, Complex Dynamical Systems and Social Sciences come together. They are particularly useful in contexts where it is impossible or too costly to have a global shared knowledge, and when adaptability due to uncertain situations is needed. In this paper, we explored the architecture of the Packet-World test bed, to understand the necessity of a layered and structured architecture. We then build new functionalities such as Memory, Energy and Task Management within this architecture. To cover information exchange between agents, a communication protocol was designed, as well as the usage of environment mediated signaling. With these built-in functionalities, we were able to resolve all proposed levels as described in the Project Work Documentation in an efficient way. By resolving the levels, a deep understanding was created on the interaction between the behavioural models of the individual agents and the resulting dynamics within the levels. Important to note is the fact that gradient fields are not used in this research as it would not add value to the flow of useful information which is already preset due to the implementation of the interaction sessions. In addition, the use of gradients fields would solve some of the complex search problems in advance for each environment and we therefore chose not to make use of gradient fields. We conclude that MAS are expected to play a significant role in future technologies, such as Full Self Driving (FSD) car interaction and Cell Biology - exciting fields where inter-agent interactions are responsible for the overall dynamics.

VIII. FUTURE WORK

Although all levels were resolved with the functionalities designed by our group, several questions remain open. First of all, we did not explore the gradient based Task Delegation approach. An implementation of such a framework would expand the flexibility and communicability of the agents, resulting in better overall performance. Another aspect that deserves future attention is the introduction of Artificial Intelligence into the Packet-World: by expanding the Perception System with Pattern Recognition, the agents will be able to learn more on their surroundings, and create behaviours on recognizing certain relevant patterns. Finally, as addressed in the Related Work section, both the introduction of Reinforcement Learning and Game Theory seem very attractive to come closer too real-life behaviour.

REFERENCES

- [1] D. Weyns, A. Helleboogh, and T. Holvoet, "The Packet-World: A Test Bed for Investigating Situated Multi-Agent Systems," in *Software Agent-Based Applications, Platforms and Development Kits*, pp. 383–408, 2005.
- [2] A. A. Abusnaina and Chan Huah Yong, "Cooperative Multi-Agent System for solving Packet World Problem in Grid," 2009 IEEE 9th Malaysia International Conference on Communications (MICC), pp. 754–758, 2009.
- [3] D. Weyns, "Architecture-Based Design of Multi-Agent Systems," Springer Berlin Heidelberg, 2010.
- [4] E. Santos, D. Guo, E. Santos Jr., W. Onesty, "A Multi-Agent System Environment for Modelling Cell and Tissue Biology," *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications*, 2004.
- [5] Missing Persons Advocacy Network, "Ground Search," 2021. Retrieved from <http://www.missingpersonsguide.com/ground-searches/>.
- [6] Anand S. Rao, Michael P. Georgeff, "BDI Agents: From Theory to Practice," *Proceedings of the first International Conference on Multi Agent Systems*, 1995.
- [7] M. N. Huhns, L. M. Stephens, "Multi Agent Systems and Societies of Agents," *Multiagent systems: a modern approach to distributed artificial intelligence*, 1998.
- [8] W. De Roover, N. Boucké, D. Weyns, T. Holvoet, "DynCNET: a negotiation and coordination protocol for dynamic task assignment," *CW Reports*, 2009.
- [9] H. Stolk, K. Gates, J. Hanan, "Discovery of Emergent Natural Laws by Hierarchical Multi-Agent Systems," *IAT '03: Proceedings of the IEEE/WIC International Conference on Intelligent Agent Technology*, 2003.
- [10] J. Growin, D. Weyns, T. Holvoet, "Design Patterns for Multi-agent Systems: A Systematic Literature Review," *Agent-Oriented Software Engineering: Reflections on Architectures, Methodologies, Languages, and Frameworks*, 2014.
- [11] Benjamin Roche, Jean-François Guégan, François Bousquet, "Multi-agent systems in epidemiology: a first step for computational biology in the study of vector-borne disease transmission," *BMC Bioinformatics*, 2008.
- [12] Fang Yu, Toshiya Kaihara, Nobutada Fujii, "A Multi-agent Based Negotiation for Supply Chain Network Using Game Theory," *IFIP International Conference on Advances in Production Management Systems*, 2011.
- [13] D. Weyns, E. Steegmans and T. Holvoet, "Protocol-based communication for situated multi-agent systems," *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*, pp. 118–125, 2004.
- [14] Craig Boutilier, Planning, "Learning and Coordination in Multiagent Decision Processes," *Theoretical Aspects of Rationality and Knowledge*, 1996.
- [15] L. Bus, oniu, R. Babuska, and B. De Schutter, "Multi-agent reinforcement learning: An overview," *Studies in Computational Intelligence*, Germany: Springer, pp. 183–221, 2010.

Solving the abstract Packet World Multi-Agent System through Direct and Indirect Communication Patterns

De Smet Lennert De Spiegeleer Pieter-Jan
R0862983 R0588561
Master of AI Master of AI

Globevnik Jonas
R0669933
Master of Engineering: Computer Science

Leenknecht Arno
R0856888
Master of AI

Abstract—The Packet World, introduced by Weyns et al., is an abstract multi-agent system application used for investigating, experimenting and evaluating different concepts and mechanisms. It allows for the introduction of various abstractions of real problems like agent communication, energy management or task delegation. These form, together with a base agent behaviour, the main issues that will be tackled throughout this paper. Additionally, in order to show that this is not a purely academic exercise, examples will be given of practical applications that could provide real-world benefits. Initial experiments limited to the base behaviour and communication will make use of a search-and-locate based strategy, which will already lead to a satisfactory completion of the base benchmarks. Moving on to the more complex issues of energy management and task coordination will then steer the process towards a complete unification into a Field-Based Task Assignment, or FiTa [1], approach, where all entities are allowed to emit gradient fields that can be followed by the agents. Not only will this prove able to provide a solution to these more intricate dilemmas, it will also further increase the performance of previous testing environments, yielding a final strategy that is both robust and shows satisfactory performance.

Index Terms—Multi-Agent System, Bidding, Energy Management, Congestion, Collision Avoidance, Gradient fields

I. INTRODUCTION

Multi-Agent Systems (MAS) are systems that tackle two common problems in global computing. First of all, MAS introduce agents that are capable of autonomous behaviour in order to successfully carry out tasks that are assigned to them. Secondly, the agents are capable of interacting with each other, using cooperation, coordination, negotiation, in order to carry out the tasks mentioned before. This is more complex when agents do not share a common goal. [2] In this paper, a MAS solution will be designed using the Packet-World test bed [3], which introduces an environment where the agents move around a two-dimensional grid. The grid contains different colored packages and destinations and the robots must move these packages to the correct destination, that of the same colour as the package. The Packet-World will be further introduced in Section II.

This basic setup can induce several problems, that will be solved in this paper. First of all, a basic autonomous behaviour must be designed for the agents to be able to locate and collect all packages. Afterwards, agents will have limited battery capacity and an energy management system will need

to be introduced in order to prevent agents from completely depleting their batteries. The agents will also need to be able to delegate tasks with one another, e.g. in order to clear blockades. Finally, to avoid deadlocks where agents block each other in narrow pathways, a congestion management feature will need to be additionally implemented.

Each of these problems will be solved subsequently. First, the basic autonomous behaviour of agents will be defined, using no forms of communication. Afterwards, using a mix of gradient fields and direct communication between agents, both of which will be explained in detail later, the energy management system is implemented. Another form of indirect communication, flags, which will also be explained later on, is used to delegate tasks to other agents. Finally, basic communication is used to solve the congestion problem.

In Section II, the ‘Packet World’ is described in detail, together with possible real world use cases of this Multi-Agent System [3] test bed. Afterwards, in Section III, the previously mentioned problems are divided into three steps that will be solved independently in Section IV. The results obtained from this section will be reviewed and discussed in Section V, after which related work is compared in Section VI. Future work that was alluded to in Section V will be discussed more in depth in Section VII. Finally, the conclusions from this paper are provided in Section VIII.

II. PACKET WORLD: A MULTI-AGENT SYSTEM TEST BED

A. General Multi-Agent System Components

The application uses different concepts to realise the aforementioned goal of the Packet World. What follows is a listing of the main concepts used in this abstract system.

a) *Virtual environment*: The system uses a virtual environment to reason about the state of a real environment. This virtual environment consists of a multi-layered grid. Each layer has its own objects and its own constraints. [3]

b) *Perception*: Perception is the ability of a robot to focus its attention to that part of the environment that it needs to realise its current goal, while ignoring other non-useful information. This is done by the agents using three functionalities: sensing, interpreting and filtering. [4]

c) *Decision making using extended free-flow trees*: The agents in the system make their own decisions about the actions they take, there is no centralised system that controls

the collective behaviour of all the agents. The model of decision making used in this system is based on free-flow trees. This allows the robot to consider multiple choices of actions, and choose the one which it considers to be the most beneficial towards the overall goal. This model is extended with the ideas of roles and situated commitments to allow agents to cooperate. Cooperating with other agents allows for the common goal to be achieved faster. It might also be necessary when a single robot cannot execute a task on its own. [5]

d) Communication: Agents can communicate with each other in order to establish collaboration. Communication can be direct, via a communication protocol, or indirect, via stigmergy. [3]

e) Synchronization: Because there is no centralised control unit, the agents must keep track of the actions of other agents. Because the relative timing and order of actions of different agents is completely arbitrary, a form of execution control is needed. This system supports three different approaches to achieve this execution control: global synchronization, regional synchronization, and synchronization based on logical time. [3]

B. Example Use Cases of Packet World

1) Mobile Agricultural Robot Swarms: One of the applications where we see a clear match with the test bed is autonomous agricultural operations, such as a swarm of planting robots. A huge advantage of operating a swarm of small machines instead of several big machines is that this is typically more reliable, more flexible to scale and easier to automate. One research example that follows this philosophy is the MARS (Mobile Agricultural Robot Swarms) project [6]. The application described in MARS aims at performing jobs, such as planting corn, by means of a swarm of collaborating agricultural robots. The software architecture in MARS is centralized, meaning that the jobs are managed and planned upfront by the central platform. However, given the complexity and variability of the environment where the robots have to operate in, we believe that a decentralized MAS may be more suitable.

Taking planting corn as an example, first of all, the robots are situated in a geographically constrained environment, i.e. a field with predefined boundaries. The second connection is the indirect communication where each robot places flags on a field map to inform others where he has seeded, as seeds under the soil are not visible. Thirdly, active perception remains very important for tasks such as collision avoidance, although it is not required for the described planting task. Last but not least, the swarm of robots applies the concept of role and situated commitment under certain situations. For example, near completion of the field there may not be enough rows for each robot to work on individually, rendering collaboration and negotiation into necessary conditions to complete the task in an efficient and optimal manner. On the other hand there are also discrepancies between the two applications, e.g. simultaneous actions are less relevant for the swarm of agricultural robots.

2) Taxi Service: A second possible practical use case of the packet world test bed could be that of a multi-vehicle taxi system capable of transporting people from and to their desired destination. This has already been described and studied in [7] in the context of a city that is modelled using a simple grid layout just as is the case in the packet world. While such a representation might generally seem to be too restrictive for application to any real city, it is important to mention that it is only used as a high level and hence abstract perspective on which the agents can do the necessary reasoning.

The most obvious mapping from the test bed to this application is certainly that of seeing the people that require transportation as the packages and their destinations as the drop-off points. Apart from using a contract based protocol for direct inter-agent communication, one can also imagine how methods of indirect communication are very much applicable as well. Indeed, take the use of flags as an example that allows for the various taxi agents to identify certain bottlenecks that should be temporarily avoided. The dynamic nature of these bottlenecks as time progresses would then correspond to the planting and removal of such flags.

There could be numerous advantages to the automation of ferrying passengers in an efficient and more organised way. Consumption of fuel might decrease together with possibly getting the people to their destinations more rapidly than would otherwise be the case. More intense communication between agents allows for more flexible and dynamic behaviour that can lead to a more robust transportation system capable of adapting to the ever changing environment.

III. PROBLEM DESCRIPTIONS

The purpose of an agent is to gather packages in the environment and deliver them to their corresponding destinations without being controlled by a central system. This task is simple at its core, but also contains some extra constraints: the agents have to charge their battery on time and they have a limited perception of the environment. However, there are also several methods to improve the efficiency of the agents' tasks, which will be shown in subsequent sections.

As a base case, an implementation where all agents work individually will be used. This is described further in Section III-A. Afterwards, the problem complexity and agent efficiency is increased in Sections III-B & III-C.

A. General Agent Behaviour

As a simple introductory problem, the agents need to pick up and deliver the packets in an autonomous manner. The agents have infinite battery capacity, but have limited perception of the environment. Defining efficient agent behaviour to solve these tasks will be done in Section IV-A.

B. Energy Management

Compared to the base problem described above, the agents do not have infinite battery life in this setting. Each turn, they lose an amount of energy depending on the action that they take in that turn. There are energy stations present in

the environment, which they can use to charge their batteries. As limited stations are available, an extra complexity lies in the fact that they need to coordinate who gets to charge their battery and who has to temporarily wait. In this phase, walls are also introduced in order to reduce the mobility options of the agents. Section IV-B will concern itself with possible solutions to these problems.

C. Task delegation between coloured agents

In all previous cases, agents could pick up any packet of any color. In this setting, agents are also assigned colors to which they are constrained. In order to deliver all packets in an environment, agents need to be able to delegate certain tasks, e.g. packets of a certain color blocking the destination of another color. The energy level of agents does not need to be managed, as energy management is assumed to be a separate and independent extension of the base problem. In Section IV-C, several features are introduced that will help resolve these problems.

IV. SOLUTIONS

The subsequent solutions all use the same base state pattern. The actual states that are defined within this design may differ between solutions, but they all represent a type of behaviour the agent can follow. Certain situations, that meet specific conditions, will make the agent switch between two behaviour states. If multiple transitions are possible, one will have priority over the others.

A. Implementing General Agent Behaviour

First the global design choices that are used to solve the problems stated in Section III-A will be highlighted, after which the implementations are discussed more in depth together with the test results. This will allow for a clear discussion on the efficiency of the implementations.

1) *Design choices and implementations:* The most important design choice is the different behaviours that an agent can use:

- **Search:** The agent uses a certain search strategy (defined in the implementation) to move around the board, looking for either a new packet or destination.
- **WalkToTarget:** The agent knows the location of its target and proceeds to take the shortest route to it.
- **Interact:** The agent interacts with the target when it reached the target location. This interaction can be picking up a target or depositing a target at the destination.

Following the above made distinction of behaviours, the first aspect to look at is that of Search. This behaviour triggers whenever the agent was not able to select a target from its memory or from its perception and as such will start walking around randomly. This random walk was implemented in two different ways:

- **Random Walk:** Every turn, an agent takes one step in a random direction (that results in a valid position).
- **Advanced Random Walk:** besides a random direction, an agent also selects a random amount of steps it will

move in that same direction (unless this is not a valid move). This improves the search capacity of unexplored space, as shown in paragraph IV-A2.

In the WalkToTarget behaviour, the agent will start to try and find a fitting target to move to first based on the situation it is in, meaning it either does not carry a packet and wants to move to one or it wants to find a fitting destination for the packet it is carrying. In both cases the agent can make use of its memory, where the absolute coordinates of any packet or destination it has encountered so far will be stored, with a limit of 10 entities that can be in the memory at any given moment in time. An implementation choice was made to, at any given moment in time, store maximally one of each type of destination and the closest packet. However, priority is given to any objects within the perception range of the agent, i.e. if it sees possible targets for its current objective, it will start moving towards the closest one. Once the agent has acquired a target, by either finding one in memory or in its perception range, it will simply walk to it. This directed walk uses the cosine similarity to order all possible steps by highest similarity to the direction of the target. If it could not select a target, the Search behaviour is triggered.

Finally, the Interact behaviour will be used once the agent has reached its target using WalkToTarget, i.e. it is next to it. This means that if a packet is carried, it will be dropped at the destination the agent has reached and in the other case it will pick up the packet it is adjacent to. Once this is done, the cycle continues by activating WalkToTarget again.

2) *Test Results:* In order to test the impact of two key implementation decisions (Memory and Advanced Random Walk), four different scenarios are considered: only random walk, only advanced random walk, memory with random walk and memory with advanced random walk. To get an average performance of each scenario, five different runs were considered and the average of their results were used.

Figure 1 shows the average amount of packets that were delivered at a certain cycle. These tests were run in the ‘basic-3-color’ environment, which contains 60 packets of varying colors. This figure provides a good insight into the impact of the both the advanced random walk and the memory. Using memory to remember destinations that the agent previously passed leads to a significant decrease in completion time. The figure also confirms that using advanced random walk allows the agents to discover new packages faster than normally. Using both features at the same time gives another performance improvement. The average turns to deliver all packets in the environment are displayed in Table I. Using advanced random walk in combination with the memory-functionality give a 92 % reduction of the average number of turns needed. All averages were calculated from 5 independent runs.

B. Implementing Energy Management

To provide an energy management system which is both efficient and robust, several features were added to the previous implementation. In the next section, these features are explored and explained. Afterwards, their performance is analysed and

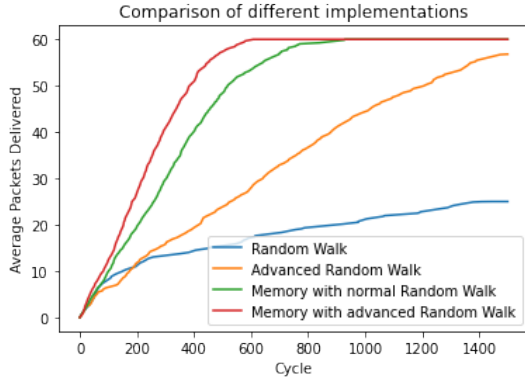


Fig. 1: Four different implementations are compared through their performance in the ‘basic-3-color’ environment.

TABLE I: Average number of turns required to deliver all packets in ‘basic-3-color’ environment, for different configurations.

	Random Walk	Advanced Random Walk
No Memory	7617.4 (-)	1621.2 (-79 %)
Memory	902.8 (-88 %)	581.2 (-92 %)

the best possible setup of the system is presented in the context of this problem.

1) *Design Choices and implementations for Energy Management:* All of the added features for energy management are presented in the paragraphs below. Their influence on the energy efficiency and robustness of the agents is presented in the section afterwards.

a) *Sharing of destination locations:* In the solution for the first problem, described in Section IV-A, the agents were able to remember the last seen destination from each color as well as the last seen closest packet. In some scenarios, like when an agent is searching for a long time before it can see a destination, it may be useful that agents can share their knowledge on the locations of destinations with one another. Whenever agents are in each other’s perception, they will send messages to each other. Each message contains the color of a destination, followed by a delimiter (‘;’) and the absolute coordinates of this destination. The agent on the receiving end can parse this message and add the location of this destination to their memory. If the agent already has a destination of the same color in its memory, the nearest one will always be remembered.

b) *Gradient fields:* Gradient fields are a form of indirect communication, which enables the agents to easily find the charging stations. This design pattern is described in [8]. Each charging station emits a gradient field, which is then combined into a single field. This gradient field is very useful for the agents, as it allows to always find the shortest ¹ path towards

the charging stations, and it allows them to calculate how far the charging station is.

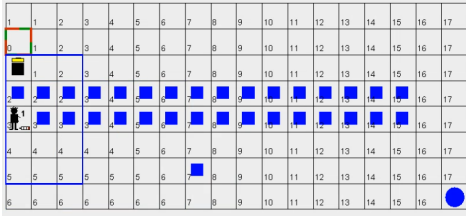
There are several degrees of freedom in designing a gradient field, an overview of the design choices is summarized below:

- **Avoid obstacles:** as an agent is not able to walk over an obstacle, such as a destination or a wall, no gradient field value is set there and thus the gradient moves around the any obstacle.
- **Dynamic gradient field w.r.t. packets:** packets are also considered as obstacles and no gradient field value is assigned to the location of a packet. Of course, the state of packets is dynamic, i.e. they can be picked up and placed down, thus the gradient field is updated every time a packet is picked up or placed down. The advantage of this approach is that the agent has a conservative estimate of the distance to the charging station. In the test worlds at hand (energy-1, energy-2, energy-3) this is less of a need as will be pointed out later in the results. However, in some cases the agent can benefit a lot from this dynamic gradient field, as can be seen in Figure 2.
- **No dynamic gradient field in function of occupancy:** one could consider to manipulate the gradient field emitted by an occupied charging station to inform the agents that it is not free at this very moment. Other agents in need of charging could then consider moving towards another charging station or wait a bit longer to charge. Therefore, other methods such as amplifying or reverting the gradient field of an occupied charging station were considered. However these methods did not improve performance and even introduced other issues such as oscillating behavior of the agents, e.g. agents continuously changing direction between two charging stations because the negative gradient from the combined field is changing direction, or inhibiting a consistently correct estimation of the required battery level to walk towards the charging station.

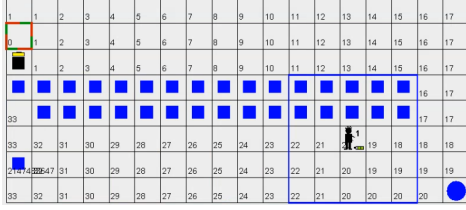
Applying the above mentioned principles, the gradient field of each charging station is calculated independently, in a breadth-first-search manner. In a final step, all the gradient fields are combined by taking the minimum of the gradient values for a given cell. Given the dynamic nature of the gradient field in function of packets, this is recalculated every cycle.

c) *Improved walking to target:* In section IV-A1 the cosine similarity methodology was introduced to walk towards a target. However, given the more challenging environment with walls, this method has some limitations. For example, if a target is in the view of the agent, but beyond a wall, the cosine similarity approach will direct the agent continuously towards the wall. Hence the reason for its replacement now with a breadth-first-search principle: the agent calculates the best path towards the target in its perception using a breadth-first-search algorithm. This is a simple algorithm, although not very memory efficient. More sophisticated planners have been described, such as the AStarPlanner [9] which is based on the well-known A* algorithm.

¹In some cases a shorter path may exist if the agent would move a packet out of the way, enabling the agent to break through a wall of packets such as in Figure 2



(a) Gradient field without dynamic update in function of presence of packets.



(b) Gradient field with dynamic update in function of presence of packets.

Fig. 2: Without dynamic updates, the agent may get stuck, while with dynamic updates the agent makes a good estimation of when he needs to walk towards the charging station (and the agent could still decide to break through the wall, if that is considered more efficient).

d) Avoiding congestion at energy stations: If the space around an energy station is limited, agents waiting to charge up could block other agents. They could leave fully charged agents with no way out or they make the charging station inaccessible to other agents that need to charge up their battery. To solve this, two new behaviours have been introduced: a first one for getting out of the way and a second one to wait. Getting out of the way simply means that the agent will step back while waiting means it will skip its turn.

When an agent is walking towards an energy station it constantly checks if agents on a location with a lower gradient value still have a way out. If this is not the case, the agent will check if walking further towards the station will solve the problem. Otherwise, the agent will change behaviours and take a step back until it is no longer blocking the exit. It will then wait until another agent has passed its previous location, meaning a space has become available, before trying again. This can be seen as reactive behavior of a single agent, which on its own is not so meaningful, however the aggregated behavior of a group of agents following this pattern is proactive and collaborative, refer to [10].

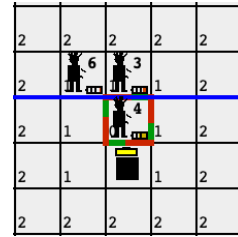
It is important that an agent will go as far as possible, in case one of the agents in front of it is dead. To avoid dying at such a crucial location in the first place, an agent that does not have enough energy to make it to the energy station will not try to do so, but will instead skip its turn. Since mostly happens when an agent is waiting, and a waiting agent is never blocking the station, the agent dying will not be a problem for the other agents. They can still walk around the dead one.

e) Charge when passing a charging station: When an agent passes close to a charging station, it may be beneficial to quickly charge up its battery before it continues its journey. The agent will then be able to have a longer journey away from the charging station before it needs to return.

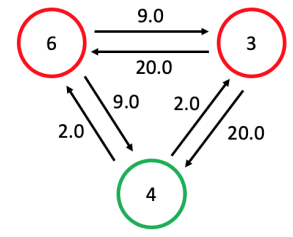
f) Use bidding to resolve urgent (conflicting) charging needs: One of the major problems remaining is when multiple agents have a charging need at the same time as this can cause one or multiple agents to run out of battery. A known pattern in MAS to deal with this scarce resource is bidding, as in [11]. A related pattern is described in [12], the market maker. Agents in need of charging and close to a charging station can raise a bid, so the agent with the best bid can go charging. As no real bidding initiator is present here, all participants send their bid to all other participants. At the end, all participants will have the same list of bids, by which they can decide themselves if they won or lost. The important design choices are:

- **Bid value:** takes in account how many cycles the agent has in reserve, before it needs to start moving towards the charging station. In this case, the lower the bid, the higher the need to start charging. To avoid that an agent leaves the energy station too fast, a bid of a charging agent will be fixed at a low value until it reaches a certain battery level. This value will still be high enough to allow agents that are in need of urgent charging to have priority over them.
- **Participants:** Only agents that are next to a charging station or charging on it are able to submit a bid. This assures all agents participate in the bid if their perception is bigger than 1, which is a fair assumption, without needing a central system to declare the winner.

An example of this bidding phase is depicted in Figure 3. Here, three participants bring out a bid, where the agent with the lowest one wins, agent 4 in this case.



(a) Energy station that has three candidates.



(b) Bids that are exchanged between the participants.

Fig. 3: A situation where three agents bring out a bid for the energy station. Agent 4 is currently charging but has not reached the desired minimum, thus this agent has a low bid and wins the bidding entirely.

2) Test Results: The features that were presented in the previous section were tested based on energy efficiency and robustness. A robust multi-agent system will have a low chance of not completing the task at hand: delivering all packages. This could be caused by all agents running out of energy, destinations being blocked off by dead agents or

TABLE II: Test results in both testing environments. The tests in ‘Energy 2’ focus on energy efficiency, the tests in ‘Energy 3’ focus on robustness. In each test scenario, only one feature is **disabled** compared to the benchmark situation, using the average of 100 runs.

(a) Test results that came from ‘Energy 2’.

Scenario	Deactivated feature	Energy Consumed	Cycles
Benchmark	\	56640.9 (-)	805.88 (-)
A	Communication about Destination Locations	58969.8 (+4.11 %)	829.44 (+2.92 %)
B	Dynamic Gradient Field	56471 (-0.30 %)	798.02 (-0.98 %)
C	Improved walking to target	56048.7 (-1.05 %)	791.06 (-1.84 %)
D	Avoiding congestion at energy stations	56366.5 (-0.48 %)	797.16 (-1.08 %)
E	Charge when passing a charging station	56441.5 (-0.35 %)	806.5 (+0.08 %)
F	Bidding	55790.1 (-1.50 %)	827.95 (+2.74 %)

(b) Test results that came from ‘Energy 3’.

Scenario	Deactivated feature	Average runs before failure	Completion Chance
Benchmark	\	41.1 (-)	97.62 %
A	Communication about Destination Locations	21.6 (-46.3 %)	95.58 %
B	Dynamic Gradient Field	26.0 (-36.7 %)	96.30 %
C	Improved walking to target	0 (-100 %)	0.00 %
D	Avoiding congestion at energy stations	1.4 (-96.6 %)	58.33 %
E	Charge when passing a charging station	19.8 (-53.3 %)	95.19 %
F	Bidding	29.4 (-28.5 %)	96.6 %

energy stations being blocked off by dead agents.

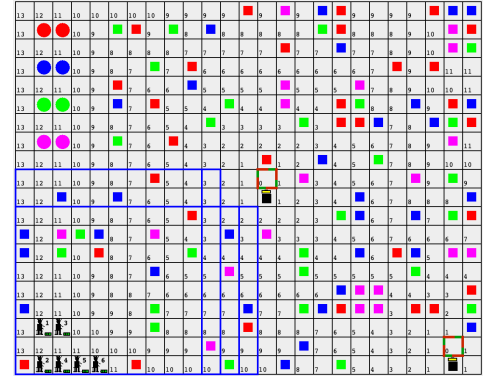
Every setup was tested in two environments, unless mentioned otherwise. The first one, which is referred to as ‘Energy 2’, mainly consists of many packages and hence will require a number of recharging cycles for every agent, which makes it a good benchmark for energy efficiency. In the second, called ‘Energy 3’, the area around the recharging stations is not very spacious and only has a single point of entry, which will mainly test performance of avoiding congestion and agent failure. Both can be seen on Figures 4a and 4b respectively. To have a better understanding of what the influence and added value of each feature is on the performance of the aggregated system, an ablation study is performed, i.e. each feature is **disabled** one by one. Their energy efficiency and robustness, quantified by the metric ‘completion chance’², are then compared to the results of the baseline system where all features are active.

a) *Disabled feature: Sharing of destination locations:*

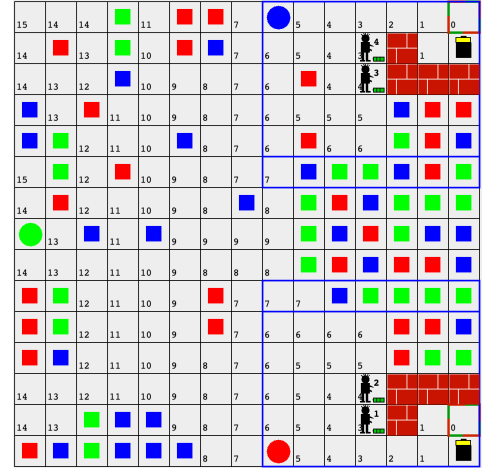
In Table IIa, test scenario A has the communication of destinations between agents disabled. As can be seen from the results, a 4% increase in energy is registered on average. This is immediately the biggest observed difference of all features. The communication seems to be especially crucial in the beginning, when the agents haven’t explored all parts of the environment themselves.

To showcase the importance of this feature further, additional tests were run on the environment depicted in Figure 5. In this environment, the agents pick up a packet of which they do not know the destination, while another agent does know this. Without communication enabled, the agents needed on average 144 cycles to deliver all packages, while with communication, they only needed 79 cycles to do the same. This is a decrease of 45%.

²Completion chance is calculated as $1 - \frac{1}{\text{Average runs before failure}}$, where a failure is defined as agents not being able to complete the task



(a) Energy 2 environment: A lot of packages are present, which will give a large test set to measure energy efficiency.



(b) Energy 3 environment: Used to test the robustness of the multi-agent system. The walls surrounding the energy station make it a very narrow entry point when multiple agents want to charge at the same time.

Fig. 4: The environments that are used to test energy management-features.

b) *Disabled feature: Gradient fields:* In test scenario B, the dynamic feature of the gradient fields (w.r.t. packets) is deactivated. From the tests in Energy 2, no conclusions can be drawn as the results are not significant enough. However, the completion chance goes down drastically. The system would now fail every 27 runs, instead of every 42 runs, a decrease of 36%. The dynamic gradient field update thus has a good contribution towards the stability of the MAS.

c) *Disabled feature: Improved walking to target:* In test scenario C, the breadth first search algorithm is now replaced by the previously used cosine similarity. As mentioned before, this was implemented to give agents the possibility to move efficiently towards their target if they’re being blocked by walls. The Energy 3-environment is thus a good testing ground for this feature. As can be seen from Table IIb, the cosine similarity implementation has a 0% completion chance. This is because after a recharge, the agent will try to move towards

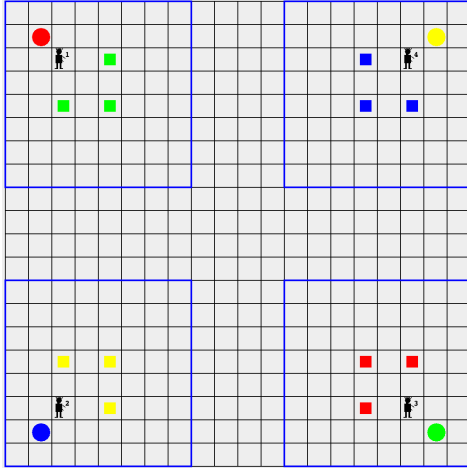


Fig. 5: This environment is used to showcase the importance of communication about destination locations.

the closest package in its vision. Without breadth first search, it would move from side to side indefinitely.

d) *Disabled feature: Avoiding congestion at energy stations:* Besides needing breadth first search to exit the charging area in Energy 3, a waiting system also needed to be implemented. If not, a lot of agents would enter this area at the same time, leading to congestion and nobody being able to move anymore. This assumption is confirmed by the results in Table IIIb for scenario D. It shows that only 58% of the runs succeed without a waiting-feature implemented into the system.

e) *Disabled feature: Charge when passing a charging station:* In Table IIa, scenario E has the ‘Charge while passing’-feature disabled, which means that they will only recharge their battery when it is sufficiently low and not out of convenience. When running these tests for scenario E in Energy 2, it occurred several times that an agent’s battery died (in 20% of the test runs). However, the other agents were still able to mostly deliver all the other packages. This behaviour is not desirable, as it could be that the agent’s battery runs out while holding a packet, which can never be delivered in that case.

On top of this unreliability in Energy 2, a decrease in robustness is observed. Only 95.19 % of the runs succeed in the Energy 3 environment. While 95.19 % does not seem much worse than 97.62 %, one must consider that this entails a decrease of 50 % of the average runs before failure. Charging while passing an energy station is thus still very useful.

f) *Disabled feature: Bidding:* Scenario F consists of all benchmark features, but with bidding for an energy station disabled. Every two runs, there is on average 1 agent that has a dead battery, while the others still deliver the other packages. From the results of the tests in Energy 3, it can also be seen that only 96.7% of the runs will succeed, which is slightly worse than the base case.

The bidding could however still be improved. As mentioned before, only agents who are next to the the energy station can

bring out a bid. In areas that are prone to congestion, some agents are forced to wait in a more roomy area. It is in these situations that most of the dead agents occur.

C. Implementing Task Delegation between Coloured Agents

1) *Design choices and implementations for Task Delegation:* Multiple possible patterns can be used to try and solve the coordination problem, with the main contenders being either FiTa [1] or DynCNET [13]. The latter extends regular contract communications by allowing these to be modified dynamically, leading to better adaptation to the varying situations encountered. The decision was made to go with the FiTa approach as it was deemed more robust than DynCNET [14]. Indeed, while the interactions in DynCNET are explicit, the choices that need to be made in their practical implementation have much less guarantee to generalise well beyond the discussed testing environments. Moreover, by generally allowing all entities in the packet world to be the source of a gradient field, previous strategies will additionally be streamlined. The main changes are as follows:

- **Gradient field:** Destinations and packets now emit a gradient field, rendering the need for a search obsolete. By specifically separating the gradient world into distinct parts for every entity, an agent can always count on being able to access the right field to follow to its target destination. For example, say an agent just picked up a red packet, then the previous behaviour of target selection still applies, though the walking to the target is now reduced to following the gradient field of the identified target.
- **Flag Placement:** More importantly for this section, agents are now able to plant flags whenever they locate an obstacle, e.g. a package that is blocking the route to their target. Flags are also a form of indirect communication, by means of environmental markers, as described in [15]. Whenever such an obstacle is identified, the agent plants a flag of the same color next to the obstacle at hand that immediately emits a gradient field. This field has a higher priority than any other currently present of that color and will trigger all agents capable of carrying a package of that color to move towards the flag and remove the obstacle.
- **Collision Avoidance:** In some cases, two agents try to pass through a narrow pathway at the same time but coming from opposite directions, as shown in Figure 6. When this happens, they will communicate with each other in order to decide who needs to back away to let the other agent through. Both agents send their current gradient value to the other and the one with the lowest gradient value is allowed to pass first. If the values are the same, the agents are ordered alphabetically to decide who can pass first. This feature will also allow agents to move around a maze-like environment relatively smoothly.

2) Test Results:

a) *Evaluating the gradient field implementation:* The move to a pure FiTa approach already gave a significant

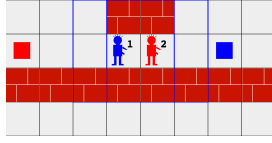


Fig. 6: Two agents try to pass through a narrow path at the same time. In this case, the red agent is further away from its packet, so it will back away and let blue pass through.

improvement in the base case environments, where the ‘basic-3-color’ benchmark for example is now solved in 326 cycles on average in comparison to 581.2 previously. Figure 7 shows how fast packets are delivered in each implementation. At the beginning, a flatter curve is observed for the implementation without gradients, as the agents here still need to perform a random walk to discover the destinations for the first time (considering no communication is used).

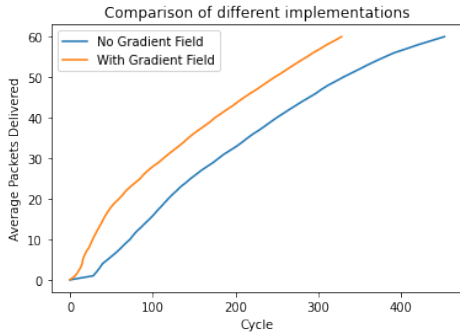


Fig. 7: A comparison between an implementation with and without gradient fields emitted from packets and destinations, taken from an average of 100 runs.

However, one has to keep in mind that maintaining these gradient fields is, in principle, a centralised operation. In contrast to this, the agents previously only had to rely on their immediate perception and personal memory. None the less, the results would suggest this to be a reasonable trade-off to make.

b) Results in the task delegation environment: Using all three major features described before, a nearly 100 % completion rate is achieved, which is a very large increase from the situation where no collision avoidance is implemented. Only very rarely, three agents will block each other. On an average of 100 runs, the agents need 7634 energy to deliver all of the packets.

c) Results in the maze environment: Without the implementation of the collision avoidance, the agent could never completely solve this environment, as it contains many narrow paths in which they need to cross each other. With this feature implemented, the agents are able to solve the maze in 67 % of the runs, which is already quite a satisfactory result. The situation depicted in Figure 8 is the major problem that remains to be solved, as the current collision avoidance-feature fails to solve this. One way this could be resolved in a future

release is passing packets along the way, for example in the figure, agent 4 could pass its packet to agents 2 etc., which solves the congestion issue at the same time. It’s important to note that this won’t work if the agents have different colours.

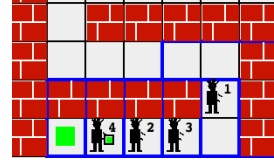


Fig. 8: In some cases, the agents are all together in one narrow pathway. Currently, this problem is not supported, but could be resolved in the future by letting the agents pass packets along the line.

V. GENERAL DISCUSSION

In the previous sections, different problems were introduced that each highlight an important aspect of a multi-agent system.

In the base problem, a general agent behaviour was implemented where agents need to pick up packets and bring them to destinations. A random search behaviour works, but can be very slow and time consuming. Making use of an improved random search and memory improves the solution time significantly, as can be seen in Figure 1, but further improvements could still be made. First of all, agents can take a long time to discover their first destination, which may be located at the other side relative to their starting position. The communication introduced in step 2 partially resolves this problem, as each agent can benefit from another agent’s knowledge. Secondly, agents have a limited memory, which is why it was decided to only remember one destination of each color and the closest packet. This causes agents to forget about some packets that could be hard to find later on. Last of all, the agents will pick up the packet closest to them, yet a potentially more optimal solution could be obtained by letting the agents pick up packets that have the same color as a destination in their memory. This would prevent agents from picking up a packet and wandering around searching for a destination.

When introducing a limited battery capacity for the agents, an energy management system needs to be implemented. The features of this system are described in Section IV-B. The main problems that needed attention were energy efficiency and robustness, i.e. consistent delivery of all packets. In Table IIa and Table IIb, it can be seen that every feature added had a positive contribution to either the energy efficiency or the robustness. Besides solving the problems that energy management introduces, one of the earlier mentioned problems has also been resolved at this stage: by implementing communication, the agents are able to tell each other the location of destinations from their memory, which can help agents who are wandering around searching for a destination.

While these features give a good improvement compared to a basic implementation, there are still some particular problematic situations. First of all, in very rare occasions, agents can

die while holding a packet. All other agents will continue to look for this last packet for an eternity. Secondly, agents are still dying, even with the bidding system in place. The latter could still be improved by looking at a more extensive solution as proposed in [11]. Last of all, agents can accidentally block off charging stations or destinations by placing down packets. They will always place down a packet when moving towards an energy station, which preserves energy, but in rare cases this can cause an incomplete run.

The need for task delegation introduced a complex problem, where the agents need to work together to deliver all packets. The introduction of gradient fields for packets and destinations gave advantages, such as less time spent randomly wandering, but also disadvantages concerning centralisation and computation time. When comparing the performances with and without gradient fields, ultimately, the advantages out-weighted the disadvantages. Flag placement was also brought in to support the task delegation, as a flag produces a gradient field that informs other agents that they need to assist. However, using flag placement alone will not assure deliverance of all packets, as agents are not able to cross each other in a narrow pathway. This was solved by a collision avoidance feature, resulting in a near perfect completion rate.

Finally, the cooperation of agents in a maze environment proved most challenging, as it contains a lot of narrow pathways. The collision avoidance feature implemented only generates a 68 % completion chance, leaving room for improvements as explored in Sections VI and VII.

VI. RELATED WORK

The problems presented in this paper could also lead to alternative solutions.

The agents mainly use direct forms of communication by sending messages to each other, apart from the flag placement, which is indirect communication. The other forms of communication are between the environment and the agents and not between the agents themselves. In [16], Chouhan et al. propose using an indirect method for all inter-agent communication. The simulated environment, a grid world similar to Packet-World but with only a single packet/agent color, has a communication module in which the agents can share information with each other through a 'share variable' by broadcasting their current state. Using this form of communication, the agents prevent collisions and/or deadlock.

In [17], Abusnaina et al. implemented yet another form of communication: a blackboard. Their agent behaviours are split into two different phases: a discovery phase in which they try to explore the packet world and a cleaning phase where they retrieve previously located packages. If an agent sees a packet during the first phase, it notes down the information on the virtual blackboard. The information on this blackboard is then used in the subsequent cleaning phase to deliver all packets. Compared to this paper, the tasks of different agents are less mixed, as the two phases are not intertwined, which leads to better energy efficiency according to Abusnaina et al.

Ketenci et al. propose the introduction of a bounded active perception in [18]. They assume that the perception is resource-bounded, which is the case for the packet world due to its limited range. Contrary to this paper, where the whole perception is processed, they propose the introduction of both top-down focus, which depends on the current goal, and bottom-up focus selection, which depends on the agent's properties and location.

A similar approach was proposed in [19], where the concept of reflective, deliberative information gathering is introduced. Being deliberative means that the agent chooses itself how to gather information. Being reflective means that the agent can self-evaluate its past sensing outcomes, guiding the deliberative process described before.

The bidding system for energy stations that was implemented in this paper was inspired by Tung Do et al. [11]. Here, they introduce several options to handle the bidding, each designed for a specific situation. All of them consist of several phases, where the bids are sent, processed and the results are sent back, while this was simplified to a single phase system in this paper.

VII. FUTURE WORK

One of the observations that was made throughout the evolution towards more complex settings was that of an increase in the amount of possible solutions that could be used to solve those more complicated problems. Even in the base case, the results of Figure 1 already show that a simple change in walking behaviour can lead to a dramatic increase in performance. When energy management is necessary, there is a similar plethora of choices available for an agent to decide when to go charging. Here, it was noticed that a slight alteration of the threshold values that determine this decision gave wildly different completion results. While all of this is most definitely still very much manageable in the limited amount of cases that were considered, one can still not be sure that the chosen strategies lead to satisfactory results in other configurations, even if they are similar. To deal with this highly dynamic nature, the use of reinforcement learning [20], [21] in multi-agent systems could be promoted. While this does have its own drawbacks, it could prove beneficial to selectively make use of the reinforcement learning paradigm to make action related decisions such as walking or charging behaviour. In combination with the information given by explicitly programmed patterns, this could lead to more robust solutions as the agent gains the ability to adapt dynamically to its environment.

To improve the performance in the maze environment, a better collision avoidance feature needs to be implemented. A good performance boost could be obtained by implementing a behaviour that allows agents to pass packets along a line in a narrow pathway. As mentioned before, this is only helpful if the agent do not have a specified color assigned to them.

VIII. CONCLUSION

Throughout this paper, multiple possible solutions to the problems that arise in the Packet World were discussed and compared. Through a unified FiTa strategy, a final solution that should be robust to changing environments was obtained. It has to be noted that, in order to really verify this claim, more testing environments should be examined, but related work has already given arguments to support this. This does not, however, mean that the given solution is always completely adequate, as was seen in the final maze environment. In any case, a global FiTa approach does seem to provide a solid foundation for future endeavours that relate to the Packet World, even though it lacks in explicitness.

REFERENCES

- [1] Danny Weyns, Nelis Boucké, and Tom Holvoet. Gradient field-based task assignment in an agv transportation system. In *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS '06*, page 842–849, New York, NY, USA, 2006. Association for Computing Machinery.
- [2] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd edition, 2009.
- [3] Danny Weyns, Alexander Helleboogh, and Tom Holvoet. The packet-world: A test bed for investigating situated multi-agent systems. In Rainer Unland, Monique Calisti, and Matthias Klusch, editors, *Software Agent-Based Applications, Platforms and Development Kits*, pages 383–408, Basel, 2005. Birkhäuser Basel.
- [4] DANNY WEYNS, ELKE STEEGMANS, and TOM HOLVOET. Towards active perception in situated multi-agent systems. *Applied Artificial Intelligence*, 18(9-10):867–883, 2004.
- [5] Danny Weyns. *Architectural Design of Multi-Agent Systems*, pages 55–92. 02 2010.
- [6] T. Blender, T. Buchner, B. Fernandez, B. Pichlmaier, and C. Schlegel. Managing a mobile agricultural robot swarm for a seeding task. In *IECON 2016 - 42nd Annual Conference of the IEEE Industrial Electronics Society*, pages 6879–6886, 2016.
- [7] Lluís Ribas-Xirgo. Multi-agent system model of taxi fleets. In Luis M. Bergasa, Manuel Ocaña, Rafael Barea, Elena López-Guillén, and Pedro Revenga, editors, *Advances in Physical Agents II*, pages 123–134, Cham, 2021. Springer International Publishing.
- [8] T. De Wolf and T. Holvoet. Design patterns for decentralised coordination in self-organising emergent systems. In *Engineering Self-Organising Systems, Lecture Notes in Computer Science*, vol. 4335, pp. 28–49. Springer, Berlin, Heidelberg, Germany (2007), 2007.
- [9] A. Chella, M. Cossentino, S. Gaglio, L. Sabatucci, and V. Seidita. Agent-oriented software patterns for rapid and affordable robot programming. In *J. Syst. Softw.* 83, 557–573, 2010.
- [10] E.A. Kendall, P.V. Murali Krishna, C.V. Pathak, and C.B. Suresh. Patterns of intelligent and mobile agents. In *Proceedings of the 2nd International Conference on Autonomous Agents*, pp. 92–99. ACM, 1998.
- [11] Thanh Tung Do, Manuel Kolp, and Alain Pirotte. Social patterns for designing multi-agent systems. In *15th International Conference on Software Engineering and Knowledge Engineering (San Francisco)*, 2003.
- [12] D. Deugo, M. Weiss, and E. Kendall. Reusable patterns for agent coordination. In *Coordination of Internet agents, chap. Reusable patterns for agent coordination*, pp. 347–368. SpringerVerlag, London, UK, 2001.
- [13] Danny Weyns, Nelis Boucke, Tom Holvoet, and Bart Demarsin. Dyncnet: A protocol for dynamic task assignment in multiagent systems. In *First International Conference on Self-Adaptive and Self-Organizing Systems (SASO 2007)*, pages 281–284, 2007.
- [14] Danny Weyns, Nelis Boucké, and Tom Holvoet. A field-based versus a protocol-based approach for adaptive task assignment. *Autonomous Agents and Multi-Agent Systems*, 17:288–319, 10 2008.
- [15] J. De Meulenaere. Stigmergy applied in the packet-world. In *Master thesis Katholieke Universiteit Leuven*, 2003.
- [16] Satyendra Singh Chouhan and Rajdeep Niyogi. An analysis of the effect of communication for multi-agent planning in a grid world domain. *International Journal of Intelligent Systems and Applications*, 4(5):8, 2012.
- [17] Ahmed A. A. Abusnaina and Chan Huah Yong. Cooperative multi-agent system for solving packet world problem in grid. In *2009 IEEE 9th Malaysia International Conference on Communications (MICC)*, pages 754–758, 2009.
- [18] Utku Görkem Ketenci, Roland Brémond, Jean Michel Auberlet, and Emmanuelle Grislin-Le Strugeon. Bounded active perception. In *Pre-proceedings of the 8th European Workshop on Multi-Agent Systems (EUMAS 2010)*, Paris, 2010.
- [19] Adam D Eck. Reflective, deliberative agent-based information gathering. 2015.
- [20] Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [21] I. Arel. Reinforcement learning-based multi-agent system for network traffic signal control. *IET Intelligent Transport Systems*, 4:128–135(7), June 2010.

Implementing Multi-Agent System behaviours for overcoming energy constraints and obstacles in The Packet-World

Jesse Geens
r0661523
jesse.geens@student.kuleuven.be

Joris Ignoul
r0706829
joris.ignoul@student.kuleuven.be

Wouter Lenaerts
r0709016
wouter.lenaerts1@student.kuleuven.be

Emeric Goossens
r0692987
emeric.goossens@student.kuleuven.be

Abstract—Situating multi-agent systems is a growing application domain. Examples of applications are autonomously driving garbage trucks and drones for rescue operations. Using the Packet-World as a test bed, some aspects of these systems are examined in depth. To complete their goals, agents need to behave autonomously and need to interact and cooperate. Agents can communicate through direct messaging as well as using gradient fields. Different types of environments introduce different aspects and peculiarities, like walls, energy constraints and different types of packets. These aspects bring forth challenges, like path-finding, priorities and task delegation for which solutions are presented in this article. Lastly, some options for future work that might improve the presented solutions are included.

Index Terms—Packet-World, situated multi-agent systems, autonomous agents, behaviour, behaviour changes, direct communication, gradient fields, energy-constrained agents, obstacles, path-finding, task delegation

I. INTRODUCTION

A multi-agent system is a system that consists of a number of autonomously operating agents, which interact with each other, typically by exchanging messages, in order to achieve a certain goal. [1] They can flexibly adapt their behaviour to changing operating conditions. [2] In this paper, a basic multi-agent system where the agents exhibit autonomous behaviour is developed. This basic autonomous behaviour is then built further upon by extending it with options for working under energy constraints and obstacle avoidance. The implementations are tested in the Packet-World [2], a rectangular grid containing coloured packets which must be transported to specific drop-off points. Examples of real-world problems mapping onto the Packet-World are automated garbage trucks and rescue drones. To realise autonomous agents, the most basic approach is a state machine with four behaviours: moving randomly, moving to a destination, picking up a packet and putting down a packet. For each of these behaviour states, transitions are defined with conditions to determine when an agent should change to a different behaviour state. The second design is an extension of the first that includes an energy aspect where the agents should charge their battery from time to time.

Gradient fields and communication were used to accomplish the charging task. In the last design agents can only carry packets from their own colour. This means that destinations can be blocked by packets they cannot pick up themselves, hence they must coordinate with each other in order to accomplish their individual tasks. This type of environment can also contain walls, meaning that the agents will have to use a more advanced algorithm for path-finding. Later on, this paper discusses the results of the different implemented behaviours. Next the approach taken is compared with related work. To conclude, the main findings are summarized and some proposals for future work are presented.

II. PACKET-WORLD

This section gives an overview of the Packet-World [2] and its real world applications.

A. Main concepts and characteristics

1) *Agent and environment*: The multi-agent system used in the Packet-World is also called a situated MAS. The main idea behind situated MAS is, firstly, to move away from a centralized solution to avoid bottlenecks. Secondly, a situated MAS takes into account the intrinsic dynamical aspect of the system [19] (e.g., a failing robot). The intelligence in the Packet-World resides in the interactions between agents that can result in complex behaviour. The agents cooperate to achieve their goals. Benefits of such a system are efficiency, robustness and flexibility.

2) *Decision-making*: To achieve a decentralized system each agent acts autonomously: it makes its own decisions and maintains its own state. The agents in the Packet-World make their decisions based on a model of free-flow trees. These trees are adjusted to include support for explicit social behaviour, i.e. they can commit to an agreement of cooperation earlier negotiated.

3) *Perception of environment*: Each agent perceives only a limited part of the environment and therefore it has only partial knowledge [22]. In the Packet-World each agent has a

range of view that expresses how far (expressed in the number of cells) it can see.

4) *Negotiation and Direct communication*: To negotiate the formerly mentioned cooperations, direct communication between agents seems natural because this is how humans interact socially. These cooperations, and therefore direct communication between agents, can improve overall efficiency in completing a goal [23]. In the Packet-World, agents can form a chain to deliver a packet at its destination. Direct communication can also be necessary to perform a task which can't be executed by only one agent. In the Packet-World some heavy packets should be carried by two agents. The interaction can be established using communication protocols existing of several protocol steps [21]. Such a protocol step consists of conditions defining whether the step is applicable and effects defining the result of the communication. The load of this communication should be as minimal as possible.

B. Real-world problems that map to the test-bed

1) *Trash collection*: A real-world example where Multi-Agent Systems may be of use is the case of automated trash collection by electric vehicles. These garbage trucks drive through the city, looking for trash to pick up and when they are full they bring it to a specific collection facility. There are multiple kinds of trash (e.g. glass, PMD, residual waste). This maps to the test bed in the following way.

Agents: the agents are automated, electric garbage trucks. Their goal is to empty all the trash cans in the environment.

Environment: the environment is a city in which there are litter bins for different kinds of litter (e.g., PMD, residual waste, GFT, glass). Each kind of litter has some collection centers where it can be disposed by the garbage truck.

Perception: all litter bins in the city are equipped with a sensor that detects when the bin is almost full. When a litter bin is almost full, it will send a signal over LoRa [11] (a network technology with a range of about 5 km). All agents that are in range can detect this signal and know that there is a full trash bin. The agents can use the `see(a, range)` function [2] to see these trash bins.

Negotiation & Direct Communication: agents can communicate with one another using radio signals, if they are within range of each other. They have to negotiate over which garbage truck will pick up which bin, so that there are no "useless" trips of multiple trucks to the same bin.

Indirect communication: the trucks are electric, so they need to be recharged regularly. The trucks will use gradient fields to locate nearby charging stations.

Synchronization: the garbage trucks can be locally synchronized, since there is little use for global synchronization.

2) *Rescue operations*: In the mountains, avalanches and earthquakes are not uncommon. Unfortunately, these sometimes involve accidents with people. These persons need to be located under the snow or under the debris. The mountainous environment presents two challenging problems. On the one hand, finding the persons is far from trivial. On the other hand, the mountains with their vegetation complicate communication

[13], making therefore a centralized solution very difficult. This application maps to the test-bed as follows.

Agents: the agents in this case are drones. These drones are used to scan the surface and the underlying snow or debris for humans.

Environment: the environment for this problem is the open air (e.g., the mountains). This environment contains people who need to be rescued.

Perception: the drones can fly and act autonomously, making use of a video stream to navigate the environment and heat sensors to locate people. These sensors have a maximum scope, that limits the perception range of a single drone. To communicate with other drones and to make exchange of information possible, each drone is equipped with a radio.

Negotiation & Direct communication: if someone is found, the drone can signal this to other drones closer to the basis which in their turn will do the same. In this way they can form a communication chain to alert the basis as soon as possible. A drone can also alert multiple drones to help searching the near area, as chances are high people were not alone during the disaster.

Indirect communication: if an area is searched, a drone can flag it in order to prevent scanning the same region again. The agents are battery-powered and need to be recharged when low in energy. Gradient fields are used to locate these battery chargers.

Synchronization: local synchronization can be useful to divide the terrain evenly between the drones.

III. THE PROBLEM

In multi-agent systems, agents typically need to perform a certain task. In the Packet-World each agent has to search for coloured packets. These packets have to be delivered to a drop-off point with the same colour. To achieve this goal, each agent needs to be able to behave autonomously. To be able to work more efficiently and to avoid hindering each other, the agents also need to communicate. In some situations, agents have limited battery and therefore should be able to detect and charge at charging stations. Finally, in some cases, agents can only pick up packets of a specific colour. The agents have to delegate tasks and request help when needed.

A. Autonomous behaviour of agents to solve tasks

To solve the task of delivering all the packets to their destination each agent should behave autonomously. Therefore agents should base their actions based on their perception of a dynamical environment. Since the goal of the Packet-World is to model agent interaction in the real world, the agents have to obey the physical laws. This implies that agents can only move a certain number of tiles per cycle (in the current implementation this is maximum one tile per cycle), agents can only carry one packet at a time and agents shouldn't collide. To achieve this autonomous behaviour, the agents possess a limited memory in which they can store data in the form of key-value pairs. The problem is now reduced to two sub-problems. On the one hand, deciding (and implementing)

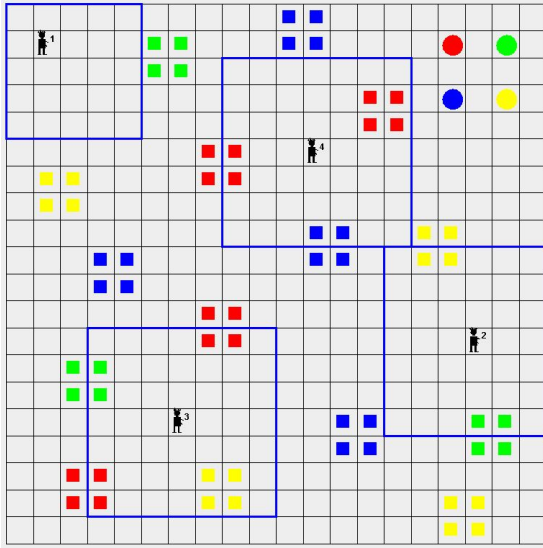


Fig. 1. The basic environment with square packets and circles as drop off points. The blue rectangles mark the end of the perceptions of the agents.

which action an agent should take based on his perception. On the other hand, using the agent's memory to improve this decision making and to make the whole process more efficient. Figure 1 illustrates a basic environment.

B. Coordination to handle energy-constrained agents

Agents in the real world often have a limited amount of energy. This energy is supplied by their batteries. Every action, except for communicating, costs a certain amount of energy. Skipping a turn has some energy cost too. The battery contains only a limited amount of energy and thus needs to be charged in time. The problem contains two aspects. On the one hand, how to let agents behave in an energy-efficient way. On the other hand, how to make sure that agents charge their battery in time. The first aspect of the energy problem is mostly an aspect of performance. This is because charging, of course, takes time. Time in which the agent cannot perform its tasks. If an agent behaves really energy-inefficient and never performs any positive contribution towards accomplishing its tasks this is no longer an aspect of performance but of solving the overall problem. The second aspect of the problem is the most crucial of the two. If the agent does not charge in time, it will die (be out of battery) and will not be able to perform its tasks anymore. If a crucial agent dies or if all agents die before its/their tasks are done, the overall problem cannot be solved. The aspect of charging in time includes that the agents need to locate the battery stations and be able to move to this stations. Locating the battery stations is itself subject to the first aspect of the problem, i.e. it needs to happen in an energy-efficient way.

C. Collaboration in an obstacle-filled environment

Environments can contain multiple types of obstacles. This can be movable obstacles, i.e. packets, fixed obstacles, i.e. walls, and finally also other agents. Furthermore, in this

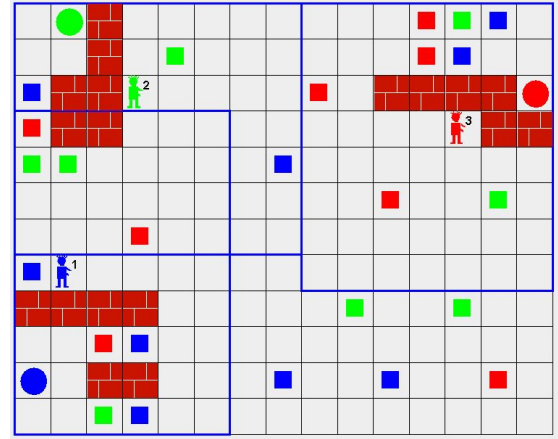


Fig. 2. Setup of the obstacle-filled environment with walls and coloured agents.

setting, agents now have a colour and are only allowed to pick up packets of their own colour. In the first part of the problem agents should be able to deal with walls in the world. This means that they should now look for the shortest path that does not go through a wall, which is obviously more challenging than a naive approach that can be used in the previous parts. The second problem is that it is possible that the only possible path to the agent's destination is blocked by packets it cannot pick up itself, as is shown in figure 2. As a consequence there should be a way to get other agents with the right colour to pick up a packet that blocks the path. The third problem for an agent is that it is also possible that multiple packets of its own colour can block the only possible path to the drop-off point of the agent's color. Finally, the overall solution should prevent deadlock. In this problem, the energy aspect will be ignored.

IV. SOLUTION

This chapter briefly introduces possible solutions for the three problems discussed above. For each of the problems, a different type of environment has been used in the Packet-World. In the first subsection, making agents behaving autonomously is tackled using an iterative approach. This approach is used when defining the behaviour of the agents (improving gradually the efficiency), as well as when working on the structure of the design (using behaviours and behaviour-changes). In the second subsection these autonomous agents are adapted to work in situations with energy constraints. For this goal, the charging stations in the environment emit gradient fields to make it possible for the agents to find them rapidly. In the meantime, agents also communicate to avoid starvation at these stations. Finally, the case with obstacles is discussed in the last subsection. The autonomous agent, without energy constraints, is adapted to yet another type of environment. An intelligent path-finding algorithm was developed for this purpose. Furthermore the communication between agents was further developed to enable them to delegate tasks.

A. Autonomous behaviour

The first challenge to tackle is to make agents, with limited perception, act autonomously. Initially, the implementation consisted of one class encapsulating the complete behaviour of the agents for every situation. This can quickly grow to a large and complex class with many interactions, which will become difficult to manage and is not scalable. Therefore, the code was divided into different classes representing behaviours and behaviour changes. This approach is based on the Behaviour-based decision pattern [14]. A behaviour states what the agent should do when it is in a certain state. For example, an agent that has no packet and is next to a packet, should pick it up. A behaviour change defines when the agent should change from one behaviour to another behaviour. An agent that is random walking and sees a new destination, should change to the behaviour "walking to a destination". In the state machine illustrated by figure 3 an overview of the different behaviour, behaviour change classes and their relations is given. On the state machine, a behaviour is illustrated as a state and a behaviour change as a transition. When satisfied, a behaviour change will cause a transition from the current behaviour to another behaviour.

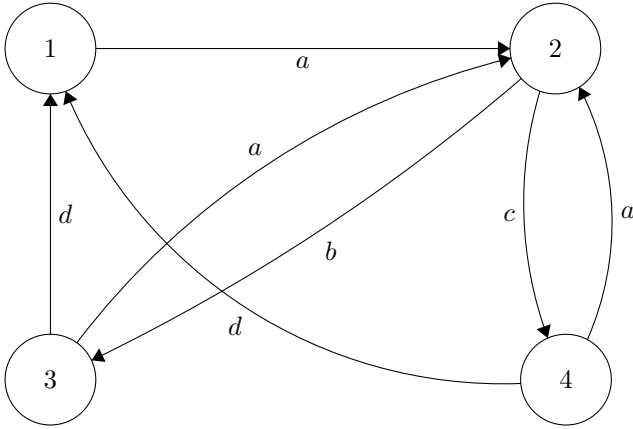


Fig. 3. State diagram for behaviour changes

a	MoveToDestinationChange	1	MoveRandom
b	ToPickPacketChange	2	MoveToDestination
c	ToPutPacketChange	3	PickPacket
d	MoveRandomChange	4	PutPacket

The behaviour of the agent is dependent on the current state of the agent. If the agent is not carrying a packet, and it has not seen one, it should look for one. This happens by scanning the entire perception, and then taking random steps (scanning only the newly added locations) until a packet is found. Once the agent has seen a packet, its destination is set to the packet's location. The agent must then take the shortest route towards it and pick it up. If the agent is carrying a packet, it should look for a drop-off point. This happens in the same manner as when looking for a packet. The predictability of the location of these drop-off points offers an opportunity for improvement.

When an agent has a packet but does not see a drop-off for this packet, it will walk randomly. Therefore a lot of time

is spent by randomly walking the grid looking for a drop-off point. For that reason, the implementation was changed to make it possible to save these in the agent's memory once they have been visited. An agent that picks up a packet of the same colour as a packet he previously delivered, will retrieve this destination from memory. This way, the agent can walk directly towards the packet's destination without having to resort to a random-walk-search. If the drop-off point has disappeared, the agent will detect this once he arrives and will search for a new one. Because these drop-off points only move very rarely, this is not considered a problem. However, in some edge-cases, this optimisation will lead to more energy-consuming behaviour. When multiple drop-off points of the same colour are present for example, the agent will move to the known one (which is potentially further away). However, this only occurs in a limited number of cases, and the improvement still results in an overall reduction of the energy consumption (measured in needed cycles). It remains important to take this into account when dealing with such environments nevertheless.

B. Energy-constrained agents

In the second solution, agents that have a limited battery life are used, and thus every once in a while they need to recharge at a charging station. It is not trivial to make sure agents do not "die" because of a lack of remaining battery power. Some encountered difficulties and how they were tackled are discussed in this subsection. The problem consists, as already mentioned, of two aspects: let agents behave efficiently and make them charge their batteries in time.

An important step towards more energy-efficient agents is making them communicate their saved drop-off points. This is done in addition to the process of discovering new ones, like described in subsection IV-A. Every cycle, each agent communicates its saved drop-off points to all other agents in its perception. If an agent receives info about a drop-off point location (with a certain colour) and for this colour the agent does not have a location stored yet, then it will store this information. The decision to store at most one location per drop-off point of a specific colour was made for the following reasons. First of all, it keeps things simple. Subsequently, the memory of the agent is not overused. Finally, this approach ensures some randomness in the points where the agents will drop off packets, so that they do not hinder each other too much. To conclude, by communicating the drop-off points agents will spend less time walking around randomly, looking for a drop-off point of a colour for which they did not yet deliver a packet.

Agents have to be charged in time and therefore have to easily locate a charging station. To make sure that agents do not wander around randomly when looking for a station, a gradient field is set up [15]. The number on a tile of the gradient field represents how many steps this tile is removed from the closest charging tile. This takes into account that an agent can only move through walkable tiles. An example of an environment with gradients is given by figure 4. This

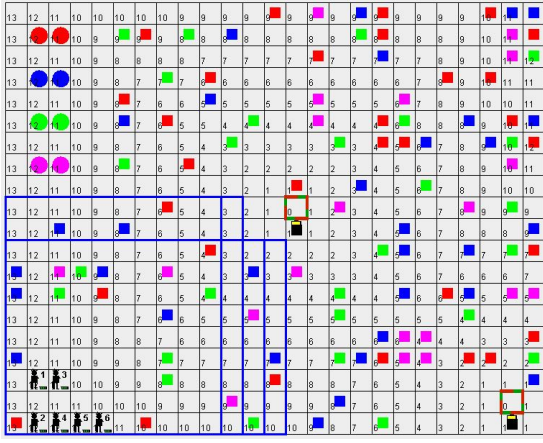


Fig. 4. Energy constrained environment with two charging stations and a gradient field represented in numbers.

gradient field is set up when initially placing the charging stations. Since the layout of the world will change over time, e.g. packets get moved, it is recalculated every time an agent charges. Initially, the first idea was to recalculate the gradient field every few cycles. Nevertheless it seemed that the framework of the Packet-World was not really suited for this, so, since time was limited, this perhaps somewhat more kludgy solution, that nonetheless gets the job done, was opted for. In the environments used for this extension, the number of charging stations is limited compared to the number of agents. When an agent arrives at a charging station, it is therefore not unusual that it is already occupied. In that case an agent waits for the charging station to become free. While waiting, an agent should waste as little energy as possible and starvation should be prevented. A few mechanisms have been implemented to handle these aspects and thus to avoid the agents from "dying" when waiting. Firstly, each agent has a lower bound for his energy level and it will start moving to the charging station if its battery goes below this bound. This lower bound is highly dependent on the gradient of the agents current position, i.e. the number of steps needed to reach the charging station. Besides, an additional energy amount is taken into account, to be able to account for delay when moving to the station. When another agent is on the shortest path for example, this agent has to move around it and will consume more energy than planned. This amount is also a bit randomised to spread out the charging of agents over time. Of course, agents wanting to charge at the same time will still occur since the approach is, for one, randomised and, secondly, agents are not fully charged instantaneously. In the case that multiple agents want to charge at the same time, one will go first and the others will wait right before the charging point. It is important to notice that they skip when waiting, since skipping a turn costs half the energy of moving without carry. This is also the reason why an agent will put its packet down next to it before walking towards the station. To further prevent an agent from "dying" in the charging queue,

it will communicate to the agents in its perception when its energy level is critical. This critical bound was chosen rather arbitrarily and an optimal value for this might be the subject of future work. If the agent currently charging receives such a message and itself is not at critical battery level, it will leave the charging station and will wait in the charging queue. An agent will always try to fully charge itself, before proceeding with the completion of its tasks. The last pitfall is that when the charging agent leaves its place, another agent takes the charging position before the requesting agent. Therefore the requesting agent sends the message also to the other agents in its perception. The other agents will when receiving this message only step into the charging area when their battery level is also critical. This causes the wanted behaviour and performs very well.

C. Obstacle-filled environment

This last subsection describes a solution for the problem described in section III-C, namely environments wherein there are walls and packets that potentially block the path to destinations. In addition, in this type of environment a new type of agent is introduced, namely "coloured" agents. Coloured agents can only pick up packets of their own colour. For these environments, the energy constraints are ignored. The solution that will be described is extending the autonomous agents without energy constraints (see section IV-A). The first challenge is to enable agents with the capability to overcome some obstacles, for example moving around walls, given their local view. An intelligent path-finding algorithm makes it possible to move around walls and to detect when packets are obstructing the path. Then subsequently other agents can be asked for help if needed. Lastly this algorithm prevents agents from getting stuck when the heuristic approach does not work. The algorithm used to accomplish this task is based on Dijkstra's algorithm [12] and uses a few extra steps to meet all the requirements mentioned above.

When an agent wants to reach a destination, it will first execute a "normal" execution of the Dijkstra algorithm in its perception. If it is able to form a walkable path to its destination, it will take that path. However, it might be that there are no walkable paths towards the destination, but that there are valid paths with some packets on. In these cases, the algorithm is run again but with a little twist: packets are not considered an obstacle. Nevertheless, an agent cannot move through packets. So the agent will locate the first packet on the path and will remove it from the path. The agent will do this either by displacing the packet itself, if it is able to, i.e. the packet and the agent have the same colour, or by asking for help as described in the second challenge. If both approaches fail to find a path, then it means that no direct path is present within the agent's perception. In that case, the agent will use a heuristic approach. It will move to the walkable cell in its perception that has the lowest Manhattan distance to its real destination. Unfortunately, in some border cases, this causes an agent to become stuck. Figure 5 shows a situation where the red agent is stuck. It wants to deliver a packet, but is stuck

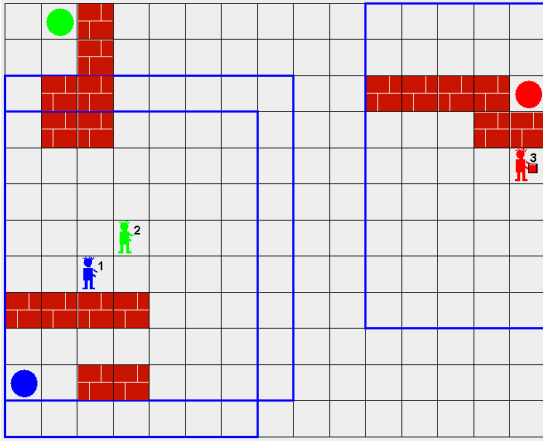


Fig. 5. Red agent is stuck because the agent is on the optimal tile according to the heuristic.

at the reachable position within its perception nearest to the red drop-off point.

Because of this, the agents employ a "stuck detection" mechanism. It detects when the agent tries to walk to a destination but does not actually move during several cycles, and is probably stuck. When the detection mechanism thinks the agent is stuck, the agent will start to walk randomly for some cycles. This approach makes it possible for the agent to move its perception and find a better path.

The second challenge is to coordinate the assistance between different agents. If an agent detects its path to be blocked by a packet with another colour, it should ask for help. The agent will construct a help message with the necessary information, such as the colour and the absolute position of the packet. If the agent has help requests from other agents in its help queue in memory, it will try to go pick the packet associated with the first help request of the queue. The agent will keep sending the request for help to all the agents in its perception until an agent sends an acknowledge message to the agent requesting help. By sending an acknowledge message that agent commits to moving that packet out of the way somewhere in the future. Therefore, an agent will only acknowledge a request for help if the packet for which help is requested is of that agent's colour. The acknowledging agent will store the location of the packet for which help is requested at the back of its queue. This means that the help queue is FIFO (first-in, first-out). If the agent's help queue is empty it will keep moving around randomly until another agent asks this one for help or until another agent acknowledges the agent's request for help. If an agent is carrying a packet and notices that the path towards its destination is blocked, then the agent makes sure it drops the packet out of the path to its destination and performs the steps described above, in the solution of the first challenge.

Finally, a last problem that needed to be tackled was deadlock detection. Because of the small "passages" in the environment, it is possible that two agents block each other's

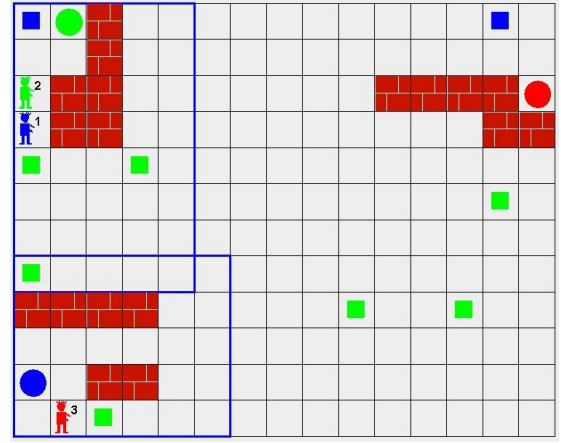


Fig. 6. The green agent wants to move down and the blue agent wants to move up, which results in a deadlock.

path as illustrated by figure 6. Without any kind of deadlock detection here, the agents would perpetually remain stuck. A deadlock resolution strategy was used here to solve this: an agent assumes it is in a deadlock when it has not moved for several cycles, using the same approach as the path-finding algorithm. If this is the case, both agents will take a random number of steps (between one and six), in order to clear the passage.

V. DISCUSSION OF THE RESULTS

This paper discusses how a multi-agent system consisting of autonomous agents has been implemented. Each agent in this system perceives its environment and acts accordingly to its current behaviour. As discussed earlier, the implementation of this first approach has been done in three phases. In the first phase all behaviour was concentrated into one single class. In the second phase, the behaviour has been split up into behaviour classes and behaviour change classes. Finally in the third phase, this implementation has been optimised, by saving the drop-off point in the agent's memory once it has visited it. Figure 7 illustrates the results of this evolution. The histogram shows for the basic environment of figure 1 the minimum, maximum and average number of cycles needed for the agents to deliver all packets. The experiment has been repeated ten times for the single class structure on the one hand, and the structured approach with the optimisation on the other hand. The structured approach without optimisation has not been shown, because this was mainly a change in the way the code was structured and thus has no noticeable impact on the performance of the system. The obtained results after the optimisation, show that little changes to the system can already have a large impact on the performance. In this case, by saving the drop-off point location, the number of cycles needed to deliver all packets has more than halved.

Agents are independent entities that only take into account their goals and perceptions. Having no communication between agents limits the agents to what they can perceive. Adding this aspect to the system can improve the performance

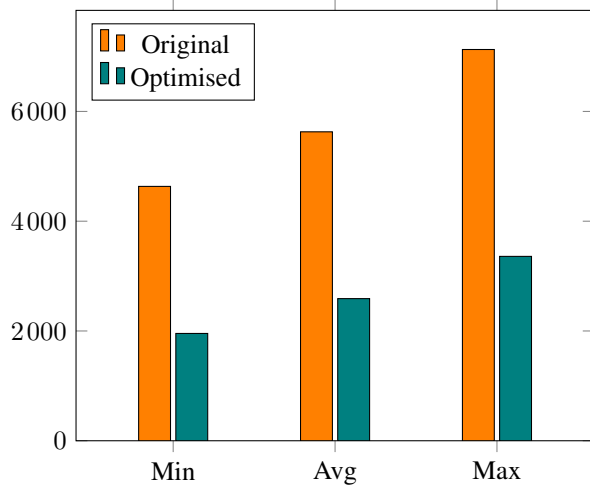


Fig. 7. Comparison of number of cycles necessary to deliver 60 packets. The experiment has been done on the basic environment over 10 runs.

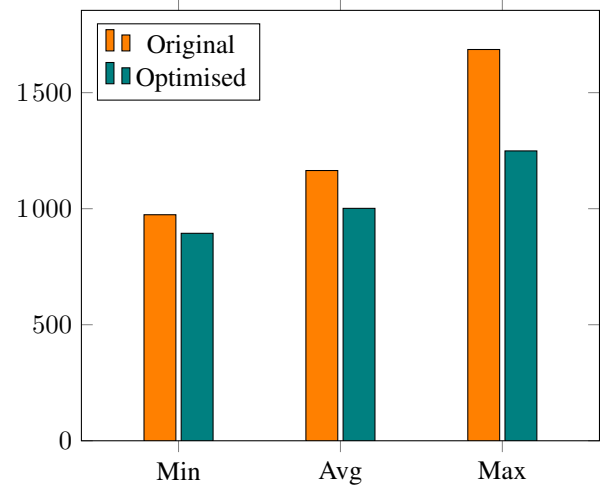


Fig. 8. Comparison of number of cycles necessary to deliver 110 packets. The experiment has been done on the energy constrained environment over 100 runs.

of the system and make the system able to handle more complicated environments. This is explored in the two extensions: energy-constrained agents and obstacle-filled environments.

The first extension handled energy-constrained agents. An approach using a combination of direct and indirect communication was chosen. By emitting gradient fields the charging stations could guide the agents to them. The gradient fields propagate only through walkable cells. This was not a very complicated implementation, but by ensuring this, the agents always easily find the shortest path to a charging station. Randomising the lower bound for the battery level also has a noticeable impact as the charging of the agents happens more evenly spread over time. When arriving at a charging station, the agent uses direct communication for coordination. Using a critical battery level and making the charging agent able to leave his place, has completely canceled the situations where agents "died" while waiting in the used environments in the experiments. Besides, the direct communication also offers some opportunity for energy saving. Figure 8 illustrates the obtained improvement when agents communicate the drop-off points they saved in memory. This results in less cycles needed looking for the drop-off point. The optimised solution averages to 1,001.52 cycles, which is a decrease of almost 14% in comparison to the non-optimized average. Further, on average, these agents use 67,883.52 energy points to fulfill their tasks. To conclude, this design has some limitations. Firstly, the agents always try charging until they are completely full. This could be highly inefficient when many agents would want to charge at the same time. This would generate big waiting queues at the charging station. Secondly, environments where the charging station would be inaccessible at the beginning, would introduce new problems.

Finally, the autonomous agents were extended to work in an environment filled with obstacles. They also were restricted to picking and putting packets of one specific colour. The path-finding algorithm based on Dijkstra worked well for most of

the situations. In particular when the agent's perception is not too small. In subsection III-C however, a situation is discussed where the heuristic approach ended up in the agent getting stuck. This is nevertheless fixed by using a stuck detection algorithm. The path-finding algorithm does not work well in all environments. An example is an environment with many walls and where on top of that the agent's perception is very restricted, e.g. a maze. In that case, the algorithm would need to use the heuristic approach most of the time, which would lead to inefficient behaviour and potentially also to loops. Next to that, agents use direct communication to delegate tasks. This enables the presented solution to operate in the obstacle-filled environment without getting stuck in a deadlock, in contrast to the basic solution that cannot deal with obstacles. To evaluate the results empirically, 100 runs have been executed on the obstacle-filled environment shown in figure 2. The resulting distribution of needed cycles is illustrated in figure 9. This amounted to an average needed cycles of 599.89 and an average energy spent of 20,677.35. However, the biggest challenge here was not the efficiency, but guaranteeing that the task could be accomplished without getting stuck in a deadlock. On the figure a right-tail can clearly be seen. This indicates the situations where some agent temporarily loops or gets stuck. This has in most cases a drastic impact on the resulting cycles.

During the design of the solutions, some design patterns were used/identified. A brief overview of this patterns will be discussed to conclude this discussion. First of all, the "Behaviour Based Decision"-pattern [14] was used to divide one big class into different behaviour classes. This structure improved greatly the readability and understandability of the code. Secondly, gradient fields [15] were used to enable efficient routing to the charging stations. Thirdly, the agents make intensively use of their own memory and thus can be classified as persistence agents [16]. Finally, in the solution

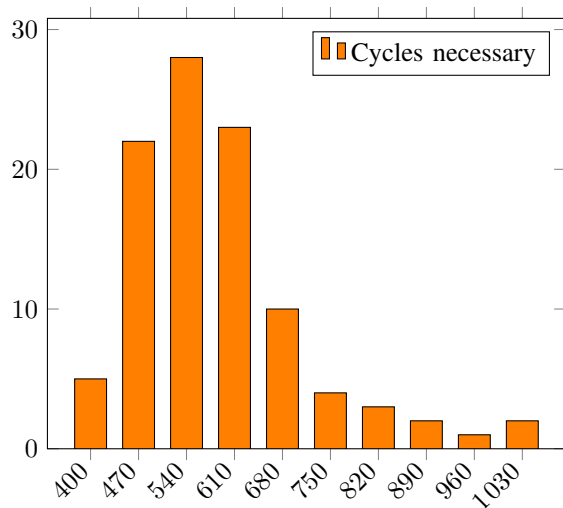


Fig. 9. Comparison of number of cycles necessary to deliver all packets in the obstacle-filled environment, split into deciles. Tested over 100 runs.

for dealing with obstacles, agents have to decompose the task of moving to a destination. If some differently coloured packet obstruct the chosen path, then the agent first has to remove it or ask help. This maps loosely to the "Task Decomposition"-pattern as described in [17].

VI. DISCUSSION OF RELATED WORK

In this section, some related papers are discussed. The approaches differ on the problem domain or on the solution domain, from the approach explained in the paper. The limitations of an agent can be set differently for example. The agents in the Packet-World are situated agents. Moreover, like Ferber and Müller [18] discuss, the agents that are used in the described solution are hysteretic agents. The agents have a memory and use this (in combination with sensing) to make decisions, in contrast to tropistic agents, who make decisions only based on what they sense in the world.

Eck and Soh [3] discuss a world similar to the Packet-World. In their work not only the reasoning and acting of the agent are dependent of the energy level, but also the perception of an agent. They discuss a mining robot with a microscope, from which the measurement accuracy is dependent of the current battery level of the agent. This differs from the Packet-World, where agents have a constant perception range and accuracy.

In their paper about Anthill, the authors Babaoglu, Meling and Montresor [4] present Anthill as a framework for agent-based peer-to-peer systems. They describe the agents as being autonomous and as acting depending on what they perceive. Their actions may modify the environment and also lead to changes in their location. This seems very similar to the approach taken in this paper. However, they make use of evolutionary computing techniques to let their agent system evolve. This has an advantage over our approach, because the agents will be able to adapt their behaviour continuously to increase performance.

Bromuri and Stathis [5] elaborated on the same Packet-World where they worked with GOLEM and cognitive agents. They describe the agents as a composite structure that defines the interaction between agents and objects. In this way they can specify the perception of the agents in a declarative and logic-based way. They state the GOLEM principles used to do this are general and reusable.

In the Packet-World agents all have the same task and are actively moving around, namely picking packets and deliver them. In their paper, Hallenborg and Demazeau [6] describe a baggage handling system in an airport. This system consists of different types of agents (Toploaders, dischargers, the route agent...) with more diverse tasks. Another interesting difference is that in their case the agents have a fixed position, but the objects that have to be handled do move.

Abusnaina and Yong [7] discuss an implementation for the Packet-World where agents work in two distinct levels: a discovering level and a working level. They make furthermore use of a global communication channel, namely Blackboard. The agents start in the discovery state and look for packets, and do this until all environment positions are visited. They store the information about the packet in a communication channel named Blackboard. When that stage is done, the agents will begin the cleaning process. They pull information about "unremoved" packets and retrieve them. This is a different approach to the same problem. In the presented implementation, the agents will constantly change from state and can be in any state at any time. There is also no global message communication possible, in contrast with their use of Blackboard. Agents can only communicate directly with agents within their perception.

Agents are not always collaborative as in the packet-world. The PIRASA framework [8] consists of seller agents and buyer agents. These agents have to negotiate with each other to sell and buy products. In that case, agents are not necessarily collaborative and therefore communication and cooperation have to be handled differently.

Gradients were used to ease the finding of the charging stations for the agents. However, the other interactions between the agents were mainly direct. Nonetheless, many indirect communication and cooperation mechanisms exist. Gorodetskii [10] discusses some of them, e.g. gradients and pheromones, by using examples from biological systems with these mechanisms. A gradient approach for robots, similar to the discussed approach with the charging stations is given by Steels in [20]. There coordination is needed to make the robots follow the vehicle.

Lastly, the concrete implementation of the multi-agent system has been done entirely in Java. Thereby the structure based on behaviours and behaviour changes was used. These classes embedded both actions and communication. Other approaches to design and implement the system are possible. In their paper, Toledo et al., discuss a platform for developing multi-agent systems, named JaCaMo [9]. It uses distinct parts to specify communication and actions. The communication and relations between agents is specified using the Moise model, while the actions are implemented through the Jason MAS

platform. This ends up in three levels of abstractions: agent, organisation and environment.

VII. PRESENT CONCLUSIONS AND FUTURE WORK

In this paper, some possible design challenges in a MAS are addressed, more specifically agents with energy constraints and environments containing obstacles. To implement, test and evaluate the solutions, the Packet-World was used. Besides, two real-world problems mapping onto the Packet-World were briefly discussed: trash collection and rescue operations. These examples mapped very well onto the Packet-World and therefore show the relevance of exploring different techniques and environments in this experimental environment, to gain a better understanding of situated agents and their cooperation. Next, the authors proposed three different behaviours for three different environments in the Packet-World, each with specific challenges. All these problems required a different approach making sure the goal of delivering all packets would be accomplished. The first, most basic behaviour only requires autonomous agents that make decisions based on the information they perceive from their environment. The performance of the agents can be optimized by utilizing their memory and taking this into account when making decisions. The results of this optimisation gives rise to two interesting observations. On the one hand, adding memory to agents can enable a lot of optimisation. On the other hand, it is also clear that little well-chosen changes, like storing only the drop-off points, can have high performance impacts. Furthermore, the use of different behaviour classes shows to be of great value, to be able to extend the basic behaviours without loosing the overview. Next, when the environment becomes more advanced, some form of coordination is needed. Direct communication was used in order to facilitate collaboration between agents and to exchange useful information. Furthermore a form of indirect communication (gradient fields) was implemented to indicate the charging stations and preventing agents to run out of energy. The basic approach served like a base for the extensions. However, this basic approach does not fit for all environments. Some conclusions can be drawn from implementing the two extensions. First, the approach for the energy-constrained agents with gradient fields, works very well. Gradient fields made it possible to efficiently route to charging stations. Also task delegation between agents showed to be powerful, because more complex environments could be solved with this mechanism, as is discussed in subsection IV-C. Finally, while testing the different solutions, sometimes agents behaved highly inefficient (all wanting to charge at the same time) or even get stuck as discussed in subsections IV-B and IV-C. During the design and implementation of the system, introducing randomness at well-chosen places was found to improve the overall efficiency of the system, and in this manner certain sub-optimal system states were avoided. The approach of adding randomness can be seen in the "stuck detection" mechanism and also in fixing the lower bound of an agents battery level (the additional energy amount is randomized).

In the future, first some work could potentially be done to make the path-finding algorithm more robust to environments with a very restricted perception (e.g. a maze). In the discussion of the energy-constrained agents, it was pinpointed that they would try to charge fully. Investigating another approach to charging could lead to more robust and efficient solutions. Charging until a high benchmark could be a possible approach. Next, it would be interesting to explore other aspects of indirect communication, like flags and pheromones, and their influence on the performance. A combination of different types of environments would probably also require a more advanced collaboration. Combining the challenges of the energy constrained-world and the obstacle-filled problem could result in for example an environment with an energy station that is blocked by obstacles. Except for improvements on the autonomous behaviour, more complex interactions could be explored as well. This can, for example, include an environment where some packets can only be carried by two agents at the same time or where agents forward messages to get a message faster at its destination. Finally, communication in the energy-constrained environments had no cost for the moment. Future research could investigate the effect on performance of giving communication a certain cost. This would also limit the rate at which messages would be sent, and therefore take this other challenges.

REFERENCES

- [1] M. Woolridge, *An Introduction to MultiAgent Systems*, 2nd edition. Wiley (2009), pp. 3-16.
- [2] D. Weyns, A. Helleboogh, and T. Holvoet, *The Packet-World: A Test Bed for Investigating Situated Multi-agent Systems*. Software Agent-Based Applications, Platforms, and Development Kits, Birkhauser Verlag, Basel - Boston - Berlin (2005), pp. 383-408.
- [3] A. Eck, L.K. Soh, *Observer effect from stateful resources in agent sensing*. Auton Agent Multi-Agent Syst 26, pp. 202-244 (2013). <https://doi.org/10.1007/s10458-011-9189-y>
- [4] O. Babaoglu, H. Meling, and A. Montresor, *Anthill: a framework for the development of agent-based peer-to-peer systems*. Proceedings 22nd International Conference on Distributed Computing Systems, Vienna, Austria, 2002, pp. 15-22, doi: 10.1109/ICDCS.2002.1022238.
- [5] S. Bromuri, K. Stathis, *Situating Cognitive Agents in GOLEM* In: Weyns D., Brueckner S.A., Demazeau Y. (eds) Engineering Environment-Mediated Multi-Agent Systems. EEMMAS 2007. Lecture Notes in Computer Science, vol 5049. Springer, Berlin, Heidelberg.
- [6] K. Hallenborg, Y. Demazeau, *Applying Multi-agent design and decision logic to a Baggage Handling system* In: Weyns D., Brueckner S.A., Demazeau Y. (eds) Engineering Environment-Mediated Multi-Agent Systems. EEMMAS 2007. Lecture Notes in Computer Science, vol 5049. Springer, Berlin, Heidelberg.
- [7] A. A. A. Abusnaina and Chan Huah Yong, *Cooperative Multi-Agent System for solving Packet World Problem in Grid*. 2009 IEEE 9th Malaysia International Conference on Communications (MICC), 2009, pp. 754-758, doi: 10.1109/MICC.2009.5431395.
- [8] J. Hopkins, Ö. Kafali, B. Alrayes et al. *Pirasa: strategic protocol selection for e-commerce agents*. Electron Markets 29, 239-252 (2019). <https://doi.org/10.1007/s12525-018-0307-4>
- [9] C.M. Toledo, R.H. Bordini, O. Chiotti, M.R. Galli (2012) *Developing a Knowledge Management Multi-Agent System Using JaCaMo*. In: Dennis L., Boissier O., Bordini R.H. (eds) Programming Multi-Agent Systems. ProMAS 2011. Lecture Notes in Computer Science, vol 7217. Springer, Berlin, Heidelberg.
- [10] V.I. Gorodetskii, *Self-organization and multiagent systems: I. Models of multiagent self-organization*. J. Comput. Syst. Sci. Int. 51, pp. 256-281 (2012). <https://doi.org/10.1134/S106423071201008X>

- [11] Bor, Martin and Vidler, John Edward and Roedig, Utz *LoRa for the Internet of Things*. EWSN '16 Proceedings of the 2016 International Conference on Embedded Wireless Systems and Networks. Junction Publishing, AUT, pp. 361-366 (2016). ISBN 9780994988607
- [12] Wang, Huijuan, Yuan Yu, and Quanbo Yuan. *Application of Dijkstra algorithm in robot path-planning*. 2011 second international conference on mechanic automation and control engineering. IEEE, 2011.
- [13] H. Li, W. He and X. He, *Prediction of Radio Wave Propagation Loss in ultra-Rugged Terrain Areas*. in IEEE Transactions on Antennas and Propagation, doi: 10.1109/TAP.2020.3045508.
- [14] K. Schelfhout, et al. *Agent Implementation Patterns*. Proceedings of the OOPSLA 2002 Workshop on Agent-Oriented Methodologies, 2002, pp. 119–130.
- [15] T. De Wolf, T. Holvoet (2007) *Design Patterns for Decentralised Coordination in Self-organising Emergent Systems*. In: Brueckner S.A., Hassas S., Jelasity M., Yamins D. (eds) Engineering Self-Organising Systems. ESOA 2006. Lecture Notes in Computer Science, vol 4335. Springer, Berlin, Heidelberg.
- [16] A. Chella, M. Cossentino, S. Gaglio, L. Sabatucci, V. Seidita, *Agent-oriented software patterns for rapid and affordable robot programming*. Journal of Systems and Software, Volume 83, Issue 4, 2010, pp. 557-573, ISSN 0164-1212, <https://doi.org/10.1016/j.jss.2009.10.035>.
- [17] S. Shu, D. H. Norrie (1999) *Patterns for Adaptive Multi-Agent Systems in Intelligent Manufacturing*. Proceedings of the 2nd International Workshop on Intelligent Manufacturing Systems, 1999, pp. 67-74.
- [18] J. Ferber and J.-P. Müller, *Influences and reactions: A model of situated multiagent systems*. Proceedings of the 2nd International Conference on Multi-Agent Systems (ICMAS-96), 1996, pp. 72–79.
- [19] D. Weyns and T. Holvoet, *A Formal Model for Situated Multi-Agent Systems*. Fundamenta Informaticae XXI (2001), pp. 1001–1034.
- [20] L. Steels, *Cooperation between distributed agents through self-organization*. First European Workshop on Modeling Autonomous Agents in a Multi-Agent World, Elsevier Science Publishers, Holland (1990), pp. 175-196.
- [21] D. Weyns, E. Steegmans and T. Holvoet, *Protocol-based communication for situated multi-agent systems*. Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems, 2004, AAMAS 2004, pp. 118-125.
- [22] D. Weyns, E. Steegmans and T. Holvoet, *Towards active perception in situated multi-agent systems*. Applied Artificial Intelligence, 2004, volume 18, number 9-10, pp. 867-883
- [23] N. R. Jennings, *Commitments and Conventions: The Foundation of Coordination in Multi-Agent Systems*. The Knowledge Engineering Review, 1993, volume 8, number 3, pp. 223-250.

A comparison of environment-optimised algorithms for situated multi-agent systems in Packet World

1st Pieter Claeys
Department of Computer Science
KU Leuven
Leuven, Belgium
pieter.claeys@student.kuleuven.be

2nd Mathijs Hubrechtsen
Department of Computer Science
KU Leuven
Leuven, Belgium
mathijs.hubrechtsen@student.kuleuven.be

3th Seppe Plomteux
Departement of Computer Science
KU Leuven
Leuven, Belgium
seppe.plomteux@student.kuleuven.be

4rd Michiel Provoost
Departement of Computer Science
KU Leuven
Leuven, Belgium
michiel.provoost1@student.kuleuven.be

Abstract—In this paper, the authors will develop several environment-optimised algorithms for multi-agent systems. The algorithms will be tested in the Packet World test bed, which is a simulator for situated multi-agent systems. The performance of these different algorithms will then be compared in terms of number of moves required as well as total energy consumed. As the test scenarios in the Packet World increase in complexity, more complex algorithms are designed for overcoming these more challenging environments. The authors then apply these more complex algorithms on simpler environments, to find out whether the benefits added by these more advanced algorithms are also transferable to less complex environments. After running these experiments, we observed that the benefits in adding more optimizations to the algorithms decreases as the complexity of these optimizations increases. This highlights the importance of a well designed base architecture that is adaptable to different environments instead of excessive optimization in specific environments.

I. INTRODUCTION

The key idea of this paper is finding ways to maneuver situated agents in a simulated environment known as the Packet World. In section II we shall explain exactly what this packet world is, and what are the main characteristics of agents, destinations and packets in this world. Then in section III we shall explore how the virtual environment relates to a real world scenario. Then we shall explore the first and most basic problem setting in section IV and its solution in section V. Then we will start expanding the problem setting in the next sections, with an energy limitation in section VI and a requirement for cooperation in section VIII. In those respective section we will also discuss the solution proposed and their result and impact on the expanded and simpler problem settings. A final expansion is then addressed in section VII where we expand the energy limitation of section VI to have include hallways, allowing agents the block portions of the map by their presence. In the discussion section, section IX, we will summarise and discuss the findings of our research endeavour, placing it into a research context in section X. In the final section, we will focus on lessons learned

from our research and what conclusions can be drawn from these. Special attention will go to limitations of our research compared to overall concerns in MAS research.

II. PACKET WORLD: BASIC CONCEPTS

The Packet World, originally described by D. Weyns, A. Helleboogh and T. Holvoet [1], is a situated MAS, meaning it consists of a set of agents acting in an environment. Additionally, each agent can be associated with a specific location where it resides for any given point in time. The packet world consists of a few key building blocks; the environment, agents, packets and destinations.

A. The environment

The environment is a rectangular regular grid that contains all other actors, objects and processes part of the Packet World. A single cell is the smallest subdivision of the environment in sub-spaces, there can be no such actor, object or process that affects or occupies only part of a single cell. The environment is organized in layers, where each layer contains entities of a single class. Examples of layers are: the agent layer, the packet layer, etc. This design allows for easy customization of the environment, as new layers with diverse effects can easily be added.

B. Agents

Agents occupy a cell in the environment and can move to adjacent cells. No two agents can be present in the same cell at the same time. Agents can communicate with other nearby agents, allowing them to cooperate and share their knowledge. However, moving costs energy. Moving with a packet is more expensive than moving without one. Standing still also requires a small amount of energy. Communication does not incur an energy cost. Agents can see a limited, square, subset of the environment, centered around the agent itself. The knowledge received from other nearby actors, combined with other input and internal state, are used by the agent to independently decide the next course of action.

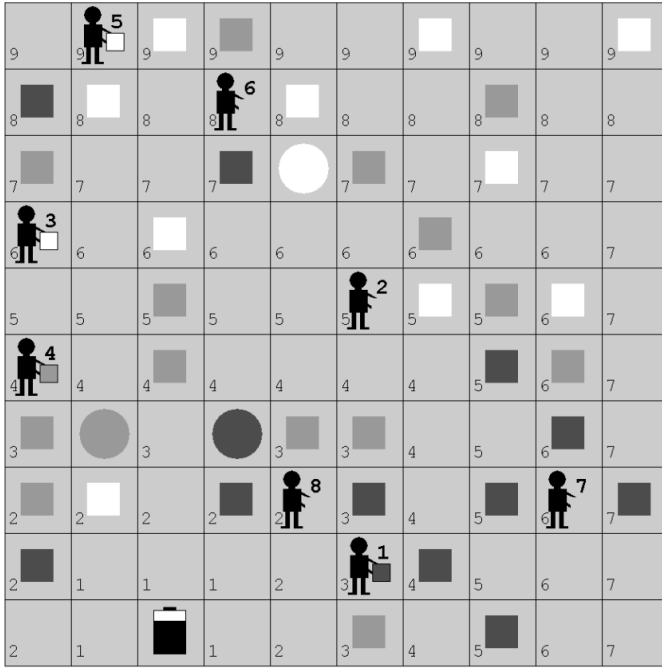


Fig. 1. A graphical representation of the Packet World

C. Packets

Packets are situated in the environment and cannot move on their own. A packet and an agent can only occupy the same cell if the agent is currently carrying the packet. An agent can only carry one packet at a time. Packets can only be picked up by agents in adjacent cells. Packets are color coded and should be deposited in a destination of equal color.

D. Destinations

Akin to agents and packets, destinations occupy a cell in the environment. Just like packets, they are color coded. Only packets with a matching color can be deposited in a certain destination. Destinations cannot be moved by agents, they can only appear and disappear through interference from outside the environment.

The goal of the agents is to deliver packets to their corresponding destination, as quickly and efficiently as possible. To measure the agent efficiency, two counters continuously keep track of the energy spent while moving and communicating. See figure 1 for a visual representation of the Packet World.

III. REAL WORLD PROBLEMS

A. Busy Restaurant

A first real world problem example that maps to the Packet World would be a busy restaurant. In the restaurant, the waiters and waitresses are the active agents and orders for a table correspond to packets. These orders have a destination (the table or, if empty, the kitchen) and can also be transferred to other agents (“dropped”). Since human beings have physical energy limitations, restaurants have spaces where their waiters can catch their breath before continuing on their task. Their

views are also limited, especially by obstacles, which limits their view range to nearby tables. A good waiter/waitress will try to optimize the amount of orders they can successfully deliver in a night (in order to gain more tips), mapping to the same goal as the packet world.

B. Naval Battle

A second problem to which the Packet World scheme is applicable is naval battles. Ships have limited energy for movement (fuel), although this is not as much the case for communication. They must carry missiles (the packets) to optimal launch positions (destinations). What is new to the environment is the fact that we also have limited knowledge of the destination, as any destination is approximate (enemy ships change position, the position is tactically less optimal than estimated). Thus making it possible for an agent’s destination to change. This adds a new dimension to the adaptability of the solution. Ships also have limited view on the underwater condition (submarines, reefs, sandbanks, ...) and will need to scan their neighbourhood and communicate terrain updates to other ships. The agent can also react differently when in enemy waters, as communication might give away their position and thus becomes scarce again. [2]

C. Autonomously Driving Taxis

A third real world example to which the Packet World can be applied is a network of autonomously driving taxis. [3] All taxis need to pick up customers and drive them to their destination, as fast as possible. These autonomous vehicles do need to share the road with other agents. Other agents include pedestrians, manually driven cars and other taxis. Minimal communication is possible between all these agents by use of for example turning signals. All agents need to follow a set of complicated traffic rules to avoid the need for more complex communication. Taxis, as all cars, have a limited range and need fuel every now and then, either as electricity or petrol. A taxi has an unlimited view of the structure of the environment by use of GPS and a limited view of other nearby agents, both unpredictable third parties and other taxis. Autonomous taxis do not directly map to the basic Packet World, as there are a lot of complicated rules and processes involved. Most notably, third party agents that do not necessarily follow all rules (people in a hurry may run a red light, or overtake someone else where that is forbidden). It may be interesting to figure out whether or not the Packet World can be easily extended to study this specific use case.

IV. PROBLEM STATEMENT

The first part of the problem is the autonomous behaviour of agents. This means that agents have to be able to act on their own without supervision. They should be able to adapt to their surroundings and make intelligent decisions on their own. In the packet world, agent behavior should be specified initially to find and retrieve packets taking into account their energy supply.

The second part of the problem is centered around energy usage; optimally using and recharging agents' batteries. Agents should be able to independently find charging station and move towards them when low on energy. Also, it's important that in a world with limited charging stations, agents communicate amongst each other to determine priority in who can use the charging station at a given time.

Finally, the third part of the problem expands on the communication between agents already touched on in the first two parts. In order to achieve an efficient system, agents should properly delegate tasks amongst each other. This can be found in charging behavior, coordination for packet colors and communicating the most efficient ways to traverse a hallway and by expansion (though not treated in depth in this work) a maze.

V. SIMPLE ENVIRONMENTS WITH LIMITED PERCEPTION

A. Environment outline

These environments are the playground of more basic algorithms. They are of little relevance to real-world problems, but give us a steady benchmark. These environments have no walls and there is even a world where an agent can perceive everything in the environment.

B. Design

For these simple environments, a few designs were created, we will go over them one by one:

1) *Wander*: The initial design is based on random choice. The behaviour is random until a packet or destination is right next to an agent, and thus accessible directly. The agent only has a limited amount out of responses: if it is holding a packet and adjacent to a destination, it will drop the packet on the destination. If it is not carrying a packet but it is adjacent to one, it will pick up the packet and in all other cases it will just take a random step.

2) *MAPE-K*: This design is based on the MAPE-K design [4]. MAPE-K consists of five components: Monitor, Analyzer, Planner, Executer, and Knowledge. The design allows for abstract treatment of the different aspects of the agent behaviour: the "Knowledge" component represents knowledge of the location of a packet, as well as a destination. The "Monitor" only has to see and report what it sees. The "Analyser" has to determine if what the agent sees is useful and the "Planner" has to decide what to do with that information.

The "Planner" has only a few states to consider: (a) agent is holding a packet or (b) agent is not holding a packet. In case of (a), the agent will either: (a.a) be near a destination, (a.b) have a destination in perception range, or (a.c) have no destination in perception range. In case of (a.a) the agent should drop off its packet, in case of (a.b) the agent should move nearer to the destination, and in case of (a.c) should perform a random step. If the agent finds itself in state (b) its primary objective would be to enter state (a). To do this, it either: (b.a) sees a packet or (b.b) doesn't see a packet. In (b.a) The action is simple: move nearer to said packet. In (b.b) we take a random step, as this behaviour makes no intelligent guesses.

3) *SmartWander*: Our next optimization scheme is to use a direction wander algorithm instead of a step wander algorithm. The difference between the two is that the direction algorithm chooses a random direction to walk along, in a straight line, until an obstacle is reached. The original step algorithm however, made a random decision every single step.

4) *original SmarterWander*: The last optimization scheme considered in simple environments, smarterWander, will be introduced here in a more primitive form. This scheme was later expanded upon to take energy usage into account, but much of its original behaviour and structure has remained. The main expansion here compared to the former optimization schemes was the fact that packets and destinations were saved in memory. A middleware was developed to support the encoding of packets and destination into single strings, while keeping a maximum capacity. This capacity was set to 100 packets and 30 destinations. These strings were then encoded in memory fragments and destinations were also transmitted between agents. The previous agent behaviour could then be extended so that it moved to the closest packet (or destination respectively) in memory if none were in perception range. In addition, the last seen packet is stored first. This organises it so that agents forget packets that were seen further into the past (if packet storage capacity is relevant), heuristically storing the packets closest to the agent. When looking for a packet (or destination) the closest packet (destination respectively) is always searched in the stored packets (or destinations respectively). In order to speed up delivery of packets, destinations (since they are static) are shared via communication between agents.

C. Implementation

The initial design is implemented in a single class named *Wander.java*, this class extends *LTDBehaviour*. Its *act(AgentImp agent)* function implements the straightforward behavior as stated in the design. It uses *agent.getPerception()* to receive the information of its environment such as other nearby agents, packets and destinations.

The first expansion is structured as explained in the design section. The only implementation details are 2 functions that check the perception of the agent and returns the nearest packer or destination respectively.

The third expansion kept the overall structure of the MAPE-K design, but had a different random wander function. The wander function has to remember the direction it is heading in and head into a new direction when it can no longer move in the current one. To be able to keep this direction, memory usage was increased, as the agent is stateless apart from its memory. This is also realistic as a real system would also have to keep a place in memory for the direction in which it's heading.

For the last expansion the behaviour grew too complex, we expanded into dedicated behavior classes: *FindDestinationBehaviour.java*, *FindPacketBehaviour.java*, and *RandomWalkBehaviour.java*. An agent uses these classes as states that specify its next actions. When a trigger occurs for a state change (such

as a packet was found), the corresponding behaviour change class will be used to load the class of the next behaviour. This in turn is not very compatible with the MAPE-K design and this design was therefore abandoned for the rest of the expansions. Storing the location of packets in memory does impose a cost: the memory could be incomplete as packets could be placed or removed. To combat this we check every time a packet in memory should be in perception range if it is still there, then adapt our memory accordingly.

Also of particular note is the serialization used to communicate between agents and store information. This is a string serialization of a list of destinations. As mentioned earlier serialization was also used on the actual agents themselves to compress the coordinates of location to a single memory fragment. This does not reduce the realism of the scenario, as in a real-world scenario, any MAS electronic device fitted with a memory would have the capacity to store a large amount of primitives such as strings (or perhaps character arrays). To make sure that this technique does not allow for exploitation and misuse infringing on the applicability of the solution, a limit is set to the amount of objects stored per fragment. This still allows for easy customization and makes it easy to explicitly reason about memory constraints. The solution is also designed to take into account a finite memory size, but this will then of course decrease efficiency as you decrease the memory size.

Another peculiarity is the fact that all `act()` methods are changed to make a more general random behaviour. This allows an agent to either: (a) do its calculated best behaviour or (b) take a step in a random direction. As we still have incomplete knowledge, this could prevent an agent from making the same mistake over and over, as any cycling or converging behaviour is broken.

D. Methodology

As we will discuss test results in this and the following sections, we will explain how these results were acquired: Using the resulting .json files from evaluating our algorithms on the given packet world demo environments, it's possible to compare and contrast the underlying mechanism of the specific algorithms. The focus of this paper will be on energy usage and movement expenditure (amount of cycles). We will always use an average of 100 runs.

E. Test Results

Looking at the original Wander without optimization in table I: average turns needed to finish was 15034 and average energy usage was 1042877. This will be used as a benchmark to compare our algorithms. Our MAPE-K, and thus vision

Algorithm	Moves	Energy
Wander	15034	1042877
MAPE-K	8281	559877
SmarterWander	418	24185

TABLE I

TEST RESULTS FOR SIMPLE ENVIRONMENTS WITH LIMITED PERCEPTION

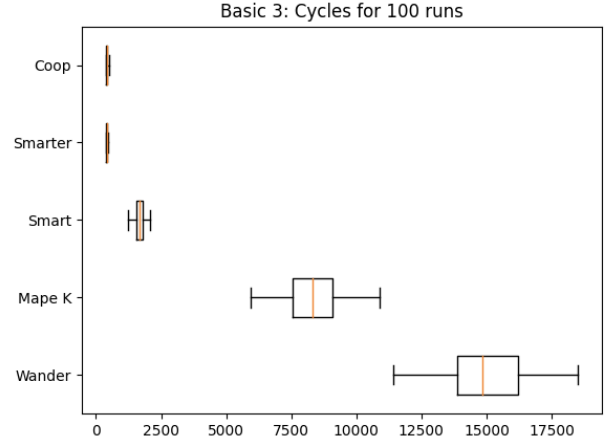


Fig. 2. A box-plot comparing the average turns needed on 100 runs for all algorithms, on the basic_3 environment

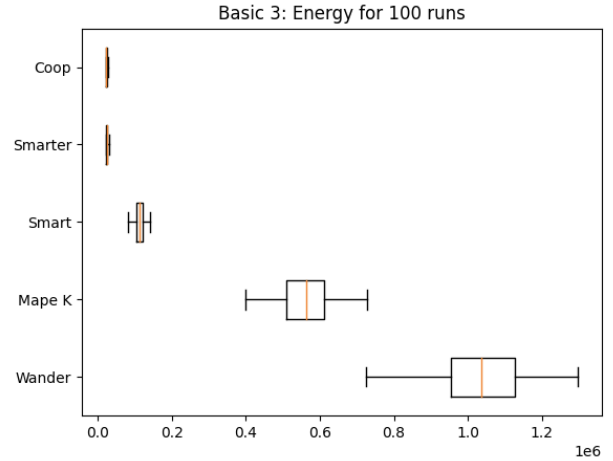


Fig. 3. A box-plot comparing the average energy needed on 100 runs for all algorithms, on the basic_3 environment

expansion, results in a decent improvement compared to the original; average turns needed to finish: 8281 and average energy usage: 559877, an overall improvement that nearly cut the previous results in half. However, the agents are still mostly reverting to random behavior when no packets are in perception range. Finally, the result of the SmarterWander; average turns needed to finish: 418 and average energy usage: 24185. This ends up improving the non-optimization design by nearly 1000%. By cutting down on random wandering, large gains are made. To contrast these improvements to each other and later solutions, all were run on the basic_3 environment and plotted in fig 2 and 3. To compare the later algorithms and compensate for the high variance on the earlier versions, an average of 100 runs was taken. The above observations still hold.

As seen, the used optimizations result in a sizable performance increase, however introducing rudimentary inter-agent

communication only resulted in a marginal improvement. This is because the information exchanged is extremely local, most of which will already be in the perception range of the agent receiving the message. On top of that, destinations are never forgotten, therefore only ramp up speed is improved, not sustained performance. It would be interesting to investigate how communication between agents can focus more on sparse, distant knowledge.

VI. ENVIRONMENT WITH MODELLED ENERGY USAGE

A. Environment outline

In this section the environment of section 1 will be extended to include support for energy costs, charging, gradient fields and walls. These are expansions to the boundaries of the simulation, not as much the environment. Adding this kind of complexity makes so that unaware agents can no longer operate in this expanded environment. To run tests on this kind of environment, the energy_1 and energy_2 environments can be used.

B. Design

First we expanded what the agent could observe and remember to include walls. This made it so that we could now remember destinations, packets, and walls – giving us a good sense of the layout of the map. To optimize pathing and allow for the inclusion of walls, Dijkstra’s shortest path algorithm was used to compute a path using all obstacles in perception and information stored in memory (walls etc.). This enables pathing to even remote, secluded areas which might be blocked off elsewhere. We also now include agents in our pathing calculations. However, only when agents are in perception range can their locations be known. It is useless to store location information on other agents as they are constantly in motion. So all agents in perception range are added to the surrounding map.

The newly desired energy behavior was split in two distinct modes: 1) finding a charge station to charge on and 2) charging on the station and relinquishing the station when another nearby agent needs it more. The findChargingStation behavior is implemented using gradient fields. An agent can then use this field to easily traverse the world to a charging station from any coordinate without previous knowledge of the charging station. This also means that agents will always converge to the nearest charging station instead of wandering to further off ones. Route determination is adjusted as well: when the charging station enters perception range, the agent no longer relies on the gradient field. It then uses Dijkstra’s shortest path algorithm to compute the shortest route using any agents and packets on the way [5]. After arriving at a charging station, the agent still needs permission to actually stand on the charging spot and start recharging. In order to do this, a broadcast protocol is used where agents transmit their battery state and intended charge location to all agents. If no agent is present on the charging station, the agent will position itself to start charging. In the other case, it will wait until the other agents

moves away from the station. During that time, it will keep communicating its battery state and charging intention.

When an agent is in charging behavior (they are located on top of a charging station and are in the process of recharging), they will charge up to the highest possible safe limit before switching back to find a new packet behavior. Do note, agents will still need to keep listening for other communicating agents that are requesting to charge. If such as a message is found, a handshake protocol is started where the currently charging agent checks the other agent’s battery state and calculates whether or not the other agent will deplete their battery while waiting on the current agent. If this is true and the current agent is sufficiently charged (so that it does not get stuck leaving the charging station), the current agent will leave the charge station, thus accepting the other agent’s request.

C. Implementation

To implement these features, the behaviour based architecture was extended. Even though only one feature was added, we have decided to create 2 different behaviours. This split is due to the intense and two-sided communication required while charging. In no other behaviour is communication utilised as heavily as while the agent is charging. In the FindChargingStationBehaviour behaviour it utilises the agent’s ability to broadcast message relating to charging to let other agents know that the current agent is on route to charge at a given station. This continuous informing of all agents on the map allows for agents that are charging to decide whether this new agent has to receive priority over the charging position. This part of the protocol is handled in a different behaviour: ChargingBehaviour. This allows for the logic of deciding the response and the logic of creating a question and interpreting the response to be distinct. As maybe other types of agents would like to respond or ask questions in a different manner. In our implementation the response is physical (move away from the station). The gradient field here is seen as an electromagnetic field pointing towards the source, thus the charging station, we have however assumed that all walls and agents are a non-conductive material and thus block and bend the field. This allows agents to determine the distance and direction to the charging station. If the agent is in the FindChargingStationBehaviour, it will blindly follow this field to the station, until it sees the station. Then it can use Dijkstra’s shortest path algorithm to find a path and see if there will be no obstructions in the future. More about this in section VII on the energy world with hallways.

D. Test Results

World layout	Moves	Energy
Energy 1	268	12758
Energy 2	724	51632

TABLE II

TEST RESULTS FOR MODELLED ENERGY USAGE

As in step 2, we tested the implementation’s performance on a collection of set packet world layouts. As said before, these

can be found in the packet world system titled as “energy-1” and “energy-2”. Our implementation averaged 268 moves and 12758 energy usage on “energy-1”. On “energy-2”, it averaged 724 moves and 51632 energy usage on “energy-2”. The performance discrepancy between these two layouts can mostly be attributed to “energy-2” being a bigger board and having a more sparse distribution of agents. While agent’s perception fields scale to a larger size, this does not reduce the complexity added by having more packets to deliver and distance to cover. It is interesting to note that the relative increase in amount of moves between “energy-1” and “energy-2” is smaller than the relative increase in energy usage. This could be attributed to the heavier energy drain of transporting packets across larger distances, as agents are efficient at scanning an area and giving each other information.

VII. ENERGY ENVIRONMENT WITH CHARGING STATION IN HALLWAYS

A. Environment outline

This environment is an expansion of the energy environment: In this scenario however, charging stations are located in hallways. This means that agents have to be patient and cooperate or else they can deadlock each other or block an other agent’s path, leading to that agent’s demise.

B. Design

Most of this design is based around the fact that it is possible for an agent to block another agent by merely standing in a location. To allow agents to be more patient and give way to each other in a manner that is useful, we defined a new behaviour. To trigger this behaviour an agent detects that it can’t walk in the direction it wants to and that an agent is blocking its path. In this behaviour, an agent has to detect whether it is standing in a hallway of a single tile wide and move out of it so that it does not block another agent. It must then decide if the other agent has passed so it can complete its journey.

C. Implementation

To implement the behaviour described above, the implementation for the energy environment was extended with MoveAsideBehaviour. This behaviour has to have a high priority as not waiting could prove disastrous for the agent. Detecting when to enter this state consists of detecting 2 scenario’s: (a) either the agent walked into another agent or (b) its path is obstructed further down the line. Detecting (a) is easy, detecting (b) requires a more complex approach. To detect if a path is obstructed by an agent, the path must have the following properties:

- 1) The path consists only of squares that have only 2 degrees of movement (we can only step forward or backward), from now on referred to as “hallways”.
- 2) In the path there is an agent that we would have to pass through.

If these conditions are satisfied, then a collision is unavoidable and the agent must move itself to a safe location. The same is

true when an agent detects that there is no path possible to its destination, as we can not risk the agent stranding in a hallway. A safe location would then be the first square while moving away from the agent (or its destination if there was no path) that is not a hallway. If such a square is located, the agent waits there until the other agents have passed or starts moving again before they passed with a small chance. As the agent is waiting on the first square that is not a hallway, any agent passing by must make 2 moves adjacent to the agent. Enabling detection of the other agent having passed. If it was not waiting for an agent, it just recalculates its route (if possible) and continues or keeps waiting. An important not is that once a safe position is reached, the agent does not reposition itself, if the square were to become a hallway. The only reason this could happen is an other agent passing by, thus this will resolve itself. One more observation must be made: the agent cannot retrace its steps while finding a safe space, if it were to have no options remaining, the agent is located in a cul-de-sac and continues with its journey assuming the agent that was blocking its path will have moved out of the way itself.

The randomness in the waiting time is also introduced in the case the other agent start heading in a different direction, so every turn it’s waiting there is a 20 percent chance that an agent continues its route without seeing the agent it collided with.

D. Test Results

As other implementations have a very low chance of solving this environment (most of the runs an agent blocks a hallway, stranding all other agents), no comparison can be made to other implementations. We can however note that our implementation took 482 cycles and 23211 energy units on average on the “energy-3” environment as seen in table III. To compare the overall complexity we can once gain refer to figure 2 and 3 as the smarterWander there was already equipped with these mechanisms. We have also run tests on the original version of smarterWander, but it was not significantly different. Therefore it is omitted on these box-plots for clarity.

World layout	Moves	Energy
Energy 3	482	23211

TABLE III

ADDITIONAL TEST RESULTS FOR THE SMARTERWANDER ALGORITHM.

VIII. ENERGY ENVIRONMENT WITH REQUIRED AGENT COOPERATION

A. Environment outline

The environment used for testing in this part is the task-delegation environment. This environment is not very different from the previous one, but it is specifically engineered that no single agent can pick a packet and create a route to that packet’s destination. This then obligates agents to cooperate and no longer work completely independently, or mostly in their own self-interest.

B. Design

The fourth expansion is centered on inter-communication of agents. Specifically for task delegation between coloured agents and maneuvering difficult task environments. Our design focuses on three main focus points: packet sharing delegation list, refactoring for more fine tuned behaviour control, and urgent packets.

The packet share delegation list is stored by each agent and updated with shared information of other nearby agents. This list contains locations, color assignments, and urgent labels of packets that are necessary for proper delegation. It informs their find packet behaviour and what packets need to be picked up at what point. The list should also be consistently updated and maintained, especially when agents drop packets for recharging, or to go and clean up an urgent packet.

We also had to extend the existing behaviours in order to ensure that agents can communicate and drop packets at the right time. An example of this is when an agent drops off a packet during their deliver packet behaviour. This dropping off of packets can occur for a variety of reasons, such as: the destination being blocked or an urgent packet needs to be picked up. However, in the previous implementation, dropping off a packet and switching to find packet behaviour would result in picking up the same packet that was just dropped off. In order to avoid this from happening, a state variable was introduced that would allow overriding this behavior. Another place where we had to adjust the existing behaviour is when an agent should not simply switch to find packet behaviour, even though it knows other packets' location, after dropping off a packet. This is because sometimes it should change to random walk behaviour to continue to explore the map and share new packet locations or urgent packet labels. If this is not done, there is a high risk that agents keep finding packets (that they can't deliver) in the same local area. If all agents remain in their "zones", they have very little chance of communicating that there is a packet that they can't move. This is implemented using a similar strategy as the drop packet strategy, in the sense that it is forced via a state variable blocking the transition to findPacket, even if it would seem to be the logical choice.

Finally, as mentioned previously, our design now also includes a notion of "urgent packets". An urgent packet is a packet that is currently blocking a destination or other coloured packets. They are sometimes packets blocking an optimal path, and sometimes blocking a location altogether. If an urgent packet is not cleared soon, it will cause some agents to become inactive and randomly wander around without being able to perform meaningful work. An agent is optimistic, in the sense that it assumes that by the time it arrives at a location, the packets blocking its path are removed. If this is not the case, It starts wandering and drops the packet it is currently holding in a location that doesn't block any hallways. Urgent packets are communicated amongst agents in order to find an agent capable of clearing a packet of the given colour.

C. Implementation

The codebase was reworked for this iteration to compensate for the inability to have postconditions (that are dependant on the state, or thus memory, of the agent) when changing states. To allow efficient delegation, it was required that agent retained information when changing state and recognized that a state change was made. To solve this, state changing was now based on a state variable saved in the agent's memory. This object is also serialized using the serialisation utility used for the packets, destinations and walls. Apart from this change, an expansion was needed to allow agents to drop packets on locations that would not hinder any agents. Therefore the hallway detection mechanism from section VII was used, determining if a packet placed on this location would block any direction of movement. If this is not the case, the agent can drop a packet and can then wander or find a new urgent packet.

We have decided not to split the behaviour of the agent when going to an urgent packet and a regular packet, as all the mechanics for the journey are identical. The distinction is made by the fact that urgent packets are retrieved from a priority list in the agent's memory. As long as this list is non-empty, no regular packets are handled.

To determine when a packet is in the way of a would-be path, Dijkstra's algorithm is adapted to make packets "walkable" but at a greater cost (a cost of 10 versus a cost of 1). This means that the agent will generate a path through packets, but prefers to walk around them. Automatically this means that the agent uses the least amount of packets possible in its trajectory. All packets within perception are then saved as "cleanup packets". These packets should be cleaned up urgently, as they block the path of the agent. The packets that the current agent cannot move itself are then communicated to other agents when they enter the current agent's perception range.

The communication of urgent packets also contains a peculiarity, in the sense that packets of a certain color are only transmitted to an agent of said color and are then removed from the sending agent's memory. This excludes the possibility that agents keep resending the same packet in a circle. This however could mean that an agent on the wrong side of the map (if it is divided into sections with agents with specific colors in both sections) receives the only notification to clean up said packet. This scenario is not present in the given environment, but could be considered a fault for the algorithm as a whole. It is however not critical as the next time the agent wants to travel to a location hindered by the aforementioned packet, it will again save the packet as urgent and can then deliver it to the right agent.

D. Test Results

Running the updated implementation on the preset packet world layout "task-delegation", an average 267 moves and 10956 energy usage was recorded. It is impossible to make a comparison, as all other algorithms fail on the environment the coop algorithm was designed for. In order to get a

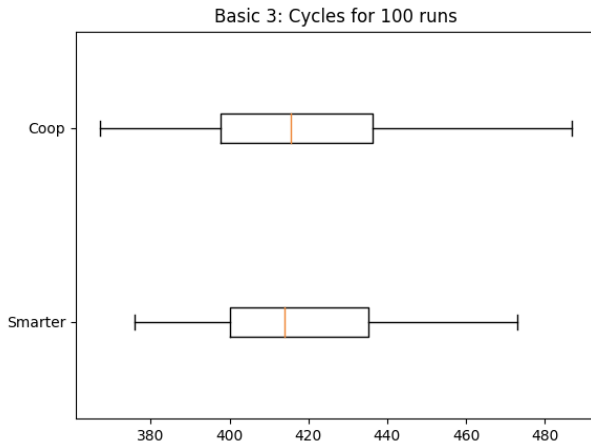


Fig. 4. A box-plot comparing the average turns needed on 100 runs for the coop and the smarterWander algorithms, on the basic_3 environment

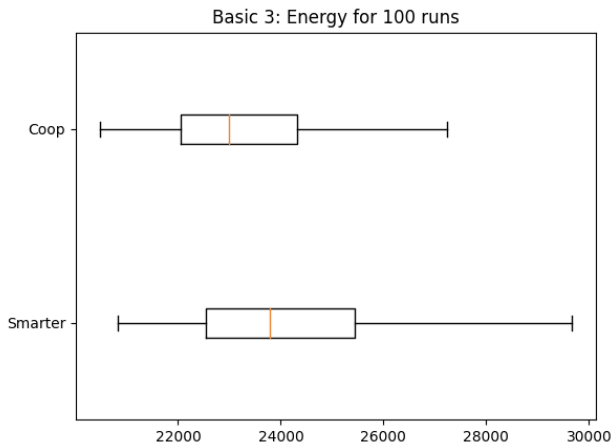


Fig. 5. A box-plot comparing the average energy needed on 100 runs for coop and the smarterWander algorithms, on the basic_3 environment

more accurate idea of the energy efficiency of the step 3 implementation, a comparison was made on the boards of step 1, more accurately on the Basic_3 board, the results of which can be seen in the box-plots on 2 and 3. It is quite hard to make out the difference between the most efficient algorithms, therefore figure 4 and 5 show a comparison of only those 2 solutions. Interestingly, compared to the implementation in section VIII on the environment in section V which did not perform any collaboration, its results stayed extremely close. This is noteworthy because this shows that in situations where energy issues do not occur and color collaboration is not necessary, the final implementation is still nearly as efficient as without the added overhead.

IX. DISCUSSION

The test results have already been mostly discussed in the relevant sections. A couple of interesting observations can be made from these results. First of all, it seems (especially

in section V) that changing algorithms has a disproportional effect on the overall performance compared to pruning of the system. By moving to random directions over random step, a much larger improvement was recorded compared to being more aware of the overall environment due to communication. Another observation was that added complexity of the environment layout can have very noticeable effects on the performance of the system. This was seen in section VI where “energy-2” led to a nearly 400% increase in energy usage compared to “energy-1”. Finally, overhead can mostly be limited in communication, reverting to more optimal behaviour depending on available obstacles.

There are however more than a few limitations with the solutions proposed as well. The first of which is that overhead does exist, not every change that improves behavior in a more complex layout will have a positive effect on a more simple layout. Relevant to this, it is difficult to cover edge cases or even predict future extensions to the design of multi-agent systems. There are many different kinds of adaptations that can occur and these often demand changes to the overall strategy used, not just an extension – but a modification. An interesting question could be, while this solution was constructed in a 2 dimensional field, many real-world applications are in 3 dimensions. Moving to a 3 dimensional design will not only introduce added complexity in layout size, but also in new previously not seen interactions. For example: how will random directions over random steps hold up in this environment? A final important limitation has to do with the given primitives and assumptions of the packet world system. The proposed designs assume significant enough computing power and storage on agents to be able to perform path finding algorithms such as Dijkstra. Also, the packet world provides certain primitives such as pheromones, gradient fields, perception fields that might not always be available.

Taking all this into account, several lessons can be drawn from the overall research. First of all: strong concepts tend to be more important than continuous optimization. Having a good software architecture leads to much needed maintainability and extensibility which can minimize the amount of work a new concept introduces. Overspecialization leads to overhead that might create a more optimal solution in one context however even with minimal changes, can create a noticeable performance drag in other contexts. Well designed algorithms however hold up a lot better across these changes and modifications. It could be more interesting to look into maintainability instead of pure optimization for packet world multi-agent systems. In other words, more attention should go to architectural design instead of implementation concerns for specific use cases.

X. RELATED WORK

The importance of multi-agent systems in real world applications has been explored before, for example in [6] several requirements were identified for multi-agent solutions. This ties in closely to the primitives provided by the packet world test bed. We have studied a limited set of related solutions,

most of which were brought to our attention by the book “Architecture-based design of multi-agent systems” [7]. For the MAPE-K design of section V, we were inspired by the design proposed in [4]. In general, software architecture techniques in multi-agent systems was a guiding principle throughout the design and implementation of our solutions. [8] Research in similar scenarios though not necessarily on the same test-bed were also done in “A Decentralized Approach for Anticipatory Vehicle Routing Using Delegate multi-agent Systems” [9], though our solution was only later influenced by some remarks in this work. Other research such as [10] has also explored the necessity of adaptability in multi-agent solutions. In order to address the large amount of complexity in environments interesting for multi-agent systems, agents should be able to adapt themselves with new knowledge. Instead of overly optimizing on simply 1 environment, agents should be able to adjust themselves to tackle other new complex problems. Another approach for communication between agents in a multi-agent system using pheromones is described in [11]. A reference architecture for multi-agent systems is discussed in [12]. Other sources consulted by this team were [13], [14] and [15] to gain a deeper insight into the topic.

XI. CONCLUSION AND FUTURE WORK

The packet world is a situated MAS designed as a test bench for multi agent solutions. It is heterogeneous in terms of the amount of real-life scenarios that can be translated to it, making it of particular interest for research. The paper focused on 3 implementation steps on top of the packet world in order to learn how easily these could be leveraged to solve MAS problems.

The implementation steps showed interesting conclusions in terms of optimal pathing, energy management, and coordination. An overall trend towards strong concepts over specialization was observed leading to the importance of good software design in MAS systems. Several limitations still exist regardless, the packet world has several inherent assumptions surrounding computing power and available primitives. Yet in its current state it is an excellent test-bed to prototype MAS solutions for pathing, energy management, and coordination.

Future work could explore how the assumptions of the packet world could be modified. One such example could be expanding to 3d environments. In general, the bigger problems in MAS solutions that can be translated to the packet world seem to center more on architectural design than pure optimization.

REFERENCES

- [1] D. Weyns, A. Helleboogh, and T. Holvoet, “The packet-world: A test bed for investigating situated multi-agent systems,” in *Software Agent-Based Applications, Platforms and Development Kits* (R. Unland, M. Calisti, and M. Klusch, eds.), (Basel), pp. 383–408, Birkhäuser Basel, 2005.
- [2] E. Kulbiej and P. Wolejsza, “Naval artificial intelligence,” pp. 291–297, 06 2017.
- [3] L. Ribas-Xirgo, “Multi-agent system model of taxi fleets,” in *Advances in Physical Agents II*, Advances in Intelligent Systems and Computing, pp. 123–134, Cham: Springer International Publishing, 2020.
- [4] D. G. D. L. Iglesia and D. Weyns, “Mape-k formal templates to rigorously design behaviors for self-adaptive systems,” *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, vol. 10, no. 3, pp. 1–31, 2015.
- [5] E. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [6] F. C. A. Groen, M. T. J. Spaan, J. R. Kok, and G. Pavlin, “Real world multi-agent systems: Information sharing, coordination and planning,” in *Logic, Language, and Computation* (B. D. ten Cate and H. W. Zeevat, eds.), (Berlin, Heidelberg), pp. 154–165, Springer Berlin Heidelberg, 2007.
- [7] D. Weyns, *Architecture-based design of multi-agent systems*. Springer Science & Business Media, 2010.
- [8] J. Juziuk, D. Weyns, and T. Holvoet, “Design patterns for multi-agent systems: A systematic literature review,” in *Agent-Oriented Software Engineering*, vol. 9783642544323, pp. 77–97, Springer, 2014.
- [9] R. Claes, T. Holvoet, and D. Weyns, “A decentralized approach for anticipatory vehicle routing using delegate multiagent systems,” *IEEE transactions on intelligent transportation systems*, vol. 12, no. 2, pp. 364–373, 2011.
- [10] D. Weyns, *An Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. Wiley - IEEE, Newark: IEEE Computer Society Press, 2020.
- [11] S. Hamichi, D. Brée, D. Mangalagiu, and Z. Guessoum, *A Multi-Agent System for production networks simulation: Toward a pheromone-based model for agents’ coordination*. 01 2012.
- [12] D. Weyns, S. Malek, and J. Andersson, “Forms: Unifying reference model for formal specification of distributed self-adaptive systems,” *ACM transactions on autonomous and adaptive systems*, vol. 7, no. 1, pp. 1–61, 2012.
- [13] D. Weyns, M. Fabien, and F. Michel, *Agent Environments for Multi-Agent Systems IV: 4th International Workshop, E4MAS 2014 - 10 Years Later, Paris, France, May 6, 2014, Revised Selected and Invited Papers*, vol. 9068 of *Lecture Notes in Computer Science*. Cham: Springer International Publishing AG, 2015.
- [14] *Convergence and knowledge processing in multi-agent systems*. Advanced information and knowledge processing, London: Springer, 2009.
- [15] A. Dorri, S. S. Kanhere, and R. Jurdak, “Multi-agent systems: A survey,” *IEEE Access*, vol. 6, pp. 28573–28593, 2018.

Multi-Agent based Warehouse Management Automation

1st Gilles Meyhi

Faculty of Engineering Science
KU Leuven

Leuven, Belgium

gilles.meyhi@student.kuleuven.be

2nd Miguel Russián Rojas

Faculty of Engineering Science
KU Leuven

Leuven, Belgium

miguelandres.russianrojas@student.kuleuven.be

3rd Rajat Sharma

Faculty of Engineering Science
KU Leuven

Leuven, Belgium

rajat.sharma@student.kuleuven.be

4th Venkataraman Subramanian

Faculty of Engineering Science
KU Leuven

Leuven, Belgium

venkataraman.subramanian@student.kuleuven.be

Abstract— Over the last few years, multi-agent systems (MAS) have gained popularity because of the numerous advantages they offer over single agent or centralized systems. In this paper, a solution for automating a warehouse management system has been proposed. Warehouse automation has seen a steady growth [6] in the form of robots handling a plethora of tasks. Such a system can be further enhanced in the case of mobile robots which can be programmed to work and collaborate amongst themselves autonomously.

Using the *Packet-World* test bed, we attempt to address the problem of establishing cooperation between several agents (MAS of robots) that work together to collect packets and deliver them to their designated stations. There exist several different approaches to tackle this issue. Three main themes were identified as pivotal in tackling the problem, namely, autonomous behaviour of the agents, energy management coordination of the agents and task delegation between the agents. The approach outlined here, uses these themes, to build upon a probable solution. The results obtained do show the effective working of the MAS, but at the same also expose some errors that reveal the drawbacks of the system. The reasons behind their occurrence and how they can be improved has also been discussed in this paper. A retrospective discussion of related work has also been carried out. Overall, the research undertaken here offers high-level insights into MAS movement and communication logistics, and how they can be developed for warehouse automation and supply chain management.

Index Terms—Multi-agent systems, *Packet-World*, Logistics, Automation, Supply-Chain Management

I. INTRODUCTION

In this paper, the work in identifying both the concepts related to multi-agent systems (where software-controlled devices interact to achieve a common goal, further explained in [7]) as well as the characteristics of the proposed test bed has been undertaken. Also, further new applications for such a system have been proposed.

Specifically, as done in [1], in this work we will focus on situated MASs, where agents only interact directly with a "local context", geographically speaking, as well as agents

within, have limited internal state, and have a notion of (geographical) position. Some benefits of situated MASs are flexibility, robustness and efficiency, although engineering complex systems based on their "short-term" behaviors is an open problem.

One natural application for situated MASs is warehouse management, where agents are tasked to move inside the warehouse to transport packages from their current position to a designated destination. This improves operational efficiency by increasing delivery throughput though introduces new complexities of agent management, some of which are discussed throughout this paper.

The *Packet-World* [1] test bed, which simulates such a warehouse including objects of interest such as packets, destinations, as well as the agents themselves, is used to describe and develop solutions to situated MAS use complexities, such as autonomous behavior, communication and coordination.

II. MODEL DESCRIPTION

A. Core components and behavior

The *Packet-World* test bed represents a 2D layered grid (the Environment), with agents that have the purpose of moving packages to their corresponding destinations (both occupying a cell) with minimal energy consumption and communication. In general, actions consume energy which must be replenished at charging stations at fixed locations, or else the agent will be blocked.

For system operation, three key components mutually interact: the *Environment*, *Agent perception* and *Agent decision making*. The Environment ensures consistency by exposing layers (agents, packets, and markers) and setting all constraints within and between, ensuring extensibility and reusability. On the other hand, following the classification in [14], Agent perception controls agent knowledge through a three-part process of Sensing, where each agent senses the environment based on their ability, Interpreting, where the data sensed are turned to knowledge for usability, and Filtering where the agent selects those items that match specific selection criteria. Finally, an agent's Decision making is a free-flow tree, used along with the perceived environment data and the current task learned from communication to choose and execute the next action.

For synchronization, three different schemes are provided in the *Packet-World* model: global, regional (within perception

range) and logical (which changes the duration of actions programmably, through an architectural extension).

B. Advanced architectural considerations

Certain forms of advanced collaboration can also be supported in the Packet World, such as simultaneous actions (e.g., carrying a packet together, one agent giving a packet to the other) which succeed if both agents have agreed to cooperate, or indirect communication by leaving markers in the environment.

For the Packet-World, markers can be Flags, Gradient Fields and Pheromones. These markers allow agents to influence each other without direct communication, which could lead to simpler agents, and the environment to communicate with an agent (e.g., gradient fields inform all agents where the nearest battery charging station is situated).

III. MODEL APPLICATIONS

In spite of its name, the Packet-World model can be applied to MAS use cases beyond warehouse management. Some examples of such use cases are described in the following sections.

A. Coordination of self-driving vehicles

In this first example MAS, agents are autonomous vehicles (AVs) that communicate with one another to achieve a global objective - transport customers, treated as packets, to their destination without any harm, and as soon as possible.

To perform their tasks effectively, along with their own sensors they can use feedback from surrounding objects that support agent interaction, comprising not only other cars, but also structures and software systems (e.g., cloud server for local traffic conditions, traffic lights advertising next changes, or gas stations listing available fuel or charger types), as proposed by De Silva et al.[10], among others.

Additionally, cars could, for example, inform each other of the obstacles (both stationary such as buildings, and temporarily stationary such as traffic jams) and customers they see, appearing as they request the service, and they can coordinate to offer a client currently in a traffic jam an alternative vehicle within walking distance, offering a convenient service in high traffic situations.

B. Restaurant waiting robots

In another possible application (in fact similar to the one proposed by René et al. [9]), agents could be robot waiters, serving dishes in a restaurant, where the environment generates clients sitting at tables, who become temporary 'packet' destinations.

Packets (such as dishes, utensils or tablecloths) become available from key locations, such as the dishwasher, kitchen or cocktail bar, and waiters must cooperate to place all necessary items at their destinations, in the right order, which can be enforced by changing the destination type (e.g., napkins to plates, drinks to first course, etc.) as soon as a packet of the preceding type is placed in it.

Agents can negotiate to fully serve different areas of the restaurant or specialize by type of packet, depending on their perception of what is most efficient.

IV. PROBLEM DESCRIPTION FOR THE PACKET WORLD

In the context of this paper, the Packet-World model will be used to demonstrate MAS concepts and techniques for warehouse management. The issues discussed henceforth can be specifically categorized as dynamic and deterministic pickup and delivery AV management problems with local information according to the classification proposed in [8].

The Packet-World consists of Agents, Packets and Drop Points in an environment where the agent should be able to pick up the packets and drop them at their destination. In addition, the agents should be able to do this in collaboration with other agents.

A. Autonomous Behavior of Agents

Agents behave autonomously and without communication, walking around to pick up packets and drop them at the destination. Agent behaviour includes the limited ability to see the environment and also perform color specific tasks (where the agent must drop a packet of a specific color in the destination belonging to that color).

B. Energy management coordination of Agents

Agents are required to recharge the battery before it is completely depleted and establish coordination with other agents to avoid a deadlock (a situation where two or more agents need to recharge immediately or *die* as a result), at the battery station. Each agent starts with 1000 units of energy, with the set of actions consisting of skip (do nothing), pick a packet, move without a packet costing 5 units and move with a packet costing 10 units. Accordingly, an energy management needs to be implemented such that an agent can recharge itself without its battery reaching zero.

C. Task delegation Between Agents

Agents are required to seek the help of other agents (delegate tasks to other agents) in order to achieve their goal. The environment is constructed in such a way that the drop point of every agent is blocked with packets of color belonging to other agents. For this, an implementation of a *delegation-logic* is required such that the agents can cooperate amongst themselves in ensuring a clear path, devoid of a packet of another agent, for each other.

V. SOLUTION FOR THE PACKET WORLD

1. Autonomous Behavior of Agents

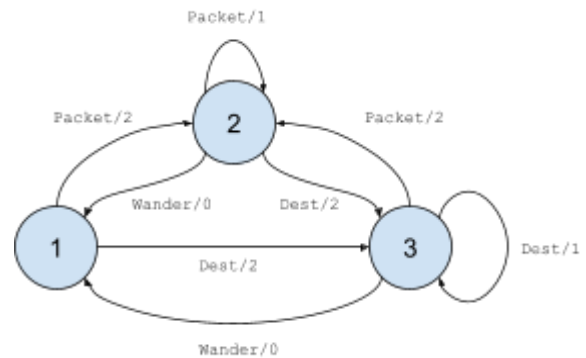


Figure 1: 'simple' agent state transition graph

Our first approach is the creation of a simple agent, whose tasks will be to explore the environment, pick packets and drop them at their destinations when they find them. This translates into clearly marked priorities: if in sight and carrying a packet, an agent will go to the specific destination

of the packet's colour to deposit it, otherwise it will go and pick up a packet if one is in sight, or by default explore the area randomly.

Implementation-wise, this was first developed into a single behavior (which in the Packet-World wraps agent logic into an agent state, producing a single action per cycle), and later into three different ones, *Wander*, *PickOrMoveToPacket* and *DropOrMoveToDestination*. Additionally, the agent configuration file has been changed to transition between these states, as is depicted in Figure 1, where in the expression 'Dest/2', 'Dest' indicates the condition for transition and 2 the priority, 'Packet' is *GoPacketCondition*, 'Dest' is *GoDestinationCondition*, and 'Wander' is the *WanderCondition*. Although this modification does not improve agent efficiency, it does improve code readability.

In an optimized version of *simple*, three improvements are proposed and implemented: first is the encouragement of exploration away from the current position in cases where an agent has nothing in its vision, implemented keeping a randomly picked but fixed direction, changed only after 10 moves for each agent, and in doing so preventing undoing a previous move. A second optimization is remembering the last seen destination encountered for each color, eliminating the need for wandering when an agent is carrying a packet having a destination of the packet's color in memory. Finally, memorizing a nearby packet if within vision when an agent picks up another also reduces random wandering after dropping a packet.

The new agent configuration resulted in the graph shown in Figure 2:

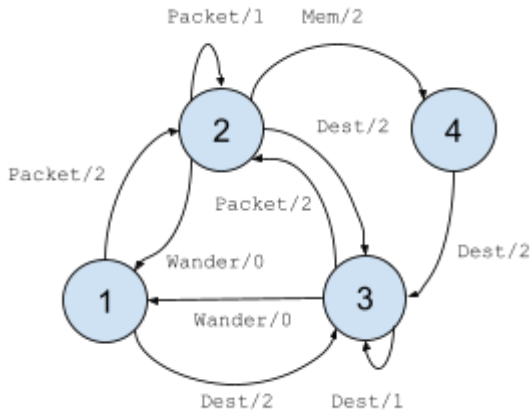


Figure 2: optimized 'simple' state transition graph

In this case, a new state *GoToMemorisedDestination* was introduced, reachable through the condition 'Mem' (*GoMemorisedDestinationCondition*), and connected by two new transitions to the previously defined states.

2. Energy management coordination of Agents

To achieve energy management coordination, we use both global coordination and local coordination between agents as a means of communication to ensure no agent's battery reaches zero. Our proposed implementation for these scenarios is named *energy_bot*.

Extending *simple*'s behavior to ensure adequate battery levels, *energy_bot* starts a global coordination protocol when its battery is in a warning state to take turns to reach the battery station, and once the agent is near it local coordination starts.

Global coordination functions as a first deadlock avoidance mechanism, seeking to prevent situations in which all agents are at a particular charging station at the same time, which could otherwise result in inefficient congestion with a risk of agent energy depletion. In this protocol, agents attempt to work smartly, reaching the station in a sequential and an orderly fashion.

This process is achieved through a decision protocol, run every 5 cycles to avoid spurious task switching, whose result is used in the agent's transition graph using the information of battery levels of all agents acquired using communication., as shown in Figure 3. To minimize memory usage, a simple threshold mechanism was used for state transitions, and the energy station location where agents intend to charge in are neither shared nor stored in memory, although this would be a reasonable improvement should memory constraints allow it.

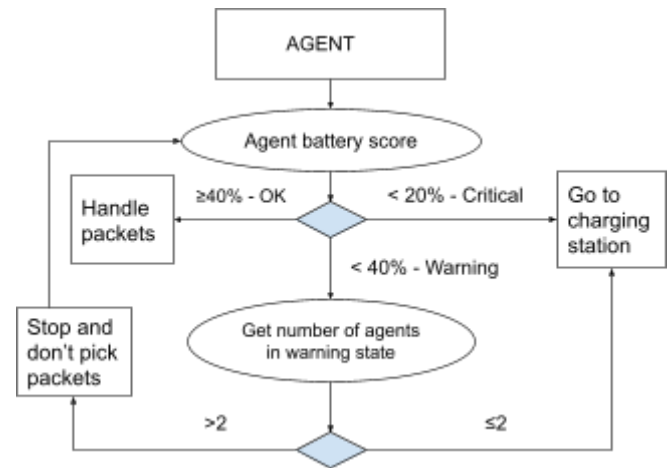


Figure 3: Global coordination flowchart ('energy_bot')

For the implementation of the logic depicted in Figure 3, handling packets signifies using *simple*'s behavior (though with a new state, *WanderOff*, instead of *Wander*), stopping corresponds to the *Wait* state, and the last state to *GoToNearestBattery*. These are explained in more detail later in this section. Finally, the battery score refers to a heuristic calculation based on gradient fields, specifically, the agent's current battery state, minus the expected energy cost of travelling to the station (i.e., the gradient value at the current position by the energy cost of a step).

Local coordination aims to manage simultaneous charging to avoid energy depletion caused by long waiting time or excessive switching between agents for charging, local coordination is performed based only on the agent's perception and current knowledge about the environment, and is materialized in two behaviours: *GoToNearestBattery* and *Waltz*.

On one hand, in *GoToNearestBattery*, an agent tours all known or perceptible energy stations first until it finds one that is not congested. Vision plays a key role in detecting congestion, i.e., four or more agents in a station's vicinity, to switch movement targets, or investigating a near but unknown energy station via gradient local minima analysis. A high level overview is provided in Figure 4.

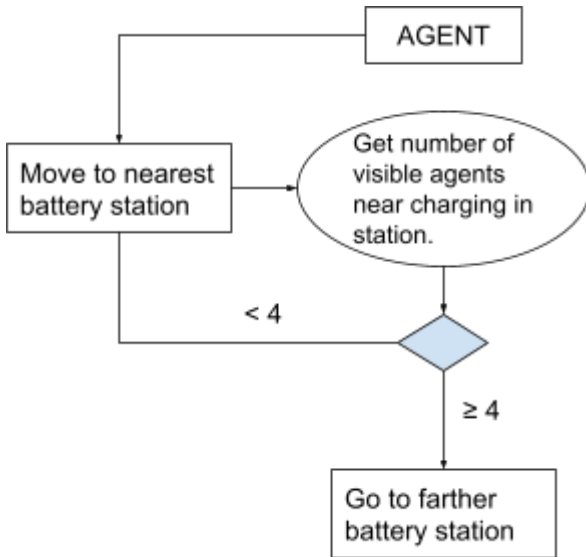


Figure 4: GoToNearestBattery flowchart

On the other hand, *Waltz*, provides a means to avoid energy depletion when multiple agents are involved by removing waiting time and instead moving in a small circle formation around a fixed point in the environment as shown in Figure 5 (arrows indicate agent movement direction), ensuring each agent charges on the colored-border cell every 8 cycles or less in a round-robin fashion.

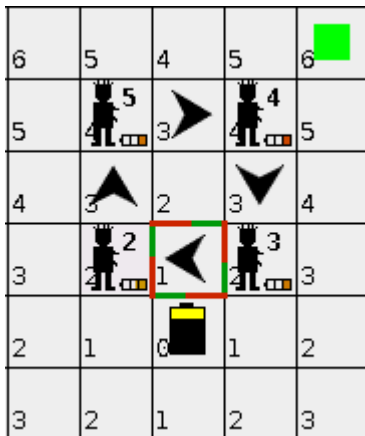


Figure 5: example Waltz formation visualization

When no agents are close to the station, the agent simply steps into the charging cell until its battery is sufficiently charged. As a precautionary measure, an agent might break formation when its energy level is too low (meant primarily for agents joining the formation). This behavior is summarized in Figure 6. It should be noted that the formation is flexible, and it can adapt to a few obstacles in or around the formation track, such as packets, walls, other agents or edges of the environment, but it can only support four agents because they cannot simultaneously exchange positions in a single cycle in the Packet-World model, meaning a margin empty cell must be left between them.

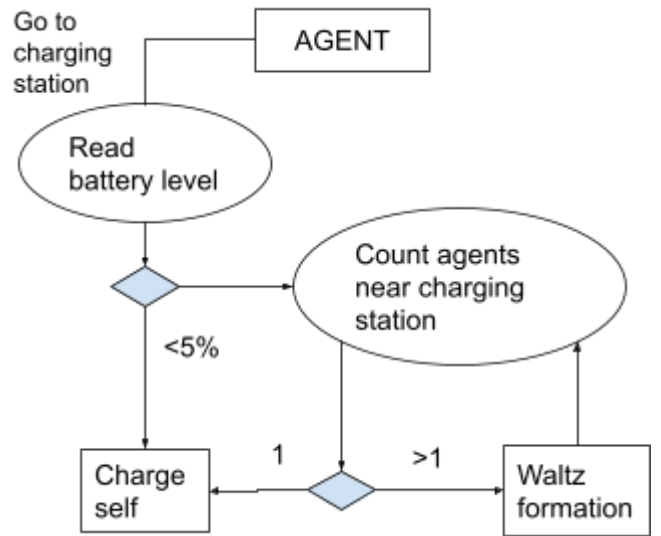


Figure 6: Waltz flowchart

From the above, it can be observed that through the use of both global and local coordination, the agents are able to recharge properly without discharging completely. This ensures that all the packets are delivered to the destination more efficiently as all agents remain operational.

For energy-related scenarios, pathfinding capabilities were also improved by adding obstacle avoidance, which was achieved by a temporary negative influence over a specific area, using a pheromone to mark it when a movement loop was detected. Considering these pheromones, along with existing obstacles such as nearby walls or the edge of the environment via their collective coordinate centroid, allows agents to avoid corners created by packet clusters regardless of agent vision range.

As some final optimizations, *WanderOff* provides a few advantages over *Wander*, such as placing avoiding walls, environment edges, and even flags in specific areas where there are no items of interest. In addition, agents share relevant information they learn with other agents by broadcasting structured messages, tagged by type. Three of these types increase collective knowledge about the environment (by sharing known packet and energy station locations and destinations).

There are a few drawbacks in the local coordination implementation, namely, that it remains unspecified what an agent would do in the event all perceivable stations were congested (the most likely outcome being switching stations until one was sufficiently free), or that when agents leave the field of vision near a battery station, an agent may believe it is less congested than another, causing the agent to change target stations without committing to a particular one. For the former case, one possible improvement could be that the agent with the highest battery level yields way to the incoming agent, either leaving or waiting outside the battery station's area, and for the latter, reengineering the internal behavior's state machine is recommended, possibly creating new states for the main agent behavior state diagram. Additionally, due to the fact that stepping consumes more energy than waiting, a likely improvement to the *Waltz* behavior would be to perform a single *Waltz* round, then stop and sequentially step into the charging station, as if in a circular queue. This should not only improve charging time (by both reducing coordination

energy expenditure and total coordination cycles) but also support more agents (e.g., forming an S-shaped queue, supporting all 9 agents possible in a 3x3 grid, rather than only 4 in the original proposal), and all without requiring recurrent agent communication.

3. Task delegation

For this task, we considered three different strategies: gathering all agents to clear the path for every specific destination, one at a time, create a priority queue for each agent, where several tasks were collected (e.g., clear the path the destination of one's own color, clear the path for another agent's destination, move a packet towards a cleared destination) and shared among agents, and finally, go towards one's own destination, and when a blockage was detected, seek another agent's help, then remain idle and clear the path for others if requested.

Of the three, the last one was chosen, and is summarized in Figure 7.

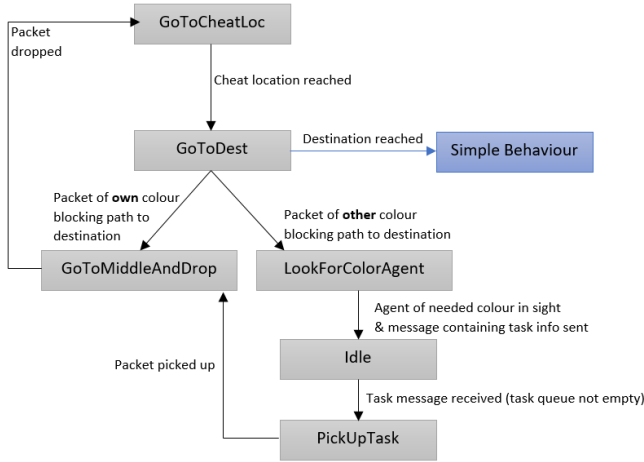


Figure 7: Agent behaviour and conditions diagram (task-delegation)

The `GoToCheatLoc` behaviour is, as the name implies, somewhat a cheating behaviour. It simply brings the agent towards a hardcoded destination. This was implemented because at the time of implementing this behaviour, we did not have a good and working pathfinding algorithm, resulting in agents not being able to go around walls getting stuck. The `GoToCheatLoc` behaviour puts each agent in a position where this does not happen anymore. It is of course not a good and robust way to handle this. If more time and resources were available, implementing a pathfinding algorithm that can go around walls would be a better way to do it.

`GoToDest` is the behaviour that guides the agents towards their destinations of their own colour. In this behaviour we take into account 3 conditions. First the condition where a packet of the agent's own colour is blocking the path to the destination. Secondly there is the condition where a packet of another colour is blocking the path towards the destination. The final and third condition is where the agent has reached their own destination, meaning that there exists a path to that destination without any other packets blocking it.

When the first condition is true, the agent takes on the `GoToMiddleAndDrop` behaviour. This behaviour takes the agent in the opposite direction of where the closest destination is (this closest destination is always updated in

the agent memory). The agent will keep stepping away for a predetermined amount of cycles (this is a configurable parameter). After this the agent will drop the packet, and if enough steps are taken away from the nearest destination the packet will be dropped in a spot where it is not blocking anymore.

When the second condition is true, the agent takes on the `LookForColorAgent` behaviour, where agent coordination is first involved: the agent remembers the location and colour of the packet that has blocked its path and starts roaming towards the middle of the map in search of the agent whose colour corresponds to that of the blocking packet. Once it finds the other agent they enter an `Idle` state to make sure the agent's delegate can find it once the task is done, and sends a "task" message containing the location of the packet that needs to be picked up. The receiving agents save this information in their task queue and proceed to execute those tasks. This is where the agents enter the `PickUpTask` behaviour, where the agent goes to the designated location and picks up the packet, then to `GoToMiddleAndDrop` once the packet is picked up, behaving as previously discussed.

When the third condition is true, which means a destination of a certain color is reachable, the corresponding agent behaves as a `Simple` agent, as explained earlier in this chapter, with some adjustments to account for the restrictions of the coloured agents, the basic principles remaining unchanged.

VI. SOLUTION TESTS

For the first two implementations (which showcase the same behavior in practice), the agents completed each scenario with the results shown in Table 1, summarizing the best performance of a small set of 3 runs. Given that most cycles are spent moving randomly, average results can vary largely.

Environment	Total cycles	Energy consumed
basic-1-tutorial	165	2180
basic-2-vision	724	21750
basic-3-color	6687	437320

Table 1: simple v1 agent best performance

It is worth noting that in the first scenario, an improvement of one less cycle was noticed when using Euclidean distance instead Manhattan distance.

The optimized behavior implementation comes with sizable improvements over the same scenarios, as shown in Table 2. These results were extracted from averaging 1000 runs on each scenario.

Environment	Total cycles	Energy consumed
basic-1-tutorial	165	2180
basic-2-vision	820	18314
basic-3-color	2520	123931

Table 2: simple v2 agent average performance

Although in the first scenario there is no improvement, owing to the fact that behavior is fully deterministic due to complete vision, in the second scenario, the task is completed three times faster on some scenarios and with only 27% of the previously used energy, whereas in the third scenario it is accomplished in average in 37.6% of the previous best time and with 28.3% of energy, clearly making a case for memory-aided optimizations.

Energy management and coordination introduce new risks in addition to previous objectives, such as having one or more agents with their battery depleted and not completing the task at hand and reaching certain deadlock situations where agents still have the resources to finish the task but they never do so.

To measure the degree of achievement for our agent implementation, results for 1000 successful runs for energy-1 and 43 runs for energy-2 have been recorded and in the process failed run results were noted as well. Unsuccessful or deadlocked runs could not be registered automatically, leading to error-prone accounting that may skew these results.

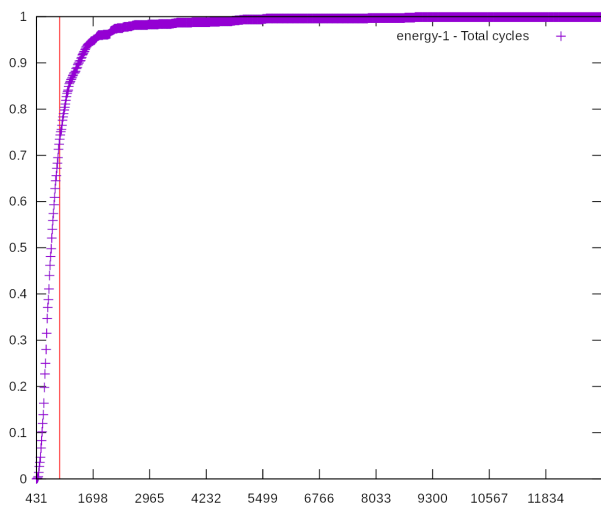


Figure 8: Total cycle cumulative distribution function (energy-1)

The first aspect to note about this implementation is that, in spite of the same starting conditions, agents may use an amount of time disproportionately larger than the median. In fact, as can be seen in Figure 8, they may consume on rare occasions (less than 0.1% of cases) two orders of magnitude more time than their best run, which may render this implementation unsuitable for quality of service-sensitive scenarios (e.g., both use cases described in chapter III). The red line in Figure 8 indicates the average number of cycles spent, and consequently what percentage of runs are above average.

This situation is naturally replicated for total agent energy consumption, shown in Figure 9. One may conclude from visual inspection that local coordination consumes the most time, which can be explained by the poor charging efficiency of 1.25% per cycle in the worst case, compared to the maximum of 10% per cycle when no coordination is needed.

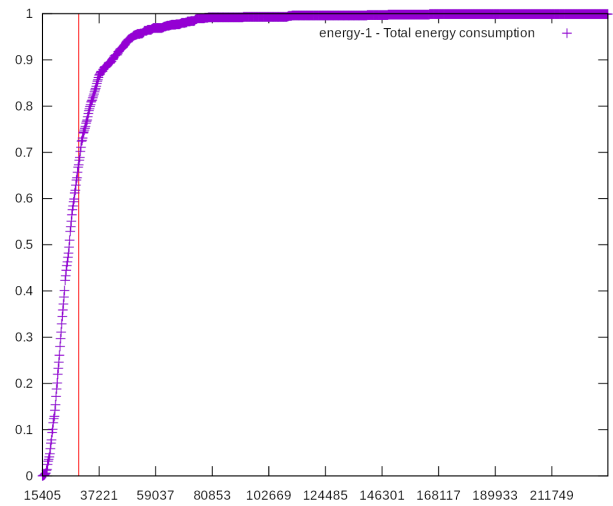


Figure 9: Energy consumption cumulative distribution function (energy-1)

Adding to the uncertainty, agents failed to complete their task in some scenarios despite our debugging efforts, which is summarized in Table 3. For this table, “failed” means that all agents had their battery depleted.

Total runs	Successful	Failed	Deadlocked
1128	1123 (99.56%)	1 (0%)	5 (0.44%)

Table 3: energy-1 outcome breakdown

The outlook for the energy-2 scenario is no less dire. In fact, less runs were made due to the aforementioned data collection limitations and because of the higher number of deadlocks or incomplete runs. A summary of the recorded ones is given in Table 4. Scenario complexity aside, some reasons for these poorer results might be the heuristic used to gauge the necessary energy units to reach the final station, which did not directly consider the number of packets to circumvent in the way, particularly high at the beginning of the run. It also failed to account for the possibility that the nearest battery station may not be available for use at all considering the long distance between the farthest battery station and the destinations, meaning agents will more often compete for the available charging station. Lastly, errors in the implementation logic (which involved 15 classes which were not always decoupled) could also be responsible for subpar results.

Total runs	Successful	Failed	Deadlocked
80	68 (85%)	4 (5%)	8 (10%)

Table 4: energy-2 outcome breakdown

All in all, the trend visible in Figure 8 is replicated in Figure 10, in which the deviation from the minimum value may be attributed to comparatively less data points, although this is

inconclusive.

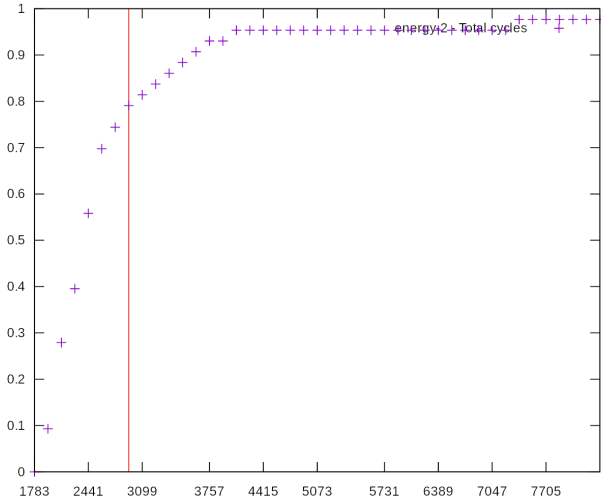


Figure 10: Total cycle cumulative distribution function (energy-2)

Interestingly, the aforementioned trend is not visible in Figure 11. This may be caused by the larger distances between energy stations and destinations, which is not accompanied by an increase in battery capacity, causing several lengthy roundtrips between destination and energy station, with few packets delivered in the process, though more runs should be performed to increase confidence in this hypothesis.

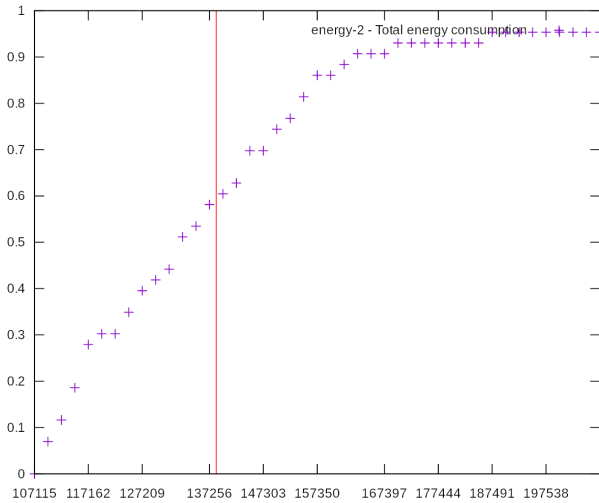


Figure 11: Energy consumption cumulative distribution function (energy-2)

A final improvement point for energy management is the number of agents whose battery was depleted in a run, detailed in Figure 12 and 13. Unfortunately, the global numbers are not informative enough on their own to locate the issue causing agents to run out of battery, and the situations in which errors arise can be elusive. Comparing the two figures, it is clear that the majority of debugging efforts should go towards energy-2, which will likely improve the performance in energy-1 as well. More specifically, the interaction between local coordination strategies should be reviewed, as during testing, it was noticed that vision, including the criteria thresholds to infer the number of agents charging, knowledge about battery stations and gradient values were jointly the source of conflicts around the very nearest battery station, followed by access obstacles for the second one.

Battery depletion breakdown

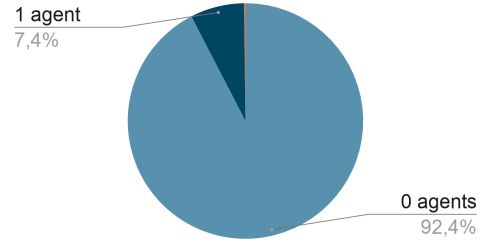


Figure 12: Battery depletion breakdown(energy-1)

Battery depletion breakdown

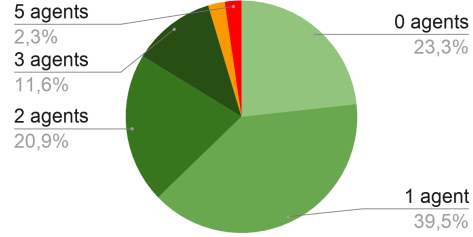


Figure 13: Battery depletion breakdown(energy-2)

With respect to pathfinding, the proposed pheromone usage only served to avoid the simplest of cases, and even created additional issues when the blockage was spurious, due to other agents. Although a traditional (e.g., as proposed in [5]) shortest-path calculation over the field of view would have been useful, particularly in scenarios such as energy-3, it would not have resolved the very issue for which it was created, namely, avoiding dead ends when there was no other information to indicate another more preferable route.

For task delegation, while an implementation is provided that eventually delivers, some state transitions issues remain due to management and time constraints, particularly when all paths have been cleared (where in practice the agent would behave like `simple`). As a result, no performance results are provided, since they would not be informative of the true agent's time efficiency.

A noteworthy addition to the `task_delegation` agent's implementation is A* pathfinding, proposed in [5], if the destination is visible, providing superior control when avoiding obstacles with a large enough vision range (as is the case for the task-delegation scenario).

VII. DISCUSSION OF RELATED WORK

In our work, communication between agents has been the key part of the agent's coordination in achieving the desired task. Recent research has explored the direction of a more decentralized (local) communication between agents without the need for a global communication between agents. This is mainly due to the unreliability of the network in a centralized system and third party attacks on the system that may cause the entire system and all the agents to fail.

Andrés et al. [2] explore this idea and establish a five layer protocol for the local communication between the agents. The first process is the characterization of the agents'

components and their relationship with the environment; the second is the analysis of transmissions between agents; the third is initializing agents for their connection to the ad hoc network; the fourth process is the generation of the activity history in each agent in the system. Finally, the fifth process describes information transmission through a header to be included in the information packet.

The major advantages of using such a system is faster communication with no adversarial attacks from third party systems. However, it removes the ability of agents to manage resources with foresight on future requirements as they will not be aware of all other agents' resource requirements.

The energy management that we have implemented uses global communication to anticipate the need to recharge without a need for longer wait time. The suggested system in [2] will potentially cause more agents to fail with battery improper management.

Further research has been done to guarantee global communication system security and reliability, which are becoming important aspects to manage agents in a complex environment. Vidya et al. [3] explore the idea of blockchain to cover both in Agent to Agent communication, protecting against adversarial attacks at the cost of scalability. Developments based on this idea are ongoing.

Multi agent coordination is another actively researched topic, as is task delegation where the agents must coordinate in order to clear the path to their drop points without communication. One example we find in the work of Cacciamani et al. [4], discussing scenarios where agents can coordinate in adversarial environments with no direct communication, akin to a game of poker once the game starts, thus using the indirect, environment-mediated one instead. They suggest use of Signal Mediated Strategies where the action of other agents can be mapped to an understanding of the other agent's needs by learning.

This strategy bears similarity with the *Waltz* behavior, where agents have an understanding of where to move only based on each other's presence. As embedded in the agent's implementation, other agent's movement in the vicinity of an energy station is interpreted as required to charge, and to resolve a potential conflict, coordination without direct messaging is achieved. Another possible use of this strategy could have been in task delegation: certain sequences of other agents' actions could have been mapped to convey need (e.g., a closed loop route beside a set of obstacles packets of different colors), availability (e.g., no movement) or want (e.g., surround an agent to indicate it should follow) to the other agent's. Thus even further messages for coordination can be conveyed to the other agents without a communication network. This signal meditation technique is similar to how humans use face modulation and body gestures to communicate without voice and only the person who receives it can map the true meaning to ensure no interception of communication.

VIII. CONCLUSION

In this paper, some key challenges of situated MASs and solution proposals for them have been discussed in the context of warehouse management automation, in particular agent Autonomous Behaviour, Energy Management and Task delegation.

To develop a working situated MAS we use the Packet-World as the test bed, which provides the middleware to interact with the environment and other agents in a simulated setup.

To manage agent Autonomous Behaviour, a task state machine was implemented onto each agent to perform basic tasks using available perception and memory as well as change roles based on environmental conditions. It is determined that shortest path calculation using Euclidean distances poses a small improvement over Manhattan distances, and that memorizing a few key pieces of information such as last seen packet or destination per color greatly improves execution times.

For energy management, agent local and global communication is leveraged to maintain agents operational. Results show that MAS implementation performance is indeterministic even in simulated environments and that, in addition to completion time, energy spent and messages exchanged, the number of agents with their battery depleted is a key consideration.

Finally, task delegation is implemented to allow it when necessary, demonstrating solutions to an increasing number of considerations require more sophisticated approaches, not only in the agent's task state machine but also in pathfinding (e.g., using A* to reach the current destination).

One common aspect of the MAS implementations in this work is the manual adjustment of thresholds, state transition conditions and even subtasks within each state. Given the static nature of the problems to be solved (e.g., move packets and avoid battery depletion) and the clear objectives (e.g., minimize completion time), optimal MAS generation could be performed procedurally through machine learning pipelines, for instance by applying policy learning with approaches such as the one proposed in [11], or semantic genetic programming as explored in [15].

Another relevant research area would be dynamic state transition management, where rather than statically defining a state graph for each agent, only basic tasks (e.g., move towards object or communicate fact) need to be defined and agents build temporary transition diagrams between them based on their interaction with other agents, in a line similar to that discussed in [12]. This would present a more scalable solution to behavior transition specification, as there are indications that implementation complexity increases with the number of objectives considered.

A final future work suggestion is the effects of bounded rationality into the implementations used here, as agents often have memory limitations that may render some approaches (such as A* for pathfinding) infeasible, applying for instance the work in [13] to non-evolutionary programming.

REFERENCES

- [1] Weyns D., Helleboogh A., Holvoet T. (2005) The Packet-World: A Test Bed for Investigating Situated Multi-Agent Systems. In: Unland R., Calisti M., Klusch M. (eds) *Software Agent-Based Applications, Platforms and Development Kits*. Whitestein Series in Software Agent Technologies. Birkhäuser Basel.

- [2] Andrés C. Jiménez, Vicente García-Díaz and Sandro Bolaños (2018) A Decentralized Framework for Multi-Agent Robotic Systems
- [3] Vidya Krishnan M., Rajesh Koduri, Sivaprasad Nandyala, Mithun Manalikandy (2020) Secure Vehicular Communication Using Blockchain Technology
- [4] Cacciamani F., Celli A., Ciccone M., Gatti N. (2021) Multi-Agent Coordination in Adversarial Environments through Signal Mediated Strategies
- [5] Hart P. E., N. Nilsson J. and Raphael B., A Formal Basis for the Heuristic Determination of Minimum Cost Paths, In IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, July 1968, doi: 10.1109/TSSC.1968.300136
- [6] ROBO Business Report (2019) <https://www.roboticsbusinessreview.com/wp-content/uploads/2019/10/RiseOfTheWarehouseRobots-LogisticsIQ.pdf>
- [7] Balaji P.G. and Srinivasan D. (2010) An Introduction to Multi-Agent Systems
- [8] Hyland M. F., Mahmassani H. S. (2017) Taxonomy of Shared Autonomous Vehicle Fleet Management Problems to Inform Future Transportation Mobility. In: Transportation Research Record, vol. 2653, no. 1, 2017, pp. 26–34.
- [9] René N., Oliveira A., Takashi T., Takeshi S., Nobutada F., Takeshi T. (2018) Multi-Agent Simulation-Based Analysis for Restaurant Service. In: Procedia CIRP, vol. 79, 2019, pp. 673–678.
- [10] De Silva V., Wang X., Aladagli D., Kondo A., Ekmekcioglu E. (2019) An Agent-Based Modelling Framework for Driving Policy Learning in Connected and Autonomous Vehicles. In: Arai K., Kapoor S., Bhatia R. (eds) Intelligent Systems and Applications. IntelliSys 2018. Advances in Intelligent Systems and Computing, vol 869. Springer, Cham.
- [11] Karimpanal T. G., Wilhelm E.. (2017) Identification and off-Policy Learning of Multiple Objectives Using Adaptive Clustering. Neurocomputing (Amsterdam), vol. 263, 2017, pp. 39–47.
- [12] Castelfranchi C., Falcone R. (1997) From Task Delegation to Role Delegation. In: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 1321, 1997, pp. 278–289.
- [13] Manson S. M. (2006). Bounded Rationality in Agent-Based Models: Experiments with Evolutionary Programs. In: International Journal of Geographical Information Science : IJGIS, vol. 20, no. 9, 2006, pp. 991–1012.
- [14] Weyns D., Steegmans E., Holvoet T. (2004) TOWARDS ACTIVE PERCEPTION IN SITUATED MULTI-AGENT SYSTEMS. In: Applied Artificial Intelligence, vol. 18, no. 9-10, 2004, pp. 867–883.
- [15] Vanneschi L., Castelli M., Silva S. (2014). A survey of semantic methods in genetic programming. In: Genetic programming and evolvable machines, 15(2), pp.195–214.

Collaborative Situated Agents for Baseline Logistics Problems on the Packet-World Testbed

Georgios Kouros, Gregoris Malekos, Bolat Tleubayev, Victor Van Wymeersch
Faculty of Engineering Science, KU Leuven
Leuven, Belgium

georgios.kouros@student.kuleuven.be, gregoris.malekos@student.kuleuven.be,
bolat.tleubayev@student.kuleuven.be, victor.vanwymeersch@kuleuven.be

Abstract—Multi-Agent Systems (MAS) enable the efficient solution of complex real world problems with flexibility and robustness to system perturbations. However, the implementation of these systems is nontrivial and requires careful consideration into software design principles, in support of the attractive properties of MAS. To this end, three key problems within MAS have been solved in this paper, namely autonomous navigation, energy management, and multi-agent coordination and cooperation. Grid search, gradient methods, and protocol-based communication are proposed as a means to enable autonomous situated agents to tackle the aforementioned tasks. The solutions are implemented on *Packet-World (PW)*, a testbed for experimentation with situated MAS, meant to be used to investigate and evaluate different perception, decision making, and communication aspects among agents that are situated in a virtual environment. The proposed solutions are evaluated with regard to total energy consumption and/or number of cycles for completing a run averaged across batch runs.

Index Terms—multi-agent systems, Packet-World, situated agents, grid search, gradients, protocol-based communication

I. INTRODUCTION

Increasingly complex industrial systems require flexible, robust, decentralized dynamic control solutions. Multi Agent Systems (MAS) are software systems that provide a solution fulfilling all of these requirements [1]. These systems are composed of multiple singular agents that work together to solve intricate problems beyond the scope of a single agent's capabilities. To develop these system solutions it is pertinent to be able to test MAS research and developments before they are applied. To this end an open-source development platform *Packet World* is used for the development of these MAS. Here the agents are required to move around in a grid-based world, communicate and coordinate to pick up and deliver various packages to defined locations [2].

This paper focuses on the development of techniques to control agent behaviour, coordination of an agent energy management systems, and communication between multiple agents.

Section II gives an introduction to PW, its components and general architecture. Section III contains a description of the problems that this assignment is aimed to solve, while Section IV presents the proposed solutions. The paper concludes with a listing of related work and a final discussion in Sections V and VII, respectively. For clarity discussions

on future work or possible implementation improvements are located within the individual parts of Section IV.

II. PACKET WORLD

Packet-World [2] with regard to its pattern language on the domain of *situated* multi-agent systems and the various aspects of this pattern language are presented in the following section.

A. Packet World Concepts and Characteristics

PW is based on the Situated Agents pattern that promotes module reuseability and decoupling of independent processes. *Agents*, *ongoing-activities*, and the *environment* are the three main abstractions of this system. The environment is observable by the agents and contains processes that regulate states and interactions. The agents can act independently based on their local perception of the environment and goals. They can communicate with each other through messages to form collaborations to jointly complete tasks when needed.

The environment of PW is a multi-layered grid, where each layer is assigned to a different entity (eg. agent, packet, flags) and has its own constraints (eg. one packet per cell). The state of the world consists of the combined grid layers and agents perceive only the layers that are relevant to them.

The perception module maps the local state of the environment to a percept for the agent. Percepts change the internal knowledge held by the agent of its environment. Percepts are provided by three subsystems, sensing, interpreting and filtering. Sensing allows the agent to focus on a specific perception ability to gain information of the world, for example through sight or smell. Sensory information is translated into percepts through internal knowledge descriptions of the world. Finally, percepts can then be filtered to exclude unnecessary information not relevant to the agents' intended search.

From local knowledge of the world, agents make decisions on which actions to perform to achieve goals with minimal energy requirements. This decision making process is represented using a free-flow tree. Role and situated commitment extensions enable social behavior. The tree nodes receive information internally, through the agents' knowledge and externally from perceptions. Nodal activity is passed down the tree to leaves where an action is selected. In the free-flow trees agents have sub-trees dedicated to cooperation-, individual-,

and battery recharging-based roles. Commitments define the relationships between roles.

Communication happens with the help of protocol steps, which are tuples (conditions, effects). Some conditions must be satisfied for an agent to send a message and some effects result from the protocol step being applied. There is a finite number of different messages which the agents can send to one another. Via those messages they can request the help of other agents to solve tasks cooperatively and answer other agent's requests for help. Information relevant to the tasks can also be shared in this way.

Advanced agent collaboration has been built into PW. Agents using explicit and implicit messaging can collaborate in many ways, such as letting other agents know where packets/goals are or setting up chains to move packets along faster. Some tasks, such as moving large packets, specifically require agent cooperation. For these tasks, messaging allows agents to collaborate in a timely manner.

B. Real-world applications

In this subsection some potential future industry applications of MAS are laid out in order to outline the relevance of MAS research to real-world problems.

In a *MAS of autonomous taxis* serving a city, each taxi is represented as an agent that is tasked with picking up and dropping off passengers. Taxis have to work together to transport large groups of passengers that sometimes might not fit into one taxi. The goal is to transport as many passengers as possible within the day while minimising total travel distance and fuel consumption. Gradient fields can be used to attract agents to areas with higher demand, or for refueling purposes.

A *MAS of waiters* serving people in a restaurant. The layout of the restaurant can be modelled within the PW grid with waiters acting as agents. Agent tasks may include taking orders, serving food, cleaning tables, represented as differently colored packets, or even cooperation for service of big tables of customers.

III. PROBLEM DESCRIPTIONS

A. Autonomous behaviour of single agents

Within PW individual agents are, among several others, required to be able to autonomously perceive the environment, navigate the terrain, pick up and drop off packages, move to recharge stations when their battery level is low, coordinate and collaborate with other agents etc. The most fundamental of these responsibilities is the ability to independently, as a single agent, perceive and navigate the environment to deliver packages to the drop-off points/destinations. The software architecture defining the agents' autonomous behaviour needs to achieve this goal in the most effective way while maintaining the benefits of multi-agent systems in general, such as flexibility and adaptability. To this end, the aim is to program optimised agent behaviours using the well-established architectural software pattern of situated agents.

B. Coordination of agents to manage energy

Creating agents that eventually are able to perform all the tasks required of them in PW does not pose a great challenge. A more challenging goal is to introduce energy constraints and program agents that are able to perform the tasks while ensuring no agent runs out of energy. To do this, the agents' behaviour when tackling their tasks must be optimised to avoid unnecessary energy wasting. Additionally, it is required to set up a system for agents to coordinate and manage energy consumption/charging. In MAS, agents coordinate and communicate directly with each other, avoiding the large overhead costs of being coordinated by a central authority which would keep track of everything and introduce a single point of failure in the system. There are two main methods for agents to coordinate [2]. The first, which is explicit, is direct messaging with requests/answers between agents. The second, which is implicit, is the use of gradient fields within the PW environment which inform agents of the existence of goals/charging points to them. In both cases the aim is for the agents to act as a coordinated unit rather than a sum of individuals competing to complete the same tasks.

C. Collaboration of agents with each other

The assignment of tasks in a MAS can be complex and difficult to manage. Often tasks cannot be completed by individual agents without the help of other agents. Moreover, agents often operate in large dynamic environments where the number of agents able to do work changes along with their goals. Due to this complexity and scale, centralised task assignment lacks the flexibility needed in MAS where the ability to rapidly switch task assignments between agents for optimal coordination and execution on a system level is crucial. Various methods exist to tackle this problem of task assignment, namely environment-based, and protocol-based. Environment-based task assignment involves populating agent environments with various gradient fields to attract them to goals and repulse them from each other. Protocol-based task assignment is more focused on messaging, where tasks send out requests for work towards all agents in the system and the agent response with the best proposal is selected for the task. Both methods allow for task reallocation, either by the update of gradient fields, or through new task requests allowing the methods to cope with variations in the system.

IV. PROBLEM SOLUTIONS

A. Autonomous behaviour of single agents

Agent behaviours were tackled in three distinct steps, building up in complexity. Starting from the aimless random wandering behaviour, agents were programmed to pick up and drop off packets should they reach the appropriate tiles. These are tiles adjacent to either packets to pick-up or drop-off destinations for packet delivery. In the second phase we refactor the wander behaviour according to the situated agent architectural pattern as shown in figure 1. In particular, the refactoring included the distribution of behaviours to distinct classes and the development of behaviour change classes that

switch from one behaviour to the other in a cyclical fashion as seen in figure 2. This transitioning between behaviours is described in more detail in section IV-A1. In the third and final phase, the wandering behaviour of the agents is optimised to improve packet delivery time and energy consumption through goal-oriented navigation. Specifics of this improvement are covered in section IV-A2 below.

1) *Design*: The design selection process involved analysis of two types of MAS architectures, namely Belief-Desire-Intention (BDI) by Rocha et al. [1] and Situated Agents [2]. We compared how would the BDI fit to our problem against the Situated Agent architectural pattern. One advantage of the BDI pattern was its focus on distributed autonomous behaviour. However, in the end, we settled on the Situated Agent architectural pattern [3] due to its innate suitability to Packet-World and its capabilities for agent-to-agent communication and coordination and agent independence (agents maintain their own behaviour and state). Also, its distributed architecture comprised of knowledge, perception, decision making, and communication components in each agent, enabled us to easily create, test, and assess increasingly more complex behaviours.

The design of the software architecture for the agent behaviour follows the form as presented in Figure 1. In this free-flow tree of a single there are two distinct behaviours, *GetPacket* and *DeliverPacket*.

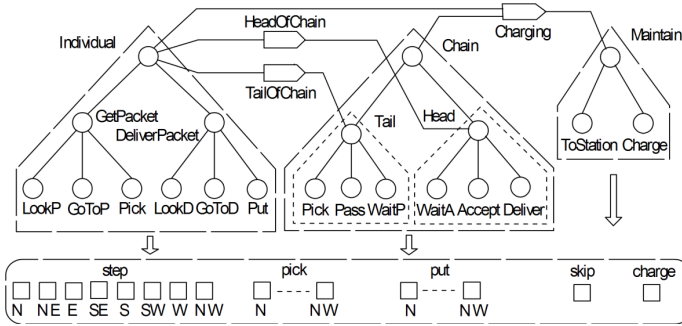


Fig. 1. Action selection free-flow tree of situated agents. Original diagram from Weyns et al. [2]

Concretely the behaviour change pattern consists of two distinct classes *BChangeGet2Del* and *BChangeDel2Get* to change agent behaviours from getting packets to delivering packets and visa-versa, based on the conditions presented in Table I. This software architecture pattern improves agent behavioural flexibility, allowing for additional behaviours such as "Recharge" to easily be incorporated into the existing code structure in the future. Furthermore, according to the situated agent action selection free-flow tree in Fig. 1, the two implemented behaviours can be hierarchically subdivided into smaller behaviours. For example, split *GetPacket* could be divided to *LookForPacket*, *GoToPacket*, *PickUpPacket* with corresponding behaviour changes.

2) *Implementation*: Implementation of the code follows directly the design as described above. However, in the final implementation an optimisation has been incorporated to

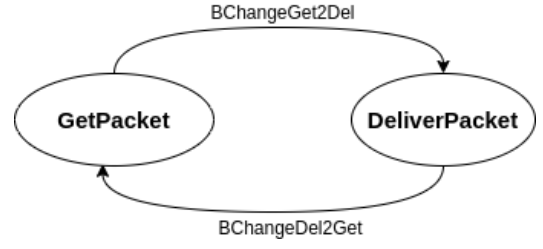


Fig. 2. Agent behaviour change classes.

improve the efficiency and performance of the agent packet collection and delivery. Instead of wandering to boxes and drop off points randomly, agents directly move to collect closest packets and drop them off at the closest drop off points with the same color. The agents always select the packets that are closest to them, provided that they are within their perception field. If the agents cannot find any packets within their perception field they perform random movements until they find one. The same approach is used for destinations, although, when a destination is found, it is stored in the agent's memory in order to avoid redundant searches. One potential future improvement would be to replace this random exploration with a targeted one that takes into account which parts of the environment the agents have already visited. This idea could be implemented quite using flags [2] placed suitably to prevent agents to re-explore already cleared areas.

To establish the best direction to walk in, agents calculate the distance from their current position to their target locations, such as packets or drop-off points. Two distance metrics were utilized namely, Manhattan and Euclidean distances. From the distance measurement agents take the best step, closest to the target destination, if possible. To prevent agents getting stuck when facing obstacles in the path to their desired locations, possible moves are ranked according to the shortest distance to the target location. Should the shortest move be blocked, the agent will take the second best step towards its goal. If the second best step is infeasible, it will take the third, etc. The results of this optimisation can be seen in the following section IV-A3.

3) *Test results*: Figure 3 shows the improvement in the speed it takes the agents to collect 60 packets of different colours in the PW environment. The optimised walking behaviours based on Manhattan and Euclidean distance measures are able to collect all 60 packets in 668, and 333 cycles, respectively. The Euclidean distance measure provides more than 41x improvement in speed when compared to the random walk. Moreover, since Euclidean distance measures the shortest path agents exploit diagonal movements, improving efficiency over the Manhattan distance measure. The total energy consumption to collect the 60 packets before and after behaviour optimisation is 983440 and 18065 units, respectively, a 54x decrease in total energy consumption. This improvement is illustrated in figure 3 in which the number of delivered packets are plotted against the Packet-World cycles for a typical run in each of the three walking behaviours.

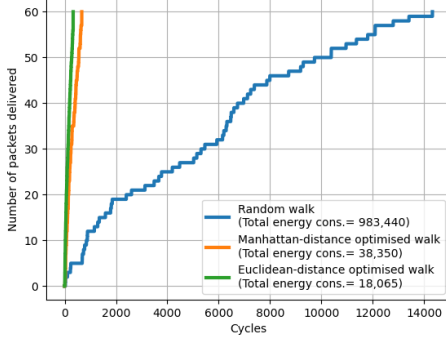


Fig. 3. Performance increase of optimised agent behaviour.

B. Coordination of agents to manage energy

Agent coordination to tackle energy management requires agents to be able to communicate either implicitly or explicitly, as well as agents to be aware of the location of charging stations in their vicinity. For agents to locate charging stations, gradient fields were implemented to guide them to the stations. The gradient value of each tile in the Grid-World was calculated using the Manhattan distance from the agent to the closest charging station, taking into consideration any obstacles like walls.

A messaging system allowing direct communication between agents was set up to manage priority in charging, which aimed to avoid agents running out of energy while waiting for another agent to finish charging. Locating and navigating to packages and goals was done in the same fashion as in section IV-A with some minor improvements detailed below.

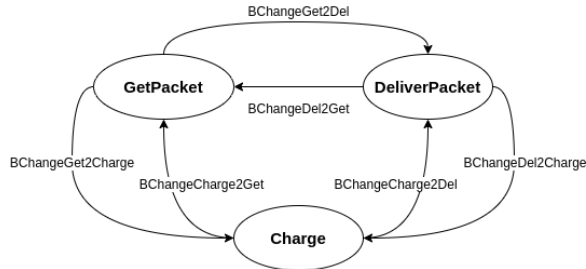


Fig. 4. Agent behaviour change classes with the added Charge behaviour.

1) *Design*: The design decisions made were based on the most intuitive ways to tackle each problem. As proposed by Weyns et Al. [2], a system of gradients was used to lead agents to the battery charging points. Once agents were below a certain battery threshold they would switch to charging mode and use this gradient field to navigate towards the closest charging station. The threshold used was dynamic depending on the distance of the agent from charging points as well as on whether or not the agent was carrying a packet. This allowed agents to be aware of whether they have enough charge remaining to complete their current task first, or if they needed to abort it in order to go recharge straight away.

Navigation was based on the Euclidean distance measure as it provided a 2x decrease in energy consumption when compared to the Manhattan distance measure as highlighted in figure 3.

Finally direct messaging was used when an agent was in urgent need of recharging. If an agents battery level was below a critical threshold they would send out a request to take priority in charging. In case another agent was using the charger, they would check whether their own battery level is above a certain yield-threshold. If both of these criteria were fulfilled the agent currently using the charging station would vacate it and give priority to the agent with the critically low battery level, while remaining in standby mode until the charging station was freed again.

2) *Implementation*: To implement the behaviours described above a new behaviour was added named Charge as presented in figure 4. This behaviour was activated when agents needed to move towards the closest charging point to recharge their batteries. In addition, four behaviour change classes were added. These classes allowed the agents to transition between getting packages, delivering packages and charging. The activation of these behaviour changes occurs based on three conditions as presented in Table II.

Priority of charging in cases where an agent needed to use the charging port urgently was implemented using agent messaging, as detailed in section IV-A1.

The dynamic threshold used by agents for switching from their current task, say delivering a packet, to moving to recharge meant that agents can more accurately judge how much power is required to reach a charging station and avoid running out of power whilst getting there. Multiple threshold checks are done at each timestep by each agent to avoid running out of battery while remaining efficient. For example, agents attempting to deliver a package compute (1) how much energy is required to reach the charging station with and without carrying their current package (based on the agents gradient field value) and (2) how much energy is required to deliver the package (based on euclidean distance and energy consumption per step) and then return to the closest charging station without a package in hand. Thereby agents can decide on the best approach to take, which is often to deliver the package and then recharge without a package in hand. Agents were implemented to carry packages to charging stations if possible to avoid creating obstacles and impassable walls. However, when agents estimated that they would not reach a charge with a margin of battery remaining they would drop their packages and go to charge empty handed as this requires less energy to move.

To take into account the existence of gradient fields the walking behaviour of agents were extended. As in section IV-A2, agents always take the best step towards their goals given obstacles that may be in their way. However, for agents following gradient fields non-optimal movements were allowed when faced with obstacles allowing agents to avoid getting stuck in gradient field local minimums.

3) *Test results*: To assess the performance of our agents, a batch run of 100 runs was made on the energy-1 and

energy-2 environments. Figures 5 and 6 respectively show histograms of the total consumed energy by the agents at the end of each run. As can be seen the consumed energy is bounded within certain limits and does not show a big variation over the runs. This is especially true for the energy-1 environment where the energy consumption remained constant on the vast majority of runs. The agents are quite stable due to the removal of random elements in their behaviour, they always make informed choices and thus their behaviour does not alter significantly between runs. The average energy consumed for energy1 was 9529.35 units and the total cycles taken to complete the tasks were 228.21 over all 100 runs. For the energy 2 environment those values were of 54868.3 units and 886.82 cycles respectively. Future work should include a behaviour allowing agents to drop packages in clusters around other packages if they have critically low battery instead of in random locations. This would decrease the variance in energy consumption between runs, as in some cases agents can create walls of packages on the way to charging stations.

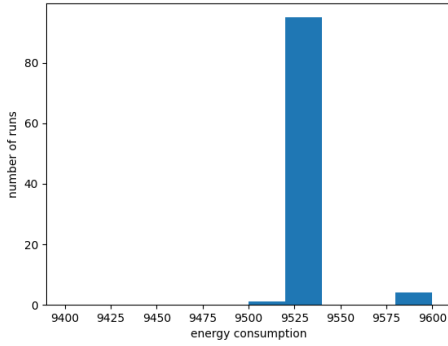


Fig. 5. Energy consumption on energy1 over 100 runs.

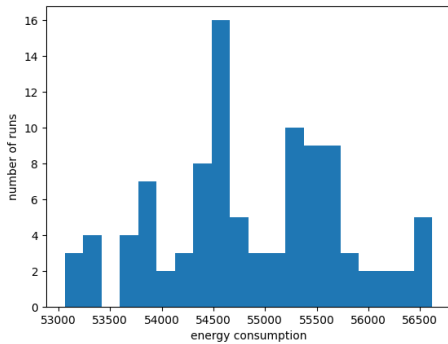


Fig. 6. Energy consumption on energy2 over 100 runs.

C. Collaboration of agent with each other

In the agent collaboration task three agents of three distinct colours are present, each able to pick up packages corresponding to their colour. Walls in the environment limit agent

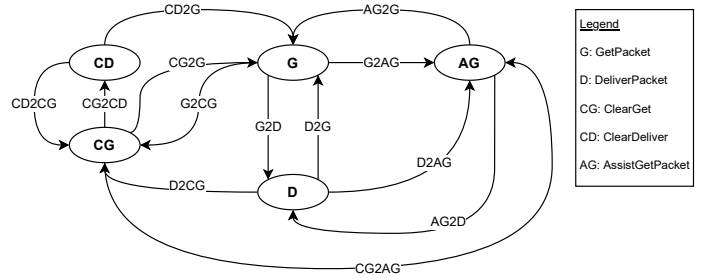


Fig. 7. Agent behaviours and behaviour changes for task III, enabling agents to clear their paths and assist other agents with theirs.

movements and packets are placed such that agents require other agents to help them in clearing the paths to their respective package drop off points. In this situation cooperation is required between agents to achieve the global goal of delivering all packets in the environment. The environment upon initialisation is illustrated in figure 9(a) below.

1) *Design:* Upon tackling the problem of agent coordination and task delegation, two basic approaches were considered. A field-based approach and a protocol-based approach.

Field based approaches rely on implicit communication through the adaption of environment. Environments can be adapted by agents by populating the grid-world with gradients, placing flags, or releasing pheromones as described by Weyns et al. [2]. These changes made to the environment are perceived by agents. From these environmental signals agents can then decide how to act. For example, when navigating down narrow corridors an agent might know to avoid paths with other agents pheromones as two agents going down the same corridor might lead them to getting stuck. Alternatively, agents may send out repulsive gradients signalling to other agents that they are in the vicinity as described by Weyns et al. [3].

Protocol-based approaches rely on explicit means of communication such as direct messaging. When an agent requires the help of another agent the agent sends a message with a request. The other agent can then either accept or decline the request based on their priorities.

Both approaches have advantages and disadvantages, however it was deemed more pragmatic for agents to directly communicate with each other instead of indirect communication by leaving markers on the environment. The large perception fields present within this task meant that agents did not struggle to find one another when in need of help. This greatly simplified the approach and made for more efficient, direct task solving.

The basic principles of this design approach is as follows. Initially agents wander around looking for drop-off locations and packets corresponding to their colour. When a goal or packet is found in their perception field, a breadth first search is used to find the shortest path to the target. If on this path to their drop-off point is blocked by a packet which the agent is unable to pick up, the agent would find an appropriate agent for help. When the correct agent is found (corresponding to the

colour of the packet blocking their path) the agent then directly messages the agent for help. The message sent requesting assistance contains the location of the packet to be moved. The process of searching for optimal paths to drop-off points and messaging agents for assistance is repeated until all the packages are delivered in the packet world.

2) *Implementation*: One of the vital components of the implemented system is Breadth First Search (BFS) inside an agents perceptive field, used for the establishment of the optimal path towards a package drop-off point and for retrieval of packets. Such a search scheme ensures that agents do not get trapped behind obstacles when using standard distance measures when walking. On Figure 8 you can see each step of the search depicted as different color. The optimal path is towards the agents goal is given by the green arrows. BFS was chosen over other search algorithm for its relative simplicity and optimality, as it always returns the shortest path to a desired position in the presence of obstacles or non-walkable cells such as walls and environment boundaries such that there are no collisions. The BFS is computed for each agent at each timestep of the simulation, which is computationally tractable since agent perceptive fields are not too large. If the perception fields of the agents were too large, the more complicated A* search could be implemented as it is both faster and has less memory requirements.

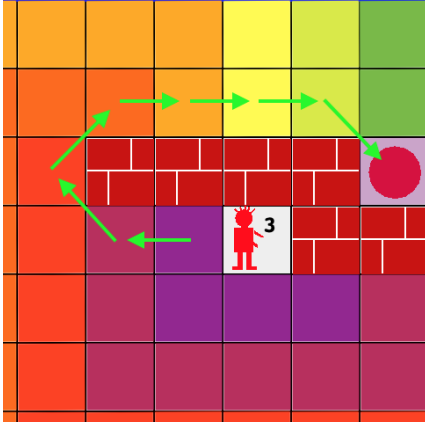


Fig. 8. Breadth First Search scheme.

Figure 7 shows all the behaviours of agents along with their corresponding behaviour change classes that are implemented. The conditions for each behaviour change is presented in Table III. The abbreviation of each behaviour is listed below along with a description of its functionality:

- **G** - GetPacket: Agents wander around the environment until they find and pick up a packet corresponding to their colour. The agents always choose the packets that are closest to them.
- **D** - DeliverPacket: Agents perform BFS to find the optimal path towards their destination and attempt to navigate towards it to drop off the packet they are holding.
- **CG** - ClearGet: When a package is blocking the agent they either get the packet to clear the path themselves

if possible or if another colour packet blocks the path, the agent finds another agent of the corresponding colour through random wandering and messages them for help along with the location of the packet to be moved. For the messaging to occur, the two agents must be within each other's perceptive fields.

- **AG** - AssistGet: Is the action taken by agents when they have agreed to assist another agent to clear their path. The request comes in the form of a message containing the location and color of the blocking packet and the ID of the agent requested to handle it so that an agent can recognize whether a request is intended for them or not. Then the agent goes to pick up that packet and deliver it to its destination. If the agent was already holding a packet, then they drop it out of the way before going to provide assistance.
- **CD** - ClearDeliver: An agent transitions to this behaviour when it has picked up a packet that was blocking the path to their destination. In this case the agent either delivers that packet if the destination is cleared or drops it out of the way and then attempts again to clear the path on their own or by asking for assistance.

3) *Test results*: From this behaviour change diagram and implementation agents are then able to solve the problem of task delegation in this complex environment, and effectively coordinate their actions to deliver all the packets. Figure 9 shows four stages during run-time with (a) corresponding to environment initialisation and (d) the termination of the program. In (b) one can see that the red and blue agents have successfully cleared the green agents path to its goal. In (c) agents have worked together in clearing all the paths to all the agents' respective drop-off locations.

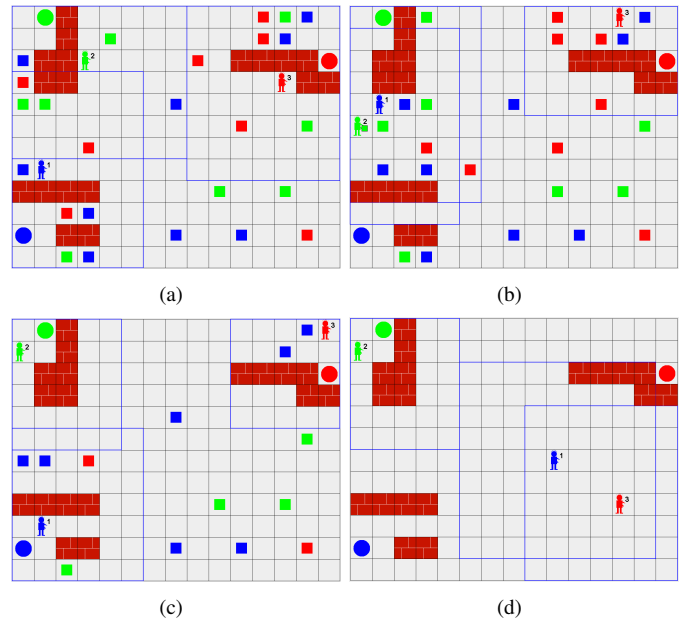


Fig. 9. Task delegation execution in packet world

Despite the implementation of the BFS, and protocol-based

task delegation, the global solution is still not always reliable. Figure 10 illustrates this over 50 runs on PW. Although most runs finished in under 1500 cycles, a considerable amount of runs took far longer, with some requiring even more than 5000 cycles. This inefficiency is mainly due to random agent wandering when searching for an agent to help and in agents waiting for their paths to be cleared by others. This inefficient behaviour could be improved by memorising the last seen locations of agents and various environmental markers. Information like flag locations would mean that agents could easily return to "memorised" locations and avoid wandering behaviour, allowing agents to indirectly communicate despite not seeing one another. Additionally, future work could also include the prioritisation of tasks among agents which would improve task efficiency.

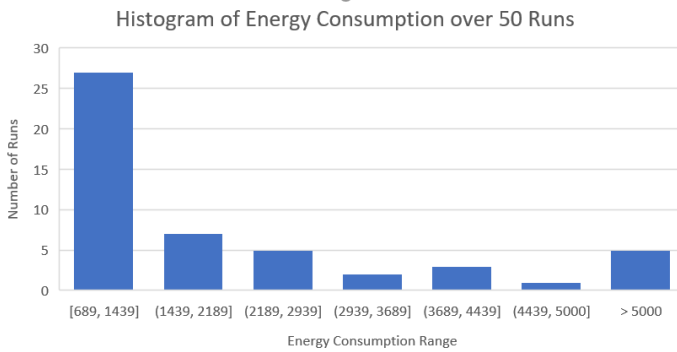


Fig. 10. Cycles taken histogram in batch run of collaboration task.

V. RELATED WORK

The field of Multi-Agent Systems (MAS) has received a significant amount of attention from the scientific community over the course of the last decades. Numerous classification approaches and design patterns were presented by a multitude of researchers. So, to gain valuable insights about the field, we have studied several significant works.

"An introduction to multiagent systems" by Wooldridge [4] provides us with an important notion on the place of MAS in the scientific world. This is done through presentation of the history of trends in computational science, followed by an examination of various scenarios, purposes, and objections to MAS. This paper does a good job of explaining the relevance of MAS solutions such as the ones seen in this paper in the future of logistics and robotics.

Rocha, Boavida-Portugal, and Gomes [1] give a detailed introduction into MAS, by presenting a general overview, classification criteria, and key characteristics of the field.

A work done by Juziuk, Weyns, and Holvoet [5] on the literature review of MAS design patterns provided us with important ideas on how to approach the development process of the project. In their work, they stress the importance of a systematic approach to the design and present a detailed overview of existing design patterns in the field of MAS. The goal of the paper is to increase the accessibility of knowledge

for practitioners in the field about design patterns. Following a systemic approach such as the one recommended allowed us to reuse large parts of our code on all three tasks solved. This also allows for the agent be further developed with more behaviours being added to the current patterns.

The main focus of Rao and Georgeff's [6] work was to explore a particular type of agents, Belief-Desire-Intention (BDI) agents. In their paper the authors laid a foundation for BDI agents from different perspectives and considered various implementations of such agents. Overall, the study presents a new paradigm for the development of MAS.

Weyns et Al. [7] provide a good interpretation of the environment as middleware in MAS. The middleware interpretation of the environment allows it to easily incorporate elements such as gradient fields and messaging which facilitate easy agent communication/coordination even between different types of agents.

VI. FUTURE WORK

Although all three presented problems were solved successfully, there is still much room for improvement to optimise these solutions in terms of turns it takes to complete the tasks and the resulting energy consumption. Furthermore, those two latter approaches failed to perform on the optional more challenging environments. In hindsight, integrating Breadth-First- Search-based navigation on the energy management task would have easily allowed the agents to navigate around walls in the energy-3 environment. The maze environment, on the other hand, presents an even bigger challenge and we hypothesized that it could have been solved using a stigmergic approach [2], such as flags or pheromones.

Approaches that include inter-agent information sharing about the explored world with the use of either direct messaging or flag placing for example could prove useful. Also, a predefined pattern through which to explore the environment in search of packets which would replace the random step exploration behaviour could greatly improve the overall performance of the solution. Some forms of indirect means of communication such as pheromones, flags or gradient placement could also be used to improve agent coordination when it came to the tasks requiring explicit cooperation.

The logical next step in this work would be to unify all the different behaviours presented in this paper in order to create a MAS able to tackle an environment that involves all the challenges faced in the three different parts combined. Such an agent would be one step closer to a solution which could eventually be implemented in real-life logistics situations.

VII. DISCUSSION AND CONCLUSION

In this paper we described three important topics or problems in MAS, namely the *autonomous behaviour of single agents*, the *coordination of agents to manage energy* and *collaboration between agents*. Within each of these areas designs are proposed and solutions implemented successfully solving the problems. From these respective solutions various key observations have been made.

TABLE I
BEHAVIOUR CHANGE CONDITIONS TRUTH TABLE FOR TASK I.

Condition	BChangeGet2Del	BChangeDel2Get
hasPacket	True	False

TABLE II
BEHAVIOUR CHANGE CONDITIONS TRUTH TABLE FOR TASK II.

Condition	BChangeCharge2Del	BChangeCharge2Get	BChangeDel2Charge	BChangeDel2Get	BChangeGet2Charge	BChangeGet2Del
hasPacket	True	False	True-	False	False	True
needsCharging	False	True	True	False	True	False

TABLE III
BEHAVIOUR CHANGE CONDITIONS TRUTH TABLE FOR TASK III.

Condition	AG2D	AG2G	CD2CG	CD2G	CG2AG	CG2CD	CG2G	D2AG	D2CG	D2G	G2AG	G2CG	G2D
hasPacket	True	False	-	False	True	True	True	-	-	False	-	-	True
pathBlocked	-	-	True	False	False	-	False	-	True	False	-	True	False
receivedRequest	False	False	False	False	False	False	False	True	True	False	True	False	False

When dealing with the autonomous behaviour of single agents, transitioning from a randomized grid traversal to a more sophisticated and targeted agent behaviour leads to a substantial improvement of the performance of the system. Total energy consumption and duration for completing the delivery of all packets is minimized significantly. However, this improvement is far from optimal. Employing selfish competitive agents results in the generation of conflicts and deadlocks between agent goals, among other problems. Two or more agents might target the same packet, but only one will pick it up, while the other will have gone towards that packet for nothing, resulting in a considerable waste of time and energy.

The problem of energy management while delivering packets requires several considerations such as battery gradient fields and communication. Gradient fields are an adequate way of guiding agents to charging points, although they provide no information about whether this charging port is currently being used or not. A basic system of messaging allows agents to never run out of battery by informing each other in case one urgently requires to charge so that the other agents may yield that agent priority. Messaging strategies such as these, the use of gradient fields, and optimised movement can help ensure that agents do not run out of power during simulations. Estimations of the energy required to complete tasks improve agent efficiency and help avoid the problem of agents running out of power on the way to charging stations.

While messaging can help manage battery demands and let critically low agents have preference to chargers, it by itself does not mean explicit agent coordination. True coordination and cooperation between agents is essential for completing tasks that cannot be completed by single agents. Agent coordination and task delegation highlight importance of integrating structured communication between the agents, enabling them to minimize their energy consumption and cooperate to solve

complex tasks that cannot be solved individually. Indirect messaging between agents, through the placement of markers on the environment, can further improve cooperation and overall MAS efficiency as agents can then respond to agent requests despite not being in range to directly communicate.

REFERENCES

- [1] Jorge Rocha, Inês Boavida-Portugal, and Eduardo Gomes. Introductory chapter: Multi-agent systems. In *Multi-Agent Systems*. IntechOpen, 2017.
- [2] Danny Weyns, Alexander Helleboogh, and Tom Holvoet. The packet-world: a test bed for investigating situated multi-agent systems. In *Software Agent-Based Applications, Platforms and Development Kits*., Whitestein Series in Software Agent Technologies and Autonomic Computing, pages 383–408. 2005.
- [3] Danny Weyns. Architecture-based design of multi-agent systems. 2010.
- [4] Michael Wooldridge. *An introduction to multiagent systems*. John wiley & sons, 2009.
- [5] Joanna Juzziuk, Danny Weyns, and Tom Holvoet. Design patterns for multi-agent systems: A systematic literature review. In *Agent-Oriented Software Engineering*, pages 79–99. Springer, 2014.
- [6] Anand S Rao, Michael P Georgeff, et al. Bdi agents: From theory to practice. In *ICMAS*, volume 95, pages 312–319, 1995.
- [7] Tom Holvoet Danny Weyns, Alexander Helleboogh and Michael Schumacher. *The agent environment in multi-agent systems: A middleware perspective*. 2009.

Refining autonomous and collaborative behaviour in the Packet World

1st Kristof De Troch

Computer Science
KU Leuven

Leuven, Belgium

kristof.detroch@student.kuleuven.be

2th Robin Spiltijns

Computer Science
KU Leuven

Leuven, Belgium

robin.spiltijns@student.kuleuven.be

3nd Emma van Doren

Computer Science
KU Leuven

Leuven, Belgium

emma.vandoren@student.kuleuven.be

4rd Maarten Wyna

Computer Science
KU Leuven

Leuven, Belgium

maarten.wyna@student.kuleuven.be

Abstract—Various test beds have been introduced throughout the literature to facilitate the exploration of fundamental concepts and mechanics in multi-agent systems. This paper aims to provide the reader with intuitive insights into these concepts and mechanics by optimising agent behaviour in one such test bed: *Packet World*. Three fundamental problems in *Packet World* are decomposed, described and solved: optimising autonomous behaviour, managing energy and delegating tasks. First, a combination of perception, communication, memory and flags is used to optimise autonomous behaviour. Next, energy management is mainly solved by using gradients. Finally, task delegation relies on a simple communication protocol. These solutions are tested, after which the results are discussed in context of their real-world applications. The results show that the specific environment in which the multi-agent system is situated has a large impact on the optimal implementation and parameters. Regardless of the environment however, the introduction of memory in agents has the biggest impact on performance.

Index Terms—multi agent systems, autonomous behaviour, managing energy, task delegation and design

I. INTRODUCTION

Multi-agent systems (MASs) are an interesting and relatively new approach to problem resolution in a distributed context. Test beds such as *Packet-World* [1] were created to explore fundamental concepts and mechanics for MASs. This paper delves into some of these concepts and mechanics by attempting to optimise the behaviour of agents situated in *Packet-World*.

By the end, the reader should have gained some intuitive insight in three fundamental problems that arise in *Packet-World* and how one can reason about solving them. These problems are optimising autonomous behaviour, managing energy and delegating tasks. Each problem will be intuitively decomposed, after which solutions to them are designed, implemented and tested. The main performance metrics will be the amount of energy and the amount of cycles required to deliver all packets in a given environment.

What follows is a brief outline of the rest of this paper. Section II introduces the *Packet-World*. In Section III, two real-world problems that loosely map onto *Packet-World* are laid out. The problems associated with basic behaviour in *Packet-World* are described and decomposed in Section IV. Section V solves the first sub-problem: optimising autonomous behaviour. Section VI then tackles energy management. In

some worlds, task delegation plays a key role, which is described in Section VII. The results of various tests conducted throughout this paper are discussed in section IX. Finally, Sections X and XI go over some related and future work.

II. INTRODUCTION OF THE PACKET WORLD

A. Agents

The agents of this MAS are situated [4], which means they can interact with each other and the environment they are placed in. Their decisions are influenced by both the environment and their communication among each other.

They gain knowledge about the environment through an ability called *perception*. This ability can be limited by a certain viewing range.

Whenever an agent performs an action, it uses energy stored in its battery. This results in the agents periodically having to charge at a charging station in the environment. If an agent's battery dies, it cannot perform any more actions for the rest of the simulation.

Additionally, agents can communicate with each other, both directly and indirectly. For direct communication, they use a given communication language to send messages to each other in order to request information about packages or destinations. Collaborations can be set up as well through direct communication. For indirect communication the agent can leave a mark in the environment. Other agents in the environment will then be able to read information from these marks. This is called *Stigmery*.

B. Ongoing activities

Ongoing activities encapsulate all behaviour that influences the environment but is not a direct product of an action by an agent. In *Packet World*, ongoing activities consist of *Stigmerys*. There are three types: flags, gradient fields [8] and synthetic pheromones [1]. Pheromones, for example, are introduced directly by agents but the rules that describe their diffusion and evaporation are encapsulated in a separate ongoing activity.

C. The environment

The environment typically enforces constraints upon the agent. For example: they can only carry one package at a time or they can not walk through walls.

Furthermore, the environment collects all influences from agents and ongoing activities. It then processes all these influences and updates its state accordingly. This new state then influences perception of the agents and the ongoing activities again.

Situated agents and ongoing activities can be active in the environment asynchronously as well as simultaneously. Without execution synchronisation, the influences of both the agents and ongoing activities would be handled in an arbitrary order. To avoid this, the environment offers options for global synchronisation, regional synchronisation and synchronisation based on logical time. These enable actions to be performed simultaneously.

III. REAL-WORLD PROBLEMS

Packet-world is an interesting test bed to optimise because it maps to various real-world problems. Two such real-world problems are described in the following sections.

A. Robots replacing waiters in restaurants

The first real world problem introduced is situated in a restaurant, as in the problem of food collection [5]. For simplicity, cleaning tables is the only task considered. The restaurant is also organised in a grid to resemble the *Packet World*.

The agents in this system are autonomous robots that are responsible for cleaning the tables. Dirty tables represent the packages, which pop up randomly in the restaurant (the environment). Agents have to detect empty tables (perception). If an agent is currently occupied, it might have to delegate the task of cleaning a perceived table.

B. Self-driving taxis

The second example concerns autonomous taxi's in a city with a grid plan. This problem is very closely related to Packet World, as long as no central task assignment system with complete knowledge is present. In the latter case, we could no longer speak of a MAS [5]. In this mapping, users requesting a ride correspond to packages in *Packet World* and gas stations represent charging stations. Occupied cars can delegate passengers to other, available cars. Road blocks could represent walls in *Packet World*.

IV. DESCRIPTION THE PROBLEM

A. Efficient autonomous behaviour

The first goal was to optimise autonomous behaviour, which means agents would only influence the environment, not each other. In an initial implementation, *random behaviour*, agents wandered around randomly, picked up a package if they were next to one and dropped the package when next to a destination. This completely random behaviour was highly inefficient in terms of both cycles needed and energy consumed. It revealed several problems that had to be solved.

Firstly, a lot of moves were wasted when an agent already perceived a packet. The agent knew the location of the packet but still wasted cycles wandering around randomly.

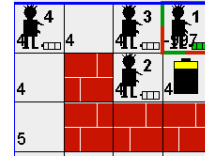


Fig. 1: A state in which agents blocked each other and died.

Furthermore, agents did not remember the locations of destinations. This caused them to waste a lot of cycles looking for a destination they have already visited.

A final problem is the so called “sparse world” problem. In a sparse world, a lot of large areas are empty. In such worlds, all the cycles and energy spent wandering in these areas in pure waste.

B. Energy limitations

When energy limitations are introduced, charging stations become highly valuable resources. This induces a new series of problems that have to be solved.

First, agents need to find charging stations quickly and reliably. Otherwise, agents might die while unnecessarily searching for a charging station.

Once agents know where to charge, they might flock to these locations. As a result, agents could block each other, especially in narrow entries near a charging station. This would cause avoidable agent deaths. Figure 1 shows an example of this happening.

Lastly, agents might arrive at a charging station with very low battery. In this case the agent might die because another agent is charging to 100% instead of letting the former agent charge.

C. Task delegation

In a final extension, an assignment of packages to agents by both agents and packets having colors. This introduces the need for task delegation, because the path of an agent can be blocked by a packet it cannot pick up. Such a scenario is shown in Figure 2.

D. Maze

The last world is a maze, shown in Figure 11. Due to their small perception, agents have trouble finding the shortest path to a desired location in this world. This results in highly inefficient behaviour, especially because dead ends are visited multiple times.

V. REACHING EFFICIENT AUTONOMOUS BEHAVIOUR

A. Design

In a first improvement to autonomous behaviour, agents will walk directly to packages within their perception-range. Agents will always prefer this action whenever possible.

Next, agents will constantly scan their perception for destinations and remember their absolute positions and colors. Subsequently, whenever an agent picks up a packet, it will move directly to its destination if it is in memory. This

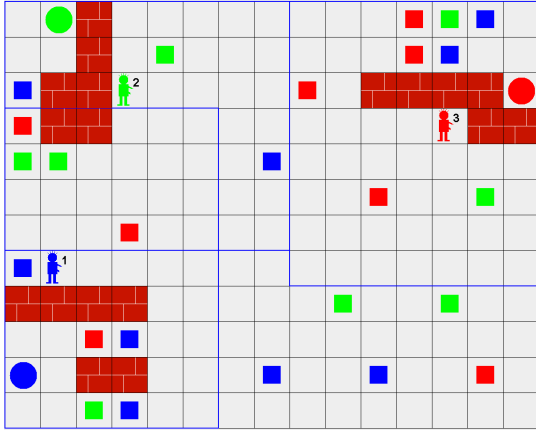


Fig. 2: Example of packets blocking a path to differently colored destinations.

also allows agents to walk directly towards destinations from the moment they enter their perception. The shortest path algorithm use to do this is described later.

Later on, the agents share their knowledge with each other upon contact. The concrete protocol for this communication is explained in subsection [V-B4](#).

To solve the sparse world problem, agents will mark empty areas of the world with flags. A flag will cover a square of nine cells. This is more efficient than using a flag for each cell, since putting a flag takes a turn. Agents will not enter the area of flags while searching for packages.

Finally, an improvement in which agents dropped pheromones on their way to a destination was tested. This would allow other agents to simply follow the strongest pheromones towards a destination. This should allow agents with a small perception to find destinations more quickly in big worlds, since direct communication won't occur as often in these settings.

B. Implementation

To solve to action selection problem [\[6\]](#), a hierarchy of behaviours with different priorities is introduced. The behaviours and their respective priority are shown in Table [I](#). Each cycle, agent will select the behaviour with the highest priority for which the corresponding condition is satisfied. The action taken in each step is fully determined by the currently selected behaviour and the current perception, memory and received messages of an agent.

There are two important things to notice regarding behaviours. Firstly, it should always be possible to select the behaviour with the lowest priority (*BaseSearch*). This is why the condition for (*BaseSearch*) will always return true. Secondly, each behaviour that all behaviours that require the agent to carry a package have a higher priority than all behaviours that don't have this requirement. This makes sense as the agent should never search for packages when it is already holding one.

	Priority
<i>BaseSearch</i>	0
<i>PutFlag</i>	1
<i>PerceptedPackageSearch</i>	2
<i>BaseDrop</i>	3
<i>KnownDestinationDrop</i>	4

TABLE I: Priorities of efficient autonomous behaviour

1) *Base search and base drop*: In the *BaseSearch* behaviour, agents walk around in straight lines and pick up packages whenever they end up next to one. *BaseDrop* is equivalent to *BaseSearch*, but for dropping of packages. If an agent can't move forward in the same direction, it will bounce off the obstacle in a random direction.

2) *Shortest path*: Once an agent knows the location of a target, it should be able to move towards it efficiently. The current implementation uses a shortest path algorithm based on Dijkstra's algorithm [\[7\]](#) with the Chebyshev distance-metric. If the target is reachable and within perception, the algorithm will find the shortest path to the target, taking obstacles into account. If the target is outside perception, the algorithm will find the reachable tile within perception that is closest to the destination and calculate a path towards that.

3) *PerceptedPackageSearch*: The *PerceptedPackageSearch* behaviour makes an agent take a step along the shortest path to the closest package within its perception. For this behaviour to be selected, the agent can not be holding a packet and a packet should be within perception.

Agents do not lock on to packets they perceive. This would be redundant since after stepping towards a packet, the same packet will remain the closest packet within perception, unless another agent has picked up the packet.

4) *Communication and memory*: Agents communicate with a simple protocol. Every agent has a memory segment that contains the names of all agents it has synced its destinations with. At the start of each action the agent checks its perception for other agents that it has not already synced with. If such an agent is found, the agent sends its complete knowledge of destinations to the other agent and adds the other agent's name to the list of synced agents.

The other side of the protocol is the reception of information. At the beginning of each cycle, the agent checks the messages in its mailbox. If a message does not contain new information, the name of sender of the message is just added to the list of synced agents. If the message does contain new destinations, each new destination is added to the memory of the agent and the list of synced agents is set to only contain the last agent. The memory of the synced agents is also reset whenever an agent encounters a new destination that is not in its memory yet.

This protocol is only complete if agents execute their communication first and then execute their acting phase. This protocol can thus be used for Look-Talk-Do behaviour based situated agents. In the *KnownDestinationDrop* behaviour agents will walk along the shortest path towards memorized

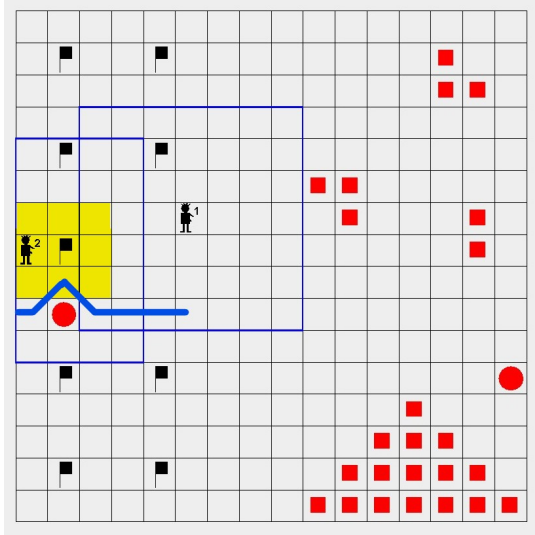


Fig. 3: Example of how a path can be blocked by a flag.

destinations whenever possible.

5) *Flags*: Flags are placed in the *PutFlag* behaviour. Badly placed flags might completely block a path to an important object. This is why there are several rules that determine the placement of a flag.

Firstly, the area that is not covered by flags should have a convex shape. This is achieved by restricting the positions of new flags to empty corners. Corners are defined by walls and the areas of other flags. Furthermore, a flag may never cover an object that is not an agent. This prevents flags from covering destinations or packages. Next, a flag should never fully block a path to a tile that is not covered by other flags. The first rule partially covers this, but a path can still be blocked in other ways. Figure fig. 3 shows a situation in which the first rule does not suffice. The area marked in yellow would block any agent that stands left to the red destination. The final rule states that an agent should be in the area of the flag he wants to place. Otherwise, the perception of the agent might be too limited to correctly evaluate the other rules.

These four rules are combined in the *PutFlag* behaviour. The conjunction of these rules is the condition required to enable this behaviour. An agent should also not be holding a packet for this behaviour to be selected.

6) *Pheromones*: Agents only drop pheromones on their way to a known destination. This way an immediate path is formed between the package and the destination. Other agents holding a package will walk towards the strongest gradient they perceive whenever possible. Because the squares close to a destination will be walked on most, these will have the strongest gradient. Pheromones also decay, which would come in handy if the locations of destinations were dynamic. We decided to leave pheromones out of the final implementation, however, since they became redundant when memory was introduced. Figure fig. 4 shows the result of placing pheromones.

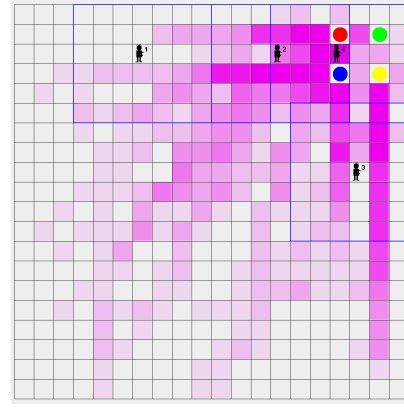


Fig. 4: Example of how the pheromones are placed

C. Test results

1) *Setup*: To test the different improvements, a test in which each improvement was subsequently added was performed. Each configuration was tested 20 times in 3 separate environments. Each of these worlds can be found in the appendix on in figures 6, 7 and 8. Table II shows the average amount of cycles needed to clear each environments, along with the relative improvement compared to *random behaviour*. All improvements were added cumulatively.

In *Perceiving packages*, the *BaseSearch*, *BaseDrop* and *PerceptedPackageSearch* behaviours were added. *Flags* adds the *PutFlag* behaviour. In *Memory*, agents will remember destinations and *KnownDestinationDrop* is added. Finally, *Communication* adds the ability of agents to communicate as described above.

2) *Results*: Walking directly to perceived packages decreases the amount of cycles significantly. In world one, agents can perceive the whole environment, so the performance gain is biggest here. No moves are wasted looking for packages. In the other worlds the perception of the agent is relatively small, so the agents still waste a lot of moves looking for packages.

Flags do not have a big impact since the tested worlds are not big enough for sparsity to be a concern. Especially in World 1, sparsity is not an issue, which is why there is almost no improvement here. Due to the way the destinations are laid out in World 3 and the agent not perceiving destinations, putting flags down increased the amount of cycles required in this case.

Adding memory to remember destinations has the biggest impact on the performance of the MAS. Agents can now both perceive and remember destinations, so there is a drastic reduction in the amount of moves spent looking for destinations. It would be interesting to test what the impact of just perceiving but not remembering destinations would be. We suspect this would make flags play a more significant role.

Finally, communication incurred a small reduction in cycles. Even without communication, there comes a point where all agents have knowledge of all destinations. Once this happens, no cycles will be wasted looking for destinations. Communication just causes this to happen sooner, which causes the

small reduction in cycles. In bigger worlds with more agents, the impact of communication would be bigger, as agents take longer to find destinations themselves.

	World 1	World 2	World 3	Avg improvement (%)
Random	871	831	3061	0
Perceiving packages	607	765	2732	16,3
Flags	602	724	3144	13,7
Memory	167	281	493	77,0
Communication	167	273	479	77,4
Total improvement	80,8	67,1	84,3	77,4

TABLE II: Average amount of cycles needed to clear the environments.

VI. MANAGING ENERGY

A. Design

At the start of the simulation, agents will try to find the destination of the battery and broadcast it to all other agents. Once the agents know a charging coordinate, they start looking around for packages. When their battery falls below some threshold, agents drop any package they might be holding (carrying a package costs battery) and go to the charger. The threshold is dynamically determined based on the agents distance to the charging station. The optimal threshold is determined in Subsection [VI-C](#).

Agents can find charging stations by following gradients. Gradients are numbers that are assigned to each square of the environment. They indicate length of the shortest path between that square and the nearest charging station.

To prevent agents from blocking each other near chargers, agents emit a gradient that pushes other agents away. These fields prevent agents that are looking for a charger from coming too close to other agents, which lowers the chance of agents blocking each other in narrow crossroads.

Gradients were chosen over a message protocol that uses locks on crossroads for various reasons. First of all, it proved very difficult to accurately define crossroads and predict possible blockages while taking every edge case into account. These crossroads are difficult to deal with and extensive studies have already been done about this phenomenon [\[16\]](#). Furthermore, the communication and logic required to prevent these blockages quickly became unmanageable as the amount of agents increased. Finally, the limitation of the perception range could cause agents to become blocked before they even perceived the possible blockage. If the agents would move on lines, we would have chosen for a message protocol that uses locks.

Gradients on the other hand require far less complex logic than communication protocols. In this approach agents looking for a battery always evade other groups of agents. Complexity also stayed constant as the amount of agents increased, since the gradients are defined by the environment and agents only have to make simple decisions based on these gradients. Finally, gradients can also easily be tuned to accommodate for different environments. In environments with a long and

narrow pass-through, the range and intensity of the gradient emitted by agents can be increased.

Emergency messages allow agents that are low on battery to notify the agent that is currently charging. When an agent receives such a message, it steps away from the charger to make way for the agent in distress. This way, unnecessary deaths due to some agent charging for too long are prevented.

B. Implementation

1) *Defining gradients*: The gradient value of a square is calculated as follows. First, each square that is walkable is assigned a value equal to the length of the shortest path from this square to the nearest charging station, taking obstacles into account. Then the values of these squares are increased based on the proximity of agents. The values of squares neighbouring an agent are increased by 3. If the distance from a square to an agent is two, the value is increased by two and if the distance from a square to an agent is three, the value is increased by 1. If multiple agents are nearby, all values are added to the square. The assignment of values to squares is made more clear by figure [5](#). To avoid agents pushing each other away from the charging station while they are relatively far away from it, the agents only start emitting their gradient fields if they are close enough to the charging station.

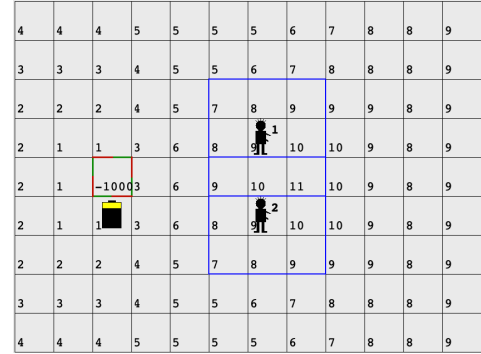


Fig. 5: Example of gradients influenced by agents

2) *Finding charging stations*: When an agent now starts looking for a charging station, it uses a combination of gradients, perception and communication in the behaviour `SearchBattery`. Whenever possible, the agent will move towards a square with a lower gradient. Some agents will perceive and memorise a charging station while others will receive its coordinates from other agents.

3) *Navigating to charging stations*: Agents navigating to a charging station in `GoToCharger` will move to a square with lower gradient whenever possible. This allows them to get closer to a charging station while evading other agents.

If no square with lower gradient is present and the agent knows the location of the nearest charging station, it will follow the shortest path to the charging station, unless the gradient of this new position is higher than the current one. In the latter case, another agent is probably close and the agent will choose to not move in its way.

Finally, if no step towards the charging station is possible, the agent will try to perform a random step that does not increase the current gradient. If no such step is possible, the agent simply skips and wait a turn.

4) *Emergency messages*: A simple communication protocol allows an agent to send an emergency signal to the agent that is currently charging in the `ChargingBattery` behaviour. The signal can only be sent if the battery of the agent is critically low and it is already close to the charging station. Any agent that receives this message interrupts its charging and moves away from the charger freeing it up for the incoming agent.

The effect of these emergency messages is discussed in the test results. The different priorities of the behaviours are summarised in Table III.

	Priority
BaseSearch	0
PutFlag	1
PerceptedPackageSearch	2
BaseDrop	3
KnownDestinationDrop	4
GoToCharger	5
ChargingBattery	6
SearchBattery	6

TABLE III: Priorities of energy management

C. Test results

A number of parameters impact both the cycles and energy required to clear certain environments. We evaluate 2 significant parameters: the required battery level for an agent to start looking for a charger and the required battery level for the agent to send an emergency message. We also investigate the effect of the inclusion of emergency messages by running one test with and one test without emergency messages. Other parameters such as the area of influence of the agent gradient fields and the threshold of activation of said field are not further investigated.

1) *Setup*: The dynamic function that determines the battery level at which an agent starts to look for a charging station is investigated first. A constant value of 400 is sufficient to prevent agents from dying. This value introduces unnecessary overhead when agents are close to the charging station however.

In this test, agents start going to a charger if their battery state B falls below a certain value $f(x)$. Here, $f(x)$ is a function that maps the current distance to the nearest battery to a certain value. The considered functions are all linear: $f(x) = 400$, $f(x) = 25x$, $f(x) = 30x$, $f(x) = 35x$, $f(x) = 25(x+5)$, $f(x) = 30(x+5)$, $f(x) = 35(x+5)$. Non-linear functions could possibly increase the performance, but this has yet to be tested. This implementation does not make use of emergency messages.

The second parameter is the the battery level at which an agent sends an emergency message. We consider the following thresholds: 40, 60 and 80. We also compare these to the implementation without the emergency message protocol. These thresholds are compared for two different values for the

first parameter: the function $f(x) = 30(x+5)$ and the function $f(x) = 35(x+5)$.

For each parameter, the test was run 100 times in the environment visible in Figure 9. This environment was chosen because it proved to be the most challenging to optimise. Other environments had at least one configuration of parameters in which no agents died. It is important to note that the specific environment has a large impact on the optimal parameters.

2) *Results*: The results of the first experiment are visible in Table IV. It shows the amount of times an agent died in a run and the average, minimum and maximum energy consumption and cycles required across runs. This implementation is very sensitive towards its parameters. Implementations in which less agents die tend require less cycles to clear the environment. The total energy consumption is not strongly impacted by agent deaths however. If an agent dies, more cycles were required, but less agents were consuming energy throughout these cycles. Some more insight could perhaps be gained by investigating how many agents exactly died each run.

$f(x) =$	DA	Avg E	Max E	Min E	Avg C	Max C	Min C
400	76	59727	64500	56120	928	1152	789
25x	100	58002	64650	55335	1441	2736	1090
30x	100	58832	63055	55595	1188	2214	929
35x	93	59179	65970	56095	1008	1333	774
25(x+5)	87	59318	64515	56270	974	1545	791
30(x+5)	25	59550	64375	55355	849	1029	762
35(x+5)	7	59043	65760	56040	817	952	759

TABLE IV: Results of the first test in the energy management (DA : Dead Agent Occurrence (in %), E: Energy, C: Cycles)

To investigate the effect of the emergency message we ran the same tests in the same world with a different threshold for the trigger of sending the emergency message. If the threshold is too high it adds extra overhead, but if it is too low more agents will die. We chose the two most promising trigger functions from the previous test. Table V shows the results with $f(x)=30(x+5)$ and Table VI with $f(x) = 35(x+5)$. These results show that adding the emergency message decreases the chance of an agent dying, but increasing the threshold can also result in an increased use of energy since more agents leave the charging station with low battery.

T	DA	Avg E	Max E	Min E	Avg C	Max C	Min C
0	25	59550	64375	55355	849	1029	762
40	15	59371	65825	56210	835	1104	755
60	13	59688	66325	56330	837	999	762
80	9	59618	64065	56915	829	979	771

TABLE V: Results of the second test in the energy management with $f(x)=30(x+5)$ (T: Threshold, DA : Dead Agent Occurrence (in %), E: Energy, C: Cycles)

VII. DELEGATING TASKS

A. Design

First, agents will determine which packet is blocking the path to a destination. If the package has the same color as the agent, the agent can pick it up himself. Otherwise, the agent

T	DA	Avg E	Max E	Min E	Avg C	Max C	Min C
0	7	59043	65760	56040	817	952	759
40	0	59453	68900	55950	817	987	748
60	2	59288	64855	56250	817	1022	761
80	0	59783	65330	56595	822	921	762

TABLE VI: Results of the second test in the energy management with $f(x)=35(x+5)$ (T : Threshold, DA : Dead Agent Occurrence (in %), E : Energy, C : Cycles)

has to delegate this task to another agent. This is done by first looking for other agents and asking for help once one is within perception.

When an agent receives a request for help and it can take up the delegated task, it immediately walks towards the received coordinate to remove the packet (unless it is low on battery or helping another agent).

B. Implementation

1) *Find blocking packet*: An agent has to determine which packet blocks the path to a destination. It does this by first searching a path to the destination. If no path is found, it will try to find the shortest path assuming it can move through packets. If such a path is found, it will select the first packet on that path as the one that has to be removed and enter into the `ClearDestination` behaviour.

2) *Clearing the package*: If an agent in the `ClearDestination` behaviour has the same color as the blocking packet, it will pick it up himself. Otherwise the agent tries to find another agent to delegate the task to. Once another agent is found, the searching agent sends a request for help containing the color and location of the blocking packet.

If the agent receiving the request has the same color as the packet, it will immediately enter the behaviour `HelpOtherAgent`, making it walk to the given coordinate to pick up the packet. Prioritising tasks received from other agents avoids deadlocks environments. Table VII lists the updated priorities. While helping an agent, communication is blocked to prevent the task from being delayed.

3) *Dropping the package*: If the path to the destination is free after picking up the packet, the agent tries to deliver the packet. Otherwise, the agent will drop the package on the first square that does not block any paths and is not influenced by a flag.

	Priority
BaseSearch	0
PutFlag	1
PerceptedPackageSearch	2
BaseDrop	3
KnownDestinationDrop	4
ClearDestination	5
HelpOtherAgent	6

TABLE VII: Priorities of task delegation

C. Test results

1) *Setup*: To test this implementation and the effect of communication, a test that compares two configurations was

performed. One in which agents can only detect blocked paths but not communicate and one in which agents can also communicate. In the first case, agents should still be able to clear the world shown in figure 2 but they will have to come across blocked paths randomly. Each configuration was run 50 times.

2) *Results*: Table VIII shows the average amount of cycles needed for each configuration in each world. There is a clear difference between the two set-ups, which makes sense since agents find blocking packages faster.

	Average number of cycles	Improvement
Without communication	654	
With communication	248	62.08%

TABLE VIII: Results task delegation

VIII. THE MAZE

A. Design

The maze is shown in Figure 11. As explained in Section IV the previous approach to reach a destination is not usable anymore. As a solution to this problem we use the gradient fields explained in Section V. Instead of charging stations emitting fields however, destinations will emit these fields. Furthermore the agent will place flags in dead ends, so the agents knows that it does not have to visit that path again.

B. Implementation

1) *Gradient fields*: Since the maze has three colors of destinations and no battery, three gradient fields will be implemented. Agents will follow the gradients corresponding to the color of the package they are holding. The gradient fields are ignored when the agent is not carrying a package.

2) *Placing flags*: Flags will mark dead ends to prevent other agents from searching there. A naive way to implement flags would be to place them the moment an agent sees a dead end. In the current implementation however, agents will remember dead ends until they reach a crossroad. When they do, they will place a flag on the first position of the alley leading to the dead end. Furthermore, flags will be interpreted as walls regarding the detection of dead ends. This enables agents to mark alleys that contain nothing but dead ends recursively.

C. Test results

1) *Setup*: For the maze, two main optimisations were introduced: destination gradients and flags covering dead ends. This test compares two configurations: one in which only *BaseSearch* and *BaseDrop* were available and one in which both flags and gradients are available. Each test configuration was run 50 times.

2) *Results*: Without gradients, the chances of finding packets and destinations are very small, especially at the end of the simulation. When gradients are added, agents can always walk to the right destination using the shortest path, which drastically reduces the amount of cycles.

Furthermore if we use flags to block the dead ends, agents will not keep on revisiting dead ends. The average amount of cycles needed for each configuration is show in Table IX

	Average number of cycles	Improvement
SimpleBehaviour	65803	
MazeBehaviour	1128	98.29%

TABLE IX: Results of maze environment

IX. DISCUSSION OF YOUR RESULTS

A. Autonomous behaviour

The aforementioned implementation shows a clear reduction in the required amount of cycles to clear a world. Memory has the biggest impact for now, as no moves are wasted looking for destinations after a given point. In bigger worlds with more agents however, communication will become even more important. Placing flags will have a more profound impact as well, since sparsity will be a bigger problem in these worlds.

The current implementation also has some limitations. First and foremost, these results were achieved in three relatively simple testing environments. All tested environments were rectangular with no alcoves or walls. The behaviours and parameters would have to be adjusted to accommodate more complex worlds. The flag pacing algorithm, for example, requires a rectangular world. Further testing in more complex environments will provide useful insights.

Another inefficiency is the behaviour of an agent when it has just placed a flag. A searching agent can not enter the flag area, but when an agent has just put a flag down, he is within the area. In this case, agents flag areas to not restrict the movement of the agent until it accidentally leaves the flag area. This problem could be mitigated by letting agents search for areas that are not covered by flags.

Furthermore, agents do not remember package locations because these are quite volatile. We assumed that this would result in wasted movement as agents would walk towards packages that were already picked up. In big worlds with few agents, however, it might be worth it to remember packet locations. This has yet to be tested.

B. Managing energy

Through trial and error, optimal parameters can be found for specific environments. Even though only a subset of parameters was covered, it is clear that the optimal parameters highly depend on the environment. Any change in the the environment such as additional walls or agents can have a major impact on performance. For example, the current implementation prevents agents from dying in the world shown in figure 10 100% of the time for any tested parameter.

It was also interesting to note that if an agent died, it always happened when the agents tried to charge for the first time. Since all agents start moving at the same time, they all get low on battery around the same time, which causes a queue at the charging station. We think no agents would die if all agents started moving at different times, but this has yet to be tested.

C. Task delegation

Asking another agent for help reduces the amount of cycles required to less than half. There are still some limitations however.

As perception shrinks, it will become harder to find the other agents to ask for help. It could be useful to introduce some sort of common meeting point. Otherwise using broadcast would be a solution.

Furthermore, the current implementation only detects the packet that blocks the shortest path to a destination. When there are multiple possible paths, it might be more efficient to clear the another one. This could be the case when the agent can clear the other path itself.

Agents could also remember blocked destinations. This prevents them from computing multiple times. We did not do this since the agents have limited memory and the computations are doable in the small tested world.

D. The maze

Having gradient fields and placing flags creates a huge reduction in cycles needed to bring all the packages to their destination. There are still some limitations however

Currently, gradient fields are generated by the environment. Another solution would be to let agents remember all visited coordinates and calculate the shortest path using that information. We chose not to do this since the memory of the agents is limited and this solution would require a lot of memory in a large world.

X. DISCUSSION OF RELATED WORK

The solution for autonomous behaviour is based on a some suggestions defined in the introduction of the *Packet World* [1]. However, the implementation and usage of these suggested approaches was refined in this paper.

The literature suggests various approaches to handle crossing agents. Mediated interaction [3] calculates the intended path of agents, after which environment detects conflicts, resolves issues and coordinates movement. This relieves agents from the burden of having to communicate and using negotiation protocols [3]. This approach is in stark contrast to our gradient based one.

Another related article on the battery charging strategies of AGV's proposes an alternative but similar approach [9]. Zhan et al. discusses the *Nearest Charging Station* strategy which is implemented here as well and also the *Minimum delay charging station* strategy. They also touch upon the charging behaviour of lithium-ion batteries in real world problems, which is more accurate but also complex than the assumptions made in the *Packet World*.

The study by Selmaier and Maurer suggests even more complex charging strategies [10]. The four strategies discussed in their research are developed for high traffic MASs. The environments we looked at and tried to solve don't warrant the overhead that these strategies induce. The strength of their approach also lies in the ability to dynamically determine the course of action based on some KPI's.

The basic protocol that is described in section VII falls under the Request message type described by Huhns and Stephens [11]. Their work proposes several other types of inter-agent communications. These types range from very simple to quite complex and agents can fulfill slave, master or peer roles in these protocols. Next to message types they elaborate on communication languages as well.

XI. FUTURE WORK

Both emergency messages and task delegation do not take lost messages into account. This is why current communication protocol could be extended to include reception acknowledgements. This would make the current solution more robust in real-world applications since messages can always get lost in real-world settings. An example implementation is proposed in [12] and [13].

Implementing a minimum delay charging station policy instead of the nearest charging station policy in *Packet World* would be interesting as well. This policy is more complex compared but could improve charging behaviour within some worlds and environments. These two policies are mentioned by Selmair et al. [10].

Another interesting improvement would be to let the agent learn from experience [14]. Using artificial intelligence, the agents can make progress and decisions based on earlier events. This is not the subject of this paper and will not be discussed, but it would be very interesting as future work.

Future work could also extend upon multiple parameters suggested in section VI. We limited ourselves to two parameters with linear models, but other thresholds and parameters could be investigated as well. More complex threshold functions could also be taken into consideration.

XII. CONCLUSION

The optimisation of agent behaviour in *Packet-World* turns out to be a layered problem that highly depends on the environment in which agents the agents are situated. This problem was decomposed into three sub-problems: optimising autonomous behaviour, managing energy and task delegation. All of these were solved with the goal of reducing the amount of cycles needed to clear various environments.

In improving autonomous behaviour, the introduction of memory reduced the amount of cycles the most. Other optimisations such as communication and flags will become more important as the size of environments increases. In this case, it will also be useful to look into more sophisticated communication protocols that allow agents to more efficiently map out these bigger worlds.

Gradients proved to be an efficient but environment-specific solution to the energy-management problem. The optimal parameters highly depend on the selected environment. Luckily, a lot of real-world applications do not require agents to function in a lot of different environments. In the context of food collection, for example, the parameters could be tuned to the specific restaurant the agents are situated in.

The current implementation of task delegation efficiently deals with blocked paths. The lack of acknowledgement could pose a problem in real-world applications however. In contrast to *Packet-world*, messages will get lost in the physical world. More sophisticated communication protocols would drastically improve robustness here.

REFERENCES

- [1] D. Weyns, A. Helleboogh and T. Holvoet, "The Packet-World: a Test Bed for Investigating Situated Multi-Agent Systems", 2005
- [2] P. Grassé, "La theorie de la Stigmergie: Essai d'interpretation du Comportement des Termites Constructeurs", 1959.
- [3] D. Weyns, A. Helleboogh and T. Holvoet, "The agent environment in multi-agent systems: A middleware perspective", 2009
- [4] D. Weyns and T. Holvoet, "A Formal Model for Situated Multi-Agent Systems", 2004.
- [5] J. Rocha, I. Boavida-Portugal and E. Gomes, "Introductory Chapter: Multi-Agent Systems", 2017.
- [6] T. Tyrrell, "Computational Mechanisms for Action Selection", 1993.
- [7] E.W. Dijkstra, "A note on two problems in connexion with graphs", 1959
- [8] L. Steels, "Cooperation between distributed agents through self-organization.", 1990.
- [9] X. Zhan, L. Xu, J. Zhang, and A. Li, "Study on AGVs battery charging strategy for improving utilisation," 2019.
- [10] M. Selmair, and T. Maurer, "Enhancing Charging & Parking Processes of AGV Systems: Progressive Theoretical Considerations," 2020.
- [11] M.N.Huhns, and L. M. Stephens, "Multiagent Systems and Societies of Agents", 1999.
- [12] D. Weyns, E. Steegmans, T. Holvoet, "Protocol-based communication for situated multi-agent systems", 2004.
- [13] D. Weyns and T. Holvoet, "A Colored Petri Net for a Multi-Agent Application".
- [14] P.Maes, "Modeling adaptive autonomous agents", *Artificial Life*, I 1-2 (1994)
- [15] W. Liu, F. Alan, T. Winfield, J. Sa, J. Chen, "Towards Energy Optimization: Emergent Task Allocation in a Swarm of Foraging Robots", 2007.
- [16] M.S.Kakkasageri, "Multiagent driven dynamic clustering of vehicles in VANETs", 2012.

APPENDIX

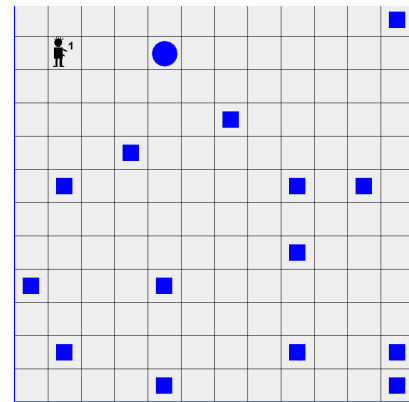


Fig. 6: Illustration of world 1

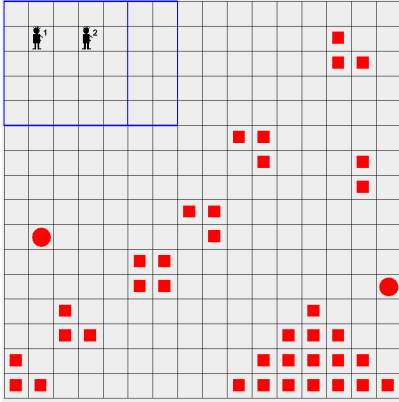


Fig. 7: Illustration of world 2

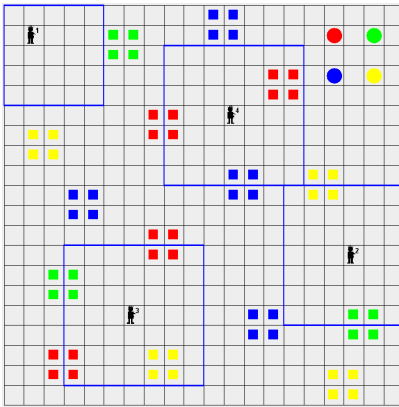


Fig. 8: Illustration of world 3

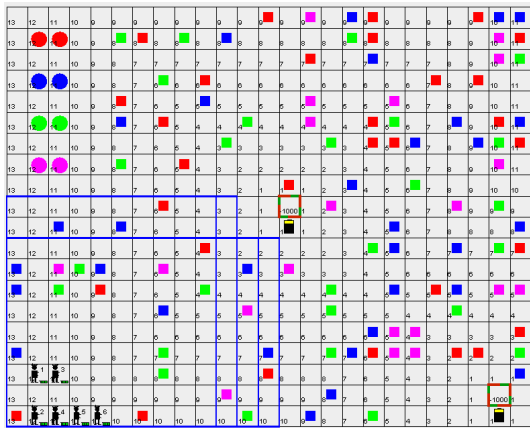


Fig. 9: World of energy management tests

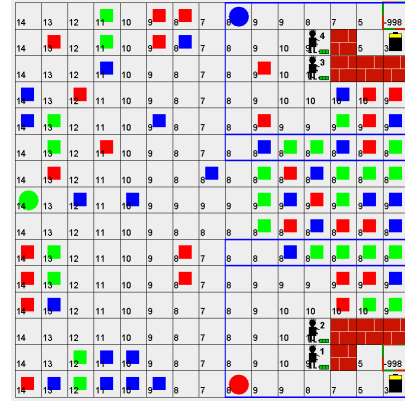


Fig. 10: Alternative world for energy management

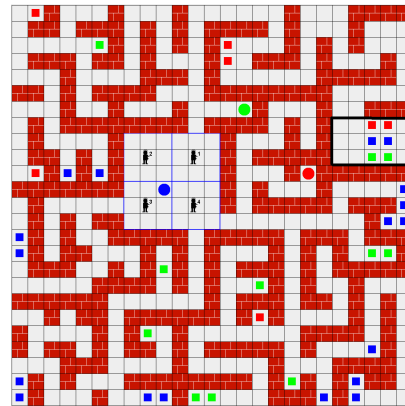


Fig. 11: Maze

Introduction and evaluation of strategies and protocols for situated multi-agents with the use of the Packet-World test bed

Tobias Baert
department of computer science
KU Leuven
Leuven, Belgium
tobias.baert@student.kuleuven.be

Kobe Kwanten
department of computer science
KU Leuven
Leuven, Belgium
kobe.kwanten@student.kuleuven.be

Sofie Landuyt
department of computer science
KU Leuven
Leuven, Belgium
sofie.landuyt@student.kuleuven.be

Kobe Reygel
department of computer science
KU Leuven
Leuven, Belgium
kobe.reygel@student.kuleuven.be

Abstract—Distributed applications can be modeled as a set of entities that communicate and cooperate to achieve their own desires and the collective goal of the application. These applications are commonly referred to as multi-agent systems. This work focuses on the coexistence and cooperation between agents by defining several strategies and protocols that dictate how the agents behave and communicate with one another for three defined problems. The three problems are the realisation of autonomous behavior of agents, energy management with limited battery capacity and task delegation for which agents need to cooperate. These problems have been realised in a setting where agents have to deliver packets to destinations of the same color in a grid world. The Packet-World, which is an abstract test bed for agents, was used to implement and evaluate the solutions that this work provides. For the first problem agents were limited to finding and delivering packets whilst sharing information about packet and destination locations. The second problem was solved by introducing a communication protocol that allows agents to determine who has the priority to charge. Lastly, the solution for the third problem has agents creating and delegating tasks to remove a packet from a path to other agents that are suited for this task. The result of this work is an introduction and evaluation of several strategies and protocols that prove to be effective in some specific scenarios that focus on the defined problems. There are some major shortcomings that were identified in the defined solutions, for which a possible solution was defined that can be implemented and evaluated in future work.

Keywords— Multi-agent systems, Packet-World, Energy management, Task delegation, Gradient fields, Autonomous agents, Behavior

I. INTRODUCTION

Multi-agent systems are typically distributed applications in which several entities or agents interact with each other to achieve their individual desires and the collective goal of the application [1]. Developing protocols and strategies for how the agents will operate comes with many challenges.

This work focuses on solving three challenges in a multi-agent setting where agents have to deliver packets to the correspondingly

colored destinations. The three challenges are: (1) the realisation of autonomous behavior of agents, (2) energy management for agents with a limited battery capacity and (3) task delegations to encourage cooperation between agents when they can not solve tasks on their own. Solving these challenges will bring more insight into the feasibility and applicability of multi-agent systems, as well as provide some strategies and protocols that can prove to be valuable for the development of real world multi-agent systems.

To ease the development and evaluation of the proposed solutions to this challenge, this work is built on the Packet-World test bed that facilitates this type of research for situated agents [2]. Situated agents are agents that are capable of responding to dynamic changes within their environment [3]. Therefore, they need to be able to perceive their environment at regular intervals and make decisions with little delay to prevent that the environment changes during decision making. The Packet-World and its features are explained in more detail in section II.

This work introduces and evaluates solutions to the challenges that were identified. These solutions were the result of an iterative development cycle where each iteration aimed to provide overall better results by modifying the existing or introducing new strategies and protocols. The three challenges are explored respectively in section IV-A, section V-A and section VI-A. The solutions and evaluation of the solutions are discussed respectively in section IV, section V and section VI. The first challenge is solved by introducing behaviors that act as commitments for the agents. Agents are committed to either finding a packet that they can deliver or delivering a packet that they are currently holding. The second challenge requires some form of communication between agents to determine which agent is allowed to use the charging station to charge its battery. Therefore, a communication protocol is introduced to determine which agents gets access to the charging station. To solve the third challenge, the agents are extended to be able to create tasks that they can delegate to other agents that are capable of performing the task, thus enabling cooperation between agents. Section III explains how some real world multi-agent system examples can be mapped to the Packet-World test bed to further motivate the importance of this work.

Some additional challenges and potential pitfalls of the proposed solutions were identified, these are explored in section VII. Section

VIII situates this work in other existing research on multi-agent systems.

II. THE PACKET-WORLD TEST BED

The Packet-World is a test bed that was developed to ease the development and evaluation of fundamental concepts and mechanisms for situated multi-agent systems [2]. It provides a setting in which situated agents are deployed on a grid world where they have to deliver packets to their correspondingly colored destinations. The packet world simulates the mechanisms by using cycles in which each agent can perform at most one action. The environment in which the agents are deployed is a collection of grid layers where each layer contains a specific type of item (e.g. destination, packet, gradient,...). Constraints are defined per layer and also between layers, for example agents can not be positioned on a square in the grid that already contains a destination. The agents can perceive this environment through a perception. The perception defines a range in which the agent can perceive things. Additionally, it allows the agent to interpret what is perceived by providing representations of the items that are found within the environment.

The agents that are situated within the environment can decide to take one of several actions each cycle. The available actions are: (1) picking up a packet in a neighbouring square, (2) moving to a neighbouring, vacant square, (3) dropping a packet on a neighbouring vacant square or a destination and (4) skipping an action for the current cycle. Agents can also communicate with each other in order to execute communication protocols or inform each other. This communication is supported through the environment that will deliver the sent message to the receiving agent. Agents can perform these actions asynchronously as an underlying reactor will merge the actions to achieve a consistent state in the next cycle.

It is possible to give batteries to agents that will contain energy that they need to spend to perform certain actions. The environment allows the agent to charge their battery by providing a grid layer with charging stations.

The Packet-World contains other layers that the agents can use to communicate indirectly with each other. Two example layers are a flag layer which contains flags that can be set and removed by agents and a gradient layer that allows agents and other items to emit gradient values in the environment.

The Packet-World facilitates the simulation and evaluation of mechanisms by allowing the application to run several simulations in a batch mode. The output of this batch mode is a file that contains events (which can also be configured) and can be used to evaluate the simulations.

III. REAL WORLD EXAMPLES

A. AGVs at automated seaport container terminals

Global trade and maritime cargo transportation have increased in recent years and they are expected to grow further still [4]. To support this growth, some harbors turn their interest to automated guided vehicles (AGVs) that can transport cargo containers from one location to another where they can be stored. The work of M. Grunow et al. [4] focuses on different dispatching strategies. This scenario can be directly mapped to the Packet-World where agents represent AGVs, packets represent containers and destinations represent the drop off locations. This means that the work in this paper can prove to be useful for a new system where the AGVs would act as individual agents.

B. Robocup

Robocup¹ is an international scientific initiative that aims to develop a team of humanoid robots that play soccer. The goal is

¹<https://www.robocup.org/>

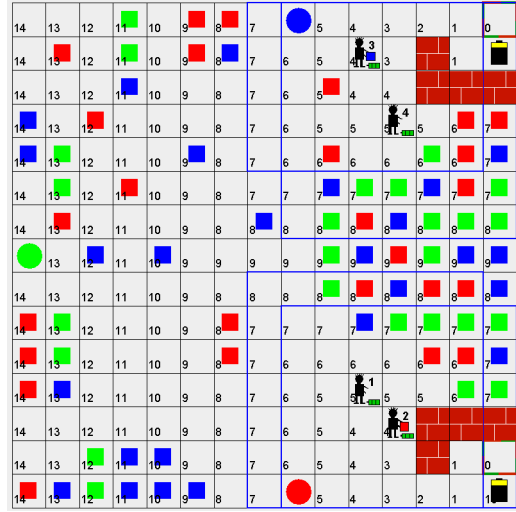


Fig. 1: Example of a Packet-World that contains agents, packets, destinations, walls, energy stations and gradients.

to develop robots that are capable of defeating the human world champion in soccer by 2050 [5].

The problem of creating robots that strategise together to win a soccer match can be mapped to the Packet-World model (introduced in section II). The different agents in the Packet-World represent the robots that play soccer, these agents are divided into two opposing teams. The ball can be modeled as a single and the only packet in the world. The goals can be represented as destinations on the west and east borders in the Packet-World grid. Message protocols can be used to coordinate amongst team members. This enables the agents to plan their positioning and perform actions like passing the ball. Attacker and defender roles can be identified which will determine the actions of the robots and allow them to create a more sophisticated strategy to win.

C. SICS marketplace

The SICS marketplace [6] framework originates from the observation that the internet evolved to now contain a lot of small storefronts that sell goods. To better connect the buyer and seller and their interests, a multi-agent system is proposed. This has the benefit of providing a unified interface for all storefronts and making it easier to find relevant offers. This can be mapped to the Packet-World as follows: the different agents represent market participants (either buyers or sellers). The packets are interests (e.g. an interest to buy a specific item at the best possible price). The destinations then represent resolutions for this interest. The environment in which the destinations and the agents are situated is a subset of the World-Wide-Web which can be seen as a graph with all existing storefronts as nodes. The agents communicate by expressing their interest in a constrained natural language called *Market Interaction Language*. The agents can then autonomously negotiate to reach the optimal resolution for the mutual interests.

IV. AUTONOMOUS BEHAVIOR

A. Problem Description

This first problem focuses on the core problem posed by the Packet-World: implementing the agent's behavior such that they move through the world, pickup packets they encounter and deliver these packets to their corresponding destinations. During this first problem agents are mainly constrained by their limited perception of the environment.

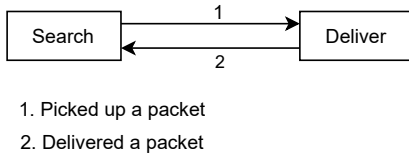


Fig. 2: State diagram of behaviors and behavior changes of search-and-deliver

B. Solution

The first step in solving this problem is to extend the wander behavior to a new `WanderAndPickup` behavior, which includes agents picking up packets they encounter and dropping these packets off when they encounter a destination of the same color.

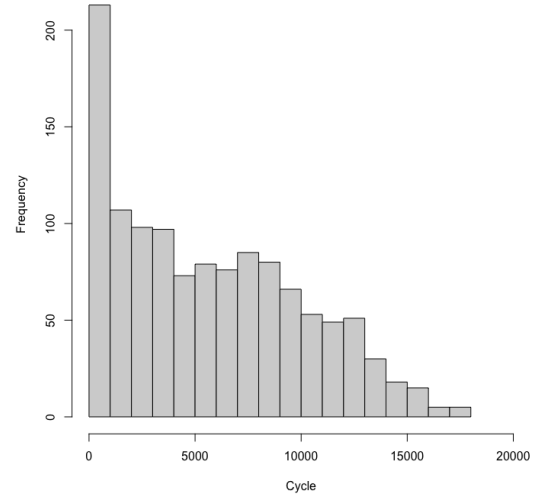
The next iteration sees the separation of behavior in two parts: a `Search` behavior and a `Deliver` behavior. This new approach is called `SearchAndDeliver`. The `Search` behavior is responsible for exploring the area and searching for packets to deliver. If the agents see a packet, they will try to go to the packet and pick it up. The `Deliver` behavior is then responsible for delivering the packet its carrying to the corresponding destination. Figure 2 shows the state diagram of these behaviors. Furthermore, each turn the agents analyze their perception and store packet and destination coordinates in memory. This newly gained knowledge allows agents to move in the direction of the last known position of a packet or destination. To ensure consistency between agents' memory and the environment, agents update their memory every cycle by comparing it to their latest perception.

The third iteration `SearchAndDeliverWithCommunication` speeds up the discovery of packets and destinations in two ways. Firstly, observations show that a purely random walk inefficiently explores the environment in search of packets and destinations. To improve this behavior agents now move in a straight line until they hit an obstacle after which they start moving in a new random direction. This type of movement was inspired by Roombas as they are often used in unknown environments where they optimally have to cover the entire area to clean the environment. Cheap Roombas determine their paths based on simple collisions or collision detectors [7]. After seeing new packets and saving them in memory agents will travel to the closest discovered package. Secondly, agents often collect complementary information as they explore different parts of the environment. To make sure that all agents quickly gain knowledge about the location of destinations, every agent communicates discovered destinations to other agents within its perception. As a result, the locations of destinations quickly spread in a gossip-like manner.

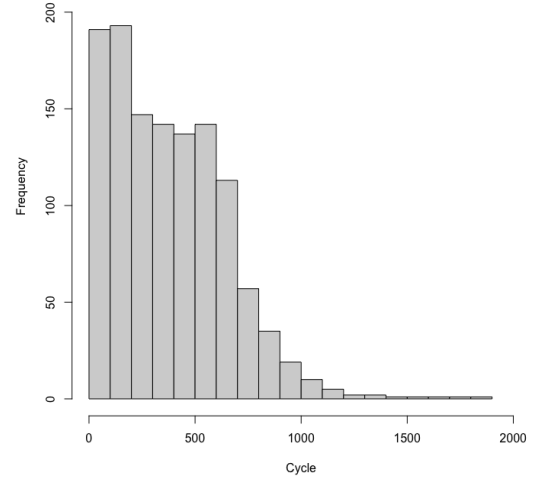
C. Testing & Discussion

To prove that each iteration improves on the last, we conduct simulations of these three behaviors and compare the produced meta-data. For the environments we use the provided `basic-2-vision` and `basic-3-color` environments. Each simulation runs until the agents have delivered all packets to their corresponding destinations. Each combination of a behavior configuration and an environment is simulated 20 times. The resulting analysis of the meta-data is shown in Table I and Table II.

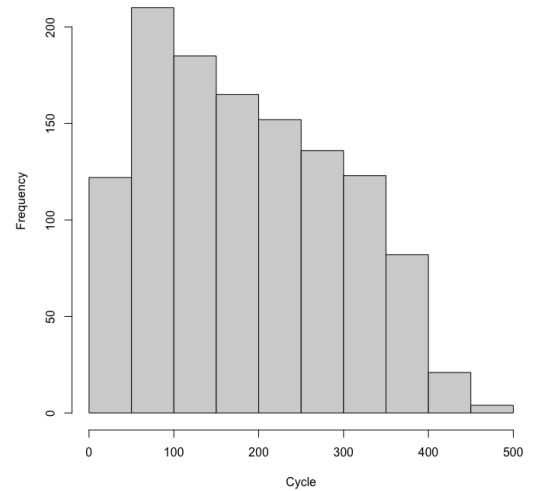
Additionally, for environment `basic-3-color` we also consider how the frequency of deliveries evolves over the course of the simulations for each behavior. Figure 3 shows the number of completed deliveries, summed across the 20 simulations, for each subsequent period of time. Note that while the vertical axis remains constant across the three figures, the horizontal axis changes as the simulations become shorter.



(a) First iteration: `WanderAndPickup`



(b) Second iteration: `SearchAndDeliver`



(c) Third iteration: `SearchAndDeliverWithCommunication`

Fig. 3: Evolution of the frequency of deliveries over cycles for each of the iterations for `basic-3-color`

Behavior	Mean energy consumed	Mean total cycles
WanderAndPickup	94208.50	3068.30
SearchAndDeliver	9209.00	372.35
SearchAndDeliver- WithCommunication	6007.50	234.30

TABLE I: Analysis of meta-data for 20 simulations of the basic-2-vision environment

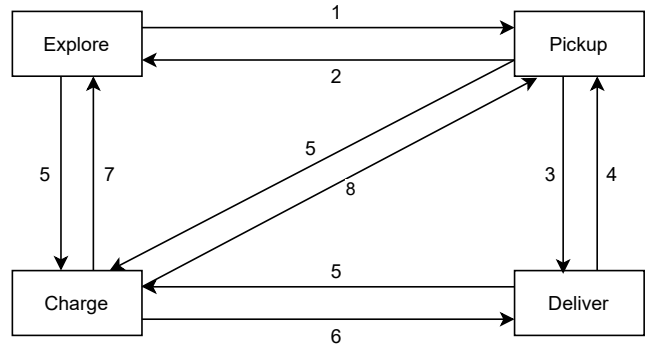
Behavior	Mean energy consumed	Mean total cycles
WanderAndPickup	1027894.00	14732.25
SearchAndDeliver	63445.50	1015.75
SearchAndDeliver- WithCommunication	23200.50	395.45

TABLE II: Analysis of meta-data for 20 simulations of the basic-3-color environment

First and foremost, table I and table II show that each iteration improves on the previous one. The introduction of memory in SearchAndDeliver yields a large improvement compared to WanderAndPickup. For environment basic-2-vision energy consumption and total cycle count are reduced by factor of 10 and 8 respectively. Switching to SearchAndDeliverWithCommunication brings both measures down by another third. For environment basic-3-color the evolution is similar yet magnified: SearchAndDeliver reduces the energy consumption and total cycle counts by a factor close to 15 compared to WanderAndPickup and SearchAndDeliverWithCommunication further reduces those numbers by nearly two-thirds. The increased effect of each iteration reducing the amount of random movement of agents. This kind of random movement to explore or navigate to packets and destinations becomes increasingly ineffective in larger environments, hence why the larger environment basic-3-color magnifies the improvements of each iteration.

We can also make several observations from figure 3. Figure 3a shows how there are many deliveries in the first thousand cycles and that the frequency of deliveries quickly drops off afterwards. This initial spike can be attributed to packets that are located close to a matching destination. For those packets many fewer random moves are required to land next to a matching destination. Figure 3b and 3c show an initial spike in deliveries but the rate of deliveries decreases more slowly in these iterations of the behavior. Figure 3b shows a long tail, indicating simulations of exceptional length. These occur in the special case that an agent spends the majority of the simulation wandering around searching for a destination, thereby increasing the workload for the remaining agents. In the third iteration such situations are avoided because of the communication of destinations between nearby agents.

The agents developed in this part still have several shortcomings. Firstly, the agents' path finding ability is limited. Agents move by greedily selecting moves towards a goal, only considering the very next move. Consequently, they can get stuck behind packets or terrain. Secondly, as these agents do not communicate about packet pickups, it occurs that agents travel towards a packet seen earlier, only to find it has been removed by another agent. Similarly, as agents do not communicate their goals to each other, sometimes a group of agents travels together to a single packet. Only when the first agent to arrive picks up the packet, the others change course. Lastly, agents never consider dropping a packet. This can be useful when an agent is struggling to find a matching destination for the packet it is carrying but has already acquired the knowledge to quickly deliver other nearby packets.



1. Knows the location of a packet to be delivered
2. Does not know a location of a packet to be delivered
3. Picked up a packet
4. Delivered a packet and knows the location of a packet to be delivered
5. Needs to charge battery
6. Charged battery and carries packet to deliver
7. Charged battery and does not know a location of a packet to be delivered
8. Charged battery and knows the location of a packet to be delivered

Fig. 4: State diagram of behaviors of energy management

V. ENERGY MANAGEMENT

A. Problem Description

The second problem builds upon the problem outlined in Section IV-A. Agents are now equipped with a battery and are characterized by the notion of an energy level. Agents need to continually monitor their own energy level while performing task, as running out of energy will render them unable to perform any actions. Additionally, the environments now feature charging stations where agents can replenish their battery level. Completely charging the battery of an agent can take multiple cycles. Lastly, the concept of walls is introduced. Walls are solid objects in the Packet-World that can not be traversed, nor can packets be placed on walls.

B. Solution

To tackle the challenges described in the previous section, the behavior is separated into four parts: Explore, Pickup, Charge and Deliver. The Explore behavior is responsible for exploring the area searching for packets. The Pickup behavior is responsible for travelling to the packets and picking them up. If the agents carry a packet, the Deliver behavior directs the agents to bring the packet to the corresponding destination. If the battery of the agents is too low, the agents adopt the Charge behavior and will start searching for an energy station to charge their battery. Figure 4 shows the state diagram of these behaviors.

In order to locate charging stations the initial approach adds a gradient field to the environment for every energy station so that each cell in the Packet-World gains a gradient value that represents the number of steps that are required to reach the closest energy station. When an agent's energy level drops below 20%, the agent stops whatever it was doing and follows the path of diminishing gradients to reach a charging station. This is further described in Section VIII-A. This behavior keeps some agents alive, but not all as there is often congestion around the charging stations.

To overcome this shortcoming, we apply a new approach. The agents use a consensus protocol inspired by [8], [9] and [10] that decides which agent is allowed to charge at a particular station. These related works are explored in section VIII-B, what follows here will be a description of the proposed solution. Each agent that is in the charging state sends a message to every agent in its perception containing: his battery state, his ID, the closest energy station to its

position and its current location. The closest energy station is used to determine which agents want to go to the same station. Between those agents that want to go to the same station the order is determined as follows:

- The agent with the lower battery level has a higher priority.
- The agent that is closest to the energy station has a higher priority.
- The agent ID is used as tiebreaker where a lower ID will result in a higher priority.

The first protocol that was used to implement these rules resulted in agents constantly having to free the charging station and reentering it. This was a direct consequence of the fact that when agents charged their battery they no longer had the lowest battery level, making them lose the priority to charge. This results in very inefficient use of the charging station and wastes many cycles.

A second decentralized protocol tries to ensure that the energy station is used more optimally by letting agents stay on the charging station as long as possible before they have to move away in order to allow another agent to charge. At the same time the protocol also tries to ensure that no agents run out of energy. If the charging agent has enough battery to survive for a couple of cycles and there is another agent with a critical battery level (defined as $< 20\%$), then the charging agent yields to make sure that the endangered agent survives. Because all agents adhere to the same rules, a consensus can be reached in a decentralized way where each agent can determine the charging order locally without the need for confirmations from other agents. The combination of these tactics is successful in keeping every agent alive most of the time.

Unfortunately, the simple movement algorithms inherited from the first problem have become a major limiting factor of the system. They hinder the evaluation of the introduced protocols because agents often get stuck behind obstacles even though there are available paths in their perception. To this end extensions are made to the way that agents move around in the environment. An A* search algorithm is implemented to find more efficient paths both to the energy stations and to packets and destinations. A* takes reachability into account, which helps the agents to move around packets and walls in the way of their goal. A* is often used in games where entities have to find paths to a target location [11]. Our work uses a modified version of the code that is available on stackabuse.com [12].

Next, an improvement is made to the condition when agents want to charge. The previous strategies still result in some agents running out of energy. Instead of simply going to a charging station when the energy level drops below 20%, an agent looks at the gradient field to determine how many steps are needed to reach the nearest energy station, and makes sure that it heads for the station so that it can arrive at the charging station when his level drops below 25%. This gives each agent some extra breathing room to coordinate and wait for their turn without their batteries running out of energy.

Finally, to further improve the utilisation of stations and minimize the congestion around a single station the agents are modified so that they each have an assigned station. The stations are assigned dynamically based on the number of stations that are known at the time. Consequently, the agents need less cycles to clear the Packet-World because less time is wasted waiting around charging stations for their turn to charge. The average number of cycles for the *energy-2* environment dropped by no less than 100 cycles even when some agents had to travel further to reach their station.

C. Testing & Discussion

In order to evaluate the quality of the provided solution for energy management, the recorded meta-data of the Packet-World is expanded to include the behavior of each agent during each cycle and to make note of cycles where agents run out of energy. A number of simulations of the *energyManagement* behavior are conducted for the provided *energy-2* and *energy-3* environments. The *energy-2*

environment is also evaluated for increased agent populations. Each environment is simulated for 100 runs. Each simulation runs until the agents have delivered all packets to their matching destinations. The resulting analysis of the aggregated meta-data is shown in Table III and Table IV.

Number of agents	6	8	10
Number of packets	110	110	110
Number of charging stations	2	2	2
Mean energy consumed per agent	70758.25	75118.75	73483.60
Mean total cycles	11793.04	9389.84	7348.36
Mean total cycles	1048.09	1412.11	1799.73
Relative frequency of behaviors			
Explore	0.02	0.01	0.01
Pickup	0.37	0.28	0.22
Deliver	0.25	0.19	0.15
Charge	0.36	0.51	0.62
Number of energy-less agents	3	152	418

TABLE III: Analysis of meta-data for 100 simulations of the *energy-2* environment

Number of agents	4
Number of packets	79
Number of charging stations	2
Mean energy consumed	23817.65
Mean total cycles	325.59
Relative frequency of behaviors	
Explore	0.01
Pickup	0.29
Deliver	0.39
Charge	0.31
Number of energy-less agents	0

TABLE IV: Analysis of meta-data for 100 simulations of the *energy-3* environment

Additionally, for environment *energy-2* we also consider which behaviors occur more frequently over the course of the simulations for each behavior. Figure 5 shows how the occurrences of each behavior, summed across the 100 simulations, are distributed over time for environment *energy-2* for 6 agents. Figure 6 shows how agent "deaths", whereby an agent runs out of energy, are distributed over time for environment *energy-2* for 8 agents, again summed over 100 simulations.

Table III first and foremost shows that the total energy consumption remains stable across the three variations of the agent count and that the total number of cycles required to fully clear the environment increases with the number of agents. This implies that the *energyManagement* implementation is not able to utilise the additional agents to achieve higher efficiency. This is likely due to a lack of coordination between agents in behaviors other than *Charge*, leading to inefficient movement to packages.

Secondly, table III shows the relative frequency of each behavior. The *Explore* behavior quickly becomes redundant due to the concentration of the destinations, as agents immediately become aware of destinations for all possible packet colors. Furthermore, the table shows that the *Pickup* behavior is more frequent than the *Deliver* behavior. This is caused by agents interrupting their delivery to travel to a charging station. After charging their battery, the agents must again follow the *Pickup* behavior to acquire a packet. Finally, it is clear that the agents spend most of the cycles in the *Charge* behavior, especially at higher agent counts. While the increased wait times around charging stations take part of the blame, we must also take into account that agents which have depleted their energy remain in the *Charge* behavior for the rest of the simulation, thereby inflating the relative frequency of the *Charge* behavior. Figure 5 shows how the frequency of each behavior evolves as the

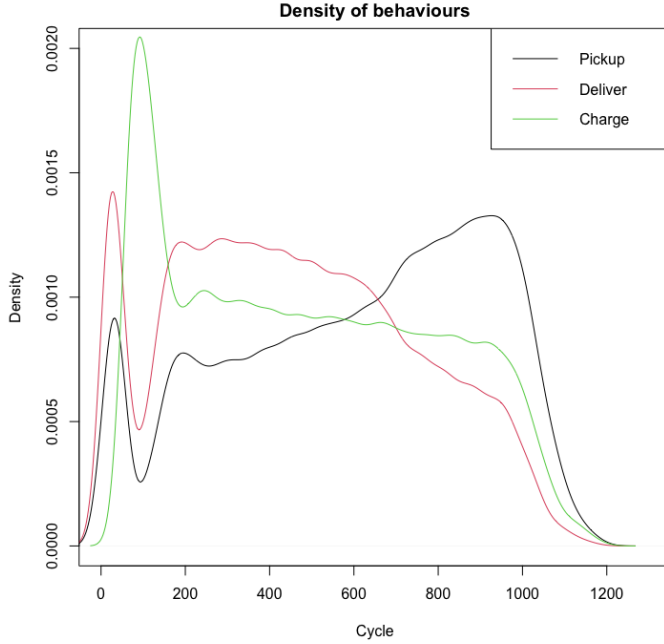


Fig. 5: Evolution of density of each behavior over cycles for energy-2 with 6 agents

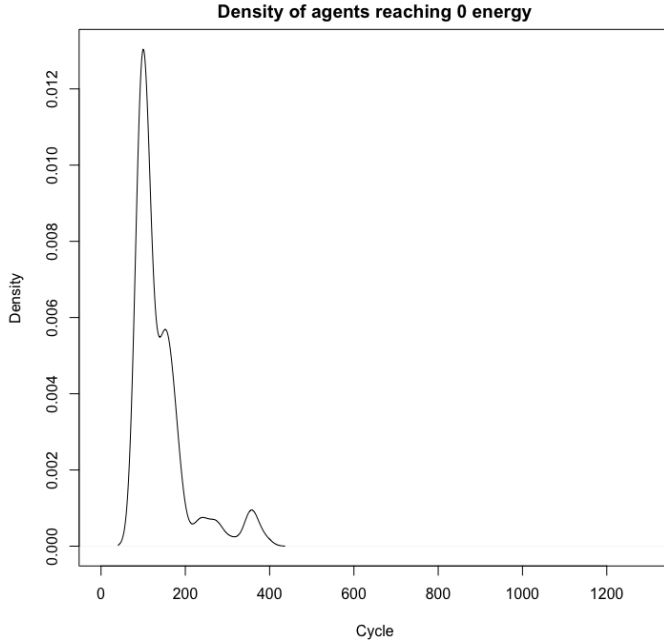


Fig. 6: Evolution of density of agents depleting their energy over cycles for energy-2 with 8 agents

simulations of environment energy-2 for 6 agents progress. Note that the `Explore` behavior has been omitted from the figure as it occurs with negligible frequency relative to the other behaviors. The evolution is similar for 8 and 10 agents.

That leads us to consider the number of energy-less agents. As can be seen in table III, that number increases strongly as the number of agents increases. This is due to contention of the charging stations. As agents start with the same battery levels and consume energy at similar pace, the agents will hit the charging threshold at around the same time. This causes the visible spike in figure 6 around cycle 100. As can be seen in figure 6, a spike in agents depleting their energy follows soon after².

VI. TASK DELEGATION

A. Problem Description

The final problem withdraws the energy concerns detailed in V-A and instead adds a color attribute to each agent. A colored agent can only pickup packets that have the same color. In the provided task-delegation environment each destination is blocked by packets of a variety of colors. As a result there is a need for coordination and cooperation between agents in order to clear paths to these destinations.

B. Solution

The solution to this problem makes the separation of the behavior in five parts: `Explore`, `Pickup`, `Wandering`, `PerformClearPathTask`, `PerformDeliverTask`. The first two have the same purpose as in section V-A. The `PerformClearPathTask` will guide the agents to perform their clear path tasks, i.e. moving a packet away from some given path, and the `PerformDeliverTask` will do the same for their deliver tasks, i.e. delivering a packet to a destination. The last behavior `Wandering` is the behavior the agents adopt when they do not have any tasks to perform but are still waiting on a clear path for their own deliver task. The agents will wander around listening for a clear path task posed by another agent or a signal that they can perform their original deliver task. Figure 7 shows the state diagram of these behaviors.

The solution keeps the A* and communication solutions from the two other problems that were explained in Section IV-A and V-A. The energy consensus protocol and gradient field elements are no longer of use and are thus removed. Because agents now can only pick up packets that match their own color and because there are destinations that are blocked by packets, the agents need to cooperate to free up a path to the destinations.

The solution that tackles this problem is inspired by [13] and [14] as described in section VIII-C. The concrete solution this paper proposes goes as follows: an agent is in an `explore` state to discover packets and destinations in the world until there is both a packet and a destination that match his color in its memory. The agent then constructs a path from the packet to the destination and goes into a `deliver` state. If there is a packet that blocks the path that the agent wanted to use, the agent constructs subtasks. Each subtask is used to define a task wherein an agent needs to clear the path. These subtasks are handed out to agents of the right color (in the right order to match the blocked path). While the agent has no tasks to perform itself, it goes into a `wandering` state looking for help and waiting for the confirmation of others that all of the subtasks that it handed out are completed. If the agent that can solve the first necessary subtask receives the call for help, it goes into a `clearPath` state. It receives info about the path that is to be cleared and moves the packet out of the way of the path. It then goes back to what it was doing, and notifies the agent that asked for help that the subtask has

²Figure 5 details behaviors for 6 agents instead of 8, however the evolution of behaviors for 8 agents is very similar.

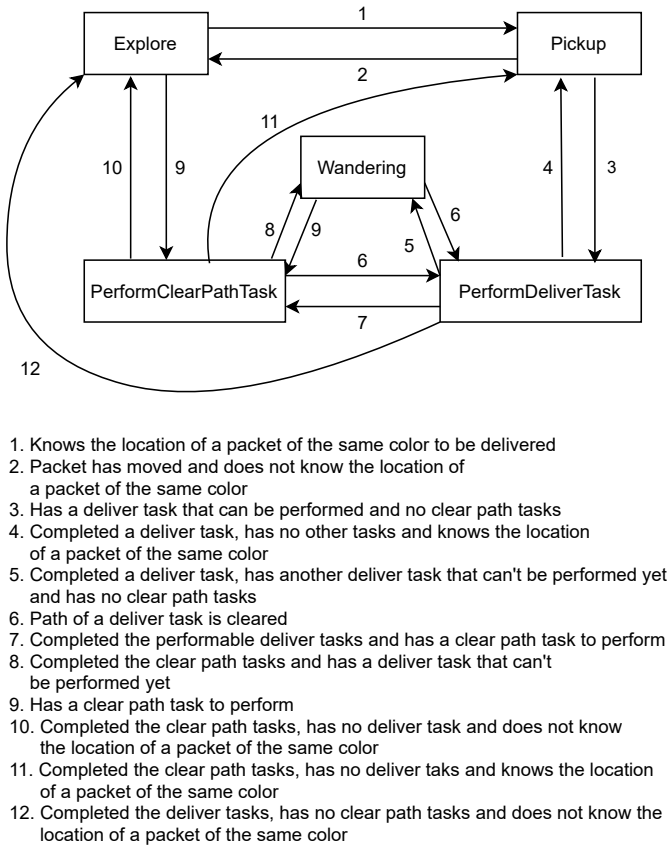


Fig. 7: State diagram of behaviors of task delegation

been completed when they come into each others range. The latter then sends out the next subtask. This is repeated until all subtasks are completed. Once all the subtasks have been completed, the agent then goes back to the `deliver` state, now with a clear path. This algorithm is successful most of the time, but there are some subtleties that sometimes provide an insurmountable challenge for the agents, causing the algorithm to fail or be inefficient. These shortcomings are listed and explored in section VII-A.

C. Testing & Discussion

As before, we evaluate the quality of the implemented behavior through simulations. This time the environment of choice is the provided `task-delegation` environment. The simulation consists of 100 runs and each run lasts until the agents have delivered all packets to their matching destinations. Runs which encounter soft deadlock (see VII-A) are omitted. We estimate that this deadlock occurs in around 7% to 9% of runs. Table V shows the relative frequency of each behavior.

Mean energy consumed	10779.65
Mean total cycles	306.18
Relative frequency of behaviors	
Explore	0.7
Pickup	0.39
PerformClearPathTask	0.08
PerformDeliverTask	0.30
Wandering	0.16

TABLE V: Analysis of meta-data for 100 simulations of the `task-delegation` environment

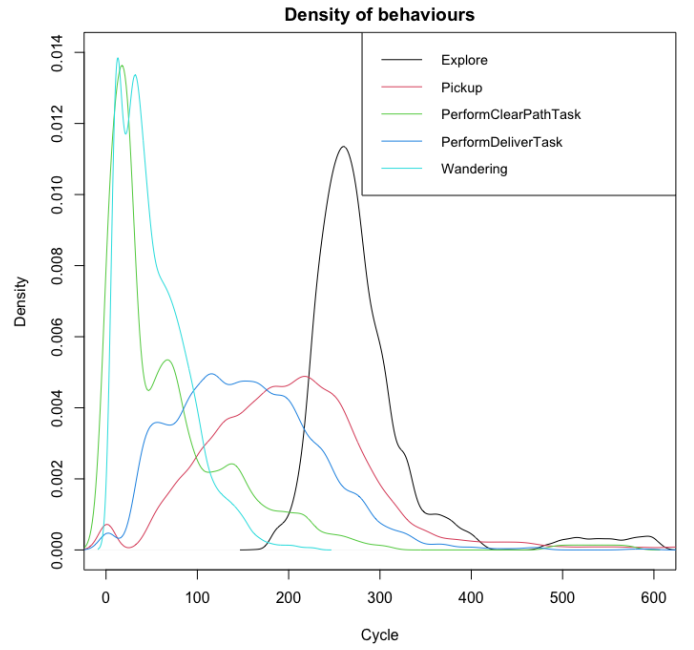


Fig. 8: Evolution of density of each behavior over cycles for `task-delegation`

Furthermore, we also consider how common each behavior is over the course of the simulations. Figure 5 shows how the occurrences of each behavior, summed across the 100 simulations, are distributed over time.

Initially, there are spikes of two behaviors: `PerformClearPathTask` and `Wandering`. The spike in frequency of `PerformClearPathTask` can be attributed to initial encounters between agents, where one agent recruits the other to clear a packet from some path. The recruited agent enters the `PerformClearPathTask` behavior while the recruiter enters the `Wandering` behavior, resulting in the apparent double spike.

Once an initial path has been cleared to each destination, the need to clear paths drastically lessens and agents start adopting the `PerformDeliverTask` behavior. As the first delivery tasks are completed agents are led to new tasks by the `Pickup` behavior, causing a delayed yet similar pattern as with `PerformDeliverTask`.

Around cycle 150 some agents run out of known packets to deliver. As a result, agents enter the `Explore` behavior, searching for new packets to deliver. In most cases, this is because there are no more valid packets in the environment. In the final part of the plot all frequencies slope downwards as fewer simulations run for these extended lengths.

VII. FUTURE WORK

A. Shortcomings

Although the solutions that were provided in this work deal with the first two problem statements fairly well (Realize autonomous behavior of agents to solve task and energy management), some major shortcomings are known. The first issue arises when multiple agents want to move down the same corridor from opposite sides. An example can be seen in Figure 9. The solution that this work provides did not include any mechanism to coordinate the movement amongst agents in this situation. This leads to a soft lock wherein the agents on both ends of the corridor endlessly try to cross it. In future work this issue can be addressed by having the agents announce their desired walking paths. Other agents can then respond in order to determine if

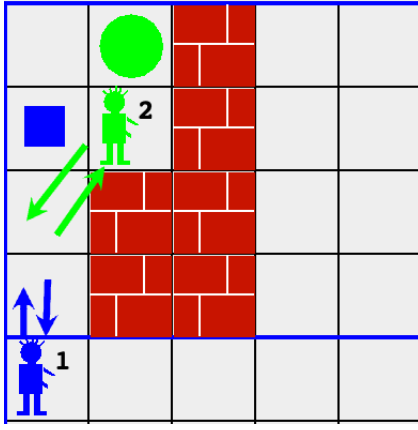


Fig. 9: An example situation where the agents are locked in a corridor.

the agent is allowed to move along its desired path. This essentially comes down to reserving resources (the positions in the grid) in a distributed environment. Some priority amongst agents needs to be decided when paths overlap. This can be based on the state of the agent, e.g. is it carrying a packet, in combination with the id of the agent.

The second problem is a direct consequence of the way in which packet information is shared. When packets are discovered they are stored within the agent's memory. This information is then shared with any agent it comes across. Packets are only removed from the agent's memory when the agent can perceive that the packet has disappeared from the location where the agent believed that it was located. This behavior can result in agents with outdated information on packet location. The problem then arises when an agent who has removed a packet from its memory repeatedly receives conflicting information from an agent with outdated information. The agent will believe this agent and use the false information which can lead to an endless loop of verification of the received info and receiving the same false information. This issue can be solved by extending or modifying the way that packet information is shared. One possible solution is to keep a timestamp with each packet location of when it was last seen or when it was last empty which can then be used to determine who has the most recent information to avoid that agents accept outdated data. Another solution is to use middleware that is built specifically to share this type of information in a distributed environment which would also reduce the complexity of the implementation in some aspects.

B. Maze environment

Next to the problems defined in Section IV an additional problem was identified, namely what happens when agents that need to deliver packets are placed in a maze environment. This work does not provide any solutions to solve this problem. However in this Subsection the fundamental ideas that were identified for a possible solution will be introduced. Some of these core ideas were inspired by the work of E. Kivelevitch and K. Cohen [15] who discuss algorithms for maze exploration in a multi-agent setting. Firstly, the agents will remember the path to the centre or starting location so that they can backtrack when they run into a dead end. Secondly, when agents find a dead end that does not contain any packets along the way they will mark the path with flags while backtracking until they meet an intersection that has other paths that are not yet explored and thus not yet marked with flags. This will ensure that agents do not endlessly explore dead ends once they have already been identified. Thirdly, agents will need to remember paths that lead to destinations in the maze so that they can find their way to the destination when they find a packet with

the corresponding color. Therefore it is important that paths that lead to a destination are never marked with flags. Finally, agents need to be able to communicate with each other to share information about destinations, packets and when they are crossing a corridor in the maze to coordinate their movement.

VIII. RELATED WORKS

A. Gradients

Agents discover energy stations with the help of gradients emitted by the stations to help guide the agents towards them. This approach is based on a work from Danny Weyns who described a solution wherein agents are attracted to tasks through the use of gradients that are emitted by these tasks [1]. The agents followed the path that leads them to the lowest gradient value (closest to the emitter of the gradient).

B. Consensus algorithm

A key component for agents to efficiently use the available charging stations is the design of a consensus algorithm to decide charging turns. For that, the agents that simultaneously want to charge can be seen as a local network of connected agents. The agents will be in each other's communication range. Consensus in such a networked MAS is explored in [10]. [8] introduces a leader-follower model, stating that having a leader proves to be efficient. Since it is unpredictable if a leader would need charging at the same time as when consensus is needed, this area was dropped. Dynamically electing a leader would turn out to be a consensus algorithm of its own. A leader-less approach is taken in [9]. The paper finds that "... if the adjacent topology of the graph is frequently connected then the consensus is achievable via local-information-based decentralized controls." This work draws inspiration from [9] by using a decentralized algorithm that always elects a clear order of charging by locally comparing information sent by each individual agent. The resulting consensus algorithm is described in full in section V-B.

C. Task delegation

The extension introduced in section VI-A introduces the need for agents to delegate tasks. To this end, [13] proposes a matrix of self-delegation consisting of four modes: economic exchange, gift exchange, authority and voting. This paper uses a mix of authority and gift exchange: each agent is an authority over the path it needs cleared, and commands other agents of the right color to clear it. This is based on a gift exchange principle: each agent knows it can request the clearing of a path and in exchange each agent knows to drop its current task to clear another agent's path if called to do so. This can also be found in [14], which proposes a trust calculation to decide whether to accept or deny incoming tasks. This paper can be seen as a simplification of this method. Each agent only calls the right agent for help at the right time. Other agents know this and assume trust, thus immediately performing the task at hand. For a concrete description of the proposed solution, refer to section VI-B.

D. Belief-Desire-Intention (BDI) agents

The key to realising autonomous behavior for agents stems from the ideas of Belief-Desire-Intention (BDI) agents that are defined in the work of A. Rao and M. Georgeff [16]. Their work defines BDI agents as agents that have certain mental attitudes that represent information, motivation and states of the agent. The environment in which the agents operate changes dynamically and agents are limited to sensing or perceiving the environment at certain intervals. This calls for the need to store information of the environment which it has sensed. This information system is called the beliefs of the agent. Our work uses this idea to store information about packet and destination locations. Each agent also needs to have some form of motivation or desire that will guide the agent in its decision making.

Our work defines this motivation as the desire to deliver all packets to their destination. The last keyword, intention, comes from the difficult situation where the agent needs to decide on which action to take before they risk that the environment has already changed. To this end, agents need to have intentions or states that represent the chosen course of action. Our work implements these states or intentions in different behaviors for agents. Agents change their behavior when certain criteria are met.

IX. CONCLUSION

This work set out to deliver some solutions to particular problems that concerns the behavior of situated agents and the coordination between these agents. More specifically, the Packet-World test bed was used to simulate and evaluate the behavior of agents that live in a grid world where they have to deliver packets. The three major challenges were movement and interaction in the packet world, energy management for agents with batteries and creating and distributing tasks amongst agents to help each other when needed. Several strategies and protocols were identified in this work that serve to solve these challenges. For the first challenge, behaviors were developed that allow the agent to pick up and deliver packets. The second challenge, energy management was solved by using communication protocols to determine which agent gets to charge. Lastly, the third challenge was solved by letting agents create and delegate tasks to each other that they themselves could not perform. Some of the provided solutions performed admirably in one case, but not so great in others. Multiple iterations of revision and improvement of the strategies and protocols helped to fine tune the solutions which lead to more optimized behavior (e.g. less energy consumed and less cycles needed to clear the world). However the solutions still contain noticeable flaws and will have to be solved in future work on this topic.

REFERENCES

- [1] D. Weyns, "Architecture-based design of multi-agent systems," 2010.
- [2] D. Weyns, A. Helleboogh, and T. Holvoet, "The packet-world: A test bed for investigating situated multi-agent systems," in *Software Agent-Based Applications, Platforms and Development Kits*, Whitestein Series in Software Agent Technologies, pp. 383–408, Basel: Birkhäuser Basel.
- [3] D. Weyns and T. Holvoet, "Architectural design of a situated multiagent system for controlling automatic guided vehicles," *International Journal of Agent-Oriented Software Engineering*, vol. 2, no. 1, pp. 90–128, 2008.
- [4] M. Grunow, H.-O. Günther, and M. Lehmann, "Strategies for dispatching agvs at automated seaport container terminals," *OR Spectrum*, vol. 28, no. 4, pp. 587–610, 2006.
- [5] F. Groen, M. Spaan, J. Kok, and G. Pavlin, "Real world multi-agent systems: Information sharing, coordination and planning," pp. 154–165, 09 2005.
- [6] J. Eriksson, N. Finne, and S. Janson, "Sics marketplace — an agent-based market infrastructure.," 1999.
- [7] B. Bennett, "Your roomba takes a weird path to clean the floors in your home. here's why it matters," June 2019. [Online; June 15, 2019 5:00 a.m. PT].
- [8] W. Ni and D. Cheng, "Leader-following consensus of multi-agent systems under fixed and switching topologies," *Systems & Control Letters*, vol. 59, no. 3, pp. 209–217, 2010.
- [9] J. Wang, D. Cheng, and X. Hu, "Consensus of multi-agent linear dynamic systems," *Asian Journal of Control*, vol. 10, no. 2, pp. 144–155, 2008.
- [10] R. Olfati-Saber, J. A. Fax, and R. M. Murray, "Consensus and cooperation in networked multi-agent systems," *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007.
- [11] X. Cui and H. Shi, "A*-based pathfinding in modern computer games," vol. 11, 11 2010.
- [12] D. Zobenica, "Graphs in java - a* algorithm." [Online].
- [13] M. Schillo, B. Fley, M. Florian, F. Hillebrandt, and D. Hinck, "Self-organization in multiagent systems: from agent interaction to agent organization," in *Third International Workshop on Modelling Artificial Societies and Hybrid Organisations (MASHO)*, pp. 37–46, 2002.
- [14] N. Griffiths, "Task delegation using experience-based multi-dimensional trust," in *Proceedings of the fourth international joint conference on Autonomous agents and multiagent systems*, pp. 489–496, 2005.
- [15] E. Kivelevitch and K. Cohen, "Multi-agent maze exploration," *Journal of Aerospace Computing Information and Communication*, vol. 7, pp. 391–405, 12 2010.
- [16] A. Rao and M. Georgeff, "Bdi agents: From theory to practice," 11 2000.

Multi-Agent Systems - Basic Delivery and Energy Handling with Packet-World

1st Ömer Yılmaz

*KU Leuven, Department of Computer Science
MSc. Artificial Intelligence, Multi-Agent Systems
Leuven, Belgium
omer.yilmaz@student.kuleuven.be*

Abstract—This paper is an introduction to multi-agent systems and aims to show its capabilities by a number of tasks. Multi-agent systems [1] include a number of agents which has restricted information about each other and aims to solve a common goal while sharing information and responsibilities. The first task is related to basic pick-up of packets and their delivery to destinations. Second task focuses on keeping track of energy during the delivery of packets without consuming all its energy and also focuses on how to reload energy whenever necessary. Finally, in the last task, collaboration is required among agents in order to achieve path planning in complex, maze-like environments. Basic pick up - deliver behaviour, battery considerations, information exchange, perception and path planning are discussed for a single agent as well as the coordination and collaboration of multiple independent agents with restricted knowledge about each other in an environment.

Index Terms—multi-agent systems, robotics, agent-oriented software engineering

I. INTRODUCTION

Multi-agent systems consist of several decision-making agents which interact in a common environment. Agents are realized with computer and typically a multi-agent system problem is addressed as a software development task [2]. For example, a group of drones could be governed with multi-agent system software and together they can solve a simultaneous localization and mapping (SLAM) problem [3]. They would create the map by information exchange, data comparison, matching and combination.

In this paper, we will show a number of problems through which we will study the multi-agent systems domain. Our motivation is to gain insights against different problem characteristics which we can meet in real-life challenges. Although our test bed is not intended to simulate the exact real life scenarios (it operates on a discrete 2D world instead of a continuous environment), we can still extract knowledge about how to select target destinations, handle obstacles along path, consider constraints in particular energy etc. In addition, we intend to share our approaches against these challenges like how we made the agent perceive and record the environment, how to keep track of energy and to reload it when necessary and how to direct to certain destinations using gradient descents etc.

We will also share our other experiences that we initially didn't consider. For example what happens if several agents goes low energy or communication topics among agents that

we don't expect to be necessary at first but happens to be critical. In addition some obstacles create complex path challenges that we didn't consider initially but face during the runtime.

In order to achieve these, we use a software called "Packet-World" [4]. In the next sections, we will introduce Packet-World, then describe the problems and tackle them. Finally, we will discuss our results and look for related works in the field gaining an insight for future work.

II. PACKET-WORLD

We can divide Packet-World into six packages as follows:

- **agent**
- **environment**
- **gui**
- **support**
- **synchronizer**
- **util**

gui includes graphical user interface related classes. *support* includes secondary classes which support certain functionalities of the application. *synchronizer* consists of classes responsible for synchronizing mechanisms of different threads in the application. Three packages we typically use are *agent*, *environment* and *util*. *agent* is responsible from agent behaviours and behaviour changes. *environment* includes classes which is responsible from the shared environment and the related worlds. *util* provides general purpose utility classes for the application.

A. Packet-World API

1) *agent Package*: *agent* package provides implementation classes of the *Agent* class and active item classes like energy stations. Agent class and other active item classes represent entities in the environment.

2) *environment Package*: *environment* package includes *Environment* class, world classes and environment-related utilities. Environment keeps all worlds which in turn keep related type items. These items can be active (moving) or passive. Worlds include representations of related types which are observable by the perception engine. Another important part of this package is the mail service. Perception and messaging is provided by the Packet-World and there is no need for further implementation.

3) *util Package*: *util* package provides extra utilities that support Packet-World coding e.g. *AsciiReader*, *Debug*, *My-Color*.

B. Batch mode

Batch mode allows the analysis of models without GUI overhead. Programmers can choose the number of runs for their session and the environment runs in the background without graphical interaction. You can see the batch mode interface from Fig. 1.

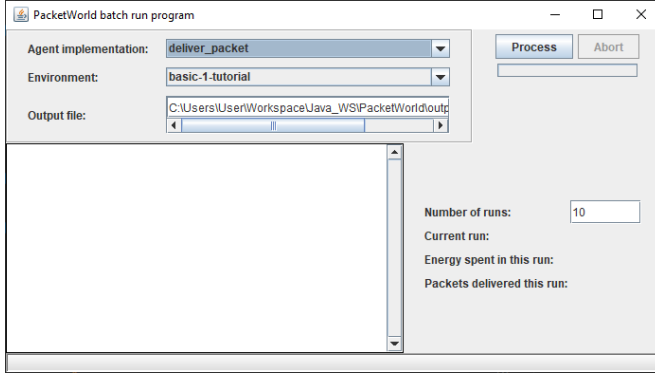


Fig. 1. Packet-World batch mode

C. World Editor

World editor allows programmers to design and build their environment. One can determine the width and height of the environment and the perception range of the agents. Agents can be inserted directly as well. Programmers can add packets and destinations of various colors. Energy stations, walls can also be added. An example is given in Fig. 2

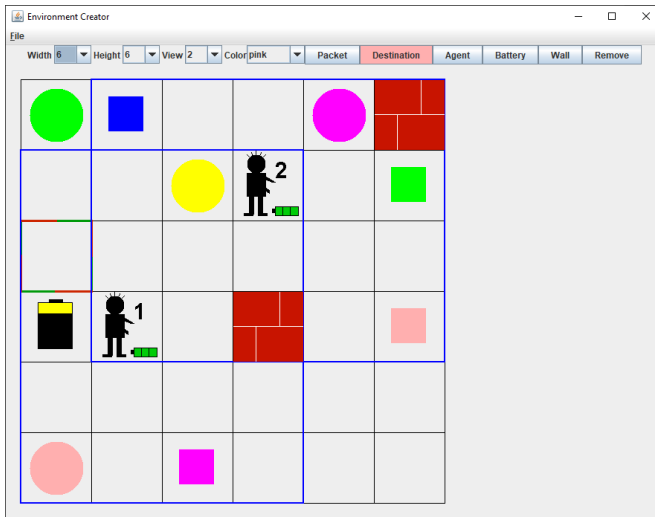


Fig. 2. Packet-World world editor

III. MULTI-AGENT SYSTEM TASKS: AUTONOMY, COORDINATION & COLLABORATION

In this section, we will present three different multi-agent system tasks focusing on basic autonomous behaviour of independent agents, coordination of agents and collaboration of them.

A. Autonomous Behaviour

Task 1 consists of a pick and put packet problem. There are 3 environments provided: basic-1, basic-2 and basic-3.

1) *Basic-1 environment*: There is one agent in this environment and there are a number of packets around as shown in Fig. 3. There is also one destination. The agent is able to perceive complete information about the environment as its view contains the whole set of cells. The task is simple: pick all packets and deliver them to the destination one by one.

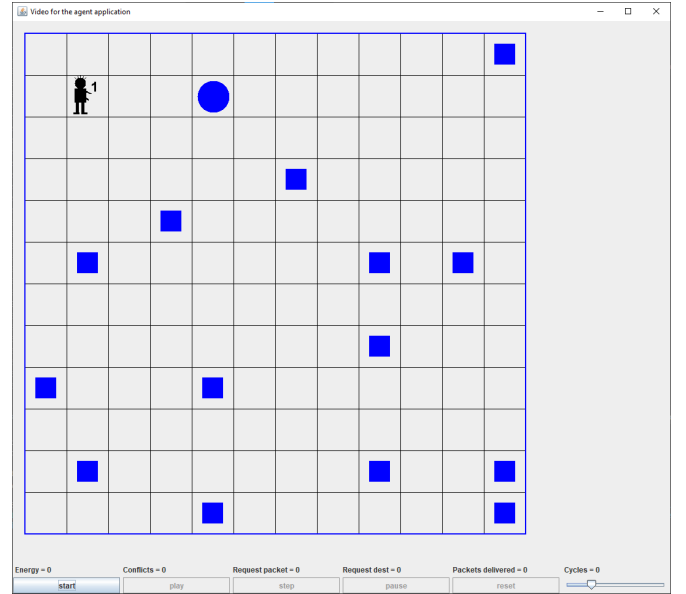


Fig. 3. Basic-1 tutorial environment.

2) *Basic-2 vision environment*: In this environment there is one more agent added in a larger environment and there are more packets to deliver as can be seen in Fig. 4. The main difference is that agents cannot see the whole environment as their view is now limited.

3) *Basic-3 color environment*: There are 4 agents in this environment and 4 different-colored destinations as shown in Fig. 5. In addition, packets have color property. 4 different colors are present and packets can be delivered to the same-colored destination.

B. Coordination

Coordination part introduces energy concepts. There are 2 environments added: energy-1 and energy-2.

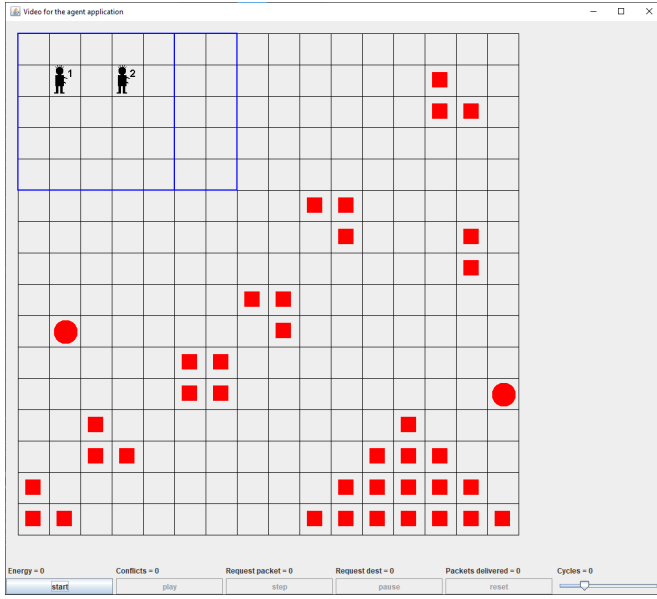


Fig. 4. Basic-2 vision environment.

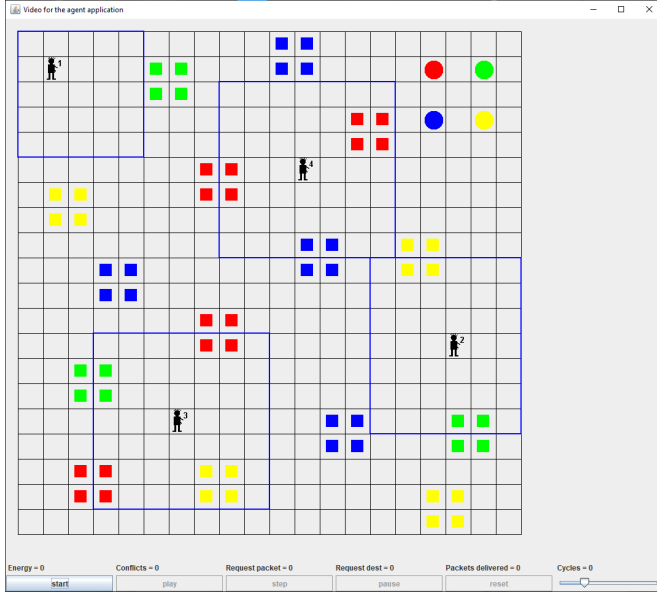


Fig. 5. Basic-3 color environment.

1) *Energy-1 environment*: Agents now have battery as seen in Fig. 6. Every step decreases the energy level. Stepping with a packet decreases more. Skipping action, which means staying still for a clock cycle, also takes some energy. Agents need to recharge their energy, otherwise they stop acting. For this, they need to go to an energy station.

In this task, gradient world is introduced in order to reach closest energy station. Mathematically gradients are vectors in the direction of steepest slope. By following the maximal gradient increase/drop, we reach to a local optima. Here, we placed energy stations to the local minimum points and embedded gradients into each cell of the gradient world. By

following the maximal gradient descent, the agents reach to energy stations. In another application, gradients could guide to different destinations.

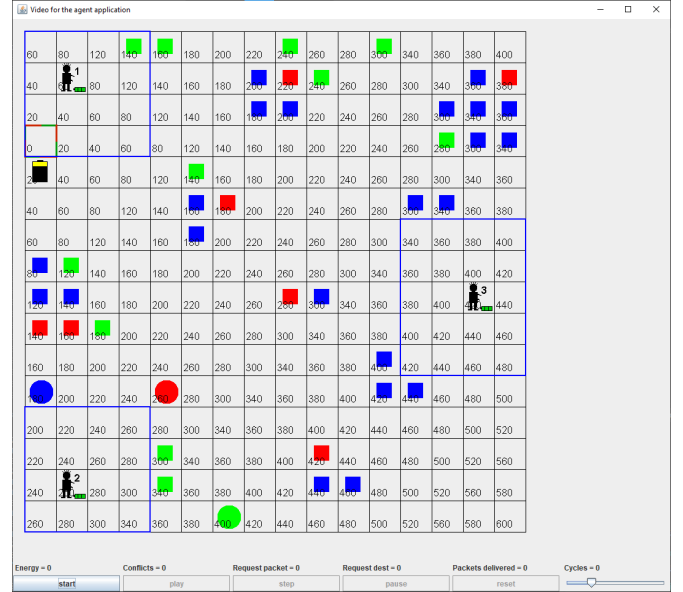


Fig. 6. Energy-1 environment.

2) *Energy-2 environment*: Energy-2 environment includes much more packets with 1 more color and 6 agents compared to 2 of energy-1 environment as shown in Fig. 7. In addition to one central energy station, there is one more. However, it is far from the destinations.

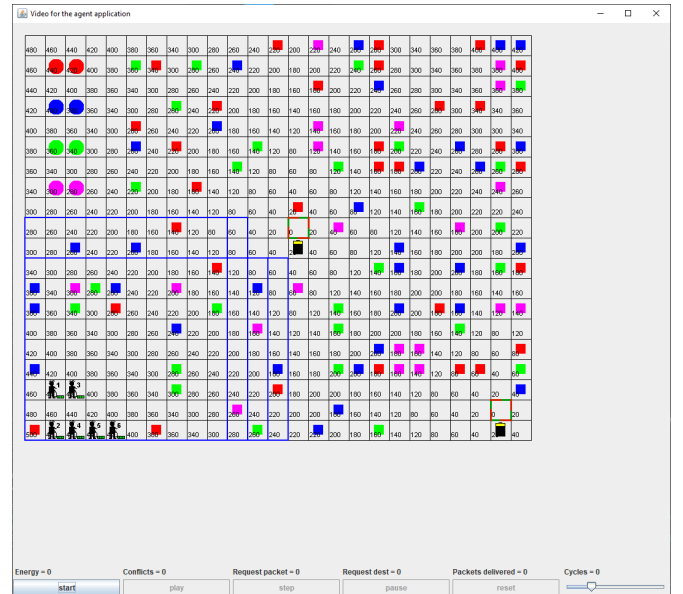


Fig. 7. Energy-2 environment.

3) *Optional energy-3 environment*: Energy-3 is optional. Walls are introduced, which can be seen from Fig. 8. Wall structures make agents stuck behind if there is no random walk or preplanning mechanisms of agents.

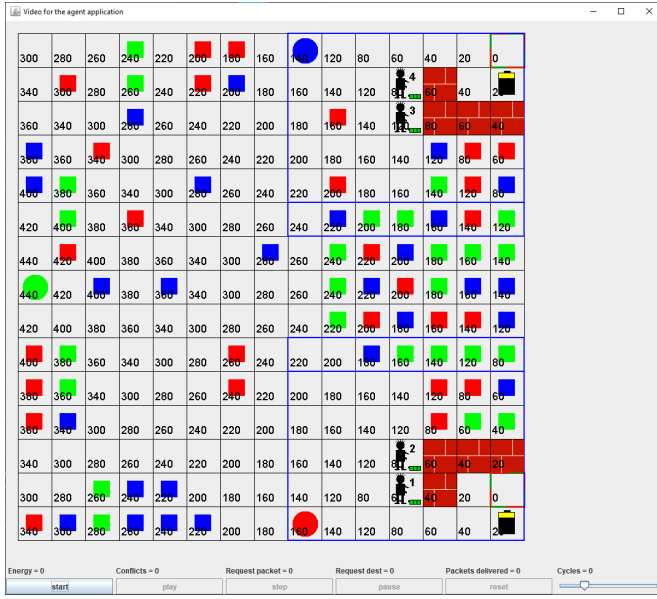


Fig. 8. Energy-3 environment.

C. Collaboration

1) *Maze environment*: As Fig. 9 shows, we have a maze now. This cannot be achieved with one-step-ahead planning. Agents need to record certain cornerstones of the environment. They may use trial and error approach. However, they also need to backtrack whenever necessary as the branchy structure of the environment shows.

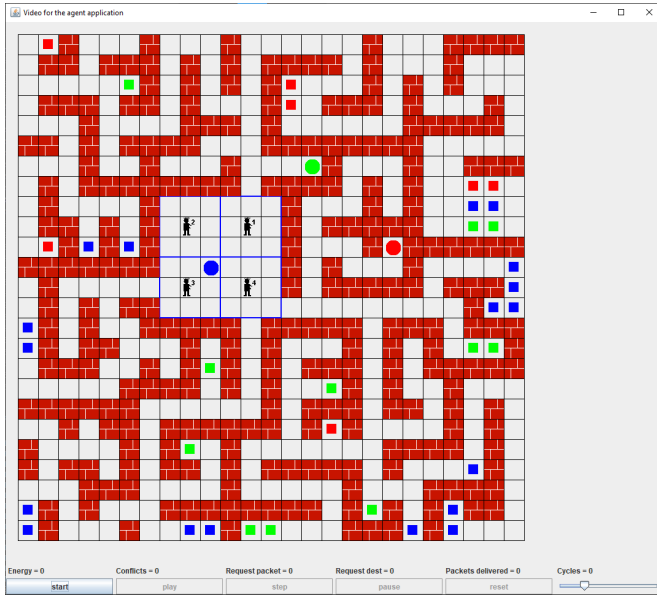


Fig. 9. Maze environment.

2) *Task delegation environment*: Task delegation environment requires collaboration among agents. When you analyze Fig. 10, you may notice that all destinations are hidden behind a number of packets. Considering an agent can only deliver

one package at a time, it is clear that the agent requires help from the other agents to move away the obstacles (other packets) from the road. Besides, agents now have a color associated. They can deliver only the same-colored packages to the same-colored destination.

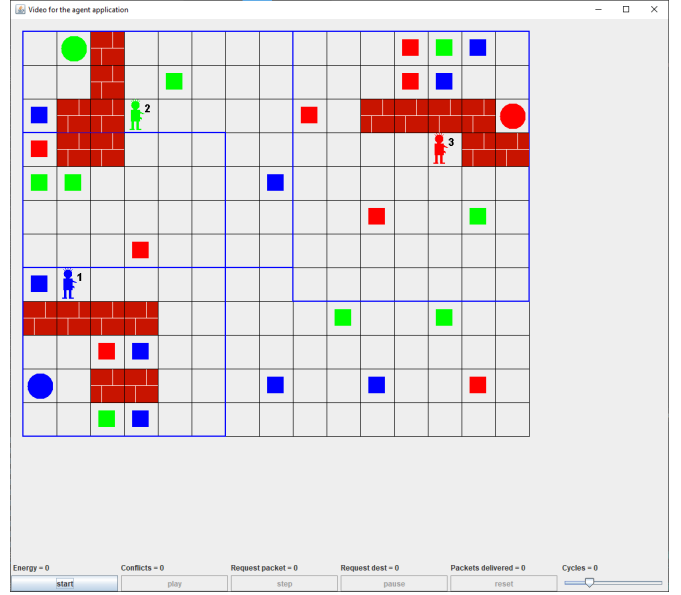


Fig. 10. Task delegation environment.

IV. HOW TO TACKLE THE TASKS

A. Autonomy Task Approach

The autonomy task has three parts as discussed:

1) *Basic-1 tutorial environment*: At this sub-task, we have tested basic perception acquisition, step-skip movements and pick-put behaviours. A random walk code was provided by Packet-World. However, we want the agent to direct to a nearby packet. That's easy though because now the agent has complete knowledge about the environment. What we have done was simply to sort the distances of packets according to the agent itself. Then, the agent is directed to the packet and when it reached to the vicinity of a packet, it picked it up. Now, the agent has a carry and it looks for the closest destination to deliver. With similar algorithm it reached to the destination and put its carry. Carrying a packet determined the state of the robot which is specified in the act method of *DeliverPacket* class. You can see the results of trials from Table I. After the given number of cycles, all packets are delivered successfully.

2) *Basic-2 vision environment*: Basic-vision environment limits the vision of agents. Agents behave according to the frame of perception. Destination addresses are critical. That's why now they are memorized. Agents have their own memory. They keep the following items:

- **visiblePackets**: stores the coordinates of packets directly visible at a clock cycle and their associated colors.
- **destinations**: stores the coordinates of known destinations and their associated colors.

Trials	All packets delivered	#Cycles	Consumed energy
1	True	162	2130
2	True	166	2180
3	True	164	2160
4	True	163	2150
5	True	163	2140
6	True	163	2150
7	True	162	2130
8	True	162	2130
9	True	164	2160
10	True	165	2180

TABLE I
BASIC-1 TUTORIAL ENVIRONMENT TRIALS

- **energyStations:** stores the coordinates of known energy stations.
- **visibleAgents:** stores the agents directly visible at a clock cycle.
- **visitedCells:** stores the coordinates of visited cells through the life of agent.
- **walkableCells:** stores the coordinates of known walkable cells.
- **goalPosition:** stores the coordinate of current target which is used to generate moves at each step.
- **energyStationPriority:** stores the priority of agent for energy station queue.

Packets, though, are dynamic and consumed. Therefore, we chose not to memorize the packets. When there is no destination discovered or no packet perceived, the agent needs to move randomly to discover the environment.

In this sub-task, we separated different behaviours of the agent and created a state machine as can be seen from Fig. 11. When the condition of "doesn't have carry" is activated, the agent is using *PickPacketToPutPacket* behaviour class. When it puts the packet, "doesn't have carry" condition is no longer activated and "has carry" condition is active. "has carry" condition is used to enter *PutPacketToPickPacket* class behaviour. For these classes we created a configuration text according to Packet-World rules. You can see the success statistics from Table II. All packets are delivered successfully in this environment as well. Here, the number of cycles increased as it could be expected from the increased complexity with limited vision.

Trials	All packets delivered	#Cycles	Consumed energy
1	True	201	5140
2	True	211	5180
3	True	330	8810
4	True	232	5880
5	True	344	9110
6	True	318	8590
7	True	244	6030
8	True	347	9280
9	True	319	8550
10	True	196	4900

TABLE II
BASIC-2 VISION ENVIRONMENT TRIALS

3) *Basic-3 color environment:* Basic color environment introduces us colorful packets and destinations. As we men-

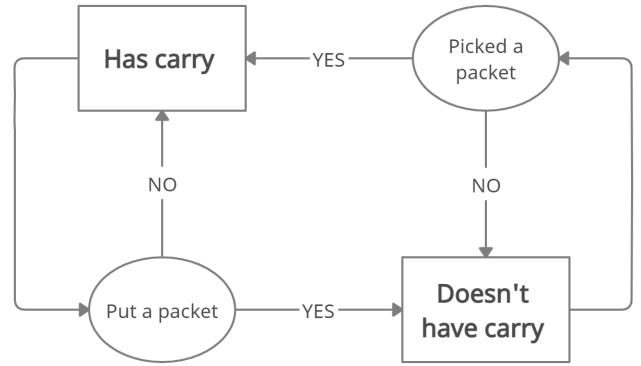


Fig. 11. Behaviour division

tioned on the Section III-A3, it is necessary to put packets to the same colored destinations. In order to address this, we queried the colors of the destinations whenever the agents see a destination. Then, their positions and colors are memorized. When the agent carries a packet, it simply looks for the closest same-colored destination. Note, there is no agent color introduced at this moment. You can see the results at Table III. All packets are delivered successfully to the same-colored destinations.

Trials	All packets delivered	#Cycles	Consumed energy
1	True	429	24470
2	True	398	22900
3	True	389	22170
4	True	488	27900
5	True	431	25400
6	True	403	22840
7	True	434	25350
8	True	413	23940
9	True	398	23150
10	True	368	20410

TABLE III
BASIC-3 COLOR ENVIRONMENT TRIALS

B. Coordination Task Approach

1) *Energy-1 environment:* Energy-1 environment requires gradient values embedded into GradientWorld. Value assignments to the cells are handled during the energy station creation. Agents move toward the least-valued neighbour cell. One important detail is that we choose the lesser value when the cell is subjected to the effects from multiple energy stations. This way the agent moves towards the closest energy station.

The behaviour is simple: at a cell if the amount of energy required to go to the closest packet/destination + to the closest energy station from there is greater than the current energy owned, just go to the closest energy station via gradients. Otherwise combine both actions: pick/put packet and after pick/put action go to the closest energy station.

We provided both single-class behaviour configuration and multiple-classes behaviour configuration. In particular, we had

```
graph TD
    Start([Start]) --> Around[Around energy station]
    Around --> Leave([Leave energy station])
    Leave -- NO --> Around
    Leave -- YES --> Carry([Carrying a packet])
    Carry -- NO --> Around
    Carry -- YES --> HasCarry[Has carry]
    HasCarry --> EnergyCriticalL([Energy level critical])
    EnergyCriticalL -- YES --> Around
    EnergyCriticalL -- NO --> PutPacket([Put a packet])
    PutPacket -- YES --> HasCarry
    PutPacket -- NO --> Around
    HasCarry --> Around
    HasCarry --> NoCarry[Doesn't have carry]
    NoCarry --> EnergyCriticalR([Energy level critical])
    EnergyCriticalR -- YES --> Around
    EnergyCriticalR -- NO --> PickedPacket([Picked a packet])
    PickedPacket -- YES --> HasCarry
    PickedPacket -- NO --> NoCarry
    NoCarry --> Around
    NoCarry --> ReachedR([Reached to energy station])
    ReachedR -- YES --> Around
    ReachedR -- NO --> NoCarry
```

In vicinity of the energy station, if there is only one agent and of course if it hasn't got near-full energy, it can directly claim the energy spot. When there are several agents exist emergency should be considered. Energy level less than 20% indicates emergency. If there is no emergency situation, other agents simply wait for the one using the station. If there is emergency situation though, the most needing one takes the turn with least amount of energy. Remember we create a sorted list of emergency. This, however, has an exception. If the loading agent has energy level smaller than 10%, it simply ignores any other agents even if they have less energy than 10%. In that case *independent agent* selfishly refuses to leave the station. Switching actions in such a very low energy situation is unnecessarily wasting energy anyways.

```

graph TD
    Start([Start]) --> Stay[Stays Around Energy Station]
    Stay --> E95{Energy state < 95%}
    E95 -- YES --> Loading([At Energy Station Loading])
    Loading -- YES --> MostNeed[Is the most needing agent?]
    MostNeed -- YES --> Stay
    MostNeed -- NO --> E10{Energy state < 10% Ignore}
    E10 -- YES --> Stay
    E10 -- NO --> End([End])
    E95 -- NO --> E97{Energy state < 97%}
    E97 -- YES --> Stay
    E97 -- NO --> Move[Moves Away From Energy Station]
    Move --> End
  
```

The flowchart illustrates the decision process for an Energy Station Agent. It begins with a start node leading to the 'Stays Around Energy Station' process. From there, it checks if the 'Energy state < 95%'. If YES, it proceeds to 'At Energy Station (Loading)'. If YES, it checks 'Is the most needing agent?'. If YES, it loops back to 'Stays Around Energy Station'. If NO, it checks 'Energy state < 10% (Ignore)'. If YES, it loops back to 'Stays Around Energy Station'. If NO, it ends. If the initial 'Energy state < 95%' check is NO, it checks 'Energy state < 97%'. If YES, it loops back to 'Stays Around Energy Station'. If NO, it proceeds to 'Moves Away From Energy Station', which then ends.

target generation requirement anyways. However, at the very first start when there is no known recorded energy station or destination, this trick is used as well.

In this part, agents now not only share emergency energy station request but also walkable cells. This allows to span the environment more rapidly. From Table IV, we can observe that all the packets are delivered in given number of steps. We can also see that an agent consumed its all energy and failed in some of the trials. However, the other agents delivered all remaining packets.

Trials	All packets delivered	#Cycles	#Stuck agents	Consumed energy
1	True	399	0	15480
2	True	569	0	23980
3	True	980	1	31655
4	True	367	0	14305
5	True	427	0	17375
6	True	853	1	26235
7	True	376	0	14520
8	True	668	1	19705
9	True	565	0	24815
10	True	398	0	16220

2) *Energy-2 environment*: Environment-2 environment has an outsider energy station which is far from destinations. Therefore, the central energy station is used more frequently

and a traffic jam is observed around it. Some agents may consume all their energies and fail, while waiting for the energy station with several other agents also waiting in the similar position and similar low state. Agents with 0% energy got stuck and become obsolete around the station. In order to tackle this, the agents communicate with each other when they are around the energy station. The agents whose battery level decreased below 20% emits a message to the visible agents indicating their ID's and battery percentage. Then, a sorted list is created among the agents around the energy station which indicate emergency. This environment too utilizes GradientWorld as it has energy stations. From Table V, we can observe the amount of required energy and time cycles in order to deliver all packets. The methods added for this task:

Trials	All items delivered	#Cycles	#Stuck agents	Consumption
1	True	1129	0	81375
2	True	1158	0	90385
3	True	1090	0	84210
4	True	1190	0	91765
5	True	1099	0	81390
6	True	1027	0	76025
7	False	-	-	-
8	True	1167	0	84985
9	True	1307	0	101900
10	True	1207	0	95455

TABLE V
ENERGY-2 ENVIRONMENT TRIALS

- **seeEnergyStations:** records the energy stations inside the perception frame.
- **checkEnergyOfPlan:** move generation refers to this function for energy station distance checking.
- **retrieveMessages:** main function for managing mails coming from other agents. It has heavy computational parts like walkable cell combination if activated.
- **generateRandomMoves:** generate moves towards a random unwalkable target (prioritizing unvisited cells if activated).
- **placeItem:** places gradients when an energy station is inserted.

V. WHAT THE RESULTS SAY?

A. Autonomous Behaviour

In the basic tutorial environment, to measure distances we could use Manhattan distance or max absolute distance. For this task, the difference is unimportant. We could also use the smallest *pick + put* distance in total and not look for them separately in each step. However, with similar argument we could also look for the best the *pick + put + pick* or *pick + put + pick + put* travel route distances as well. In fact, it is subject of search tree and executing steps separately is simpler and more readable.

Agents finished the delivery of Packets with the closest packet/destination heuristic at the basic vision environment without aiming to random targets if there is a perceived target. Even when there is no perceived target, random selection is smart. It never targets walkable cells. Therefore, the delivery of

packets achieved in near optimal time (for one step lookahead as described above). Using separate classes for the different behaviours and creating a state machine for them simplified the design. Design simplification as a part of overall architecture design process is one of the fundamental concepts in multi-agent systems. We could arguably make agents share destination information with each other. However, message sending and retrieval is of later tasks' problem.

The 4 destinations of the basic color environment are very close to each other. Therefore, care is needed to confirm that the agents deliver to the correct color as expected; because the deliveries could be targeted to color a but can be delivered to b while the agent is nearby both destinations of color a and b. Therefore, simple approach algorithms do not suffice and an actual color check nearby the target is necessary. Other than that, similar arguments to the basic vision environment hold for this environment as well. There is no e-mail mechanisms or any other information exchange yet. Considering communication limitations, all packets being delivered to their corresponding destinations with a distance heuristic is more than sufficient for this part.

B. Coordination

We calculate distances while creating GradientWorld. It could be Manhattan distance as well and we added a small extra distance per step in order to compensate for uncertain dynamics and unobservable effects that diverges the agent from the original path. One zone of the energy-1 environment requires special planning. As can be seen in Fig. 14, an agent at the bottom of the red zone wants to load energy. However, our distance algorithm cannot guide the agent towards right in order to pass the packets block. Because the right side is not any closer to the directed energy station than the left cell. Therefore, at least a shuffling mechanism is necessary to get out of the local area whenever agents get stuck. However, our aim is to use a complete planning package as we will describe in the future work section.

The methods added for this task:

- **seePackets:** records the packets inside the perception frame.
- **seeDestinations:** records the destinations inside the perception frame.
- **generateMoves:** generates possible single steps directed to a target.
- **generateRandomMoves:** generates possible single steps directed to a random target which is not a member of walkable cells, prioritizes unvisited cells.

Energy-2 environment contains much more agents and packets compared to earlier ones. The behaviour around the central energy station is problematic. That's because there is only one energy station close to the destinations (the other energy station is far from the destinations). In order to keep every agent active, we created the queue of low energies around the energy station. This, however, required certain amount of fine tuning. We also added fine tuned distance values. Because the battery decrease expectation decides the packets to which

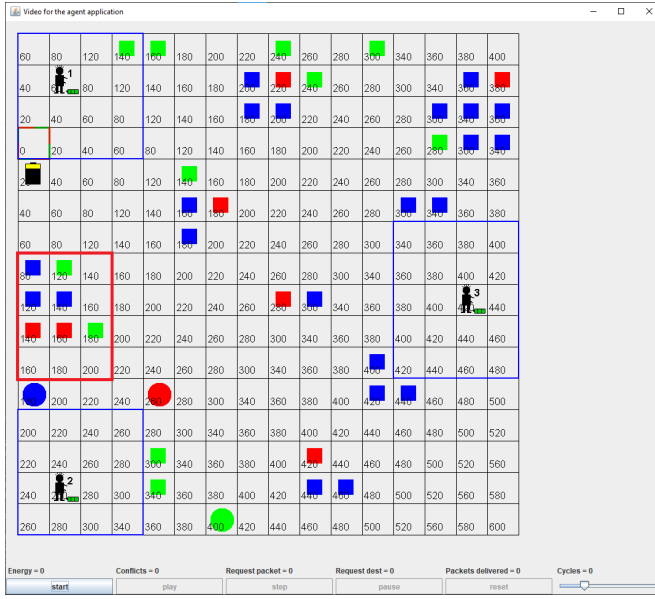


Fig. 14. Energy-1 environment zone where the agents get stuck.

agents can reach. If the total energy required to reach a packet and then to reach an energy station is more than the currently stored energy, result could be 100% energy consumption for the agent, a failure. Although agent may discover new energy stations during the travel, this approach is not safe. Similarly, if the total energy required to reach a destination for delivering and then to reach an energy station is more than the currently stored energy, result would be 100% energy consumption.

Energy environments shares walkable cells. Whenever two agents see each other, they send their whole list of walkable cells. The unknown walkable cells coming from the other agent are added to the current list of walkable cells. Basically agents synchronize their information. However, the computational burden of walkable cell sharing was considerable. That's why we didn't share any other lists of agents. One may choose to share energy stations or destinations with a very similar approach to the one we have provided (code) for walkable cells in the implementation. An agent can send one message during the communication. That's why we have provided an encoding scheme allowing to combine different lists into one message and inside each list a separation mechanism for different items of lists. Receiving agent parse the combined message during communication phase before action.

VI. FUTURE WORK AND RELATED STUDIES

In the collaboration part one-step-planning is no longer sufficient and we need to plan for future steps. Because, "the directed random steps" can get stuck in certain branches of the maze such that they try the same movements repeatedly and do not discover sufficiently simply because they don't foresee the destinations hidden behind longer paths. The algorithm uses the basic distance and not the path distance while approaching. For planning we can make use of a classical planning system like "Fast Downward" [5] or even better we

could use "PDDLStream" [6] which combines Fast Downward program with planning of agent actions including continuous parameters. For example, it is no longer necessary to plan on a grid cell but we can use continuous pose values. PDDLStream combines both classical planning with continuous parameter streams that generate continuous domain arguments for certain actions like `moveTo(x,y)` where `x` and `y` are not necessarily discrete integers. These actions can be anything like pick, put, move, grasp etc... It is clear that Packet-World, as given in the tasks, lacks continuous domain support. It would be useful for industrial applications if it supported continuous domain. Meanwhile, grids could still be used to granularize the worlds.

In fact we already started to implement the Packet-World - PDDLStream combined application. We have already achieved the following:

- We have created a Python script that starts both programs correctly.
- The script includes a Flask code and starts web server.
- Flask code accepts agent id and starts a PDDLStream session for each agent.
- Agent behaviour class sends a json file via http POST request.
- Agent behaviour class sends a http GET request expecting solution.

Necessary things are:

- Agent behaviour class should embed information from all worlds into json.
- Flask script should parse the json and turn it into *initial state* for PDDLStream.
- We should create "*domain.pddl*" [7] file content for the Packet-World environment which determines the predicates and actions available for classical planning [8], [9] in the Packet-World environment e.g *moveLeft*, *moveRight*.
- We should clarify goal achievement conditions e.g. an implementation containing *noPacketLeft*, *packetFound*, *energyStationFound* predicates.
- Solution steps should be returned to Packet-World in the http GET response and be parsed into correct agent steps by the agent behaviour class in the communication phase.

Further expectations could be:

- To introduce continuous coordinates to Packet-World in order to fully use PDDLStream's "classical planning with continuous parameters" capability.
- Introduce other items and world to use and experiment with other predicates like *grasp*.
- Try to conceptualize the software for a real world application e.g. SLAM [3] and drone delivery.

The environments we introduced could be a subject of reinforcement learning applications as well. Reinforcement learning agents have a limited "perception" as in the Packet-World. The agents behave according to their limited world view. Very similar applications can be found [10] in the literature. Because of the partially independent behaviour of agents, several reinforcement learning guided agents can be

placed to the environment and work similarly to the case of decentralized multi-agent systems.

Packet-World has been guiding the researchers as in [11] which introduces a new multi-agent actor-critic method, counterfactual multi-agent (COMA) policy gradients.

Network packet routing is naturally modeled as multi-agent reinforcement learning problem. In [12] which is about stabilising experience replay in Q-learning, Packet-World has been useful as it is a packet delivery test bed.

In addition Bromuri has been using Packet-World repeatedly [13], [14] and his focus was mainly situated cognitive agents in a distributed agent environment. He also tried to represent distributed agent environment with logic-based formalism. Interaction between agents, objects, and containers are supported in his Ambient Event Calculus (AEC) and they show how to specify interactions within the GOLEM agent platform.

Also, Weyns further introduces us the Packet-World with [15] and in the book [2].

REFERENCES

- [1] Weiss, Gerhard, ed. *Multiagent systems: a modern approach to distributed artificial intelligence*. MIT press, 1999.
- [2] Weyns, Danny. *Architecture-based design of multi-agent systems*. Springer Science and Business Media, 2010.
- [3] Durrant-Whyte, Hugh, and Tim Bailey. "Simultaneous localization and mapping: part I." *IEEE robotics & automation magazine* 13.2 (2006): 99-110.
- [4] Weyns, Danny, Alexander Helleboogh, and Tom Holvoet. "The packet-world: A test bed for investigating situated multi-agent systems." *Software Agent-Based Applications, Platforms and Development Kits*. Birkhäuser Basel, 2005. 383-408.
- [5] Helmert, Malte. "The fast downward planning system." *Journal of Artificial Intelligence Research* 26 (2006): 191-246.
- [6] Garrett, Caelan Reed, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. "PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning." *Proceedings of the International Conference on Automated Planning and Scheduling*. Vol. 30. 2020.
- [7] Strobel, Volker, and Alexandra Kirsch. "Planning in the wild: modeling tools for PDDL." *Joint German/Austrian Conference on Artificial Intelligence (Künstliche Intelligenz)*. Springer, Cham, 2014.
- [8] Helmert, Malte. "Concise finite-domain representations for PDDL planning tasks." *Artificial Intelligence* 173.5-6 (2009): 503-535.
- [9] Bylander, Tom. "The computational complexity of propositional STRIPS planning." *Artificial Intelligence* 69.1-2 (1994): 165-204.
- [10] Karpathy, Andrej. "<https://cs.stanford.edu/people/karpathy/reinforcejs/>"
- [11] Foerster, Jakob, et al. "Counterfactual multi-agent policy gradients." *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. No. 1. 2018.
- [12] Foerster, Jakob, et al. "Stabilising experience replay for deep multi-agent reinforcement learning." *International conference on machine learning*. PMLR, 2017.
- [13] Bromuri, Stefano, and Kostas Stathis. "Situating cognitive agents in GOLEM." *International Workshop on Engineering Environment-Mediated Multi-Agent Systems*. Springer, Berlin, Heidelberg, 2007.
- [14] Bromuri, Stefano, and Kostas Stathis. "Distributed agent environments in the ambient event calculus." *Proceedings of the Third ACM International Conference on Distributed Event-Based Systems*. 2009.
- [15] Weyns, Danny, and Tom Holvoet. "A reference architecture for situated multiagent systems." *International Workshop on Environments for Multi-Agent Systems*. Springer, Berlin, Heidelberg, 2006.

APPENDIX

ENVIRONMENTS USED DURING THE RESEARCH

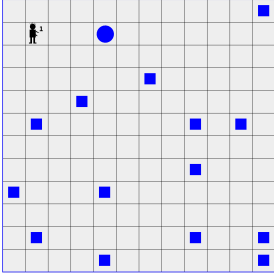


Fig. 9. Set up of the *basic-1-tutorial* environment used for testing basic functionalities. [Section IV-A]

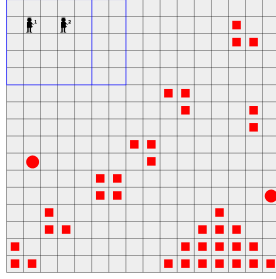


Fig. 10. Set up of the *basic-2-view* environment used for testing view constraints. [Section IV-A]

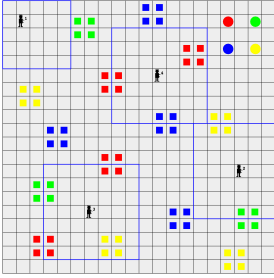


Fig. 11. Set up of the *basic-3-tutorial* environment used for testing color constraints. [Section IV-A]

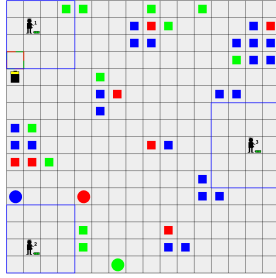


Fig. 12. Set up of the *energy-1* environment used for testing energy coordination with small view. [Section IV-B]

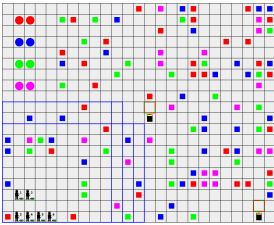


Fig. 13. Set up of the *energy-2* environment used for testing energy coordination with large view. [Section IV-B]

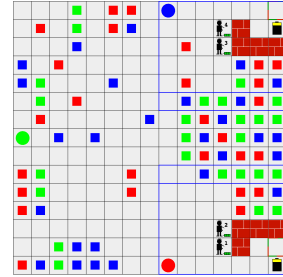


Fig. 14. Set up of the *energy-3* environment used for testing advanced energy coordination. [Section IV-B]

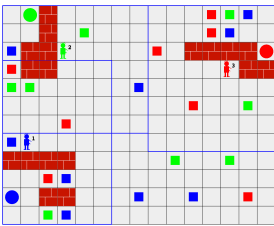


Fig. 15. Set up of the *task-delegation* environment used for testing cooperation with colored agent constraints. [Section IV-C]

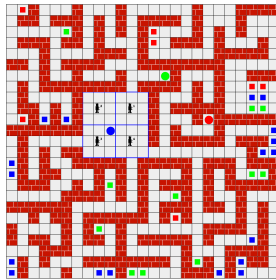


Fig. 16. Set up of the *maze* environment used for testing cooperation with small view and tricky map. [Section IV-C]